INTI Parking System, IICP Park Finder

Name: LIM NI FEIH

Coventry Student ID: 12672774

Supervisor Name: Usha Jayahkudy

Bachelor of Science (Hons) in Computer Science, 3+0 in Collaboration with Coventry University

School of Computing, INTI International College Penang

8th November 2024

6001CEM Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detecti.

Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialise products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (https://ethics.coventry.ac.uk/) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking)

Signed: LNF Date: 6 November 2024

Please complete all fields.

First Name:	Ni Feih
Last Name:	Lim
Student ID number	P22013772
Supervisor Name	Usha Jayahkudy

This form must be completed, scanned and included with your project submission to Turnitin. Failure to append these declarations may result in your project being rejected for marking.

ACKNOWLEDGEMENT

Firstly, I would like to thank my parents for allowing me to study in INTI International College Penang as a Bachelor of Science in Computer Science student as I would not have been able to continue my studies and experience the journey as a college student today if it were not for them. Not only that, but I am also extremely grateful for the emotional support and understanding shown by them which has not only gotten me through my final year project but through my entire degree as well.

I would also like to thank my supervisor Ms. Usha Jayahkudy who agreed to be my supervisor for my final year project. Without her support and supervision, I would not have been able to complete the project on time and turn it into a successful system. Her patience and guidance not only helped me plan and execute my project well but also strengthen my mental wellbeing to complete the project especially when faced with the many challenges and stressful situations for the past two semesters. It is also with the help of Ms. Usha that I managed to collaborate with Mr. Syazwan, the Head of Admin and Facilities Management department for this project.

Lastly, I want to thank Mr. Syazwan for spending the time and effort through all the meetings from discussing the requirements of the project to executing the user acceptance testing for the web application despite his busy schedule. It is only with Mr. Syazwan's help that I was able to gain insights into the current parking system in INTI to determine the requirement of the admin role for the system and the mobile application can show an accurate display of parking lots in the map with the layout provided by him. Additionally, I would also like to thank him for being very patient and supportive throughout the project from start to end which made the project journey enjoyable.

Table of Contents

ACKNOWLEDGEMENT iii
LIST OF TABLES ix
LIST OF FIGURES xi
LIST OF ABBREVIATIONS xvii
ABSTRACT xix
CHAPTER 1: INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 2
1.3 Research Questions 7
1.4 Objectives 7
1.5 Client, Audience, Motivation 7
1.6 Scope 8
1.7 Conclusion 8
CHAPTER 2: LITERATURE REVIEW 9
2.0 Definition of Topic 9
2.1 Image Processing Methods 16
2.1.1 Background Subtraction 16
2.1.2 Edge Detection and Line Segmentation 18
2.1.3 Feature-based Object Detection 20
2.1.4 Comparison of Image Processing Methods 23
2.2 Shortest Path Algorithm 26
2.2.1 Dijkstra's Algorithm 26
2.2.2 Bellman-Ford Algorithm 27
2.2.3 A* Search Algorithm 28
2.2.4 Comparison Between the Shortest Path Algorithms 30
2.3 Existing Vacant Parking Space Detection System 32
2.3.1 Penang Smart Parking Infrared Sensor 32
2.3.2 Ultrasonic Sensor in Shopping Mall Parking Lot 33
2.3.3 Comparison between Existing Systems and IICP Park Finder 35
2.4 Database 39
2.4.1 Google Cloud Firestore 40
2.4.2 MongoDB Atlas 41
2.4.3 Comparison between Google Cloud Firestore and MongoDB Atlas 43
2.5 Proposed Work 45

2.6 Summary 47
CHAPTER 3: RESEARCH METHODOLOGY 48
3.1 Introduction 48
3.2 System Development Methodology 48
3.2.1 Requirements Phase 49
3.2.2 Design Phase 50
3.2.3 Development Phase 51
3.2.4 Testing Phase 51
3.2.5 Deployment Phase 52
3.2.6 Maintenance 52
3.3 System Design 53
3.3.1 Use Case Diagrams 53
3.3.2 Module Flowchart Diagrams 56
3.3.2.1 Mobile Application Module Flowcharts 56
3.3.2.2 Web Application Module Flowchart 62
3.3.3 Data Model Diagram 67
3.4 Gantt Chart 69
3.5 Development Tools 72
3.5.1 Flutter 72
3.5.2 Firebase Database (Firestore) 73
3.5.3 Vue.js 74
3.5.4 Google Maps API 75
3.5.5 Google Earth Pro 76
3.5.6 QGIS 76
3.5.7 Yolov8 77
3.5.8 DeepSort (PyTorch) 79
3.5.9 GitHub (Version Control) 80
3.6 Summary 80
CHAPTER 4: PROJECT IMPLEMENTATION 81
4.1 Introduction 81
4.2 Mobile Application 81
4.2.1 Login 81
4.2.2 Register 82
4.2.3 Forgot Password 85
4.2.4 Parking Page 86

4.2.5 Contact Page 92
4.2.6 Inbox Page 94
4.2.7 Settings Page 95
4.2.8 Profile Page 96
4.2.9 Change Password Page 97
4.2.10 Car Plate Number Page 98
4.2.11 Contact Us Page (AFM) 101
4.2.12 About IICP Park Finder Page (Terms and Conditions) 101
4.2.13 Logout 102
4.3 Web Application 103
4.3.1 Login 103
4.3.2 Parking Statistics Page 104
4.3.3 Student Lists Page 107
4.3.4 Fines Page 109
4.3.5 Log Details Page 113
4.3.6 Logout 114
4.4 Object Detection Model 116
4.4.1 Yolov8 Implementation (Detect Objects) 116
4.4.2 Deep SORT PyTorch Implementation (Track Objects) 118
4.4.3 Parking Lot Bounding Box Drawing 120
4.4.3 Firebase Connection (Update Vacancy Status of Parking Lots) 122
CHAPTER 5: TESTING AND EVALUATION 125
5.1 Introduction 125
5.2 Functional Testing 125
5.2.1 Unit Testing 125
5.2.1.1 Mobile Application 125
5.2.1.1 (a) Login Page 125
5.2.1.1 (b) Register Page 127
5.2.1.1 (c) Forgot Password Page 130
5.2.1.1 (d) Parking Page 132
5.2.1.1 (e) Contact Page 135
5.2.1.1 (f) Inbox Page 139
5.2.1.1 (g) Profile Page 140
5.2.1.1 (h) Change Password Page 141

5.2.1.1 (i) Car Plate Numbers Page 143
5.2.1.1 (j) Contact Us Page 147
5.2.1.1 (k) About IICP Park Finder Page 148
5.2.1.1 (l) Logout 149
5.2.1.2 Web Application 150
5.2.1.2 (a) Login Page 150
5.2.1.2 (b) Parking Statistics Page 152
5.2.1.2 (c) Student Lists Page 155
5.2.1.2 (d) Fines Page 157
5.2.1.2 (e) Log Details Page 163
5.2.1.2 (f) Logout 165
5.2.2 Integration Testing 166
5.2.2.1 Mobile Application 166
5.2.2.1 (a) Login Activity to Register Activity 166
5.2.2.1 (b) Login Activity to Forgot Password Activity 167
5.2.2.1 (c) Login Activity to Parking Activity 168
5.2.2.1 (d) Parking Activity to Contact Activity 169
5.2.2.1 (e) Contact Activity to Inbox Activity 170
5.2.2.1 (f) Inbox Activity to Settings Activity 170
5.2.2.2 Web Application 173
5.2.2.2 (a) Login Activity to Parking Statistics Activity 173
5.2.2.2 (b) Parking Statistics Activity to Student Lists Activity 174
5.2.2.2 (c) Student Lists Activity to Fines Activity 174
5.2.2.2 (d) Student Lists Activity to Log Details Activity 175
5.2.2.2 (e) Home Activity to Log Out Activity 176
5.3 Non-Functional Testing 177
5.3.1 User Acceptance Testing 177
5.3.1.1 Mobile Application 178
5.3.1.1 (a) Interview with Participant Number One 178
5.3.1.1 (b) Interview with Participant Number Two 179
5.3.1.1 (c) Interview with Participant Number Three 180
5.3.1.1 (d) Interview with Participant Number Four 182

5.3.1.1 (e) Interview with Participant Number Five 183
5.3.1.1 (f) Questionnaire Survey 184
5.3.1.1 (h) Conclusion for User Acceptance Testing for Mobile Application. 193
5.3.1.2 Web Application 194
5.3.1.2 (a) Interview with Head of Admin and Facilities Department 194
5.3.1.2 (b) Questionnaire Survey..... 196
5.3.1.1 (c) Conclusion of User Acceptance Testing for Web Application..... 197
5.4 Conclusion..... 197
CHAPTER 6: DISCUSSION..... 199
6.1 Achievements of this Project..... 199
6.2 Limitations 200
6.2.1 Recorded Video Limitations..... 200
6.2.2 Lack of Training Data for Object Detection Model..... 200
6.2.3 Fixed User Location for Navigation 200
6.3 Future Enhancements 201
6.3.1 Implement Real-Time Video Footage 201
6.3.2 Collect Custom Data for Model Training 201
6.3.3 Implement Real-Time Location for Navigation 201
6.3.4 Improve User Interface for Mobile Application 201
6.4 Conclusion..... 201
CHAPTER 7: REFLECTION 203
7.1 Personal Reflection 203
7.2 Challenges and Solutions 204
7.3 Conclusion..... 204
CHAPTER 8: CONCLUSION 206
References 207
Appendix..... 214

LIST OF TABLES

Table 2. 1 Summary of Literature Review on the Comparison between CNN and SVM..... 14
Table 2. 2 Comparison of Image Processing Methods..... 23
Table 2. 3 Comparison Between the Shortest Path Algorithms..... 30
Table 2. 4 Comparison Between Existing System and IICP Park Finder..... 35
Table 2. 5 Comparison Between Google Firestore and MongoDB Atlas..... 43
Table 3. 1 Relationship Between the Collections in Firebase Database..... 67
Table 3. 2 Gantt Chart..... 69
Table 5. 1 Unit Testing For Login Page..... 125
Table 5. 2 Unit Testing for Register Page..... 127
Table 5. 3 Unit Testing for Forgot Password Page..... 130
Table 5. 4 Unit Testing for Parking Page..... 132
Table 5. 5 Unit Testing for Contact Page..... 135
Table 5. 6 Unit Testing for Inbox Page..... 139
Table 5. 7 Unit Testing for Profile Page..... 140
Table 5. 8 Unit Testing for Change Password Page..... 141
Table 5. 9 Unit Testing for Car Plate Numbers Page..... 143
Table 5. 10 Unit Testing for Contact Us Page..... 147
Table 5. 11 Unit Testing for About IICP Park Finder Page..... 148
Table 5. 12 Unit Testing for Logout..... 149
Table 5. 13 Unit Testing for Web App Login Page..... 150
Table 5. 14 Unit Testing for Web Parking Statistics Page..... 152
Table 5. 15 Unit Testing for Web App Student Lists Page..... 155
Table 5. 16 Unit Testing for Web App Fines Page..... 157
Table 5. 17 Unit Testing for Web App Log Details Page..... 163
Table 5. 18 Unit Testing for Web App Logout..... 165
Table 5. 19 Integration Testing for Mobile App Login Activity to Register Activity..... 166
Table 5. 20 Integration Testing for Mobile App Login Activity to Forgot Password Activity.. 167
Table 5. 21 Integration Testing for Mobile App Login Activity to Parking Activity..... 168
Table 5. 22 Integration Testing for Mobile App Parking Activity to Contact Activity..... 169
Table 5. 23 Integration Testing for Mobile App Contact Activity to Inbox Activity..... 170
Table 5. 24 Integration Testing for Mobile App Inbox Activity to Settings Activity..... 170
Table 5. 25 Integration Testing for Web App Login Activity to Parking Statistics Activity..... 173
Table 5. 26 Integration Testing for Web App Parking Statistics Activity to Student Lists Activity
..... 174
Table 5. 27 Integration Testing for Web App Student Lists Activity to Fines Activity..... 174

Table 5. 28 Integration Testing for Web App Student Lists Activity to Log Details Activity .. 175
Table 5. 29 Integration Testing for Web App Home Activity to Log Out Activity 176
Table 5. 30 User Acceptance Testing with Participant Number One 178
Table 5. 31 User Acceptance Testing with Participant Number Two..... 179
Table 5. 32 User Acceptance Testing with Participant Number Three..... 181
Table 5. 33 User Acceptance Testing with Participant Number Four..... 182
Table 5. 34 User Acceptance Testing with Participant Number Five 183
Table 5. 35 User Acceptance Testing with Head of Admin and Facilities Department 195

LIST OF FIGURES

Figure 1- 1 Problem Statement of Project 2
Figure 1- 2 Frequency of Students Utilizing the Parking Facilities in INTI Penang..... 2
Figure 1- 3 Time Spent by Students to Find a Vacant Parking Space..... 3
Figure 1- 4 Alternatives Sought by Students When Unable to Find a Parking Space..... 3
Figure 1- 5 Number of Students Who Drive to Campus..... 4
Figure 1- 6 Number of Rounds Driven Around the Campus for a Parking Space..... 5
Figure 1- 7 Challenges Faced by Students Parking on Campus 6
Figure 2- 1 Process of Vacant Parking Space Using Image Processing..... 9
Figure 2- 2 Incorrect Angle View of Parking Lot for Image Processing (Paula, et al., 2019)... 9
Figure 2- 3 Correct View of Parking Lot for Accurate Image Processing (iStock, 2024)..... 9
Figure 2- 4 Example of Detecting People Using Background Subtraction (Phoaivu, 2011).. 16
Figure 2- 5 Example of Detecting Vehicle with Edge Detection and Line Segmentation (Li, et al., 2016)..... 18
Figure 2- 6 Example of Detecting Vehicles Using Feature-Based Object Detection (Kaan, 2023) 20
Figure 2- 7 Dijkstra's Algorithm Diagram and Table (ScienceDirect, 2023) 26
Figure 2- 8 Bellman Ford's Algorithm (GeeksforGeeks, 2024)..... 27
Figure 2- 9 A* Search Algorithm (Codecademy, 2024) 28
Figure 2- 10 Penang Smart Parking App and Infrared Sensor (Mei, 2021)..... 32
Figure 2- 11 Ultrasonic Sensors in Shopping Mall's Parking Lot (AKE Parking Technology, 2020) 33
Figure 2- 12 Firestore Logo (Supun, 2020) 40
Figure 2- 13 Mongo DB Atlas Logo (Amazon Web Services, Inc. , 2024)..... 41
Figure 2- 14 Proposed Work Timeline for IICP Park Finder 45
Figure 3 - 1 Waterfall Model (Dhruv, 2021)..... 48
Figure 3 - 2 Concerns Raised for IP Camera Implementation 49
Figure 3 - 3 How to Retrieve Video Footage from IP Camera..... 50
Figure 3 - 4 How to Run Code on Recorded Footage..... 50
Figure 3 - 5 Use Case Diagram for IICP Park Finder's Mobile Application..... 53
Figure 3 - 6 Use Case for IICP Park Finder's Web Application 54
Figure 3 - 7 Register, Login & Forgot Password Module Flowchart 56
Figure 3 - 8 Parking Page Module Flowchart 57
Figure 3 - 9 Contact Page Module Flowchart 58
Figure 3 - 10 Inbox Page Module Flowchart..... 58
Figure 3 - 11 Settings Page Module Flowchart..... 59
Figure 3 - 12 Profile Page Module Flowchart 60
Figure 3 - 13 Car Plate Number Module Flowchart 61
Figure 3 - 14 Login Module Flowchart..... 62
Figure 3 - 15 Parking Statistics Module Flowchart..... 63
Figure 3 - 16 Student Details Lists Module Flowchart..... 64
Figure 3 - 17 Car Plate Number Fine Module Flowchart..... 65
Figure 3 - 18 Logging Details Module Flowchart 66
Figure 3 - 19 Data Model Diagram 67
Figure 3 - 20 Flutter Logo (Flutter, 2024)..... 72
Figure 3 - 21 Cloud Storage for Firebase Logo (Google for Developers, 2024)..... 73

Figure 3 - 22 Firebase Functionalities 73
Figure 3 - 23 Vue.js Logo (Vue.js, 2024) 74
Figure 3 - 24 Google Maps API Logo (GoogleforDevelopers, 2024) 75
Figure 3 - 25 Google Earth Pro Logo (Neowin, 2024) 76
Figure 3 - 26 QGIS Logo (QGIS, 2024) 76
Figure 3 - 27 Yolov8 Logo (Ultralytics Inc., 2024) 77
Figure 3 - 28 Comparison of YOLO Models (Ultralytics, 2024) 77
Figure 3 - 29 Loss and Top-1Error Rate of Testing And Validation of Deep SORT Model 79
Figure 3 - 30 GitHub Logo (Munira, 2018) 80
Figure 4- 1 Login Page 81
Figure 4- 2 Terms And Conditions Page Upon Registering 82
Figure 4- 3 Terms and Conditions Page Upon Registering (Scrolled to the Bottom) 82
Figure 4- 4 Register Page 83
Figure 4- 5 Verification Loading Page 84
Figure 4- 6 Verification Email 84
Figure 4- 7 Forgot Password (Page 1) 85
Figure 4- 8 Forgot Password (Page 2) 85
Figure 4- 9 Reset Password Email 85
Figure 4- 10 Reset Password 85
Figure 4- 11 Parking Page 86
Figure 4- 12 Snackbar Appear when Parking Lot is Occupied 87
Figure 4- 13 Dialog Box to Confirm Navigation 87
Figure 4- 14 Navigation 87
Figure 4- 15 Code Snippet of Calculating the Centre Location of Parking Lots for Dijkstra's Algorithm 88
Figure 4- 16 Code Snippet for Using the Calculated Centroid to Calculate Distance for Weight in Dijkstra's Algorithm 88
Figure 4- 17 Code Snippet of Helper Function to Calculate Distance using Haversine Formula 89
Figure 4- 18 Code Snippet of Dijkstra's Algorithm Part 1 89
Figure 4- 19 Code Snippet of Dijkstra's Algorithm Part 2 90
Figure 4- 20 Dialog Box Prompt When Tapping on Entrance 90
Figure 4- 21 Dialog Box Confirming Arrival 91
Figure 4- 22 Contact Page 92
Figure 4- 23 Error Hide Plate Number 93
Figure 4- 24 Error Car Plate Not Found 93
Figure 4- 25 Contact Found 93
Figure 4- 26 Navigate to Phone App 93
Figure 4- 27 Inbox Page 94
Figure 4- 28 Notification Details 94
Figure 4- 29 Settings Page 95
Figure 4- 30 Profile Page 96
Figure 4- 31 Edit Profile Details 96
Figure 4- 32 Change Password Page 97
Figure 4- 33 Error Old Password Invalid 97
Figure 4- 34 Dialog Box Success in Changing Password 97
Figure 4- 35 Car Plate Numbers Page 98

Figure 4- 36 Add Car Plate Numbers Page 98
Figure 4- 37 Updated List of Car Plate Numbers 98
Figure 4- 38 Edit and Delete Car Plate Numbers 99
Figure 4- 39 Edit Car Plate Number 99
Figure 4- 40 Confirmation Dialog to Delete Car Plate Number 99
Figure 4- 41 Cannot Delete with One Car Plate Number 99
Figure 4- 42 List with Clamped Car Plate Number 100
Figure 4- 43 Warning Dialog When User Edit or Delete Clamped Plate Number 100
Figure 4- 44 Contact Us Page (AFM) 101
Figure 4- 45 About IICP Park Finder Page (T&C) 101
Figure 4- 46 Logout Confirmation Dialog 102
Figure 4- 47 Admin Login Page 103
Figure 4- 48 Admin Login Successful 103
Figure 4- 49 Home Page with Statistics Dashboard 104
Figure 4- 50 Data Picker to View Day Statistics 104
Figure 4- 51 Statistics for Chosen Date 105
Figure 4- 52 Code Snippet for Displaying the Line Graph 105
Figure 4- 53 Statistics Dashboard for Month 106
Figure 4- 54 Statistics Dashboard for Selected Month and Year 106
Figure 4- 55 Code Snippet to Calculate Average Available Parking Lot for Each Day in the Month 107
Figure 4- 56 Student Lists Page 107
Figure 4- 57 Search Student List by Car Plate Number 108
Figure 4- 58 Search Student List by Student ID 108
Figure 4- 59 Fine Page 109
Figure 4- 60 Add Fine 109
Figure 4- 61 Updated Fine List after Adding Fine 110
Figure 4- 62 Edit Fine 110
Figure 4- 63 Pay Fine 111
Figure 4- 64 Select File to Upload as Receipt 111
Figure 4- 65 Fine Mark as Paid 112
Figure 4- 66 Receipt Uploaded 112
Figure 4- 67 Delete Confirmation Dialog 113
Figure 4- 68 Log Details Page 113
Figure 4- 69 Search Category for Log Details 114
Figure 4- 70 Admin Logout 114
Figure 4- 71 Code Snippet of Modified Yolov8 DetectionPredictor Class (Part 1) 116
Figure 4- 72 Code Snippet of Modified Yolov8 DectectionPredictor Class (Part 2) 117
Figure 4- 73 Code Snippet of Modified Yolov8 DetectionPredictor Class (Part 3) 117
Figure 4- 74 Parameter to Tune the Deep SORT Algorithm 118
Figure 4- 75 Deep SORT PyTorch Package Files 118
Figure 4- 76 Import Deep SORT to Integrate with Yolov8 119
Figure 4- 77 Setting Parameters for Deep SORT 119
Figure 4- 78 Implement Deep SORT Algorithm 119
Figure 4- 79 Code Snippet for Drawing Parking Lot Bounding Box 120
Figure 4- 80 Add Parking Lot to Database Collection 121
Figure 4- 81 Save Parking Lot Drawing and Update Database 121

Figure 4- 82 Parking Lots Coordinates in parking_lots.json File 121
Figure 4- 83 Image Used for Drawing Parking Lots 122
Figure 4- 84 Code Snippet to Load Parking Lot Boxes in Video 122
Figure 4- 85 Code Snippet to Draw Parking Lot and Bounding Box on Detected Cars 123
Figure 4- 86 Code Snippet to Compare Detected Car Bounding Box and Parking Lot 123
Figure 4- 87 Object Detection Model Running on Recorded Video 124
Figure 5 - 1 Load to Parking Page After Successful Login 126
Figure 5 - 2 Validate Login Empty Field 126
Figure 5 - 3 Validate Login Email and Password 126
Figure 5 - 4 Registration Successful, Waiting for Verification 128
Figure 5 - 5 Verification Email for Registration 128
Figure 5 - 6 Confirm Email Verified 128
Figure 5 - 7 Account Registered Successfully 128
Figure 5 - 8 Reject Empty Field 128
Figure 5 - 9 Reject Invalid Email Address 128
Figure 5 - 10 Reject Mismatched Password 129
Figure 5 - 11 Reject Email Already Registered 129
Figure 5 - 12 Reject Car Plate Already Registered 129
Figure 5 - 13 Reject Password Too Short 129
Figure 5 - 14 Navigate to Waiting Page for User to Reset Email 131
Figure 5 - 15 Email Sent to Reset Password 131
Figure 5 - 16 Reset Password Page from Firebase 131
Figure 5 - 17 Password Reset Successful 131
Figure 5 - 18 Reject Empty Field 131
Figure 5 - 19 Show Vacant and Occupied Parking Lots, and Available Lots in Parking Page 133
Figure 5 - 20 Snackbar Shown When Parking Lot Chosen is Occupied 133
Figure 5 - 21 Confirm Navigation 133
Figure 5 - 22 Start Navigation 133
Figure 5 - 23 Confirm Stop Navigation 133
Figure 5 - 24 Navigation Stops and Snackbar Shown 133
Figure 5 - 25 Confirm to Navigate to Nearest rking Lot of Chosen Entrance 134
Figure 5 - 26 Map Not Centered 134
Figure 5 - 27 Map After Tapping on Recenter Button 134
Figure 5 - 28 Map Not Centered When Navigating 134
Figure 5 - 29 Map After Tapping on Recenter Button When Navigating 134
Figure 5 - 30 Toggle Update When Show 136
Figure 5 - 31 Show Field in Database Update to True 136
Figure 5 - 32 Toggle UI Update to Hide Car Plate Number 137
Figure 5 - 33 Show Field in Database Update to False 137
Figure 5 - 34 Reject Empty Field 137
Figure 5 - 35 Car Plate Number Does Not Exist 137
Figure 5 - 36 Owner Hid Car Plate Number 137
Figure 5 - 37 Car Plate Number Found 138
Figure 5 - 38 Call Car Plate Number 138
Figure 5 - 39 Show List of Notifications For User 139
Figure 5 - 40 Show Details of Notification Selected by User 139

Figure 5 - 41 List Updates Color After Reading the Notification 139
Figure 5 - 42 Show Details of User Profile Detail 140
Figure 5 - 43 Edit Name 140
Figure 5 - 44 Edit Phone Number 141
Figure 5 - 45 Profile Changes Saved Successfully 141
Figure 5 - 46 Reject Empty Field 142
Figure 5 - 47 Reject Incorrect Current Password 142
Figure 5 - 48 Reject New Password Shorter Than 6 Characters 142
Figure 5 - 49 Reject Mistmatched Password 143
Figure 5 - 50 Password Changes Successfully 143
Figure 5 - 51 List of Car Plate Numbers Owned by User 145
Figure 5 - 52 Edit Car Plate Number (MALAYSIA123) 145
Figure 5 - 53 List After ‘MALAYSIA123’ Edited to ‘MALAYSIA’ 145
Figure 5 - 54 Snackbar Shown When User Key In Existing Car Plate Number 145
Figure 5 - 55 Delete Car Plate Number 145
Figure 5 - 56 List After 'MALAYSIA' Deleted 145
Figure 5 - 57 Warning When User Edit or Delete Clamped Car Plate Number 146
Figure 5 - 58 Navigate to Add Car Plate Number Page 146
Figure 5 - 59 Error When Car Plate Number Added Already Exist 146
Figure 5 - 60 List Updated After 'MALAYSIA123' Is Added 146
Figure 5 - 61 Show AFM Details 148
Figure 5 - 62 Launch Phone App With AFM Number 148
Figure 5 - 63 Email Copied to Clipboard After Long Press 148
Figure 5 - 64 Show Terms and Conditions 149
Figure 5 - 65 Scroll Terms and Conditions to the Bottom 149
Figure 5 - 66 Prompt User to Confirm Logout 150
Figure 5 - 67 Successful Admin Login 151
Figure 5 - 68 Reject Incorrect Password and Email 151
Figure 5 - 69 Reject Unauthorised User 151
Figure 5 - 70 Show Line Graph for Available Parking Lots of Current Date 152
Figure 5 - 71 Select Date 153
Figure 5 - 72 Show Available Parking Lots Throughout the Date 153
Figure 5 - 73 Show Line Graph of Average Available Parking Lot throughout Current Month 154
Figure 5 - 74 Show Line Graph of Average Available Parking Lots throughout Chosen Month 154
Figure 5 - 75 Show List of Car Plate Numbers and Student Information In Ascending Order 155
Figure 5 - 76 Show List of Details Based on Car Plate Number Query 156
Figure 5 - 77Show List of Details Based on Student ID Query 156
Figure 5 - 78 Show Fines for Vehicle 157
Figure 5 - 79 Add FIne Modal 158
Figure 5 - 80 Reject Empty Fields 158
Figure 5 - 81 Fine List Update After Add 158
Figure 5 - 82 Edit Fine Modal 159
Figure 5 - 83 Reason and Car Park Level Changed to Updated Value 159
Figure 5 - 84 Add Receipt File 159

Figure 5 - 85 Reject Empty File View 160
Figure 5 - 86 Choose Receipt File 160
Figure 5 - 87 Payment Status and Receipt Status Changed After Payment 161
Figure 5 - 88 Confirm Delete Fine 161
Figure 5 - 89 Fine Deleted 162
Figure 5 - 90 View Fine Receipt 162
Figure 5 - 91 Show Log Details 163
Figure 5 - 92 Show Log Details on Searched Car Plate Number 164
Figure 5 - 93 Show Log Details on Searched Student ID 164
Figure 5 - 94 Logout Button 165
Figure 5 - 95 Confirm Logout 165
Figure 5 - 96 Graph for Optimal Sample Size for Qualitative Usability Studies 177
Figure 5 - 97 Terms and Conditions of User Acceptance Testing Survey 185
Figure 5 - 98 Expectation of Login Feature 185
Figure 5 - 99 Ease of Using Login Feature 186
Figure 5 - 100 Ease of Registering Account 186
Figure 5 - 101 Clarity of Error Messages for Registration 187
Figure 5 - 102 Forgot Password Functionality 187
Figure 5 - 103 Ease of Resetting Password 188
Figure 5 - 104 Intuitiveness of Parking Page 188
Figure 5 - 105 Usefulness of Double-Parking Function 189
Figure 5 - 106 Ease of Calling Function 189
Figure 5 - 107 Ease of Accessing Inbox Feature 190
Figure 5 - 108 Delay in Loading Messages 190
Figure 5 - 109 Ease of Changing Settings 191
Figure 5 - 110 Settings Saved and Applied Correctly 191
Figure 5 - 111 Useful Features 192
Figure 5 - 112 Suggestions for App Flaws 192
Figure 5 - 113 Likeliness to Use the Application 193
Figure 5 - 114 Suggestions for Additional Functionalities 193
Figure 5 - 115 User Acceptance Testing with Head of Admin and Facilities Department 194
Figure 5 - 116 Questionnaire Survey for Web Application 196

LIST OF ABBREVIATIONS

AFM	Admin and Facilities Management
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
BFS	Breadth-First Search
CNN	Convolutional Neural Networks
DBMS	Database Management System
DFS	Depth-First Search
ERD	Entity Relationship Diagram
GCP	Google Cloud Platform
GLCM	Gray-level co-occurrence matrix
GPS	Global Positioning System
HOG	Histogram of Oriented Gradients
HSV	Hue, Saturation, Value
ITS	Information and Technology Services
KNN	K-Nearest Neighbors
LBP	Local binary patterns
LSD	Line Segment Detector
R-CNNs	Region-based convolutional neural networks
RANSAC	Random Sample Consensus
RGB	Red, Green Blue
ROI	Region of Interest
RPNs	Region Proposal Networks
RTSP	Real-Time Streaming Protocol
SDKs	Software Development Kit
SIFT	Scale-Invariant Feature Transform
SQL	Sequel
SSD	Single Shot Detectors
SURF	Speeded-Up Robust Features

SVM	Support Vector Machines
UAT	User Acceptance Testing
UI	User Interface
VMS	Variable Message Signs
YOLO	You Only Look Once

ABSTRACT

INTI Penang’s parking system faces challenges including limited parking spaces, lack of real-time information on space availability, and inefficient space usage due to improper parking. This study presents the IICP Park Finder, a parking system designed to help students locate vacant parking spaces more efficiently with computer vision capabilitites. By conducting literature review, it has been decided to use Dijkstra's shortest path algorithm for the system to guide users to the nearest available space based on their selected entrance, minimizing search time. The solution integrates image processing with the integration of YOLOv8 and Deep SORT algorithm to detect open spaces and alerts users, while data management through Google Cloud Firestore ensures scalable and real-time updates on space availability. For administrative use, a web application displays parking trends via line graphs, enabling better traffic flow and resource planning. Developed using the Waterfall model, IICP Park Finder underwent extensive unit and integration testing, both of which yielded successful results, as well as user acceptance testing, where feedback was largely positive with minor suggestions for UI improvements. This study contributes to the field of intelligent parking systems, demonstrating the potential to enhance campus parking mobility and efficiency.

CHAPTER 1: INTRODUCTION

1.1 Background

Parking space management has become a crucial aspect of urban planning and development, particularly in institutions with high traffic volumes such as universities. The efficient management of parking spaces is essential to reduce congestion and minimize traffic chaos. In recent years, the increasing number of vehicles on the road has led to a surge in demand for parking spaces, making it a challenging task for institutions to manage their parking facilities effectively.

Traditional parking systems rely on manual monitoring (using guards to manage traffic flow) often lead to frustration among drivers as the lack of real-time information on parking availability can result in drivers circling around the parking lot, contributing to traffic congestion, air pollution, and wasted fuel. Moreover, the inefficient use of parking spaces can lead to a shortage of parking spots, causing inconvenience to the drivers.

Advancements in camera technology, computing power, machine learning, IoT, and cloud computing have accelerated the application of image processing in intelligent parking systems. These advancements have enabled the development of sophisticated and reliable systems capable of real-time parking space detection, occupancy monitoring, driver guidance, automated payment, and traffic flow improvement. By leveraging computer vision and machine learning algorithms, image processing can be used to detect vacant parking spaces, track parking usage patterns, and provide real-time information to drivers.

INTI International College Penang, a leading private institution of higher learning in Malaysia, faces similar parking challenges. With a large student population and limited parking spaces, the institution struggles to manage its parking facilities effectively. The lack of real-time information on parking availability often leads to congestion, frustration, and wasted time among students and visitors. Therefore, there is a need for an intelligent parking system that can detect vacant parking spaces and provide real-time information to users.

While image processing has been successfully applied in various parking management systems, there is a lack of research on its application in the context of INTI Penang. This project aims to bridge this gap by developing an image processing-based system to detect vacant parking spaces in INTI Penang. The objective of this study is to design and implement an intelligent parking system that can provide real-time information on parking availability, optimize parking space utilization, and enhance the overall parking experience in INTI Penang.

1.2 Problem Statement



Figure 1- 1 Problem Statement of Project

Finding a parking space in campus is a daily struggle for students when classes commence for the semester. From the statistics of the survey conducted in the University of Alabama, 39.9% of students take more than 10 minutes to find a vacant parking space while 43% of students park their vehicles illegally when the students cannot find a place to park (Sents, 2023). 11% of the students even gave up attending classes and decide to go home. Additionally, survey results reveal that 74.7% of students are not satisfied with the parking situation, and 70.9% find it difficult to find parking in Taylor's University (Filzani, et al., 2020) where 73% of students reported spending 15-30 minutes searching for a parking spot. To address these issues, 65.3% of students strongly agree that a better parking system is needed. This study has prompted for a survey to be conducted to determine the seriousness of the case in INTI Penang.

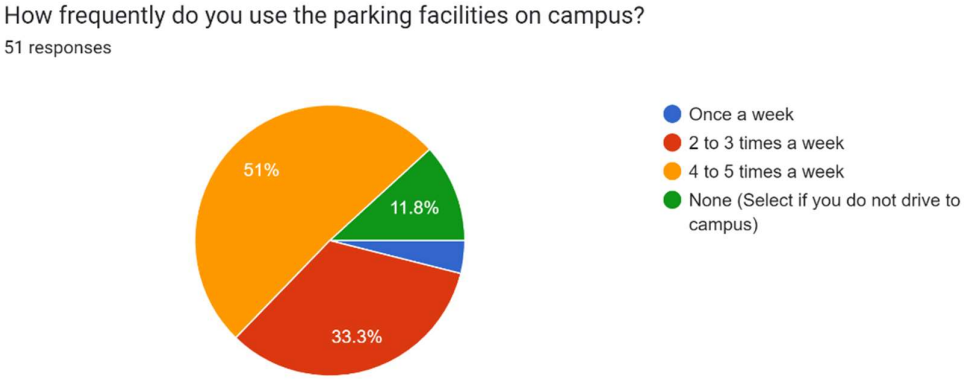


Figure 1- 2 Frequency of Students Utilizing the Parking Facilities in INTI Penang

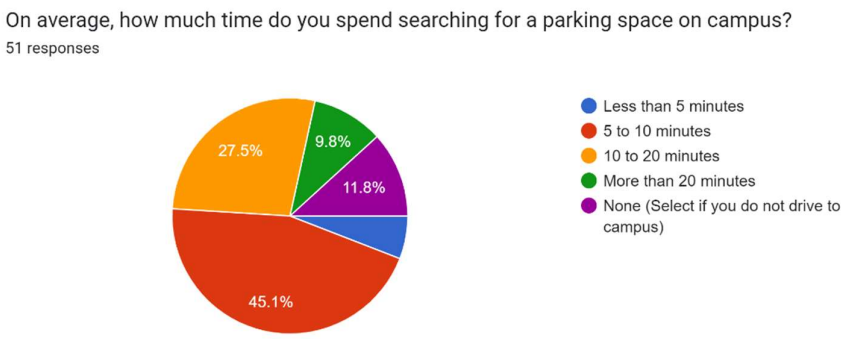


Figure 1- 3 Time Spent by Students to Find a Vacant Parking Space

A survey was conducted from 29 April 2024 until 3 May 2024 via a Google Form to determine the statistics regarding the percentage of students utilising the parking facilities on campus and the problems faced by them when finding for parking spaces in INTI Penang. The survey received 51 responses from INTI Penang's students who are enrolled in different courses. According to the survey conducted, the students who drive to campus utilize the parking facilities at least twice a week and 45.1% of the students take 5 to 10 minutes to find for parking spaces while 37.3% (27.5% takes 10 to 20 minutes and 9.8% takes more than 20 minutes) of them spends more than 10 minutes (refer to Figure 1-2 and Figure 1-3 above).

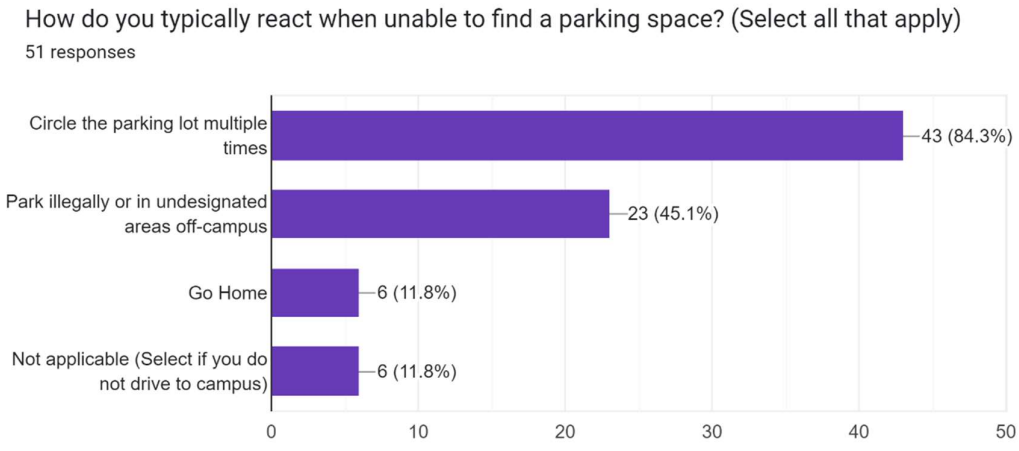


Figure 1- 4 Alternatives Sought by Students When Unable to Find a Parking Space

In addition, while 84.3% of INTI Penang students opt to circle the parking lot multiple times when unable to find a vacant parking space, 45.1% of students decide to park illegally, and 11.8% opt to go home and skip class as shown in Figure 1-4 .The inability for students to find vacant parking spaces easily not only causes INTI students to waste time finding for parking

but also lose motivation to attend classes. Some even park by the roadside and receive parking fines from the government officials due to illegal parking. It is evident that the students of INTI International College Penang are not an exception to the alternatives sought due to the unavailability of parking spaces and hence have to bear the consequences of fines and skipping classes.

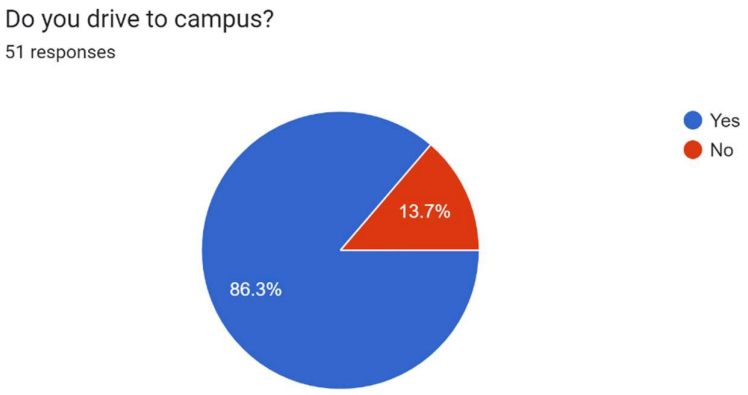


Figure 1- 5 Number of Students Who Drive to Campus

Lack of parking spaces in universities and colleges have been a long-standing issue due to the high number of students against the low number of available parking spaces in campus. According to a survey conducted by (Ezarik, 2022), 60% of the undergraduate students (out of 2000 correspondence) find that there is inadequate amount of parking spaces in their campuses. This problem is especially true and troublesome for students of INTI Penang as 86.3% of the students studying in INTI Penang drive to campus while the remaining 13.7% uses other means of transport to reach campus as shown in Figure 1-5. Not to mention, the campus is located on a hill, distance away from any government parking spaces or residential areas so students do not have any choice except to park their vehicles on campus grounds with limited parking spaces.

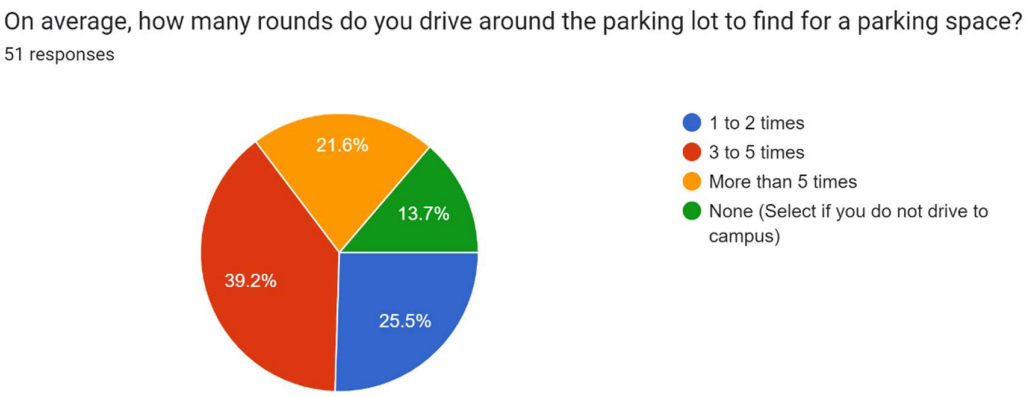


Figure 1- 6 Number of Rounds Driven Around the Campus for a Parking Space

Besides the lacking number of parking spaces provided by the campus, the lack of information regarding the availability of parking spaces also contributes to the challenge of finding a vacant parking space. As shown in Figure 1-6, 60.8% (39.2%+21.6%) of the students drive around the parking lot at least 3 times to find a parking space, wasting their fuel and time. Currently, the only method of information provided to the students regarding the availability of parking spaces are from the guards. The guards are stationed to be in the parking lot especially during peak hours typically around 9 a.m. to 12 p.m. When students drive to campus, the guards will show hand signs to indicate if the parking lot is full and advice the students to explore other floors. However, when the student drive to the other floors, the guards there also say that the parking spaces are fully utilized causing the student to drive back to the previous floor hoping there is a vacant parking space. Sometimes the guards say that the parking lot is full and did not realise there is a vehicle leaving causing the student to miss the parking lot. Therefore, it is evident that the lack of information regarding the available parking spaces causes students to keep driving aimlessly and sometimes miss their opportunity to find a vacant parking spot.

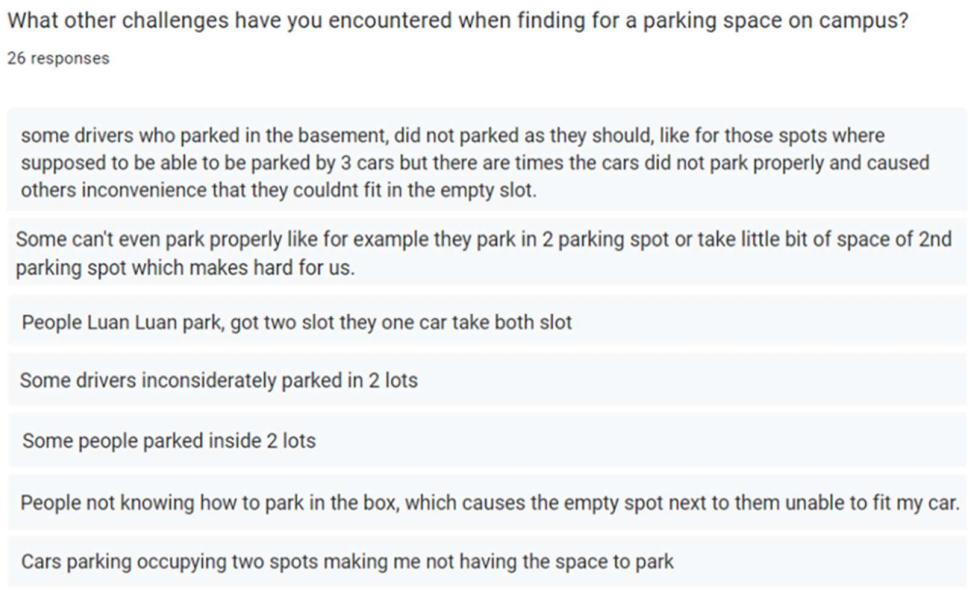


Figure 1- 7 Challenges Faced by Students Parking on Campus

Finally, the high numbers of students parking their vehicles in multiple parking spaces also causes other students to have difficulties acquiring a vacant parking space. This problem can be seen from Figure 1-7, where 14 respondents state in the survey that there are students who take up two parking lots causing them to not be able to park. Despite the insufficient number of parking lots provided, some students tend to park in the middle of two parking lots as the students fear other vehicles will be parked too closely to them and causes damages to their cars.

In conclusion, finding a vacant parking space is a struggle for many students in INTI Penang due to the high number of students against the low number of parking spaces, the lack of information regarding the available parking spaces, and students occupying two parking spaces for one vehicle. IICP Park Finder aims to counter these problems to ease INTI students' parking process on campus.

1.3 Research Questions

1. How can a web application that utilizes a shortest path algorithm be developed to help INTI students find vacant parking spaces nearest to entrance points?
2. How to provide information regarding available parking spaces to reduce the time searching for parking space for students?
3. How can statistics be effectively implemented in the web application to provide administration staff with parking usage patterns for reducing wait times and improving traffic flow during peak hours?

1.4 Objectives

The objectives derived for the project are as follow:

1. To develop a mobile application allowing INTI students to find a vacant parking space nearest to entrance points using Dijkstra's shortest path algorithm.
2. To integrate the image processing capability within the car park so that it could alert the users on the available parking spaces.
3. To implement line graphs in the web application for administration staffs to strategize on reducing wait times and improving traffic flow during peak hours.

1.5 Client, Audience, Motivation

The client for this project is the students of INTI Penang. The institution has a large student population and limited parking spaces, making parking management a significant challenge. With IICP Park Finder, the students will not only be able to find the vacant parking spaces available on campus in the web application without going in circles around the parking lot but also find one that is nearest to the entrance points so the students do not have to walk so far. Not only that, the students will also receive a notification if the parking lot is full so the students do not waste time driving around and can immediately switch to other floors to find for the available parking spaces.

Besides the students, the project's client also includes the administration staffs of INTI Penang. The line graphs implemented in the web application will be used by administration staff to strategize on reducing wait times and improving traffic flow during peak hours. This information can be used to spread the classes of students to avoid traffic congestions during the starting and ending time of the classes. This reduces the competition of acquiring a parking space for students and reduce the manpower needed (guards) to smoothen the traffic for the administration staffs.

The motivation for this project is to address the parking challenges faced by INTI Penang and improve the overall parking experience for the institution's community. The project aims to develop an intelligent parking system that can optimize parking space utilization, reduce congestion, and enhance user satisfaction. By leveraging image processing technology and a shortest path algorithm, the project aims to provide information on parking availability, to help users find vacant parking spaces quickly, and improve traffic flow during peak hours. The project also aims to contribute to the existing body of knowledge on intelligent parking systems and their application in the context of universities.

1.6 Scope

The scope of the project entails the development of android mobile applications exclusively tailored for use within INTI Penang, necessitating integration with a customized map system. This initiative also mandates the incorporation of IP cameras and Wi-Fi access points for functionality, alongside a requisite internet connection. Real-time data transmission may pose challenges due to INTI's operational constraints where the IP camera might not be able to proper Internet connection due to the distance from the nearest Wi-Fi access point is too far, potentially impacting implementation feasibility. The video used for the project will be recorded as using real-time footage will require access to the campus' router for port-forwarding which will potentially cause security breaches. This is further discussed in the Requirements Phase of the Waterfall Model in Chapter 3.

1.7 Conclusion

In conclusion, this chapter has presented the background and motivation for the project to determine vacant spaces in INTI Penang using image processing. The project aims to develop a web application that allows INTI students to find vacant parking spaces nearest to entry points using a shortest path algorithm, integrate image processing capability within the car park to alert users on available parking spaces in real-time, and implement line graphs in the web application for administration staff to strategize on reducing wait times and improving traffic flow during peak hours. The project is motivated by the need to optimize parking space utilization, reduce congestion, and enhance the overall parking experience for students, staff, and visitors. The next chapter will present the approach used to achieve these objectives.

CHAPTER 2: LITERATURE REVIEW

2.0 Definition of Topic

Car park availability detection using image processing involves analysing video or image data from cameras installed in parking lots or garages to determine the occupancy status of each parking space. This information can then be used to guide drivers to available spaces or provide real-time parking occupancy data.

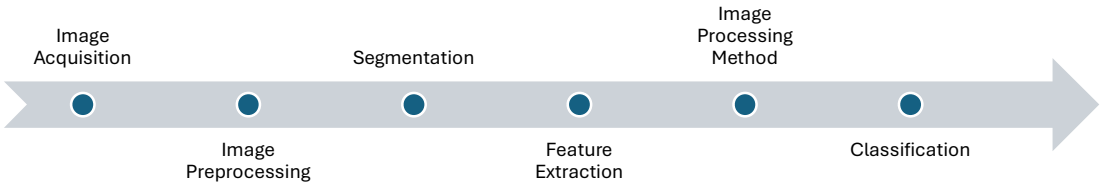


Figure 2- 1 Process of Vacant Parking Space Using Image Processing

As shown in Figure 2-1, the process detecting vacant parking spaces using image processing involves the following steps:



Figure 2- 2 Incorrect Angle View of Parking Lot for Image Processing (Paula, et al., 2019)

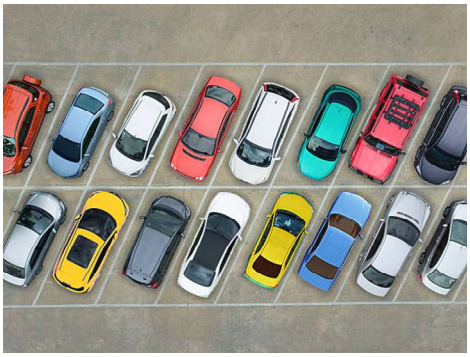


Figure 2- 3 Correct View of Parking Lot for Accurate Image Processing (iStock, 2024)

1. **Image Acquisition:** To take clear pictures or videos of the parking spots, cameras are positioned at key points across the parking lot. For the cameras to properly record the parking spots and cars, they need have a high enough resolution and frame rate. The more precise the detection and processing, the greater the resolution and frame rate that the camera can record. To reduce occlusions and guarantee that every parking spot is viewable, the camera positioning and viewing angles should also be carefully considered. Even if there are other parking spots filled, the camera should be able to see every parking space. In contrast to Figure 2-3, which shows all parking spaces visible without obstruction, Figure 2-2 shows a parking spot in the upper right corner of the image that cannot be completely seen, leading to erroneous readings of the availability of parking spaces. Depending on the use case, processing technique, network, and parking lot premises, any type of camera, including CCTV and IP cameras, can be utilised to record video of the lot.

2. **Image Preprocessing:** The acquired images are pre-processed to enhance their quality and remove noise, shadows, or other unwanted artifacts that could interfere with accurate analysis (Krig, 2016)

   Common preprocessing techniques include:

   - **Noise removal**: Using filters (such as Gaussian, Median, and Bilateral) to lessen noise and enhance picture quality. Because of things like transmission problems, ambient influences, and poor sensors, digital photographs frequently contain different kinds of noise analysis (Krig, 2016). Accurately detecting and analysing elements like parking spots or cars can be challenging due to noise, which can also add visual distortion, diminished contrast, colour distortion, and undesired artefacts. The overall clarity and quality of the image are improved by using noise reduction filters, such as Gaussian or median filters, which help to reduce noise while maintaining crucial image information.

   - **Color space conversion:** Converting an RGB image to greyscale or another colour system (such as Hue, Saturation, Value (HSV)) that could be more suited for particular algorithms is known as colour space conversion. Colour information is represented differently in different colour spaces. According to Jashojit et al. (2016), the RGB colour space depicts colour as a blend of red, green, and blue components, whereas the HSV colour space depicts colour as a blend of hue, saturation, and value components. Because the hue, saturation, and value components of the HSV colour space are easier to manipulate than the red, green, and blue components of the RGB colour space, it is frequently employed for tasks involving color-based segmentation or object

tracking. Thus, RGB to HSV is a viable colour space conversion for an image processing-based parking spot detecting system.

- **Perspective correction**: According to Wojciech et al. (2007), perspective correction is a method used in image processing to fix distortions in a picture brought on by the viewpoint of the camera. The parking spots may seem warped or twisted when a camera is positioned at an angle. By adjusting for this angle, perspective correction helps to make the parking spots look more precise and rectangular. Additionally, perspective correction helps eliminate lens distortion, which can cause straight lines to look bent or curled. In order to make parking spots simpler to identify and examine, perspective correction aims to change the picture such that they look as though they were taken from a straight overhead viewpoint.

3. **Segmentation:** In order to distinguish the areas of interest (parking spots) from the backdrop in the obtained picture, the segmentation method is essential. Partitioning the picture into many segments or areas, each of which represents a significant aspect of the image, such as parking spots, cars, or other items, is the main objective of segmentation.

The segmentation process for vacant parking space detection typically involves the following steps:

1. **Region-based segmentation:**. This method creates uniform zones by combining pixels with comparable characteristics, such colour, texture, or intensity (Cloudinary, 2024). Region growth, split-and-merge algorithms, and thresholding are often employed techniques for region-based segmentation. a. Thresholding: This technique uses intensity values to classify pixels as being in a certain zone. In a greyscale picture, for instance, pixels with intensity values over a particular threshold can be regarded as belonging to a parking spot, whereas pixels with values below the threshold are regarded as the backdrop. b. Region growing: Using a collection of seed points (pixels) as a starting point, this method iteratively expands areas by include nearby pixels that meet specific similarity requirements, such colour or intensity (Balasubramanian, et al., 2016). c. Split-and-merge: This method repeatedly divides and combines sections according to predetermined homogeneity criteria, such variance or edge strength, after first treating the entire picture as a single region.
2. **Edge-based segmentation**: To determine the edges or borders of objects in the picture, this method makes use of edge detection algorithms (Cloudinary, 2024).). The

parking spots may subsequently be distinguished from the background using the edges that have been identified. The Canny edge detector, Sobel operator, and Prewitt operator are examples of common edge detection techniques.

4. Image Processing Method: Image processing involves identifying and locating objects (vehicles) within an image. This can be achieved through background subtraction method, edge detection and line segmentation, or feature-based object detection**.**

a) Background Subtraction: Using the background subtraction approach, each incoming frame is subtracted from a backdrop model of the empty parking lot. Foreground objects are the residual pixels or areas that can be further processed for object categorisation and detection. Typically, this approach includes linked component analysis, post-processing, foreground identification, and background modelling (Junzhao, et al., 2013). When the camera is still and the illumination is somewhat steady, background removal can work well. It could, however, have trouble with issues like dynamic backdrops, shadows, and shifting lighting.

b) Edge Detection and Line Segmentation: To determine the limits of parking spots inside an image, this method combines line segmentation techniques with edge recognition algorithms (Satya, n.d.). The process entails identifying edges using edge detection algorithms (such as Canny, Sobel, and Prewitt), extracting line segments from the detected edges using methods like the Hough Transform or Line Segment Detector (LSD), and then examining the extracted line segments and their intersections to find rectangular areas that might be parking spaces (Vipin, 2024). Post-processing techniques may also be used, such as applying geometric restrictions or heuristics based on the parking lot layout to refine the identified parking space locations. This approach may work effectively in parking lots that are well-organised and have clearly defined spots, but it may have trouble with occlusions, faded or missing.

c) **Feature-based Detection**. Geometric, colour, texture, edge, and context are some of the characteristics that are extracted from the segmented regions and used as input to an object recognition algorithm that is based on machine learning or deep learning (Sumitkrsharma, 2023). Typically, it entails steps like feature extraction, applying non-maximum suppression to eliminate redundant or overlapping detections, fine-tuning the detected bounding boxes, or filtering out false positive detections using additional post-processing techniques. Parking spaces are then detected using the extracted features and either traditional machine learning algorithms (like SVMs, Random Forests) or deep learning-based object detectors (like Faster R-CNN, YOLO, SSD). Because the algorithms can learn to recognise patterns and characteristics linked to empty parking spots, feature-based identification may be more

resilient to changes in parking lot layouts, occlusions, and lighting conditions. However, it can need a significant amount of computer resources and labelled training data, particularly for deep learning-based methods.

The image processing methods will be further discussed and compared in Chapter 2.1.

5. Classification

Classification involves determining whether the detected objects are vehicles or other entities (JavatPoint, 2024). In object detection, after the bounding boxes are predicted, each bounding box must be classified to determine what object it contains. This can be done using machine learning models.

a) Support Vector Machines (SVM): SVM uses data analysis to make classifications. Finding the hyperplane that best divides the data into classes is how it operates (Sungchul & Hwangjo, 2012). A collection of labelled photos, each of which is identified as either empty or having a car has to be collected. The SVM is then trained using the labelled pictures, and the features that are taken out of the images are used as input into the model together with the respective labels. Features such as the edge detection and the Histogram of Orientated Gradients (HOG) are extracted from images to distinguish between vehicles and empty spaces. The trained SVM is used to categorise areas of a picture using the same set of characteristics that were extracted. Based on the learnt hyperplane, the SVM will forecast if each region of interest (ROI) in the parking lot is filled with cars or empty.

b) Convolutional Neural Networks (CNN): CNNs are deep learning models designed for image classification tasks. The model consists of convolutional layers that automatically learn spatial hierarchies of features from input images (GeeksforGeeks, 2024). A large dataset of images are collected and labeled with either 'Vehicle' or 'Empty'. The dataset should cover images of various conditions such as different lighting, weather, and angles. A CNN architecture suitable for the task will then be designed, typically including multiple convolutional layers followed by pooling layers and fully connected layers (GeeksforGeeks, 2024). The CNN is then trained using the labeled dataset and learn to identify features such as edges, textures, and complex patterns that differentiate vehicles from empty spaces. Techniques such as data augmentation is used to improve generalization and prevent overfitting. ROIs of the parking spaces that correspond to individual parking spaces must be defined. Pass each ROI through the trained CNN, which will classify it as either 'Vehicle' or 'Empty' based on the features learned during training.

Table 2. 1 Summary of Literature Review on the Comparison between CNN and SVM

No.	Title	Authors	Dataset	Method	Results (Accuracy)	Findings
1.	Comparison of CNN and SVM for Ship Detection in Satellite Imagery	(Jati & Arrie, n.d.)	Optical Satellite Images to Detect Ships	CNN	99.12%	In terms of ship detection, CNN has demonstrated its superiority over SVM, with an average training accuracy of 0,9912, or 99.12%, and a validation accuracy of 0,9798, or 97,89%. SVM achieves an accuracy of 94,38%, or 0,9438.
				SVM	94.38%	
2.	Image Classification using SVM and CNN	(Yeshwanth, et al., 2020)	Images of Dalmations, Dollar Ball, Pizza, Soccer Ball, and Sunflower	CNN	93.57% on large data	A relatively tiny dataset was used with SVM in order to reach a 93% accuracy rate. Even with SVM's great strength, attaining this kind of high accuracy is still unusual. The reason the results had such high accuracy was because there wasn't a big enough dataset. Thus, more than tripled the size of the dataset has been employed using data augmentation. An accuracy of 82% has been obtained while using SVM, causing a notable drop. Deep learning method was then chosen as the outcomes were not satisfying. Upon effectively integrating CNN, an astounding 93.57% accuracy rate was attained using the identical dataset.
				SVM	93% on small data 82% on large the data (triple the size)	

3.	Comparison Between Convolutional Neural Network CNN and SVM in Skin Cancer Images Recognition	(Zaid, et al., 2021)	320 Clinical Images of the Different Diseases	CNN	97%	The ResNet-50 model was used to train the DLCNN, and it demonstrated accurate classification of the benign and malignant skin categories. With testing images, the ResNet-50 DLCNN achieved a strong identification rate of over 97%, demonstrating that the skin scans with benign and malignant lesions are correctly identified. 86.9% accuracy was attained by the support vector machine (SVM) classifier for the classification of skin cancer during the feature extraction phase. This indicates that,CNN is more accurate by 11%.
				SVM	86.9%	

Table 2.1 shows the experiments conducted by researchers on comparing CNN and SVM with various types of dataset. According the research, CNN's accuracy is consistently higher than SVM especially when applied to large numbers of dataset as seen in the experiment by Yeshwanth, Nanda, & Rao in 2020 where the accuract of SVM drops significantly when the number of dataset was increased by more than triple while the accuracy of the CNN algorithm remains high at 93%.

2.1 Image Processing Methods

Image processing involves techniques and methods used to analyze, manipulate, and transform digital images to enhance their quality, extract information, or prepare them for further tasks like image classification. It is widely used in fields such as computer vision, medical imaging, remote sensing, and multimedia.

2.1.1 Background Subtraction

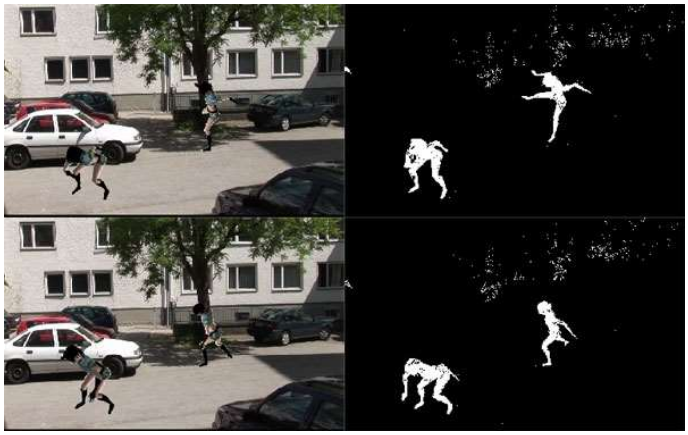


Figure 2- 4 Example of Detecting People Using Background Subtraction (Phoaivu, 2011)

Background subtraction is a computer vision technique used to detect moving objects in a video or image sequence by separating the foreground (moving objects) from the background (static scene) as shown in Figure 2-4 (Piccardi, 2004). It is commonly used in applications such as video surveillance, human-computer interaction, and traffic monitoring.

The basic idea behind background subtraction is to create a reference background model of the static scene and then compare each incoming frame with this background model (Sabih, 2022). The pixels that deviate significantly from the background model are considered part of the foreground (moving objects).

The process of background subtraction typically involves the following steps:

1. **Background Modeling**: The first step is to estimate the scene's background in order to generate a background model. This may be accomplished by taking many pictures with nothing moving or by modelling the backdrop pixel values over time using sophisticated methods like kernel density estimation or Gaussian mixture models (Sabih, 2022).

2. **Foreground Detection**: The current frame is compared to the backdrop model for every new frame in the video sequence. Foreground pixels are defined as pixels that display a considerable departure from the backdrop model (Sabih, 2022). This is usually accomplished by using a threshold to identify whether a pixel is in the background or foreground after calculating the absolute difference between the current frame and the background model.
3. **Foreground Mask Generation**: A binary mask, with foreground pixels represented by ones and background pixels by zeros, is the product of the foreground detection stage (Sabih, 2022). The moving items and the static backdrop are separated by this mask.
4. **Morphological Operations (Optional)**: Morphological operations like erosion, dilation, opening, and closing can be applied to the foreground mask to remove noise, fill in holes, or smooth the boundaries of the detected foreground objects (Ghassem, et al., 2013).
5. **Object Tracking (Optional)**: Once the foreground objects are detected, additional techniques like object tracking can be applied to follow the movement and trajectory of the objects over time.

Background subtraction techniques work best when the background is stationary or slowly changing, and the lighting conditions are stable (Piccardi, 2004). Thus, this method can face challenges in scenarios with dynamic backgrounds (e.g., waving trees, water surfaces), gradual illumination changes, and camouflage (when the foreground and background have similar colors).

2.1.2 Edge Detection and Line Segmentation

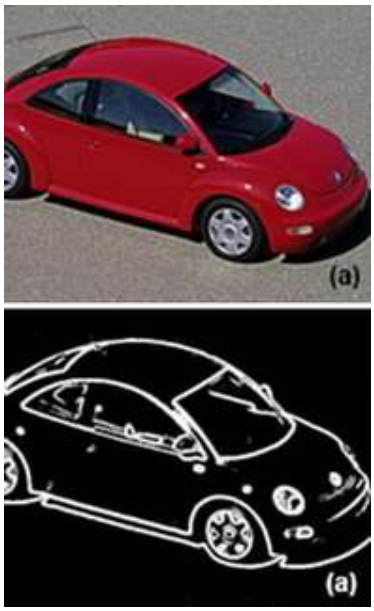


Figure 2- 5 Example of Detecting Vehicle with Edge Detection and Line Segmentation (Li, et al., 2016)

Edge Detection: Edge detection is a technique used to identify the boundaries or edges of objects within an image as shown in Figure 2-5 above. Edges are defined as when there is a sharp change in pixel intensity or color, that indicates the boundary between different objects or regions in the image as seen in Figure 2-5 where the outline or edge of the car is extracted from the top image to the bottom image.

The process of edge detection involves applying mathematical operators (filters) to the image to identify and highlight these sharp changes in pixel values. Some common edge detection algorithms are gradient-based methods, Laplacian-based methods and canny edge detection.

Gradient-based approach determines the rate of change in pixel brightness in both horizontal and vertical directions. Combining these numbers yields the edge strength and direction for each pixel, effectively highlighting locations where intensity changes suddenly. Sobel, Prewitt, and Roberts are popular gradient-based operators that use different strategies to generate gradients and aid highlight edges in various image-processing tasks (Jamil & Mahgoub, 2016).

Laplacian-based algorithms detect edges by looking for zero-crossings in the image's second derivative, which is produced using the Laplacian operator. This method catches regions where intensity changes abruptly, making it perfect for edge localisation. Laplacian-based

algorithms frequently provide accurate edges, which are useful for applications requiring great precision (Mikhail & Partha, 2008).

Canny Edge Detection is one of the most used edge detection techniques because it works well in complicated pictures. This approach involves several steps, including noise reduction, gradient computation, non-maximum suppression, and hysteresis thresholding. Canny edge detection, which combines these procedures, provides thin and well-defined edges, decreasing noise and improving clarity (OpenCV, 2024).

The output of edge detection algorithms is typically a binary image, where pixels belonging to edges are represented as white (1) and non-edge pixels are represented as black (0).

Line Segmentation: Line segmentation is the process of identifying and extracting line segments or straight lines from an image or a set of edge points. It is often used in applications such as document analysis, object recognition, and object tracking. Once edges have been detected, line segmentation algorithms can be applied to group these edge points into meaningful line segments. Hough transform, ransom sample consensus (RANSAC), and spilit-and-merge alogirhtms are common line segmentation techniques.

The Hough Transform is a prominent approach for finding lines in images that involves translating image points from Cartesian to a parameterised Hough space. Each point in the Hough space represents a possible line in the original picture, therefore points that align in this space may be collected and recognised as line segments. This method is effective at detecting lines in pictures that are broken or have tiny variations in direction, making it a valuable tool for pattern recognition and computer vision applications (Priyanka & Bidyut, 2014).

Random Sample Consensus (RANSAC) is an iterative model-fitting approach that uses a random subset of data points to fit a line model. RANSAC refines the line model through multiple iterations by determining the number of inliers (data points that closely match the model within a specified tolerance). This procedure continues until a good model is discovered, with a sufficient number of inliers to corroborate the observed line. RANSAC is very useful in noisy pictures since it iterates and refines several subsets of data to get the best-fitting line.

Split-and-Merge Algorithms adopt a different method, beginning with considering each edge point as a separate line segment. The technique combines collinear segments via iterative

evaluation, joining those that align with one another and dividing those that deviate depending on parameters such as angle or distance. This method is effective for spotting segmented lines or structures in photos in which various segments of the line may differ in orientation or continuity.

The output of line segmentation algorithms is typically a set of line segments, represented by their start and end points, or by their slope and intercept parameters. Edge detection and line segmentation are fundamental techniques in computer vision and are often used as preprocessing steps for higher-level tasks such as object recognition, feature extraction, and image understanding.

2.1.3 Feature-based Object Detection

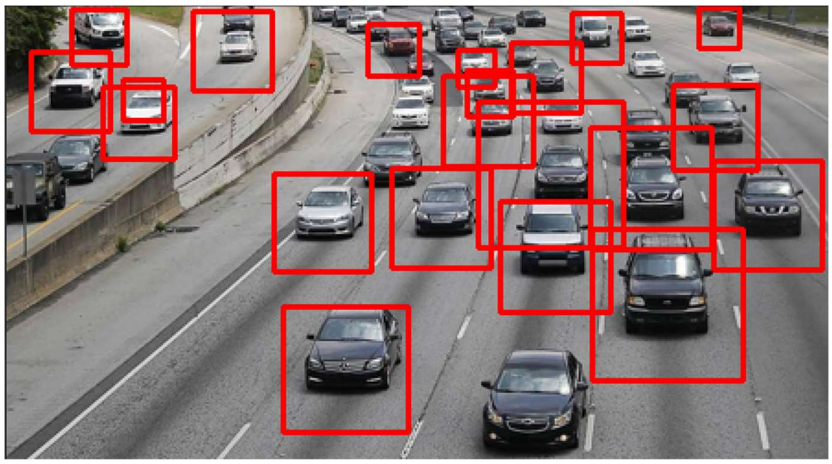


Figure 2- 6 Example of Detecting Vehicles Using Feature-Based Object Detection (Kaan, 2023)

Feature-based object detection is a computer vision approach that identifies and locates certain items or patterns inside an image or video frame, as seen in Figure 2-6. It entails extracting distinguishing features from a picture and comparing them to a pre-trained database of features characterising the items of interest (Sharma, 2023). This approach extracts distinguishing characteristics from an input picture or video frame. These traits might be based on numerous qualities such as edges, corners, forms, textures, or colour distributions. Depending on the technique, feature extraction may play a variety of functions while training a model.

a. Traditional Machine Learning Pipelines

Traditional machine learning pipelines require manually extracting characteristics from photos. This extraction uses well-defined methods including Scale-Invariant Feature Transform (SIFT), Speeded-Up Robust Features (SURF), Histograms of Orientated Gradients (HOG), and Local Binary Patterns (LBP). These approaches are intended to capture unique aspects of picture data, such as edges, corners, forms, textures, or colour distributions, which are critical for recognising and distinguishing objects within an image. Once retrieved, these characteristics are fed into classic machine learning methods such as Support Vector Machines (SVMs), Random Forests, or K-Nearest Neighbours (KNN) (Nicholas, n.d.). These algorithms use the extracted characteristics to build a model that can categorise or predict outcomes based on previously learnt patterns. For example, in a parking spot identification system, HOG characteristics might be employed to describe the form of vehicles. This method relies on manually developed features that are tuned to capture the unique qualities of the items of interest.

b. Deep Learning Pipelines

Deep learning-based object identification is a computer vision technology that use deep neural networks to identify things in digital photos or movies. It entails training a deep learning model on a large dataset of labelled pictures, with the objects of interest marked with bounding boxes or pixel-wise segmentation masks.

The procedure begins with Data Preparation, which involves collecting and annotating a huge collection of photos. Each image is labelled with information on the existence of objects, their location, and, in many cases, their class, which serves as the foundation for training a detection model. Following data preparation, the Model Architecture stage selects a deep neural network architecture optimised for object detection tasks. Region-based convolutional neural networks (R-CNNs), such as Faster R-CNN, and single-shot detectors, such as YOLO (You Only Look Once) and SSD (Single Shot MultiBox Detector), are popular designs, with each providing distinct trade-offs in speed and accuracy (GeeksforGeeks, 2024).

The next step is Feature Extraction, which involves passing the input picture through convolutional layers to extract key features at several scales and resolutions, improving the model's capacity to recognise objects of different sizes and shapes (GeeksforGeeks, 2024). Following feature extraction, the Region Proposal stage finds prospective areas of the picture that may contain items of interest. This is often accomplished using approaches such as selective search or region proposal networks (RPNs), which serve to narrow down the regions

of attention, hence lowering the complexity of future processes (GeeksforGeeks, 2024).

In Classification and Localisation, the model divides the suggested areas into several item categories and fine-tunes the bounding boxes or segmentation masks to better encompass the discovered objects. This stage is crucial for accurate item recognition and precise localisation (GeeksforGeeks, 2024). The Model Training phase then fine-tunes the network by minimising a loss function that balances classification accuracy with localisation precision, frequently employing backpropagation and optimisation techniques such stochastic gradient descent (GeeksforGeeks, 2024).

Finally, during inference, the trained model is used to recognise objects in previously unseen photos or videos. It uses the previously learnt feature extraction, region proposal, and classification/localization procedures to identify objects in real time or in batches for a variety of applications (GeeksforGeeks, 2024). These processes provide a thorough pipeline that enables deep learning-based object identification models to learn complicated visual patterns and efficiently generalise to fresh data.

Deep learning-based object detection has been instrumental in various applications, such as autonomous driving, surveillance systems, robotics, and augmented reality, where accurate and real-time object detection is crucial. It has achieved state-of-the-art performance on benchmark datasets and continues to be an active area of research in computer vision and machine learning. In deep learning pipelines, the process of feature extraction is integrated within the architecture of the neural network itself, typically through convolutional layers.

Convolutional Neural Networks (CNNs), which are commonly used in this context, are designed to automatically detect patterns such as edges, textures, and shapes by applying convolutional operations followed by pooling (Lev, 2024). During training, the CNN learns hierarchical features where lower layers capture basic patterns like edges and textures, while deeper layers capture more complex structures like shapes and object parts. This automatic feature learning allows the model to adapt to the specific characteristics of the data it is trained on, often leading to more accurate and robust predictions. In a parking space detection system, a CNN can learn to identify and distinguish vehicles from their surroundings, thus detecting whether parking spaces are occupied or vacant. This method benefits from the capability of deep learning to handle large and complex datasets, making it a powerful tool for image-based tasks.

Feature-based object identification has various advantages, making it a popular choice in settings that need dependable and adaptive detection. One of the primary advantages is its

resistance to occlusion, as it focusses on local characteristics, allowing partial object recognition even when other sections are obscured. Furthermore, many feature extraction algorithms are intended to be scale and rotation invariant, allowing objects to be recognised at a variety of sizes and orientations. This adaptability is further strengthened by efficient matching approaches like k-d trees or approximate closest neighbour searches, which expedite the process and make feature-based algorithms very successful in real-time or resource-constrained contexts (Lev, 2024).

Feature-based object identification has several limits, though. The method is often sensitive to changes in viewpoint; major changes in an object's appearance or angle might diminish detection accuracy. Furthermore, the reliance on training data means that performance is inextricably linked to the quality and diversity of data used in feature generation, which might restrict generalisation to new or unknown objects. Finally, computational complexity might be an issue, since extracting and matching characteristics, especially in high-resolution pictures or video streams, requires substantial computing power, thus restricting its application in real-time detection scenarios (Engati Technologies, 2024):

Feature-based object detection is widely used in various applications, including object recognition, image retrieval, augmented reality, and robotic vision systems. However, in recent years, deep learning-based object detection methods have gained popularity due to their superior performance and ability to learn complex features directly from data.

2.1.4 Comparison of Image Processing Methods

Table 2. 2 Comparison of Image Processing Methods

Aspect	Background Subtraction	Edge Detection and Line Segmentation	Feature-based Object Detection
Purpose	Detect moving objects by separating foreground from background	Identify edges and line segments in an image	Identify and locate specific objects or patterns in an image
Input	Video sequence or image sequence	Single image	Single image. Can be used for video sequences, but it typically requires additional steps

Output	Binary mask separating foreground and background	Binary edge map and set of line segments	Locations and scales of detected objects
Techniques	Background modeling, foreground detection, and morphological operations	Edge detection filters (Sobel, Canny, etc.), Hough Transform, RANSAC, Split-and-Merge algorithms	**Traditional Machine Learning Pipelines:** (SIFT, SURF, HOG, etc.), feature matching, geometric verification. **Deep Learning Pipelines:** Implement deep neural networks architecture such as Faster-RCNNs, YOLO, and SSD.
Advantages	Real-time object detection, works well for stationary backgrounds	Provides low-level image features for further processing, can detect object boundaries	Robust to occlusion, scale and rotation invariance, efficient matching
Limitations	Sensitive to dynamic backgrounds, illumination changes, camouflage	Cannot detect semantic information, sensitive to noise	Dependent on training data, sensitive to viewpoint changes, computationally expensive
Applications	Video surveillance, human-computer interaction, traffic monitoring	Document analysis, object recognition, object tracking	Object recognition, image retrieval, augmented reality, robotic vision

Background Subtraction is effective for real-time object detection in video sequences or image sequences, particularly in scenarios with stationary backgrounds. However, it is sensitive to dynamic background changes, illumination variations, and camouflage, which can limit its applicability in more complex environments.

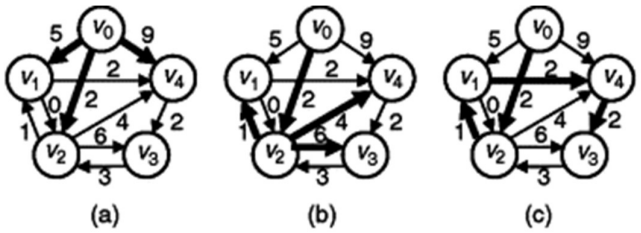
Edge Detection and Line Segmentation techniques are useful for extracting low-level image features, such as edges and line segments, which can aid in object boundary detection and subsequent processing. While these methods cannot directly detect semantic information or handle noisy data effectively, the techniques provide valuable input for higher-level processing tasks, such as document analysis, object recognition, and object tracking.

Feature-based Object Detection methods, particularly those based on deep learning pipelines, have shown remarkable performance in identifying and locating specific objects or patterns in images and video sequences. These methods are robust to occlusion, scale, and rotation invariance, making them well-suited for applications like object recognition, image retrieval, augmented reality, and robotic vision. However, their reliance on large training datasets, sensitivity to viewpoint changes, and computational complexity can pose challenges in certain scenarios.

2.2 Shortest Path Algorithm

A shortest path algorithm is a computational method used to determine the shortest path between two nodes in a graph (Magzhan & Hajar, 2013). The graph is typically composed of nodes (vertices) connected by edges, which may have associated weights representing distances or costs. The goal of the algorithm is to find the path that minimizes the total weight (distance or cost) from a starting node to a destination node. These algorithms are widely used in various fields, including routing and navigation systems, network optimization, social network analysis, and many others. The algorithm used for IICP Park Finder is to determine the nearest parking space to the entrance points on campus so that students do not need to walk a far distance after parking. Therefore, only algorithms that incorporates weighted graphs and single source, shortest-path algorithms will be explored for this section such as the Dijkstra's algorithm, Bellman-Ford algorithm, and A*-Search algorithm.

2.2.1 Dijkstra's Algorithm



vertex	Predecessors					Shortest-Path Estimates				
	v_0	v_1	v_2	v_3	v_4	v_0	v_1	v_2	v_3	v_4
non	NIL	NIL	NIL	NIL	NIL	0	∞	∞	∞	∞
v_0	NIL	v_0	v_0	NIL	v_0	0	5	2	∞	9
v_2	NIL	v_2	v_0	v_2	v_2	0	3	2	8	6
v_1	NIL	v_2	v_0	v_2	v_1	0	3	2	8	5
v_4	NIL	v_2	v_0	v_4	v_1	0	3	2	7	5
v_3	NIL	v_2	v_0	v_4	v_1	0	3	2	7	5

Figure 2- 7 Dijkstra's Algorithm Diagram and Table (ScienceDirect, 2023)

Dijkstra's algorithm is a greedy algorithm that solves the single-source shortest path problem for a graph with non-negative edge weights (Magzhan & Hajar, 2013) as shown in Figure 2-7 above. It finds the shortest path from a given source node to all other nodes in the graph.

The algorithm starts by initialising tentative distance values for each node in the graph, setting the source node's distance to zero and all others to infinity, and keeping separate sets for visited and unvisited nodes. It continually picks the nearest unvisited node as the current node, permanently assigning its tentative distance value. The tentative distances of each unvisited neighbour are then updated by adding the current node's distance and the weight of the edge linking it to the neighbour. If the determined number is less than the neighbor's current

approximate distance, it is changed appropriately. After an update, the current node is marked as visited. This procedure is repeated, picking the nearest unvisited node and updating neighbours until all nodes have been visited or the target node has been reached (Magzhan & Hajar, 2013).

The setback of this algorithm is it assumes that all edge weights in the graph are non-negative, so using negative weights may provide inaccurate results. Additionally, the approach is only intended to identify the shortest path between a single source node and all other nodes in the network (Amgad, et al., 2017):

The time complexity of Dijkstra's method is determined by the implementation and data structure used to store the graph and tentative distance values. A binary heap or Fibonacci heap implementation has a time complexity of O((V + E) log10 V), where V is the number of nodes (vertices) and E is the number of edges in the graph (GeeksforGeeks, 2024).

The algorithm is commonly used in routing and navigation systems, network routing protocols, social network analysis, and a variety of other applications that involve determining the shortest path between nodes in a graph.

2.2.2 Bellman-Ford Algorithm

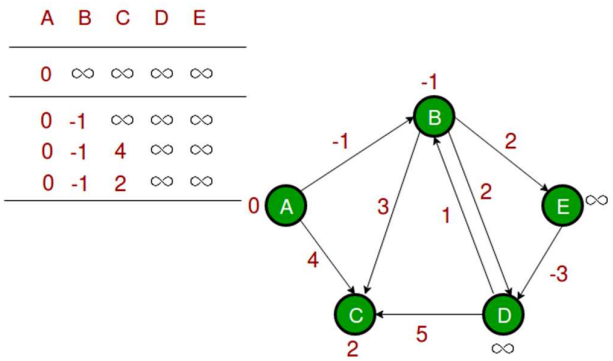


Figure 2- 8 Bellman Ford's Algorithm (GeeksforGeeks, 2024)

The Bellman-Ford algorithm is a shortest path algorithm that can handle graphs with negative edge weights (GeeksforGeeks, 2024). It is used to find the shortest path from a single source node to all other nodes in a weighted graph. Unlike Dijkstra's algorithm, which cannot handle negative edge weights, the Bellman-Ford algorithm can handle both positive and negative edge weights. Figure 2-8 shows how Bellman-Ford algorithm is illustrated in table form according to the graph.

The algorithm begins by assigning a distance value to each node in the graph: the source node is set to zero, and all other nodes are assigned to positive infinity. It also includes a predecessor array that records the preceding node in the shortest path from the source. The algorithm iterates through all edges in the graph and updates node distances based on the condition that if the sum of a node's distance u u and the weight of the edge (u,v) is less than the current distance of node v, the distance of v is updated to this smaller value, and u u is set as the predecessor of v v. The relaxation stage is done V−1 times (where V is the total number of nodes) to ensure the shortest path between nodes may be determined within V−1 edges. Finally, a final relaxation phase looks for negative cycles; if any distance value changes again, the algorithm recognises a negative cycle and states that there is no solution. (Andrew & Tomasz, 1993)

The Bellman-Ford algorithm can handle networks with negative edge weights, whereas Dijkstra's approach is limited to non-negative weights. This makes Bellman-Ford especially valuable in cases with probable negative values. However, it is less efficient than Dijkstra's for dense networks with non-negative weights, as its time complexity is O(V∗E), where V is the number of vertices and E is the number of edges. Despite its increased complexity, Bellman-Ford remains adaptable, as it can handle edge circumstances such as zero-weight edges and self-loops, which can be difficult for other shortest path algorithms (GeeksforGeeks, 2024).

The Bellman-Ford algorithm is useful in situations where negative edge weights are present, such as in certain network routing protocols or in analyzing financial systems with negative interest rates.

2.2.3 A* Search Algorithm

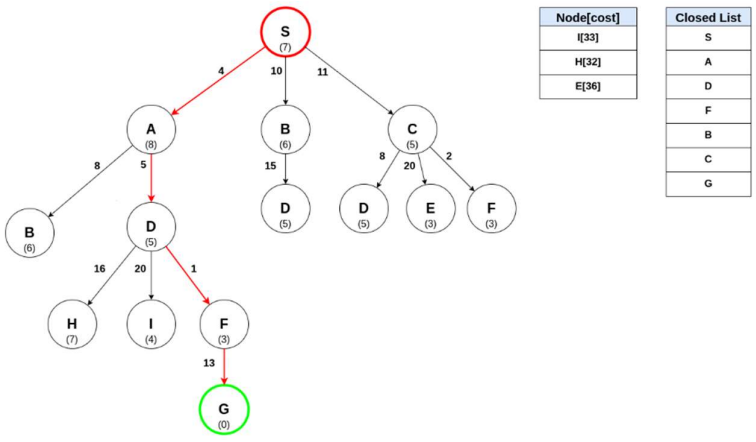


Figure 2- 9 A Search Algorithm (Codecademy, 2024)*

The A* (A-star) Search algorithm is a best-first search algorithm used to find the shortest path between two nodes in a weighted graph. It is an informed search algorithm, meaning that it uses heuristics to guide the search process and improve efficiency (GeeksforGeeks, 2024). The A* algorithm combines the ideas of Dijkstra's algorithm and heuristic functions to explore the most promising paths first.

The method starts by creating two lists: an open list of nodes to be assessed and a closed list of nodes that have already been reviewed, with the start node added to the open list. It then generates an evaluation function f(n) = g(n)+h(n) for each node in the open list, where g(n) is the real cost from the start node to node n n, and h(n) is a heuristic estimate from n n to the objective. The node with the least f(n) value is chosen from the open list as the most promising to investigate further. This node is removed from the open list and added to the closed list; if it is the target node, the algorithm finds the shortest path and ends. Otherwise, it examines the neighbours of this node and calculates a preliminary cost: g (neighbor) = g(current) + cost(current, neighbor). If a neighbour is not currently on the open or closed list, or if the new tentative cost is lower than a previously recorded cost, it is added to the open list with an updated g (neighbour) and g(neighbor) value. Steps 3-5 are repeated until the target node is reached or the open list is empty, indicating that no path exists between the start and the goal (Franz, et al., n.d.).

The A* Search method uses a heuristic function h (n) to avoid overestimating the cost from the current node to the objective. This ensures that the algorithm can identify the shortest path if one exists. Given an acceptable heuristic and a path from beginning to end, A* assures finding the shortest path, making it both optimum and complete—it will discover a path if one exists or properly declare that no path is conceivable if none exists. Furthermore, A* outperforms uninformed search algorithms such as Breadth-First Search (BFS) and Depth-First Search (DFS) because it employs heuristics to prioritise and explore the most promising pathways first, considerably minimising wasteful exploration (Javatpoint, 2024).

The A* Search algorithm's time and space complexity are determined by its heuristic function and network topology. In the worst scenario, when the heuristic function is not successful, the time complexity might be exponential in the number of nodes (O(b^d), where b is the branching factor - the average number of edges from each node, and d is the number of nodes on the resultant path ((Graham, 2024). However, with an effective heuristic function, the algorithm can be substantially more efficient than ignorant search methods. The A* Search method is extensively utilised in a variety of applications, including pathfinding in video games and route planning in navigation systems.

2.2.4 Comparison Between the Shortest Path Algorithms

Table 2. 3 Comparison Between the Shortest Path Algorithms

Algorithms	**Dijkstra's Algorithm**	**Bellman-Ford Algorithm**	**A* Search Algorithm**
Handles Negative Edge Weights	No	Yes	Yes (with admissible heuristic)
Finds Shortest Path Between	Single source to all nodes	Single source to all nodes	Specific source to destination
Time Complexity (E = number of edges and V= number of vertices)	O((V + E) $\log_{10} V$)	O(V * E)	Depends on heuristic and graph structure. Worst case: *O(b^d)*, where *b* is the branching factor – the average number of edges from each node, and *d* is the number of nodes on the resulting path
Key Properties	▪ Greedy approach ▪ Non-negative edge weights • Single-source shortest path	• Handles negative edge weights • Detects negative cycles • Relaxation approach • Dynamic programming approach	• Informed search algorithm • Uses heuristics to guide search • Optimality guaranteed with admissible heuristic • Completeness guaranteed
Applications	▪ Network Routing Protocols	• Distance-Vector Routing Protocols • Currency Trading Platforms	• Robotics Navigation • AI Pathfinding for Games

	▪ Mapping and GPS Navigation ▪ Network Optimization		• Interactive Map Applications • Delivery Route Optimization

While Dijkstra's Algorithm is efficient for graphs with non-negative edge weights and finds the shortest path from a single source to all nodes, the Bellman-Ford Algorithm can handle negative edge weights and detect negative cycles. The A* Search Algorithm is an informed search algorithm that uses heuristics to guide the search for the shortest path between a specific source and destination, providing optimality and completeness guarantees with an admissible heuristic.

2.3 Existing Vacant Parking Space Detection System

Existing vacant parking space detection systems leverage a range of technologies to identify and manage available parking spots. These systems can broadly be categorized based on the technology they use: sensor-based, camera-based, and data-driven systems.

2.3.1 Penang Smart Parking Infrared Sensor



Figure 2- 10 Penang Smart Parking App and Infrared Sensor (Mei, 2021)

The Penang Smart Parking Sensor system is an initiative implemented by the Penang Island City Council (MBPP) in Malaysia to address parking challenges and improve the overall parking experience for motorists in Penang. The system utilizes infrared sensor technology to detect the availability of parking spaces in various parking areas within Penang Island as shown in Figure 2-11 above.

The Penang Smart Parking Sensor system involves the installation of magnetic sensors in each parking bay, embedded in the ground. The sensors detect the presence of a vehicle and send a signal to a central control system when a parking space is occupied, and similarly, they register when a space becomes vacant. This data is processed in real-time by the control system, which maintains an updated record of each parking bay's occupancy status, showing the total number of available and occupied spaces in the area. The processed occupancy data is then shared with motorists through various channels, including smartphone apps, variable message signs (VMS) at strategic locations, or digital displays within parking facilities, helping drivers locate available parking efficiently.

The technology provides real-time parking availability information, allowing vehicles to easily identify open spots, decreasing the time and annoyance associated with parking. This efficiency can also enhance traffic flow by reducing congestion caused by vehicles waiting to park. Furthermore, sensor data may be analysed to discover patterns, peak usage hours, and

high-demand regions, giving local officials vital insights into improving parking management and future infrastructure development. Furthermore, the system supports dynamic pricing, which allows parking prices to alter in real time based on demand, boosting effective use of space and potentially generating additional cash for the city (Buletin Mutiara, 2023).

The Penang Smart Parking Sensor system, part of the broader Penang Smart City initiative, faces several challenges. Each infrared sensor only covers a single parking spot, requiring numerous installations to monitor an entire facility, leading to high installation and maintenance costs compared to image processing systems that can cover more ground with fewer cameras. Additionally, infrared sensors are sensitive to environmental factors like sunlight, temperature, and obstructions which can result in false readings and compromise reliability, while image processing systems tend to be more resilient. Infrared sensors also have limited data output, typically providing only binary (vacant or occupied) status without details like vehicle type or license plate information, reducing the potential for advanced parking features and insights. Moreover, they are less adaptable to layout changes, requiring significant adjustments and costs for reconfiguration, unlike image processing systems, which can be recalibrated more flexibly. Infrared sensors' exposed placement in parking areas makes them vulnerable to tampering and damage, whereas strategically positioned cameras are generally more secure and resilient.

2.3.2 Ultrasonic Sensor in Shopping Mall Parking Lot

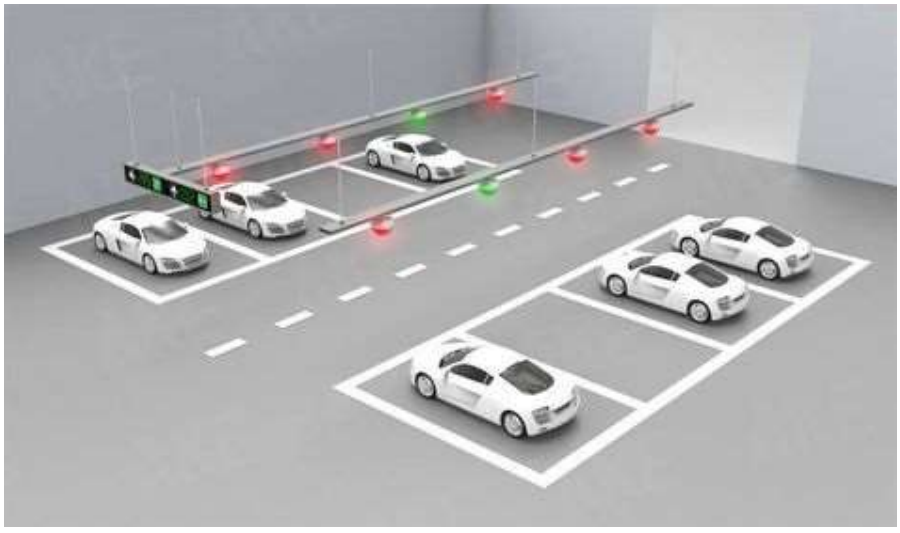


Figure 2- 11 Ultrasonic Sensors in Shopping Mall's Parking Lot (AKE Parking Technology, 2020)

The ultrasonic sensor used in shopping mall parking lots uses sensor technology that is widely employed in parking facilities to detect the presence or absence of automobiles in parking spots. This sensor operates on the principles of ultrasonic waves, which are high-frequency sound waves that humans cannot hear (Norlia, 2016). These sensors identify vehicles accurately, even in low-light or tough settings, thanks to time-of-flight measurements of ultrasonic waves. Their resilience allows them to resist a variety of weather conditions, making them appropriate for both indoor and outdoor parking applications. Furthermore, ultrasonic sensors are often less expensive than other parking sensor technologies, particularly when applied in big buildings, providing a good mix of performance and price (MatBotix, 2019).

The ultrasonic parking system works by strategically installing sensors across the parking lot, at overhead facing the regions being detected (parking lots). Each sensor transmits high-frequency ultrasonic waves (40 kHz to 250 kHz) to the parking area. When these waves strike a parked automobile, they bounce off its surface and return to the sensor. The sensor calculates the distance to the object by measuring how long it takes the waves to travel to and from it. If the measured distance falls within a particular range, it indicates that a vehicle is occupying the space; if it exceeds that range, the region is considered empty. The sensor then communicates the occupancy status (occupied or unoccupied) to a central control system or parking management software, allowing real-time tracking of parking availability (Norlia, 2016).

Ultrasonic sensors also have a number of drawbacks. The sensors have a narrow range and can be influenced by temperature, humidity, and surface qualities of target objects, which can result in missed detections or false positives. In surroundings with a high concentration of sensors, interference or crosstalk can occur, resulting in contradicting data, especially in big facilities where precise placement is required. Ultrasonic sensors generally give basic occupancy information without any extra context, restricting their potential to enable sophisticated features such as vehicle type recognition or license plate scanning, which might improve parking management. They are also susceptible to environmental conditions like wind and temperature, which can distort data and reduce accuracy. Finally, installing and maintaining these sensors can be time-consuming and expensive, as each sensor requires exact alignment and routine maintenance to ensure constant operation (Yong, et al., 2021).

While ultrasonic sensors are commonly used in shopping mall parking lots, the method may be combined with other sensor types, such as magnetic sensors or video-based systems, to improve overall accuracy and reliability in detecting parking space occupancy.

2.3.3 Comparison between Existing Systems and IICP Park Finder

Table 2. 4 Comparison Between Existing System and IICP Park Finder

Feature / Characteristic	Penang Smart Parking Infrared Sensor	Ultrasonic Sensor in Shopping Mall Parking Lot	IICP Park Finder
Technology	Infrared sensors	Ultrasonic sensors	Image Processing
Principle of Operation	Uses infrared light to detect the presence of vehicles by sensing changes in infrared radiation.	Uses ultrasonic waves to detect the presence of vehicles by measuring the time-of-flight of sound waves.	Image Processing Method with object detection via deep learning
Installation	Typically embedded in the ground or mounted overhead in each parking space.	Typically mounted overhead or on the ground, facing the parking spaces.	Mounted overhead for bird's eye view of multiple parking spaces.
Data Transmission	Transmits occupancy status to a central control system in real-time.	Transmits occupancy status to a central control system or parking management software.	Transmit data through RTSP protocol with port forwarding for remote access to data if using IP Camera. Transmit data through Ethernet cable if using traditional cameras such as CCTV.

Accuracy	Generally accurate but can be affected by environmental factors such as dirt or sunlight.	Generally accurate and performs well even in low-light or challenging environmental conditions.	Depends on classification model's dataset and accuracy.
Cost	One sensor to one parking space.		A single camera to monitor multiple spaces simultaneously
Installation Complexity	May require more complex installation processes, especially for ground-embedded sensors.	Easier to install as they can be mounted overhead or on the ground without major infrastructure changes.	Easy to install if using IP Camera as cabling is not required. Only one installation for every few parking spaces.
Maintenance	May require regular maintenance to ensure sensors are clean and unobstructed.	Generally low maintenance, but periodic checks for sensor alignment and functionality are needed.	Low maintenance.
Environmental Durability	Can be affected by harsh environmental conditions, such as dirt accumulation or direct sunlight.	Rugged and durable, suitable for a wide range of indoor and outdoor conditions.	Durable if use IP 66 Cameras and above with weatherproof and rainproof for outdoors.

Real-Time Information	Provides real-time updates on parking availability through various channels.	Provides real-time updates on parking availability through a central system or management software.	Provides real-time updates. Depends on internet connection if using wireless camera.
Detection Range	Limited to the specific parking space where the sensor is installed.	Limited to the specific parking space the sensor is monitoring; distance is calculated based on reflection time.	Limited to range of parking spaces viewed by the camera.
Integration With Other Systems	Can be integrated with smartphone apps, VMS, and digital displays.	Can be integrated with parking management software and other sensor types for enhanced detection.	Can be integrated with smartphone apps, VMS, and digital displays.
Data Processing	Central system processes data to provide real-time occupancy status and analysis.		
Adaptability	Well-suited for parking bays with clear line-of-sight for infrared sensors.	Adaptable to various parking layouts without obstructing vehicle movements.	Well adaptability with proper lighting and without obstructing view of the parking spaces.

Operational Environment	Primarily outdoor, but can be used indoors with proper installation.	Suitable for both indoor and outdoor environments.	Suitable for indoor more than outdoor due to varying light and weather conditions which may affect the accuracy of dectection.
Benefits	Reduces parking search time, helps in traffic management, and supports dynamic pricing.	Non-intrusive, accurate detection.	Easy to install, cost-effective, high potential for advanced features like license plate recognition and parking violation detection
Limitations	Performance can vary based on environmental noise and requires proper alignment for accurate detection.		

Image processing technology offers a compelling alternative to traditional sensor-based methods for detecting vacant parking spaces. Unlike infrared and ultrasonic sensors that require individual installation and maintenance for each space, image processing utilizes a single camera to monitor multiple spaces simultaneously, making it a cost-effective and scalable solution. Its flexibility allows for adaptation to various parking layouts, while the rich information extracted from images, such as vehicle type and size, can provide valuable insights beyond mere occupancy.

Furthermore, image processing reduces the need for extensive physical infrastructure and can easily adapt to changes in parking layouts or lighting conditions. The potential for advanced features like license plate recognition and parking violation detection, combined with real-time monitoring capabilities, makes it a powerful tool for efficient parking management. As a non-intrusive method, image processing minimizes disruption to parking spaces, offering a comprehensive and technologically advanced solution for optimizing parking utilization.

2.4 Database

A database is a structured data collection that is organized and maintained in such a way that information can be saved, retrieved, and manipulated efficiently (Oracle, 2020). According to Klause and Andreas (2001), the essential components of a database are data, which is the actual information saved in the database and may take many forms, including text, numbers, dates, photos, and documents. The schema serves as a design for the database's organization, specifying tables, columns, data types, and inter-table interactions. Tables are essential units in which data is structured into rows (records) and columns (fields), with each row representing a distinct entry and each column containing particular information. Queries are commands that obtain, insert, update, or remove data. They are often expressed in languages such as SQL. Indexes are data structures that enable quick searching and retrieval by producing a sorted representation of certain data segments. Finally, the Database Management System (DBMS) is the software that maintains and regulates database access, offering a user and application interface, assuring data integrity, security, backup, and performance optimization. The database used for an empty parking spot detection system is determined by a number of criteria, including the system's size, projected data volume, necessary performance, and the types of queries that must be handled.

Database Requirements

1. **Real-Time Updates**: Users need real-time information on parking space availability.
2. **Fast Querying**: Mobile applications require quick response times for a smooth user experience.
3. **Scalability**: The system should handle increasing numbers of users and parking spaces.
4. **Data Integration**: Combining different types of data, including sensor readings, user preferences, and location data.
5. **Flexibility**: Ability to adapt to changes in the data model as the application evolves.

Therefore, using a cloud database would be a suitable choice for developing a system for vacant parking space detection with a mobile application. Cloud databases offer real-time data synchronization capabilities, which are crucial for updating parking space availability information as sensors or users report changes in occupancy (Buchanan Technologies, 2024). The databases are designed to be extremely scalable, allowing for an increase in the number of users and parking spots while maintaining speed. Cloud databases are available worldwide, allowing customers to get parking space information from their mobile devices regardless of location. Furthermore, cloud providers frequently provide managed database services, which reduces operational expenses and maintenance duties for the development team. Cloud databases are meant to be adaptable and scalable, making it simpler to accommodate changes in the data model as the application matures.

While adopting a cloud database may incur additional expenses, the benefits of scalability, real-time updates, and worldwide availability frequently exceed the price, particularly for a mobile application with high needs (Google Cloud, 2024). A few cloud databases, such as the Google Cloud Firestore and MongoDB Atlas, provide a free tier with restricted consumption and flexibility.

2.4.1 Google Cloud Firestore



Figure 2- 12 Firestore Logo (Supun, 2020)

Google Cloud Firestore is a flexible, scalable NoSQL document database provided by Google Cloud Platform. It is part of the Google Cloud Platform's free tier, which includes 1GB of storage and 50,000 reads, 20,000 writes, and 20,000 deletes per day for Firestore (Google for Developers, 2024). Firestore is a document-oriented database that stores data as JSON-like documents arranged into collections, allowing for customizable data structures. It provides real-time updates by automatically syncing data changes across all connected clients, making it perfect for applications that require real-time synchronization, such as parking spot detection systems. Firestore also provides offline functionality, storing data locally so that mobile applications may run without an internet connection, with modifications synced when reconnected. Its geographic query capabilities enable location-based inquiries, which are critical for finding nearby available parking spots.

Additionally, Firestore is built for scalability, distributing data across different data centers while handling big data volumes and heavy traffic loads. Security features include data validation, user authentication, and fine-grained access control. Firestore works easily with other Google Cloud Platform services (such as Cloud Functions, Cloud Storage, and Firebase), easing application development and deployment. Multi-region replication improves data availability and latency by dispersing data across geographical regions. Google offers SDKs and frameworks for a variety of platforms, including iOS, Android, Web, and server environments, making Firestore integration easy across numerous apps (Google for Developers, 2024).

Using Firestore also has its disadvantages, including vendor lock-in, because it binds the application to the Google Cloud Platform environment, making it difficult to transfer to other

providers or self-hosted solutions. Firestore's querying capabilities are restricted when compared to other databases, necessitating extra client-side processing or assistance from complementing services like as Cloud Functions for complicated searches and aggregations. Although there is a free tier, the pricing mechanism might be prohibitively expensive for large-scale applications requiring high throughput or substantial data storage and transfer capabilities. Firestore, as a document-oriented database, may not be the best fit for applications that need complicated relational data models or multi-document transactions (Ben, 2023).

Google Cloud Firestore can be a good choice for the vacant parking space detection system due to its real-time updates, offline capabilities (for mobile apps), scalability, and integration with other Google Cloud services.

2.4.2 MongoDB Atlas



Figure 2- 13 Mongo DB Atlas Logo (Amazon Web Services, Inc. , 2024)

MongoDB Atlas is a fully-managed cloud database service provided by MongoDB, Inc. It is a cloud-hosted version of the popular open-source MongoDB database, offering several features and advantages for applications that require a scalable, high-performance NoSQL database solution (MongoDB Atlas, 2024).

MongoDB Atlas is a fully managed service in which MongoDB, Inc. manages database cluster provisioning, scalability, and maintenance, eliminating operational overhead and freeing developers to focus on application development. It provides high availability and automated failover by duplicating data across numerous nodes and regions, ensuring access even when hardware or networks fail. Atlas enables horizontal scalability and auto-scaling based on thresholds, making it flexible to expanding data and traffic volumes. Atlas supports numerous regulatory criteria while improving security with its sophisticated security and compliance capabilities, which include encryption, IP whitelisting, and monitoring. The technology enables global data distribution across several cloud locations, ensuring low latency access for consumers globally. Atlas connects with key cloud providers, including AWS, Google Cloud

Platform, and Microsoft Azure, enabling for seamless use with other cloud services. It also offers data migration tools for migrating data from current MongoDB installations or other databases, as well as automatic backups, point-in-time recovery, and queryable backups, all of which ensure complete data protection and quick restoration (MongoDB Atlas, 2024).

Using MongoDB Atlas has certain issues, including the pay-as-you-go pricing approach can be expensive for large-scale systems with significant storage and throughput requirements, since costs rise with data volume and traffic. Atlas's fully-managed service provides little control over the infrastructure and configuration, which may be a disadvantage for enterprises that want granular management for compliance or security reasons. With eventual consistency, Atlas may delay data propagation among copies, thereby necessitating additional steps for applications that require tight consistency. Its document-oriented paradigm, while adaptable, has a learning curve that requires developers to adjust to new querying and data modeling methods. Some sophisticated capabilities, including as sharding and query optimization, may need manual configuration as compared to self-hosted MongoDB environments, where administrators have more control. While Atlas works well with major cloud providers, it may have limited interoperability with third-party tools outside of the MongoDB ecosystem. Furthermore, being a shared cloud service, Atlas may experience performance unpredictability, particularly in shared cluster setups, which might affect application responsiveness (Jonathan, 2024).

MongoDB Atlas can be a good choice for applications that require a scalable, high-performance NoSQL database solution with robust security and compliance features. It offloads the operational overhead of managing database infrastructure, allowing development teams to focus on building applications. However, it's important to consider factors such as pricing, integration with existing infrastructure, and specific application requirements when evaluating MongoDB Atlas or any other cloud database service.

2.4.3 Comparison between Google Cloud Firestore and MongoDB Atlas

Table 2. 5 Comparison Between Google Firestore and MongoDB Atlas

Feature / Characteristic	Google Cloud Firestore	MongoDB Atlas
Service Type	Fully-managed NoSQL document database by Google Cloud Platform.	Fully-managed NoSQL document database by MongoDB, Inc.
Data Model	Document-oriented, JSON-like documents organized into collections.	
Real-Time Updates	Supports real-time updates, synchronizing changes automatically across clients.	No native real-time updates, but can be implemented with additional services like Change Streams.
Offline Support	Provides offline support with local caching and automatic synchronization.	Limited offline support; typically requires custom implementation or additional tools for full functionality.
Geospatial Queries	Supports geospatial queries, useful for applications like location-based services.	
Scalability	Automatically scales with traffic, distributing data across multiple regions and instances.	Horizontally scalable with auto-scaling capabilities, adding or removing nodes as needed.
Integration	Seamless integration with Google Cloud services (Cloud Functions, Firebase, Cloud Storage).	Integrates with Amazon Wed Services, Google Cloud Platforms, Azure, and supports various cloud services.
SDKs and Libraries	Available for iOS, Android, Web, and server-side environments.	SDKs and drivers available for multiple languages and platforms.
Query Capabilities	Limited querying capabilities; complex queries may need additional processing or use of Cloud Functions.	Rich querying capabilities, including aggregations and complex queries, but some may require manual setup.
Pricing Model	Free tier available; can become expensive for large-scale use with high throughput or data requirements.	Pay-as-you-go pricing; costs can escalate with increased storage and traffic volumes.
Data Modeling	Suited for JSON-like document structure; may not fit complex relational models or transactions spanning documents.	Flexible data model but can require adjustments for complex queries or transactional needs.

Consistency Model	Provides eventual consistency, with potential delays in change propagation.	Eventual consistency; strong consistency can be achieved with additional measures.
Infrastructure Control	Limited control over infrastructure due to being a fully-managed service.	Limited control over infrastructure with a focus on managed service.
Compliance & Security	Built-in security features and integration with GCP's security framework.	Strong security and compliance features, including auditing and monitoring tools.
Backup and Restore	Supports data export but may require custom solutions for comprehensive backup strategies.	Automated backups, point-in-time recovery, and queryable backups.
Learning Curve	Generally easier to learn with its document model and integration with Firebase for mobile apps.	May require learning new querying and data modeling techniques; learning curve can be steeper.
Performance	Performance scales automatically with usage but has limits on maximum reads/writes per second.	Performance depends on cluster configuration and can vary in shared environments.
Additional Features	Real-time updates, easy integration with Firebase for mobile app development.	Robust global distribution, advanced features like sharding, and detailed monitoring tools.

Google Cloud Firestore is advantageous for applications needing real-time synchronization, offline support, and easy integration with other Google Cloud services, particularly for mobile applications. It's highly scalable and provides robust geospatial queries, but has limited querying capabilities and may be costly at scale. **MongoDB Atlas** on the other hand is ideal for applications requiring flexible data modeling, high availability, and global distribution. It offers strong security and compliance features, advanced querying, and supports a variety of cloud platforms. However, it has a steeper learning curve and potential performance variability in shared environments.

2.5 Proposed Work

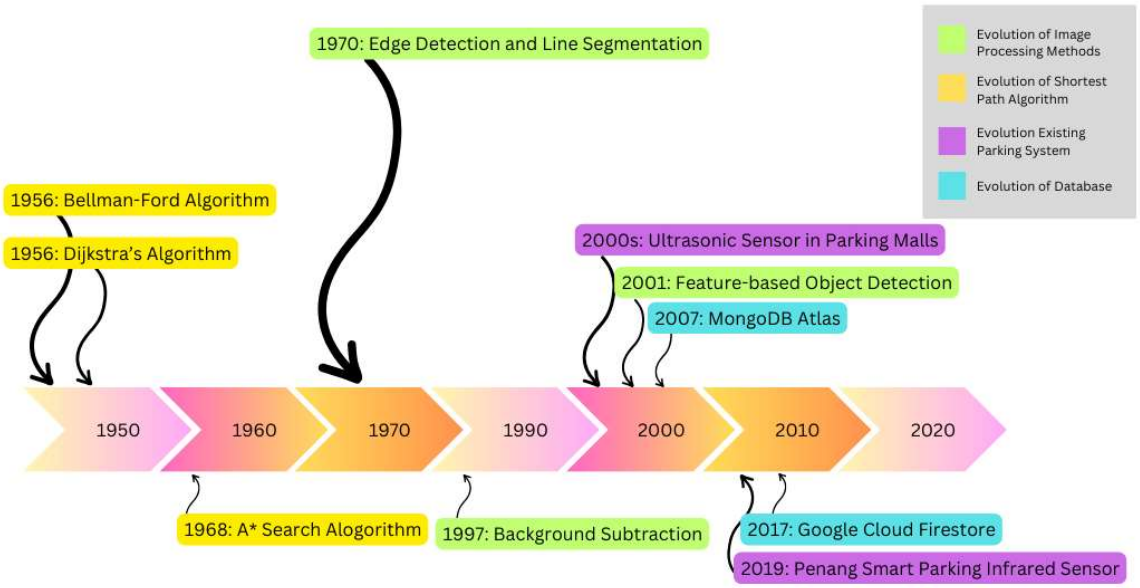


Figure 2- 14 Proposed Work Timeline for IICP Park Finder

IICP Park Finder, a parking space detection system, has chosen to implement the following components: feature-based object detection, Dijkstra's Algorithm, and Google Cloud Firestore. Figure 2-13 shows the timeline of the topics discussed in Chapter 2 to conclude the propose work of IICP Park Finder.

The primary objective of IICP Park Finder is to accurately detect and locate available parking spaces. Feature-based object detection algorithms are well-suited for this task because they can reliably identify and classify objects (in this case, parking spaces) based on their unique visual characteristics, such as shape, color, and context. Feature-based object detection methods analyze the image data to extract relevant features (e.g., edges, corners, textures) and then match these features against a pre-trained model to identify the objects of interest. These algorithms are robust to variations in lighting conditions, viewpoints, and occlusions, making them ideal for real-world scenarios like parking space detection.

Once the available parking spaces have been detected, IICP Park Finder needs to provide users with the most efficient route to reach the chosen entrance from the nearest parking spot.

Dijkstra's Algorithm is a widely adopted and effective pathfinding algorithm that calculates the shortest path between two points in a weighted graph. In the context of IICP Park Finder, the parking lot or area can be represented as a graph, with nodes representing intersections or decision points, and edges representing the paths or roads connecting them. The weights assigned to each edge can represent factors such as distance, traffic conditions, or other relevant costs. By applying Dijkstra's Algorithm, IICP Park Finder can determine the optimal route from the user's current location to the desired parking space, minimizing the travel time and distance. This algorithm ensures that users can quickly and efficiently navigate to their destination, enhancing the overall user experience and reducing frustration associated with finding parking.

IICP Park Finder is likely to handle a large volume of data, including parking space availability, user information, and potentially other related data. To ensure scalability, reliability, and efficient data management, the system requires a robust and flexible database solution. Google Cloud Firestore, a NoSQL document database provided by Google Cloud Platform, is an excellent choice for this purpose. Firestore offers several advantages that make it well-suited for IICP Park Finder, including scalability, real-time data synchronization, offline support, robust security features, and seamless integration with other Google Cloud services. By leveraging Firestore, IICP Park Finder can provide a seamless user experience, even in areas with poor network connectivity, while ensuring that sensitive information is protected and accessible only to authorized users.

By leveraging these three key components (feature-based object detection, Dijkstra's Algorithm, and Google Cloud Firestore) IICP Park Finder can provide an accurate, efficient, and user-friendly parking space detection and navigation system that meets the needs of modern urban environments. The combination of these technologies allows IICP Park Finder to accurately identify available parking spaces, determine the optimal route to reach those spaces, and manage and synchronize data effectively, ensuring a seamless and reliable experience for users.

Image processing methods for detecting vacant parking spaces in IICP Park Finder, is also chosen instead of relying on ultrasonic or infrared sensors as image processing techniques offer several advantages over traditional sensor-based approaches. Firstly, image processing allows for a broader and more comprehensive coverage area, as a single camera can capture and analyze multiple parking spaces simultaneously, unlike individual sensors that typically monitor a single spot. Secondly, image processing algorithms can leverage advanced object detection and computer vision techniques, enabling more accurate and reliable detection of

vacant spaces, even in challenging scenarios with varying lighting conditions, occlusions, or complex parking layouts. Furthermore, image-based systems are generally more cost-effective to install and maintain compared to deploying and maintaining a large number of individual sensors across a parking facility. Overall, the decision to utilize image processing methods aligns with the goals of providing a comprehensive, accurate, and scalable parking space detection solution.

2.6 Summary

Chapter 2 of this report delves into the literature review of existing research and technologies relevant to the development of the IICP Park Finder system. It begins by defining car park availability detection using image processing and outlining the process. Three image processing methods are explored: background subtraction, edge detection and line segmentation, and feature-based object detection, each discussed in detail with their principles, advantages, and limitations compared in a table.

The chapter also examines shortest path algorithms, focusing on Dijkstra's, Bellman-Ford, and A* search algorithms. Their characteristics and complexities are analyzed and compared to determine the optimal choice for IICP Park Finder. Existing systems like the Penang Smart Parking Infrared Sensor and Ultrasonic Sensor in shopping malls are reviewed, highlighting their workings, benefits, and drawbacks. Finally, the proposed work for IICP Park Finder is presented, justifying the chosen technologies of feature-based object detection, Dijkstra's algorithm, and Google Cloud Firestore, emphasizing their advantages for accurate, efficient, and user-friendly parking space detection and navigation.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Introduction

The system development methodology, development tools, and use case diagrams related to this project will be discussed in this chapter.

3.2 System Development Methodology

The waterfall model is the system development methodology that has been chosen for this project. The term "linear-sequential life cycle model" refers to the waterfall model, which only advances to the subsequent stage or phase upon the conclusion of the preceding one (Tutorialspoint, 2024). This methodology is chosen as the objectives and scope of the project are clearly defined and it is easy to implement. This is said because meetings were conducted with the AFM manager and ITS manager who assisted in determining the objectives (requirement of the administrative department for the web application) and the scope (resorting to recording instead of real-time as port forwarding may potentially result in security breaches and time required to request access and approval from the head of management).

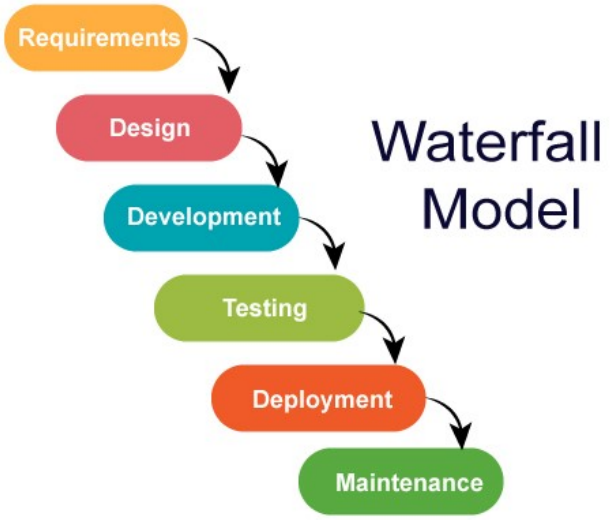


Figure 3 - 1 Waterfall Model (Dhruv, 2021)

Figure 3-1 shows the process of the waterfall model. This model divides the system development cycle into 6 phases, which included the Requirements phase, Design phase, Development phase, Testing phase, Deployment phase, and Maintenance phase. The phases applied to the project will be further discussed in this chapter.

3.2.1 Requirements Phase

In this phase, the requirements of the project are gathered from stakeholders to determine the objectives and scope of the project. This phase started by collecting data via a Google Form targeting the students of INTI Penang regarding the issues faced when finding for parking spaces in INTI Penang which the results are shown in the problem statement. In the Requirements phase of this project, it is found that the students INTI International College Penang have to spend a lot of time finding for parking space when attending classes. This has caused students to attend classes late, parking in illegal areas and even return home after giving up on finding for a vacant parking space. The results of the survey can be seen in the problem statement of Chapter 1. After establishing the problem statement, the objectives of the project were determined to solve the problems stated.

Besides the students, the management team is also a stakeholder of the project as the team oversees maintaining the system and improve the traffic during peak hours. The greatest challenge of this phase was to determine the scope of the project as the footage from INTI parking spaces is required. Hence, meetings with the manager of the Admin and Facilities Management (AFM) department, Mr. Syazwan and the manager of the Information and Technology Services (ITS) were conducted as the scope of the project is in INTI and thus involves the management team. Initially, the video footage used for the project was supposed to be real-time in which the manager of AFM allowed cameras to be installed to record the parking spaces dedicated to the campus shuttles. This will require the use IP camera as the parking lot is outdoors so the cameras used must be waterproof and weatherproof.

Concerns
Power Socket required for IP Camera 2
Bandwidth for IP Camera to connect
Access to router for port forwarding to access camera remotely (ITS)

Figure 3 - 2 Concerns Raised for IP Camera Implementation

To utilise IP camera, there were a few concerns raised for the installation as shown in Figure 3-2. One of the concerns is that the installation will require a power socket which was raised to Mr. Syazwan where he mentioned that extension cords are available and is to not be an issue. The second concern is the IP camera requires connection to internet where the bandwidth in the installation area has to be strong enough for the IP camera to connect. This was addressed by Mr. Syazwan as well where there is a router near the point where the camera is to be installed and the connection is possible. The third and more troubling concern is that to have remote access to the footage, port forwarding is required which will require the

ITS department to open a port on the campus router. This however will cause potential security breaches which meant the higher management will need to be involved and greatly lengthen the time to conclude requirement stage of the project.

```
from ultralytics import YOLO

# Load the model
model = YOLO("yolov8n.pt")

# Create a generator for the RTSP feed
results = model.predict(source='rtsp://admin:admin @alex-gpt.168.15.6:554/onvif1', stream=True)

# Iterate over the generator to process and display each frame
for result in results:
    result.show()  # Display the frame with detections
```

Figure 3 - 3 How to Retrieve Video Footage from IP Camera

```
python predict.py model=yolov8l.pt source="videos/inti_top_view_two_edited.mp4" show=True
```

Figure 3 - 4 How to Run Code on Recorded Footage

As shown in Figure 3-3 and Figure 3-4 model.predict (source = rtsp number) is used to retrieve the video footage from an IP camera while predict.py source= video path is used to retrieve a recorded footage. This means that the calling method to run the video footage is the same whether it is a real-time footage from the IP camera or a recorded footage. Thus, it has been decided that the video footage used for the project will be recorded instead of real-time as there is no difference to the system and quicken the step to the next phase.

3.2.2 Design Phase

During this phase, the required specifications for the development were studied. This includes the literature review of the definition of the project's topic, image processing methods to be used, shortest path algorithms, databases of the system, and existing vacant parking spaces detection systems. Each topic is discussed and compared to determine which of the discussed tools are to be implemented in the proposed work. The literature review were discussed in Chapter 2 of this report.

Besides, UML (Unified Modeling Language) diagrams and data model diagram (equivalent of Entity Relational Diagram for NoSQL database) are designed to show how the system flows and how the database is set up. The UML diagrams designed for this project are use case diagrams and module flowcharts for mobile application and web application which will be shown and further discussed in the Chapter 3 of this report.

In addition, the system requirements such as the software development life cycle, development tools, and system architecture used for the project are defined in this phase as well. The timeline of the project is defined in a Gantt Chart according to the Waterfall model, with one phase completed before proceeding to the next. The development tools for the project include Flutter with Dart programming language for mobile application development, Woosmap API for mapping, Firestore as database, and JavaScript with SCSS styling for web application development. Besides, GitHub is used for version control of the project. This is further discussed in Chapter 3 of the report.

3.2.3 Development Phase

In this phase, the requirements of the proposed project that have been studied and analyzed will be implemented into the development of the system. This is the phase where the programmers make use of the information provided in the previous stages to create a functional system. The code is usually implemented in small components which will then be integrated at the end of the phase. The application UI is based on the prototype while the codes are based on the class diagram, the flow of the application is based on the use case diagram, and the database is built based on the data model diagram for NoSQL. For IICP Park Finder, the object detection system, mobile application for students, and web application for the management team will be developed.

3.2.4 Testing Phase

The fourth stage of the Waterfall methodology is the testing phase. In this phase, the system is tested to ensure that it functions properly, does not contain any errors, and meets all the project requirements before it can be released to clients and users. Evaluation of the system is necessary if any major issues arise. This phase is crucial for quality assurance, as it ensures the software meets both functional (what it's supposed to do) and non-functional (how well it performs) requirements.

Non-functional and functional testing are conducted in this phase to guarantee a high-quality product that functions correctly and efficiently. Functional testing, implemented through unit testing (verifying individual components) and integration testing (verifying the integrated system), ensures that the software behaves as expected according to the specified requirements. Non-functional testing, on the other hand, assesses how well the software performs under various conditions, evaluating aspects like usability (how easy it is to use), performance (speed and responsiveness), security, compatibility, and scalability. User acceptance testing will be carried out for the non-functional testing to determine how easy and useful the application is to use.

By identifying errors, bugs, and defects early on, testing allows for timely correction, preventing issues from escalating and becoming more costly to fix later. It also verifies that all project requirements are met, aligning the software with client or user expectations. Moreover, testing helps uncover potential risks or vulnerabilities in the software, such as security flaws, compatibility issues, or scalability problems, allowing for early mitigation.

3.2.5 Deployment Phase

Once the testing phase is completed and the system no longer has errors or bugs, the mobile application and web application for IICP Park Finder can be deployed for the students and management team to utilize. The system's source code will be made available for download on Github, but users will need to run it independently in Android Studio before it can be installed and used on a mobile device. This is due to the fact that certain elements of a mobile app that are released require funds, and the app's security features, including data encryption, are not professionally implemented. As a result, there will be no online app store release for the mobile app. Once the system has sufficient capabilities, it goes online and becomes accessible to end users. Furthermore, the web application can only be accessed locally because installing it on a server would cost money due to cloud service fees. After the deployment, the end users' input will be collected and compiled for the following stage. This will enable the completion of the next stage.

3.2.6 Maintenance

Maintenance is the final stage of the Waterfall model where the system is already delivered to the client and is being used. The developers of the software must create patches and updates to address the issues that arise while using the software. During this stage, users of the mobile app may experience system issues, bugs, or defects. Therefore, the best course of action for resolving these issues is to continuously support and maintain the system. The identified flaws in this mobile application should be inspected right away, and whenever possible, the appropriate countermeasures should be implemented right away. New updates for the mobile app will be released to fix the flaws and malfunctions once they have been resolved. In summary, the goal of the most recent versions released was to enhance the system.

3.3 System Design

This section documents the use case diagram, module flowcharts, and data model diagram designed for the system.

3.3.1 Use Case Diagrams

There are two use case diagrams for this project which is the use case for the mobile application and web application respectively as shown below.

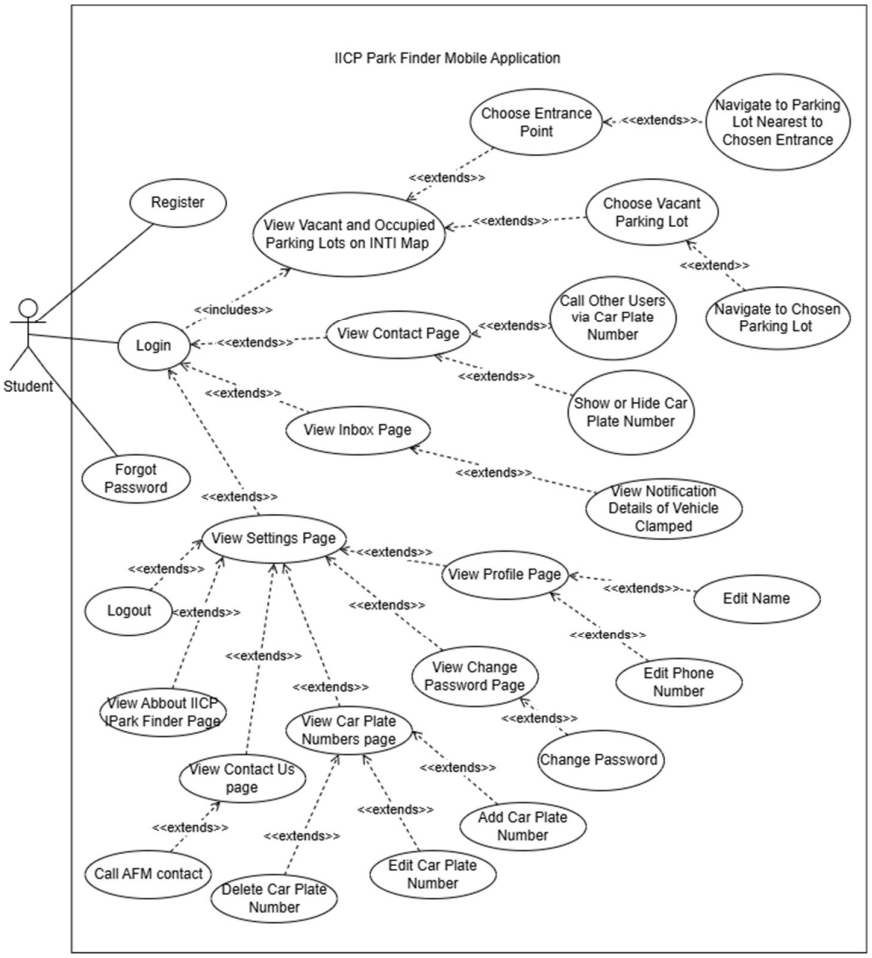


Figure 3 - 5 Use Case Diagram for IICP Park Finder's Mobile Application

The use case diagram for the IICP Park Finder Mobile Application illustrates how an INTI student interacts with the application. When the student first enters the application, the student can start by registering or logging in. Upon logging in, the student will view the map of parking lots to see vacant and occupied parking spaces on campus, choose an entrance point, and navigate to the nearest parking lot or choose a vacant parking space and navigate there. The student can also navigate to the contact page where the student can contact the owner of the

vehicle blocking their vehicle via the car plate number. The student can also decide to show or hide their car plate number depending on if the student has double parked other vehicles.

Additionally, the student can view the inbox page where the student can see the notification when their vehicle has been clamped. Finally, the user can view the settings page where several functionalities are available. The student can edit their profile details, change their password the account password, add, edit, or delete the car plate number details, view information of Admin and Facilities Management details (Contact Us page) in case of vehicle clamp, and view the terms & conditions (About IICP Park Finder page) of using the mobile application. The logout functionality is also accessible in the settings page allowing the student to securely exit the application.

In the case where the student forgot the account password, the student can attempt to reset the password with the forgot password use case.

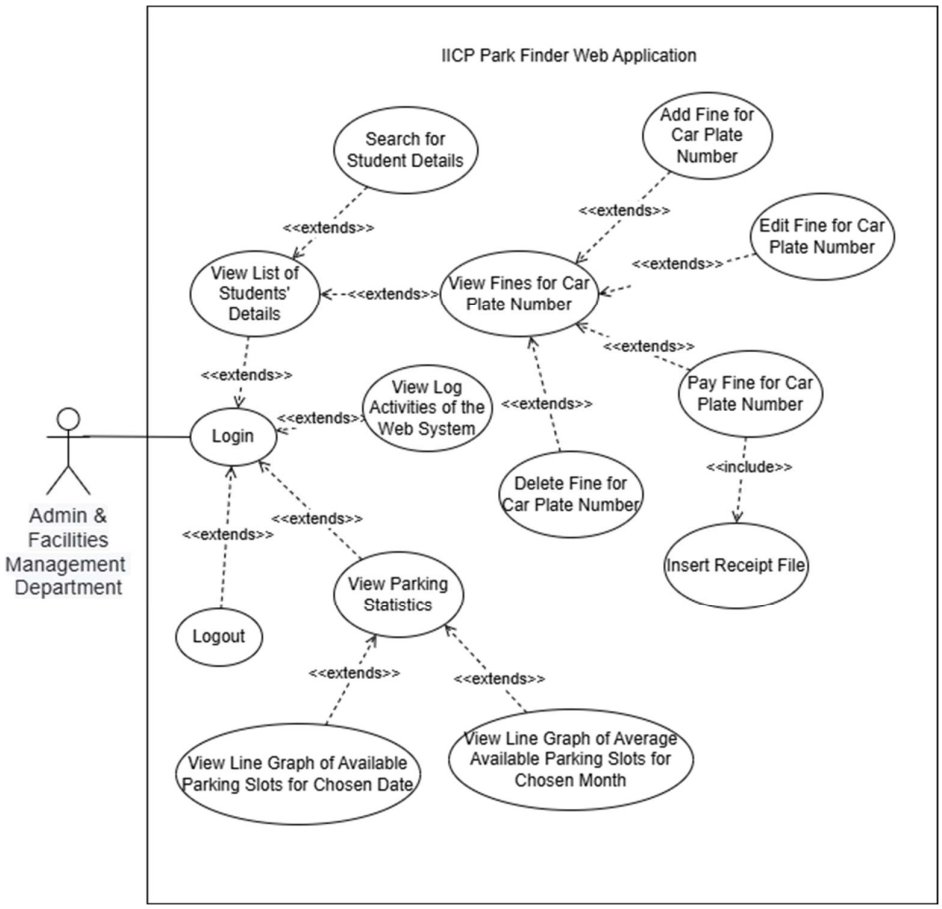


Figure 3 - 6 Use Case for IICP Park Finder's Web Application

The use case diagram for the IICP Park Finder Web Application depicts the interactions of the Admin & Facilities Management Department with various functionalities of the web application system. The user can log in to access core features such as viewing the list of students who registered in the application, parking statistics and account logging. When viewing the list of student details, the user can search based on the student ID or car plate number. From there, the user can view the fines of the car plate number, add fine, edit fine, and delete the fine. If the owner of the vehicle that has been fined has paid the fine, the admin user can insert the receipt and indicate that the fine has been paid. The log activities function is automatic when the admin user edit, delete or pay the fine to ensure that if any suspicious activity has been carried out, the admin can check the logging to see if anyone has tampered with the system. Additionally, the admin user can view parking statistics which includes viewing line graph of available parking spaces throughout the day of the selected day or the average available parking space throughout the month. This function is to allow the admin user to determine when the peak hour of users parking in campus is and plan to improve the traffic accordingly. These interactions allow the department to efficiently manage parking information, monitor the status of parking spaces, and ensure effective oversight of parking operations.

3.3.2 Module Flowchart Diagrams

This section shows the module flowcharts of the mobile application and web application.

3.3.2.1 Mobile Application Module Flowcharts

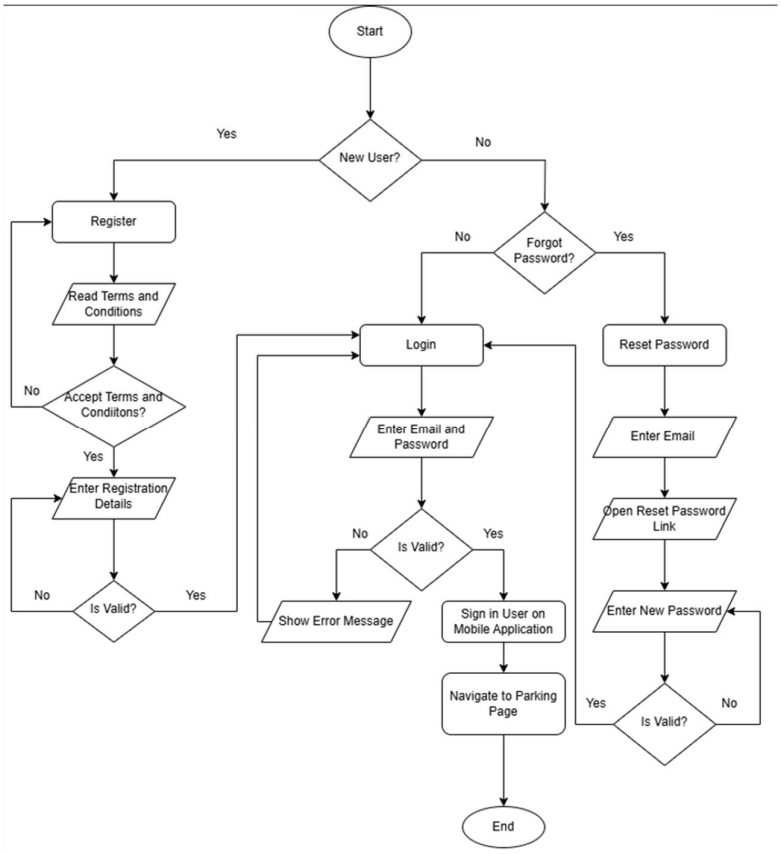


Figure 3 - 7 Register, Login & Forgot Password Module Flowchart

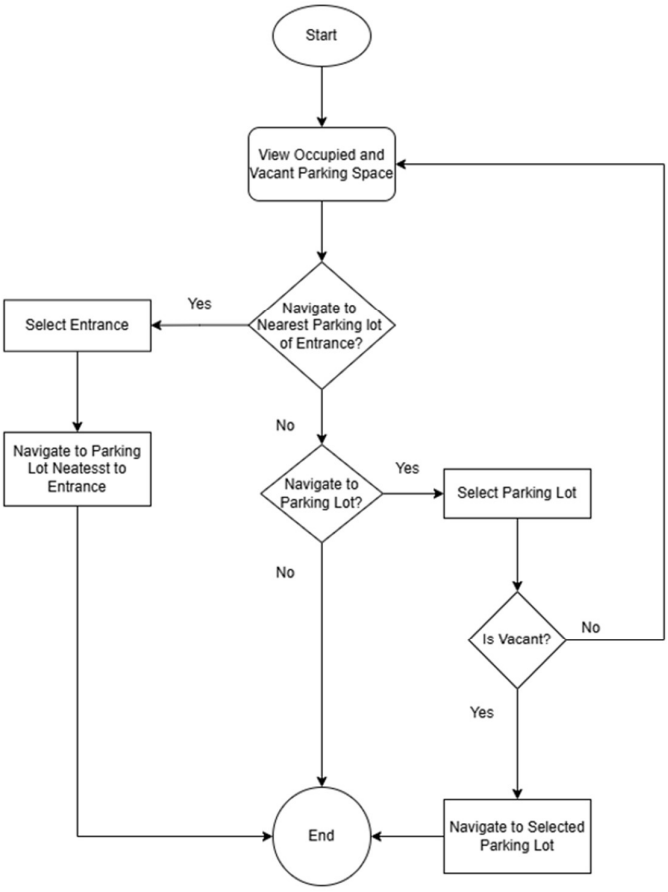


Figure 3 - 8 Parking Page Module Flowchart

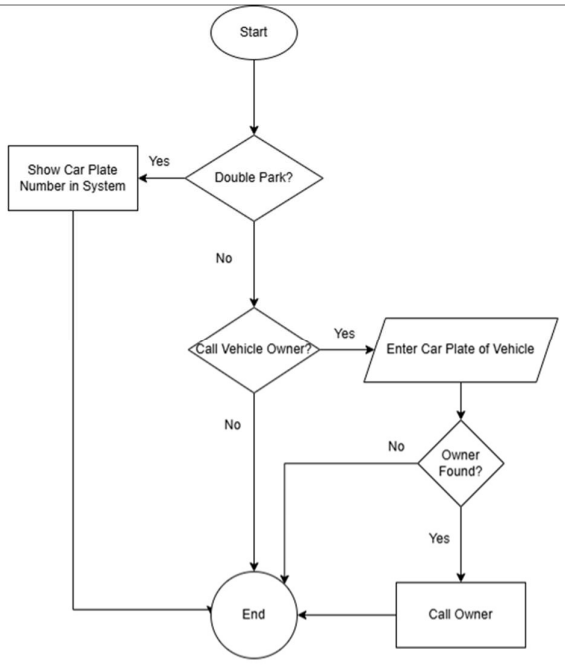


Figure 3 - 9 Contact Page Module Flowchart

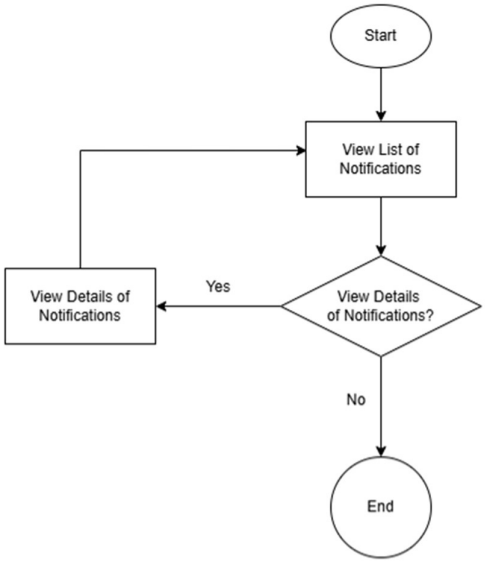


Figure 3 - 10 Inbox Page Module Flowchart

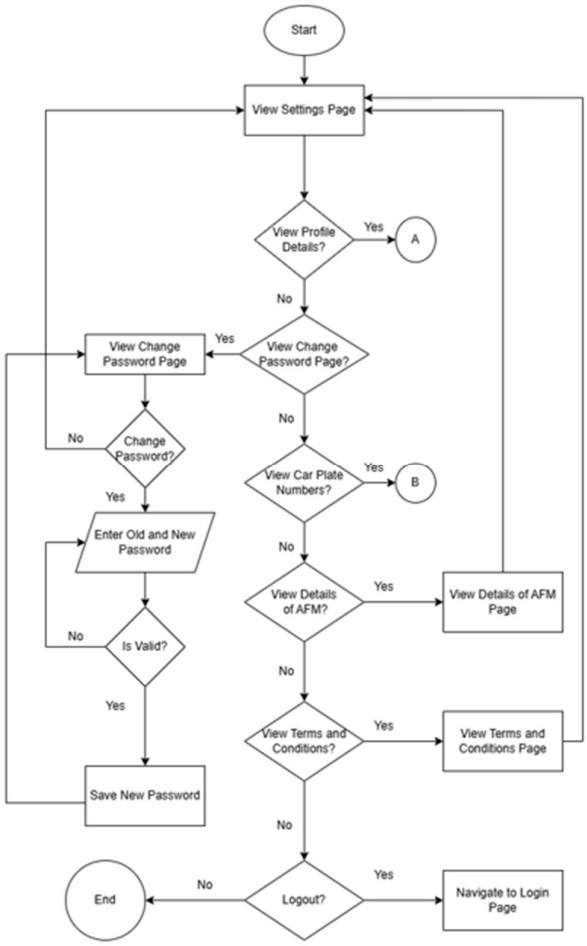


Figure 3 - 11 Settings Page Module Flowchart

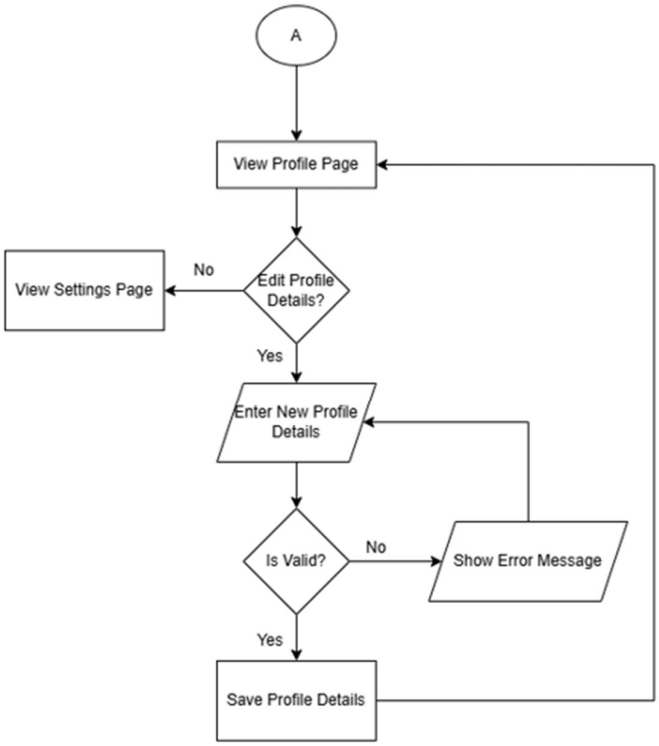


Figure 3 - 12 Profile Page Module Flowchart

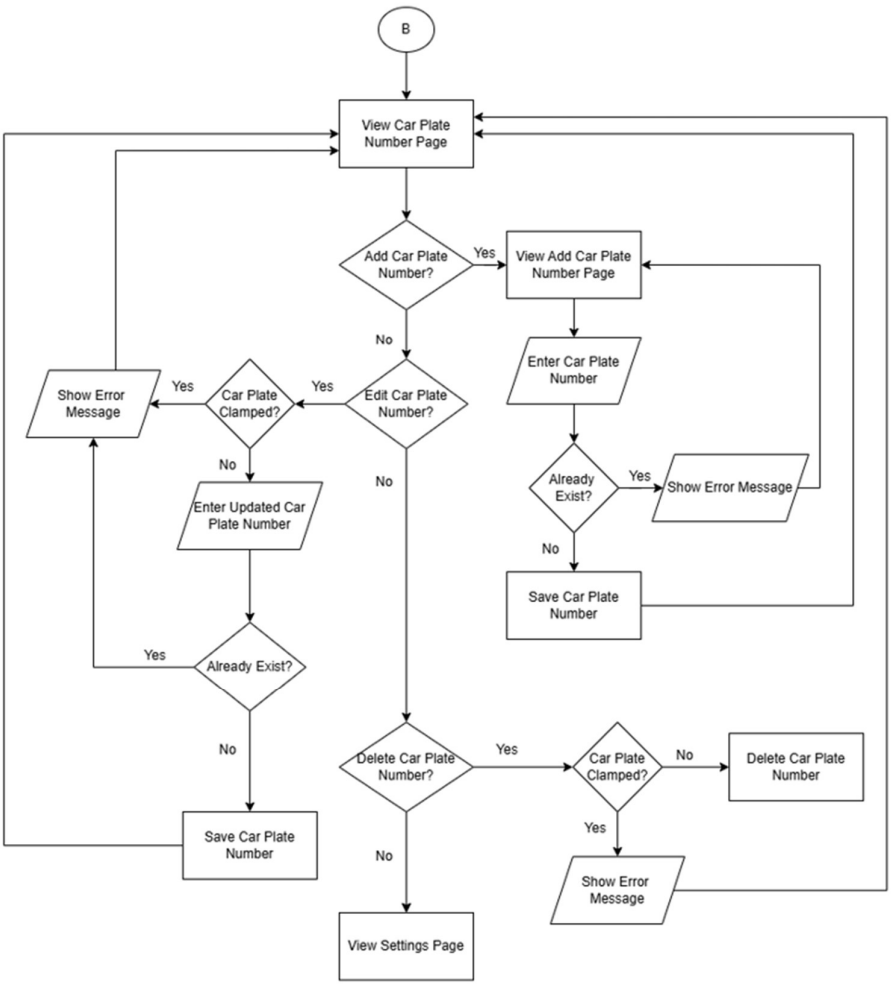


Figure 3 - 13 Car Plate Number Module Flowchart

Figure 3-7 until Figure 3-13 shows the module flowcharts of the functionalities in IICP Park Finder mobile application which includes the register, login, forgot password, parking page, contact page, inbox page and settings page functions. The settings page module flowchart is displayed in three diagrams as it is too complex to fit into one. The end and starting points A and B represents the profile page and car plate numbers respectively indicating the user can go to the profile page and car plate numbers page from the settings page.

3.3.2.2 Web Application Module Flowchart

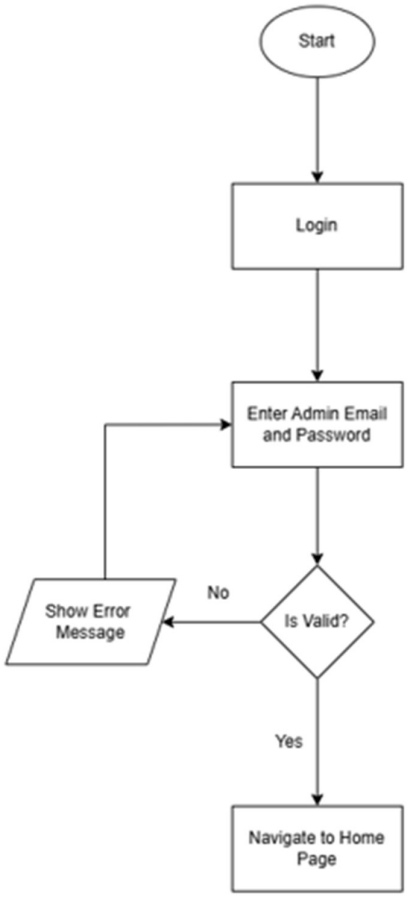


Figure 3 - 14 Login Module Flowchart

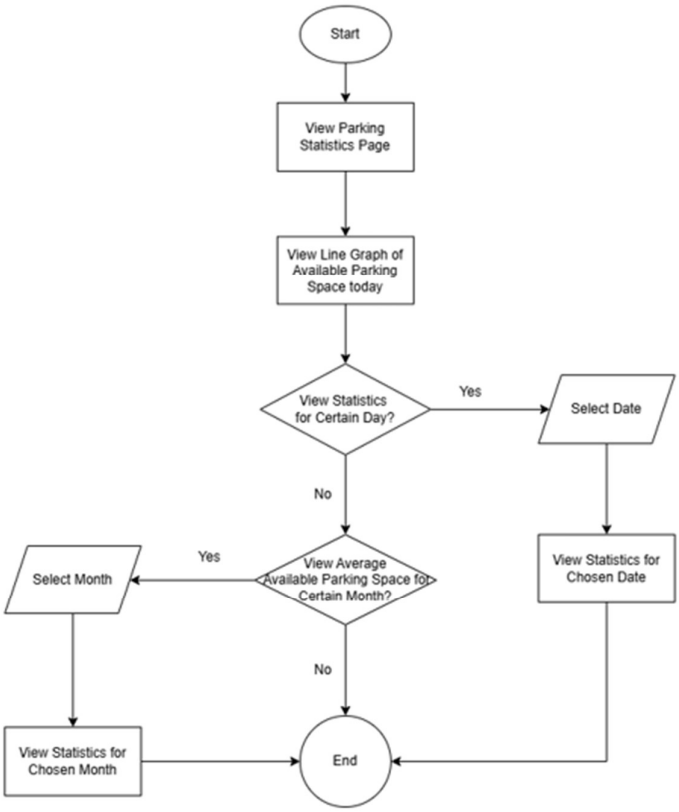


Figure 3 - 15 Parking Statistics Module Flowchart

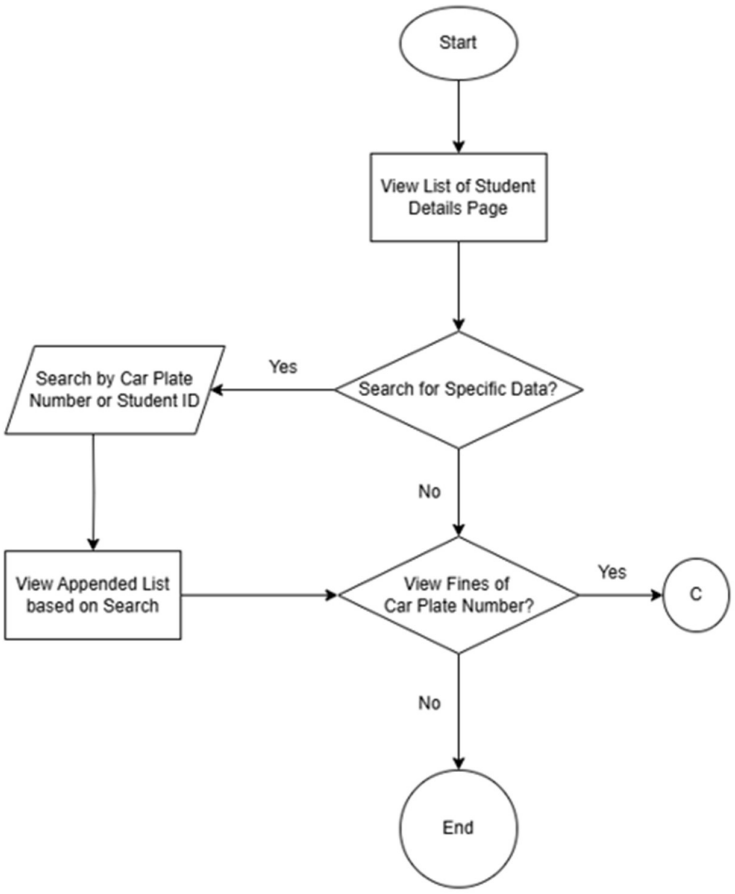


Figure 3 - 16 Student Details Lists Module Flowchart

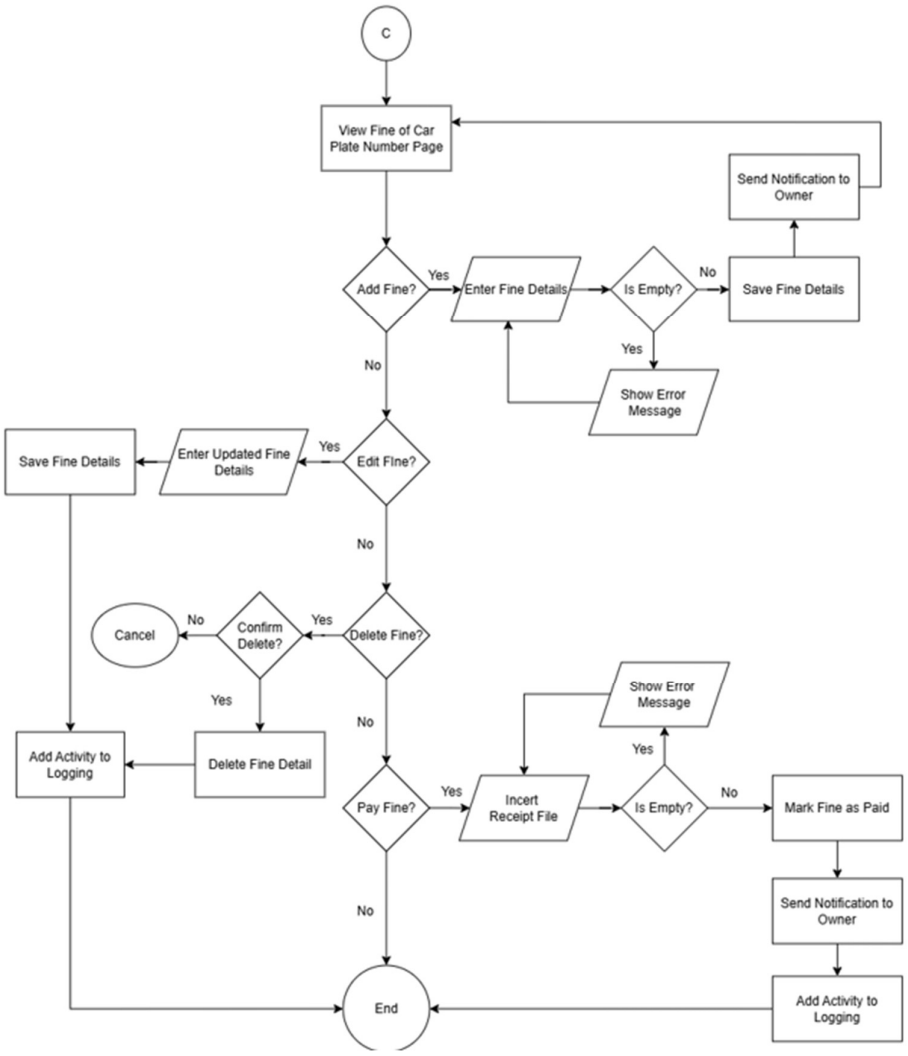


Figure 3 - 17 Car Plate Number Fine Module Flowchart

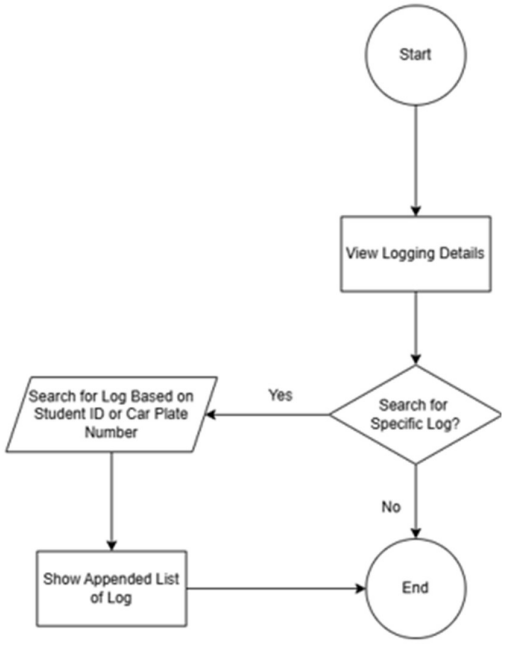


Figure 3 - 18 Logging Details Module Flowchart

Figure 3–14 until Figure 3-18 shows the module flowcharts of the functionalities in the web application. The module flowcharts include the login, parking statistics, student list details, car plate number fines, and logging details. The car plate number fines can only be accessed from the student list details module thus why it is connected with the C end and start point.

3.3.3 Data Model Diagram

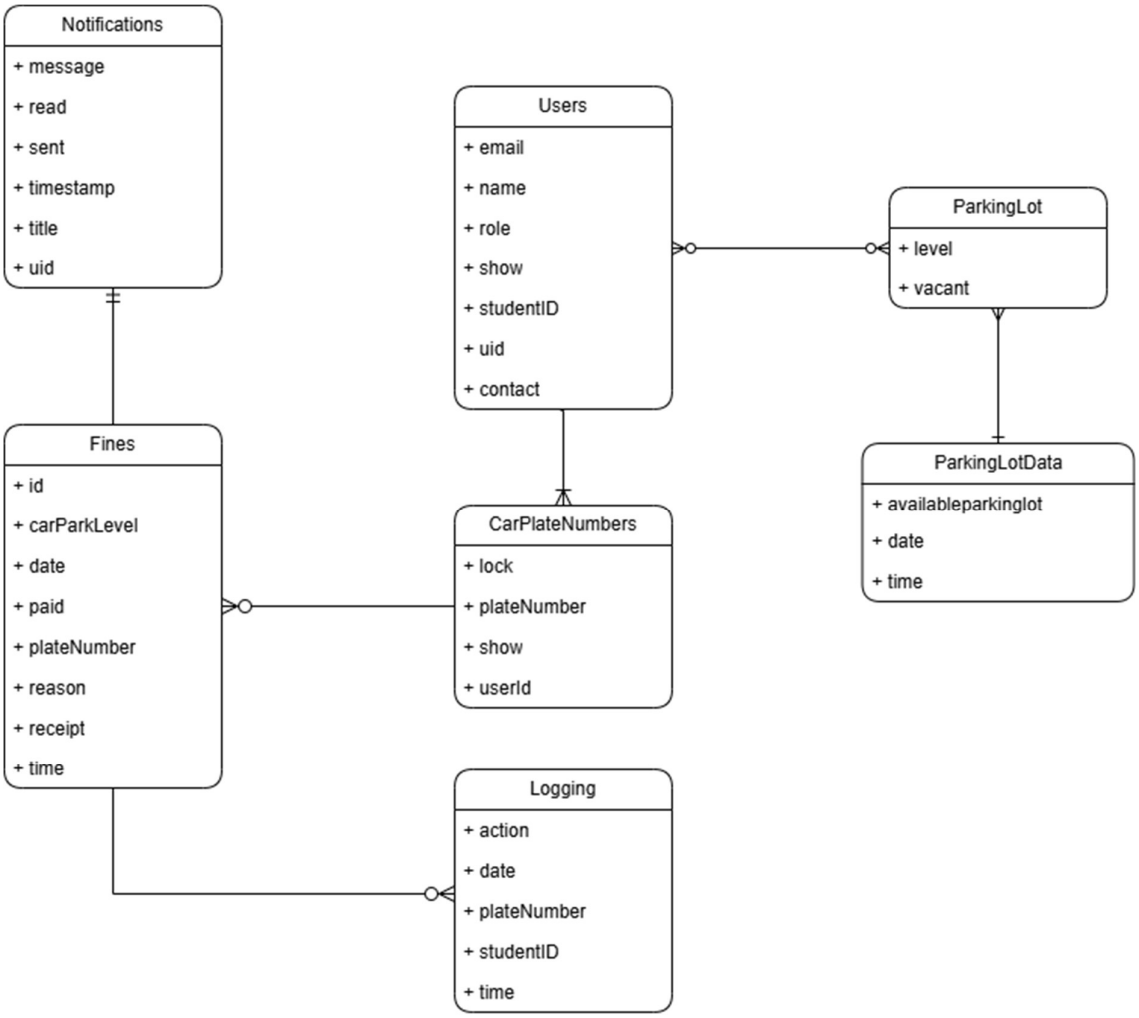


Figure 3 - 19 Data Model Diagram

Figure 3-19 shows the Data Model Diagram (database model) for IICP Park Finder which is the equivalent of ERD for NoSQL and non-relational database management system. The database is stored in Firebase Database and uses a NoSQL query with seven collections. Unlike ERD, the collections do not have primary and secondary keys. The relationship between the collections is shown in the table below.

Table 3. 1 Relationship Between the Collections in Firebase Database

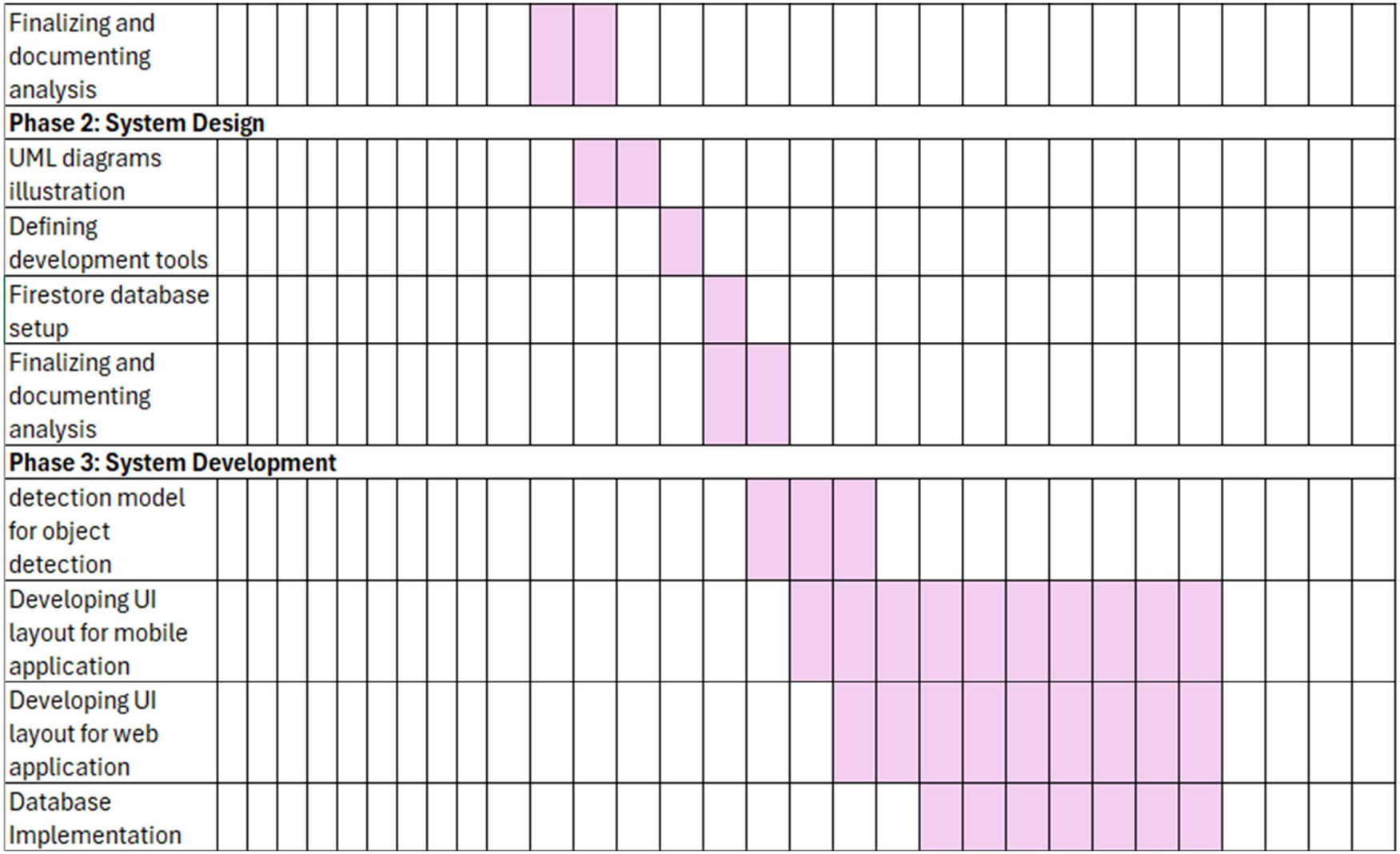
From	To	Relationship	Description
Users	CarPlateNumbers	One to One or Many	One user must have at least one car plate number

Users	ParkingLot	Zero or Many to Zero or Many	Zero or many users can view zero or many occupied and vacant parking lots
CarPlateNumbers	Fines	One to Zero or Many	One car plate number can have zero or many fines
Fines	Notifications	One to One or Two	One fine can deploy one or two notifications only (Add and Pay fine)
Fines	Logging	One to Zero or Many	One fine can have zero or many logging (Logging is not saved when fine is added, only added when edit,delete, or pay fine)
ParkingLot	ParkingLotData	Many to One	Many parking lot contributes to one parking lot data

3.4 Gantt Chart

Table 3. 2 Gantt Chart

Activity	WEEK																												
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
Phase 1: Requirement Analysis																														
Determine project title and objectives																														
Approval of project allocation form																														
Research on image processing process for vacant parking space																														
Research on image processing methods																														
Research on shortest path algorithms																														
Research on databases																														
Research on existing vacant parking space detection systems																														


Finalizing and documenting analysis
Phase 2: System Design
UML diagrams illustration
Defining development tools
Firestore database setup
Finalizing and documenting analysis
Phase 3: System Development
detection model for object detection
Developing UI layout for mobile application
Developing UI layout for web application
Database Implementation

Validation implementation																													
Finalizing System																													
Documentation of Development																													
Phase 4: Testing																													
Documentation of Testing																													
Unit Testing																													
System Testing																													
Usability Testing																													

Table 3.2 shows the Gantt Chart designed to manage the project timeline according to the Waterfall model to ensure the project remains on track at any time.

3.5 Development Tools

This section discusses the tools to be used in the development of the project.

3.5.1 Flutter



Figure 3 - 20 Flutter Logo (Flutter, 2024)

Flutter, Google's open-source UI software development kit, was the clear choice for this project due to its numerous advantages. Its ability to create high-performance cross-platform mobile applications from a single codebase significantly streamlines development, reducing both time and resources required (Flutter, 2024). Flutter's utilization of the Dart programming language and its reactive framework further enhances efficiency, allowing for seamless code implementation across various platforms. This flexibility is crucial to the diverse technological landscape of IICP Park Finder.

Another compelling reason for choosing Flutter is its widget-based architecture, which offers a comprehensive collection of pre-built UI elements. This rich toolkit simplifies the creation of visually appealing and complex layouts, essential for delivering a superior user experience. In addition, Flutter's "Hot Reload" feature dramatically speeds up development by enabling instant visual feedback on code changes. This invaluable tool fosters rapid iteration and debugging, making Flutter an ideal choice for projects with tight deadlines.

Flutter's growing popularity within the developer community also played a role in the decision. Its active community ensures robust support, frequent updates, and a wealth of resources, guaranteeing a smooth and efficient development process. Overall, Flutter's combination of efficiency, flexibility, rich UI capabilities, rapid development features, and strong community support solidified its position as the optimal choice for this project.

3.5.2 Firebase Database (Firestore)



Figure 3 - 21 Cloud Storage for Firebase Logo (Google for Developers, 2024)

Google offers Firebase, a full-featured development platform with a range of cloud-based services and APIs for developing online and mobile applications (Google for Developers, 2024). Firestore is the online database used in this project to store data collected from the application. One of the main components of this project is the Firebase real-time NoSQL database. Developers can use it to store and sync data in real time between several clients.

Another significant component of utilizing Firestore is it provides an authentication API that makes user authentication and identity management implementation easier. It supports several authentication techniques, such as anonymous authentication, social logins (Facebook, Google, etc.), and email and password. Firestore is also chosen as it enables developers to store material created by users such as image files. Using a straightforward API, additional functionalities like uploading, downloading, and maintaining data saved in the cloud can be accomplished. Not only that, Firebase has specific initialization for each platform such as android, Ios, web, and flutter which eases the integration for database setup for IICP Park Finder which uses many platforms. To sum up, Firebase Storage is secure, more scalable, and provides access control integration with Firebase Authentication.

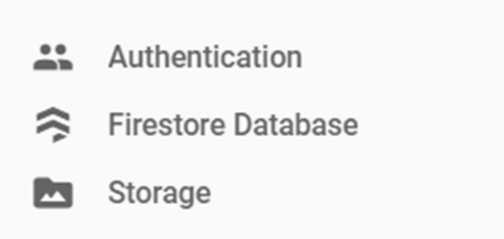


Figure 3 - 22 Firebase Functionalities

Firebase will be utilized for its authentication, firebase database and storage. The authentication function is to hash and store the passwords of users to secure the data from

any breaches. The firebase database is used to store all the data utilized by IICP Park Finder system from the user details to the fines and parking lot data. Finally, the storage is used to store all the receipts for the fines paid by the students for easy access and data storing.

3.5.3 Vue.js



Figure 3 - 23 Vue.js Logo (Vue.js, 2024)

Vue.js is a progressive JavaScript framework that is particularly well-suited for the development of user interfaces in single-page applications. Vue is built to be adaptable, beginning with a straightforward foundation and enabling users to incorporate additional complexity as required. This architecture is founded on components, which promotes modularity and facilitates the effective organisation of applications, code management, and component reuse. Vue's reactivity system adjusts the user interface (UI) in response to data changes which simplifies the process of managing application state and developing dynamic, responsive interfaces without the need for intricate configurations (Vue.js, 2024). Vue's reactivity system is straightforward, as it necessitates minimal configuration.

Additionally, Vue's provides utilities such as Vue Router for complex navigation management and Vuex for centralised state management. These tools maintain a moderate learning curve while ensuring scalability for larger applications. It is accessible and supports long-term development and maintenance due to Vue's detailed documentation and active community. The framework is an exceptional option for the development of contemporary, interactive web applications due to its ease of integration, strong community support, and versatility. Therefore, Vue.js will be used as the framework for the web application of IICP Park Finder.

3.5.4 Google Maps API



Figure 3 - 24 Google Maps API Logo (GoogleforDevelopers, 2024)

The Google Maps API is a great tool for developers who wish to incorporate dependable, high-quality mapping functionality into their applications. It offers millions of users worldwide access to Google's comprehensive mapping data, which is accurate, frequently updated, and trusted. Furthermore, its compatibility with a variety of platforms guarantees that maps are effectively displayed on mobile applications. Google Maps API will be implemented in IICP Park Finder to display the map of INTI International College Penang where custom parking lot polygons and pathway are integrated into the system for customizability.

When it comes to displaying maps with custom polygons and paths, the Google Maps API is particularly beneficial. The API enables developers to overlay polygons to define specific areas, designate boundaries, and add custom shapes that are interactive and responsive. Paths can also be incorporated to depict routes, trails, or connections between points, rendering it an ideal choice for applications that necessitate personalised geographical representations. These features enable a high degree of customisation, providing users with a personalised map experience that is tailored to the application's specific requirements. Additionally, the Google Maps API allows for the adjustment of colours, transparency, and interactivity for both polygons and paths. This simplifies the process of emphasising specific areas or routes, adding labels, and providing context that improves user engagement and navigation.

3.5.5 Google Earth Pro



Figure 3 - 25 Google Earth Pro Logo (Neowin, 2024)

Google Earth Pro is a great tool for producing exact geographic overlays, particularly where polygon and path coordinate precision is critical. Its robust functionality and high-resolution satellite imagery enable the mapping of areas with precise detail, guaranteeing that the overlays are in perfect alignment with real-world locations. This level of detail is essential for IICP Park Finder which necessitates spatial accuracy, such as the mapping of parking lots, where each polygon and path must align precisely with tangible spaces.

By overlaying a parking lot image with Google Earth Pro, it is possible to create polygons and paths that precisely correspond to the layout. This enables the establishment of boundaries, the assignment of specific areas, or the creation of navigation paths that correspond to the space's actual architecture. The software's tools facilitate the precise marking of these shapes, thereby supplying the developer with a set of coordinates its capacity to export these coordinates and integrate them into other platforms. This precision allows the mapping of the parking lots and custom path of INTI International College Penang to be accurately drawn in the mobile application.

3.5.6 QGIS



Figure 3 - 26 QGIS Logo (QGIS, 2024)

QGIS is a vital tool for the conversion of spatial data into formats that are compatible with a variety of applications, with a particular emphasis on the conversion of files from Google Earth Pro to a mobile-friendly format. QGIS, an open-source geographic information system, offers a variety of robust data transformation tools, such as the capacity to convert KML files to

GeoJSON. GeoJSON is a widely used format in mobile applications due to its lightweight structure. Therefore, this procedure is crucial when transferring data generated in Google Earth Pro into IICP Park Finder's mobile application.

The accuracy and detail of the geographic data, such as polygons and paths, are preserved when they are imported into a mobile application by employing QGIS for this conversion. This method enables the application to render map overlays with precise coordinates as originally drawn in Google Earth Pro, thereby facilitating seamless integration. The workflow is simplified and data loss during conversion is minimised when KML files are loaded, reformatted, and exported in GeoJSON, all of which are made simple by the user-friendly interface of QGIS.

3.5.7 Yolov8

Figure 3 - 27 Yolov8 Logo (Ultralytics Inc., 2024)

YOLOv8 is renowned for its high accuracy and real-time performance, which allow it to swiftly and efficiently detect vehicles without the necessity of additional training. This model can be directly applied to identify vehicles in each frame, enabling the system to accurately identify occupied and vacant spaces, by utilising pre-trained YOLOv8 weights. The parking information can be kept current and real-time insights into parking availability can be provided to users by relaying this data to a database.

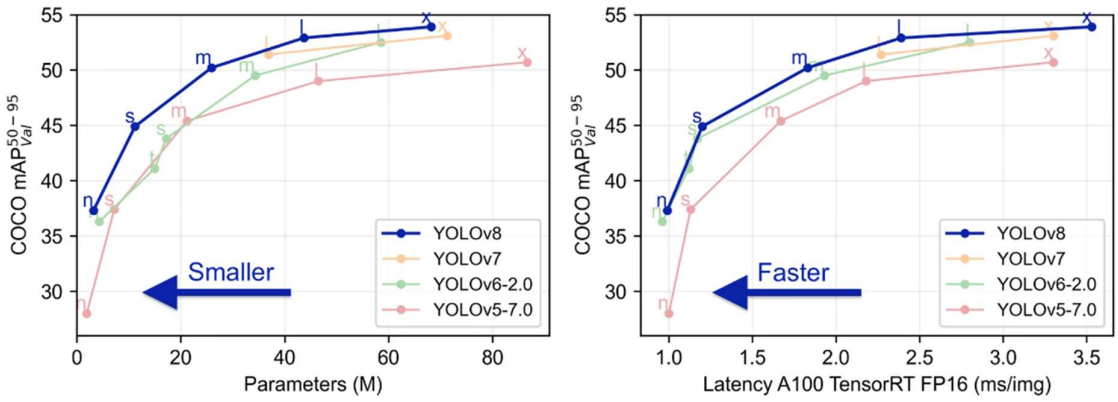


Figure 3 - 28 Comparison of YOLO Models (Ultralytics, 2024)

The two graphs in Figure 3-28 compare the performance of various YOLO (You Only Look Once) model versions — specifically YOLOv5, YOLOv6, YOLOv7, and YOLOv8. These comparisons are based on the models' parameter counts, accuracy, and inference latency. Each point in the graphs represents different variants within each YOLO version (n, s, m, l, x), which range from smaller, faster models to larger, more accurate ones.

The left graph plots parameters (in millions) on the X-axis against accuracy (mAP, or mean Average Precision) on the Y-axis. This graph shows how each YOLO model version performs in terms of accuracy for a given model size. YOLOv8's line sits above the other versions, indicating that it achieves higher accuracy at each parameter count, particularly in the larger models suggesting that YOLOv8 is more efficient than previous versions and outperforms the older YOLO models in terms of accuracy.

The right graph, on the other hand, plots inference latency (milliseconds per image) on the X-axis against accuracy (mAP) on the Y-axis. Latency measures how quickly each model processes an image, with lower values indicating faster inference. Here, YOLOv8 also demonstrates its efficiency by achieving higher mAP values at each level of latency compared to YOLOv5, YOLOv6, and YOLOv7. This means that, for a given inference speed, YOLOv8 provides better detection accuracy.

In summary, YOLOv8 offers the best balance between speed and accuracy among the versions compared. Therefore, YOLOv8 would be a preferred choice for applications that require either high-speed processing without compromising accuracy or high accuracy without excessive latency such as the IICP Park Finder application.

Additionally, the integration of YOLOv8 with the Deep SORT (Simple Online and Realtime monitoring) algorithm offers a reliable monitoring solution that improves the accuracy and reliability of detections. Deep SORT is an optimal solution for preserving vehicle identity as vehicles enter or exit parking spaces, as it employs a combination of motion and appearance data to monitor objects across frames. The YOLOv8 model's detections are not only accurate but also consistently tracked as a result of the integration of Deep SORT. This reduces misidentification and facilitates a more seamless data update to the database.

3.5.8 DeepSort (PyTorch)

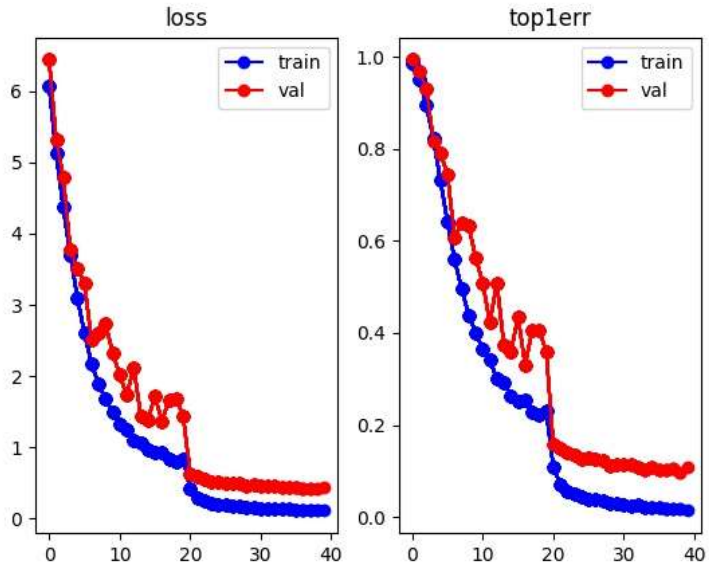


Figure 3 - 29 Loss and Top-1Error Rate of Testing And Validation of Deep SORT Model

PyTorch's Deep SORT (Simple Online and Realtime Tacking with deep association metric) is a pre-trained tracking algorithm that employs a combination of appearance descriptors and motion prediction to identify objects across multiple frames. Figure 3-29 shows the training and validation performance of the DeepSORT model (or a similar deep learning model) over multiple epochs. The left plot tracks loss, with both training (blue) and validation (red) loss values decreasing over time, indicating that the model is learning effectively and improving its predictions. The right plot shows the top-1 error rate, measuring how often the model's highest-probability output is incorrect. Both the training and validation error decrease similarly, suggesting that the model generalizes well without overfitting. As both loss and error stabilize towards the end, this indicates that the model has converged, reaching its optimal performance level.

This PyTorch implementation is particularly beneficial for ensuring the continuity of detected objects, even in complex or congested environments such as parking lots. Deep SORT improves the fundamental detection capabilities by assigning a unique ID to each vehicle, which enables precise tracking as it travels. The detection results can be refined by integrating Deep SORT with YOLOv8, which ensures a higher level of accuracy in updating the parking database with real-time occupancy status and reduces errors in tracking due to overlapping or similar-looking objects.

3.5.9 GitHub (Version Control)



Figure 3 - 30 GitHub Logo (Munira, 2018)

GitHub is a web-based platform that serves as a central hub for software development projects. Leveraging Git, a powerful version control system, GitHub allows teams of developers to collaboratively manage and track changes to their codebase over time. This ensures that everyone is working on the latest version of the code, while also preserving a complete history of revisions for reference or rollback (GitHub, 2024).

Beyond version control, GitHub offers a rich set of features for project management, code review, and community building. Developers can create issues to track bugs and feature requests, submit pull requests for peer review of code changes, and engage in discussions through comments and forums. This is why GitHub has been chosen to maintain the version control for the development of IICP Park Finder. The repository will be split into web application and mobile application for easy readability.

3.6 Summary

In conclusion, the IICP Park Finder project is best developed using the waterfall model of system development life cycle. This is because there are multiple stages in the construction of this system which should not duplicate, and the requirements of the projects are clearly defined. Additionally, this chapter includes a discussion of the system designs for clear visualization of the flow of the application, Gantt Chart for the project timeline, and development tools used for the system.

CHAPTER 4: PROJECT IMPLEMENTATION

4.1 Introduction

The project implementation or also known as development phase, is conducted as the third phase of the waterfall model. This chapter will discuss the features and include screenshots of the mobile application, web application and object detection model of IICP Park Finder. Code snippets of the system will only be included for the features related to the objectives discussed in Chapter 1.

4.2 Mobile Application

This section documents the development of the mobile application IICP Park Finder.

4.2.1 Login

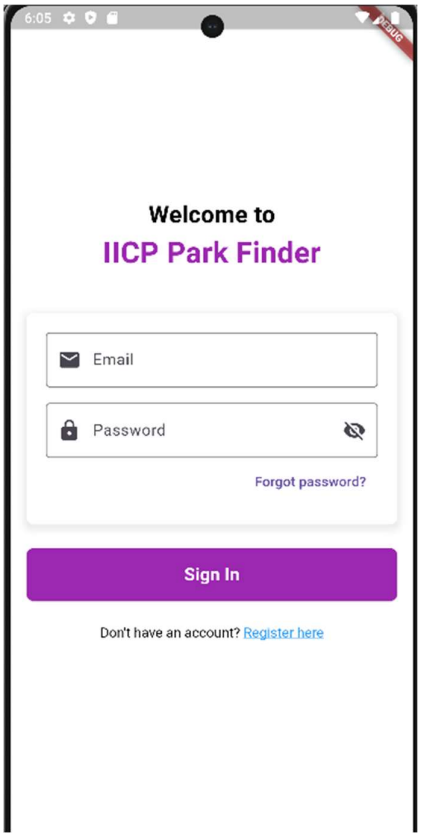


Figure 4- 1 Login Page

On this login page, users see a message introducing "IICP Park Finder" and the app's login form. The login form includes fields for email and password. For the password field, users can toggle visibility, allowing them to show or hide the characters as they type. If either field is left blank, an error message prompts them to fill in both fields. If the email or password is incorrect,

an error message appears, letting users know they can try again. Successful login takes users directly to the main page, where they can begin using the app.

A "Forgot password?" link is also provided for users who need password recovery. A "Register here" link at the bottom offers new users the option to set up an account by navigating to the registration page.

4.2.2 Register

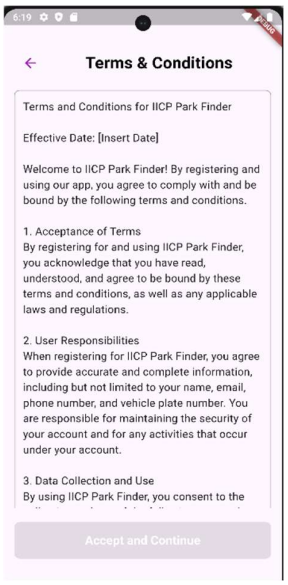


Figure 4- 2 Terms And Conditions Page Upon Registering

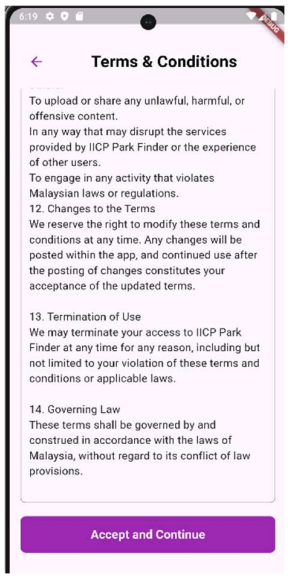


Figure 4- 3 Terms and Conditions Page Upon Registering (Scrolled to the Bottom)

The registration feature comprises of a few pages. Figure 4-2 and Figure 4-3 above shows the Terms and Conditions (T&C) page that users will navigate to when tapping on the ‘Register here’ in the login page. The user can only tap the Accept and Continue button after scrolling to the bottom of the texts indication that the user has read and accepts the T&C of using the application. The T&C in the application is only a draft and has been approved by Mr. Syazwan (Head of AFM department).

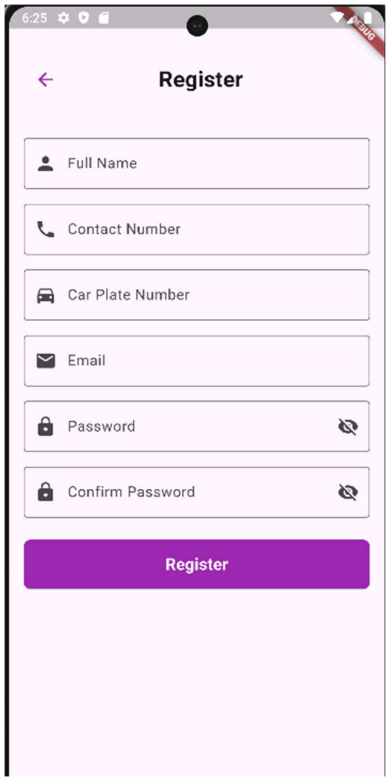


Figure 4- 4 Register Page

After accepting the T&C of the application, the user will navigate to the Register Page as shown in Figure 4-4 where the user has to fill in the name, contact number, car plate number, student email, password, and confirm password. The registration page incorporates multiple layers of validation to ensure the accuracy and completeness of user information before an account can be created. Upon initiating registration, the app first verifies that all required fields are filled out, if the password and confirm password fields match, the email provided ends with "@student.newinti.edu.my," ensuring that only users with a valid student email address can register and queries Firestore to check if the car plate number already exists in the database. If any of the validation is not passed, respective dialog error messages based on the validation checked will pop up prompting the user to fix the error.

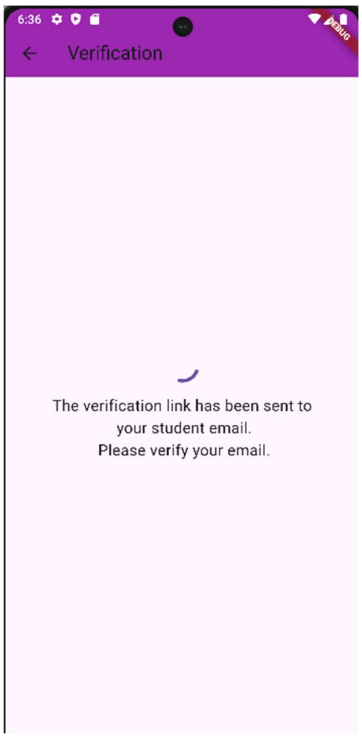


Figure 4- 5 Verification Loading Page

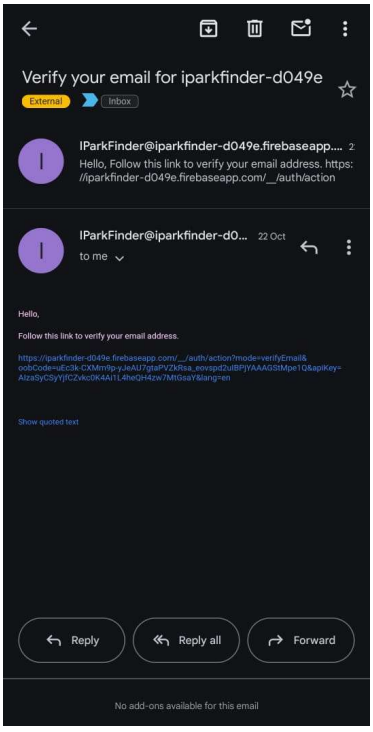


Figure 4- 6 Verification Email

After registering, the user will navigate to the verification loading page as shown in Figure 4-5 and receive a verification email as shown in Figure 4-6. After the user verifies the email, the application will show a dialog box saying that the email is verified and return to the login page.

4.2.3 Forgot Password

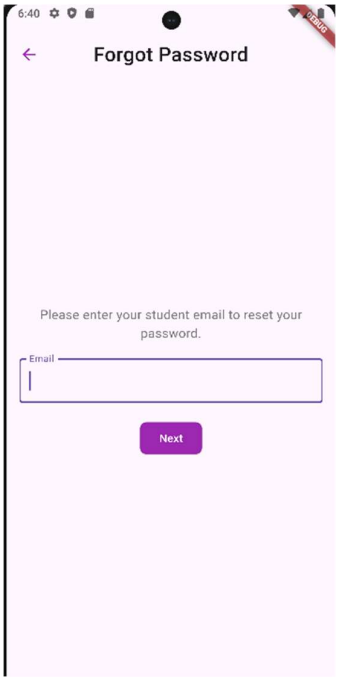


Figure 4- 7 Forgot Password (Page 1)

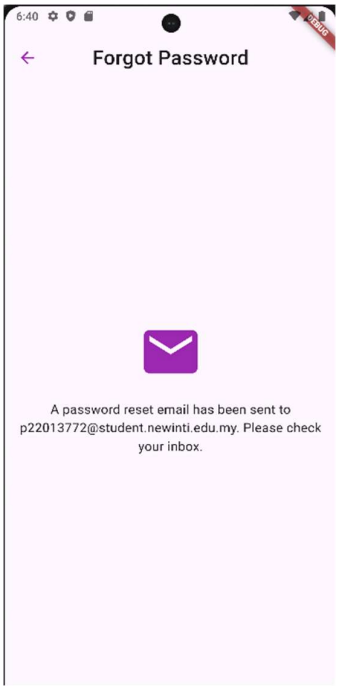


Figure 4- 8 Forgot Password (Page 2)

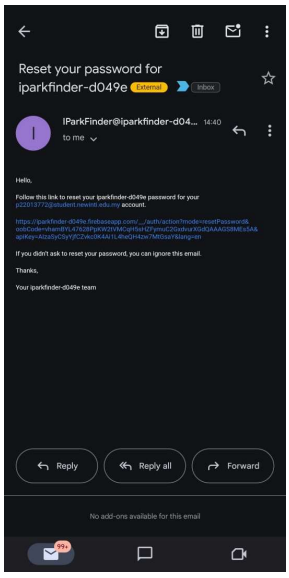


Figure 4- 9 Reset Password Email



Figure 4- 10 Reset Password

The forgot password feature comprises of a few pages. After tapping on the 'Forgot Password?' link in the login page, the user will navigate to the Forgot Password Page 1 where the user must type the student email address. If the email is not found in the database then the application will show an error message that the account is not found. If the email passes the validation, the application will navigate to the second page as shown in Figure 4-8 telling the user that a password reset link has been sent to the email address. Figure 4-9 shows the reset password email sent to the user while Figure 4-10 shows the reset password function provided by Firebase Authentication. Once reset, the email will let the user know that the password is reset where the user will go back to application to login.

4.2.4 Parking Page

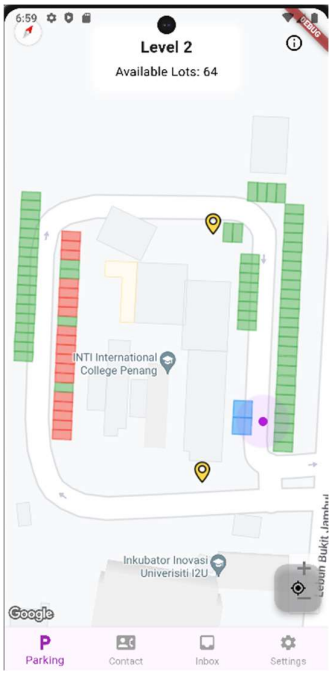


Figure 4- 11 Parking Page

After logging into the application, the user will navigate to the Home page users can navigate to the Parking, Contact, Inbox, and Setting page in the navigation bar as shown in Figure 4-11. The default page when entering the Home page is the Parking page where the user can see the vacant and occupied parking lots in INTI International College Penang with the green lots meaning vacant and red slots meaning occupied. The blue lots are the OKU parking lots. The two yellow markers represent the entrances to the campus. The page also shows the available parking lots for the level including the OKU parking lots. The mobile application currently only caters to the level 2 parking lot. The vacancy status is updated when the object detection model detects changes in vacancy and updates the database.

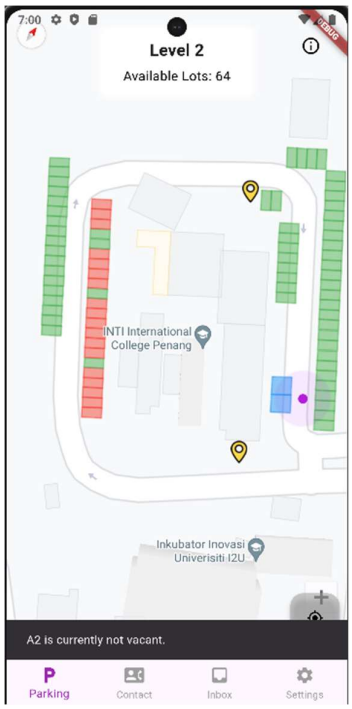


Figure 4- 12 Snackbar Appear when Parking Lot is Occupied

If the user taps on a red parking lot (occupied parking lot), a snackbar will appear at the bottom of the screen stating the parking lot is not vacant.

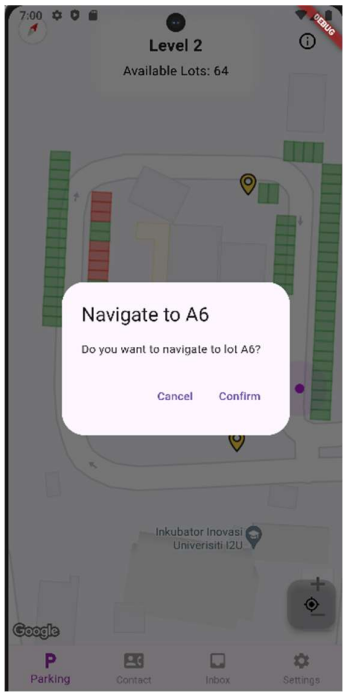


Figure 4- 13 Dialog Box to Confirm Navigation

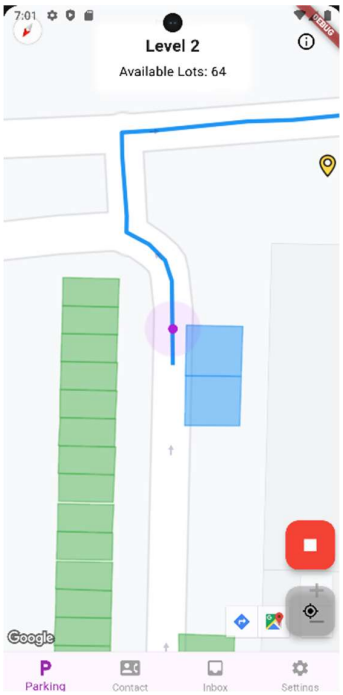


Figure 4- 14 Navigation

If the user taps on a green parking lot (vacant parking lot), the application will show a dialog box as shown in Figure 4-13 and navigates the user to the selected parking lot if the user taps on ‘Confirm’. Currently due to the testing purposes, the navigation mimics a user moving along the path to reach the destination as shown in Figure 4-14. The red floating button allows the user to stop the navigation by prompting a dialog box confirming the action. The grey floating button will recenter the map to the user’s location and bearing.

```
static LatLng calculateCentroid(List<LatLng> points) {
  double latitudeSum = 0;
  double longitudeSum = 0;

  for (var point in points) {
    latitudeSum += point.latitude;
    longitudeSum += point.longitude;
  }

  double centroidLat = latitudeSum / points.length;
  double centroidLng = longitudeSum / points.length;

  return LatLng(centroidLat, centroidLng);
}
```

Figure 4- 15 Code Snippet of Calculating the Centre Location of Parking Lots for Dijkstra’s Algorithm

```
static void initializeGraph() {
  print("Initializing graph...");
  entrances.forEach((entranceName, entranceLatLng) {
    graph[entranceName] = {};
    parkingLots.forEach((lotName, lotData) {
      if (lotData.containsKey('location') && lotData.containsKey('vacant')) {
        LatLng centroid = lotData['location']; // Use centroid
        double distance = _calculateDistance(entranceLatLng, centroid);
        graph[entranceName]![lotName] = distance;
        graph[lotName] = {entranceName: distance};
      }
    });
  });
  print("Graph initialized: $graph");
```

Figure 4- 16 Code Snippet for Using the Calculated Centroid to Calculate Distance for Weight in Dijkstra's Algorithm

```
// Haversine formula to calculate distance between two LatLng points
static double _calculateDistance(LatLng point1, LatLng point2) {
  const R = 6371; // Radius of the Earth in kilometers
  double latDistance = (point2.latitude - point1.latitude) * pi / 180;
  double lonDistance = (point2.longitude - point1.longitude) * pi / 180;
  double a = sin(latDistance / 2) * sin(latDistance / 2) +
      cos(point1.latitude * pi / 180) * cos(point2.latitude * pi / 180) *
          sin(lonDistance / 2) * sin(lonDistance / 2);
  double c = 2 * atan2(sqrt(a), sqrt(1 - a));
  return R * c;
}
```

Figure 4- 17 Code Snippet of Helper Function to Calculate Distance using Haversine Formula

```
// Dijkstra's algorithm for shortest path, only considering vacant parking lots
  static String? findShortestPath(String startNode) {
    if (graph[startNode] == null) {
      print("Error: Start node $startNode does not exist in the graph.");
      return null;
    }

    Map<String, double> distances = {};
    Map<String, String?> previousNodes = {};
    Set<String> unvisitedNodes = {};

    // Initialize distances
    graph.keys.forEach((node) {
      distances[node] = double.infinity;
      previousNodes[node] = null;
      unvisitedNodes.add(node);
    });
    distances[startNode] = 0;

    while (unvisitedNodes.isNotEmpty) {
      // Find the node with the smallest distance
      String? closestNode;
      double shortestDistance = double.infinity;
      for (String node in unvisitedNodes) {
        if (distances[node]! < shortestDistance) {
          shortestDistance = distances[node]!;
          closestNode = node;
        }
      }
```

Figure 4- 18 Code Snippet of Dijkstra's Algorithm Part 1

```
  if (closestNode == null) break;

  // Remove the closest node from unvisited nodes
  unvisitedNodes.remove(closestNode);

  // Calculate distances for neighbors of the closest node
  graph[closestNode]?.forEach((neighbor, distance) {
    double alternativeRoute = distances[closestNode]! + distance;
    if (alternativeRoute < distances[neighbor]!) {
      distances[neighbor] = alternativeRoute;
      previousNodes[neighbor] = closestNode;
    }
  });
}

// Find the nearest vacant parking lot from the starting node, excluding OKU1 and OKU2
String? nearestParkingLot;
double shortestPathDistance = double.infinity;
parkingLots.forEach((parkingLot, lotData) {
  // Only consider vacant parking lots, excluding OKU1 and OKU2
  if (lotData['vacant'] && distances[parkingLot]! < shortestPathDistance && parkingLot != "OKU1" && parkingLot
    shortestPathDistance = distances[parkingLot]!;
    nearestParkingLot = parkingLot;
  }
});

if (nearestParkingLot != null) {
  print("Nearest vacant parking lot to $startNode is $nearestParkingLot with distance $shortestPathDistance");
  return nearestParkingLot;
} else {
```

Figure 4- 19 Code Snippet of Dijkstra's Algorithm Part 2

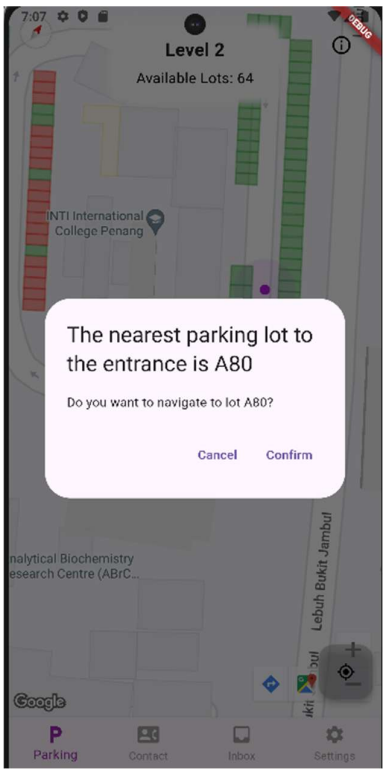


Figure 4- 20 Dialog Box Prompt When Tapping on Entrance

Figure 4-20 shows the dialog box prompt when the user taps on an entrance marker. The application will use Dijkstra's algorithm to find the nearest parking lot to the entrance and prompt the user if the user wants to navigate there. If the user confirms, then the application will navigate to the parking lot.

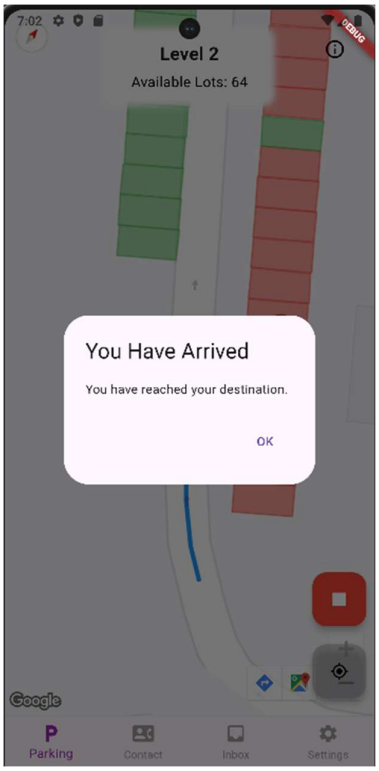


Figure 4- 21 Dialog Box Confirming Arrival

If the user chooses to navigate to the location, a dialog box prompt will pop up alerting the user has reached the destination.

4.2.5 Contact Page

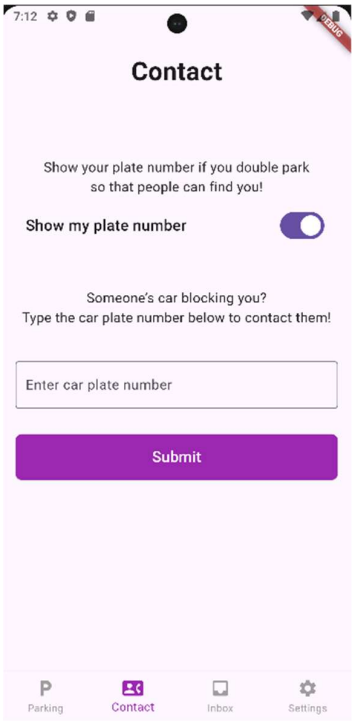


Figure 4- 22 Contact Page

Figure 4-22 shows the contact page where the user can toggle the 'Show my plate number' to show or hide their car plate number in the system for privacy issues. This function allows the user to replace the practice of putting their phone number on the dashboard when double parking. The user can also type the car plate number of the vehicle that double parked them to call the owner.

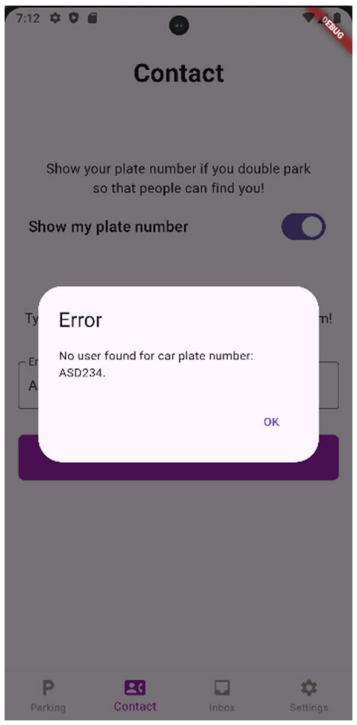


Figure 4- 23 Error Hide Plate Number

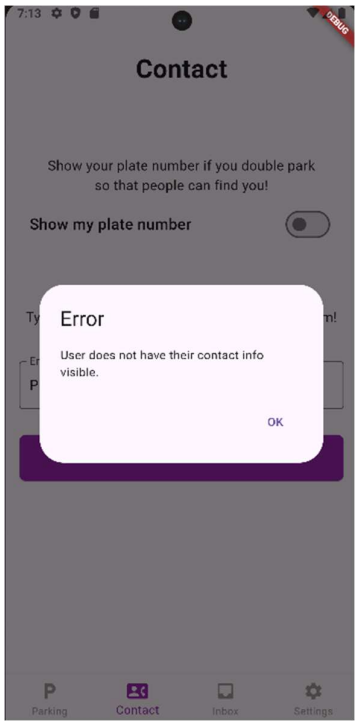


Figure 4- 24 Error Car Plate Not Found

Figure 4-23 and Figure 4-24 shows the error message when the vehicle owner did not register the vehicle in the system and when the vehicle owner hide the contact number respectively.

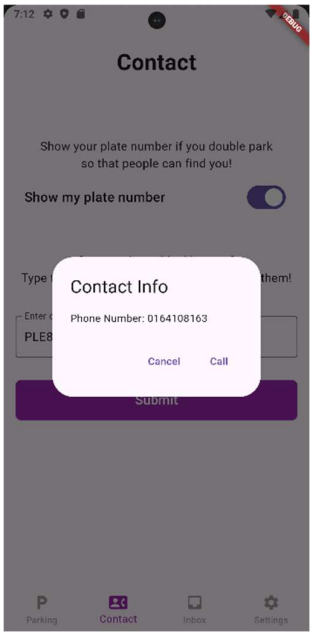


Figure 4- 25 Contact Found

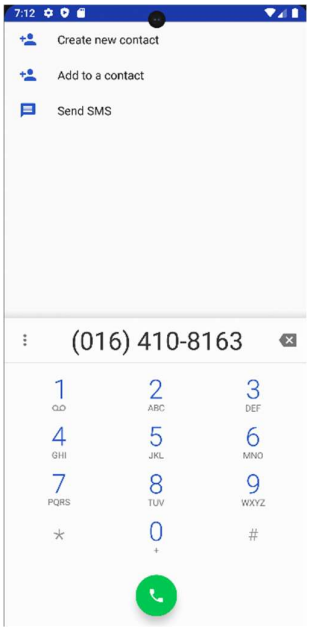


Figure 4- 26 Navigate to Phone App

Figure 4-25 shows the dialog box when the car plate number keyed in has the phone number visible allowing the user to call the owner. When the user tap on call, the application will launch the phone app which automatically keys in the contact number.

4.2.6 Inbox Page

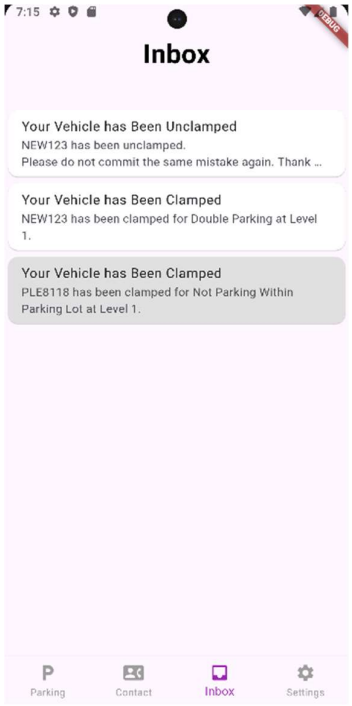


Figure 4- 27 Inbox Page

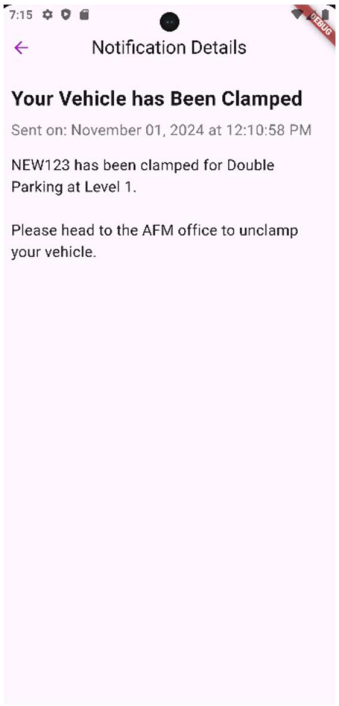


Figure 4- 28 Notification Details

Figure 4-27 shows the Inbox page where the user will receive notification when the vehicle has been clamped or when the user has paid the fine and unclamped the car. White tile means the message has not been read while grey means the notifications has been read. Figure 4-28 shows an example of a notification for vehicle clamp.

4.2.7 Settings Page

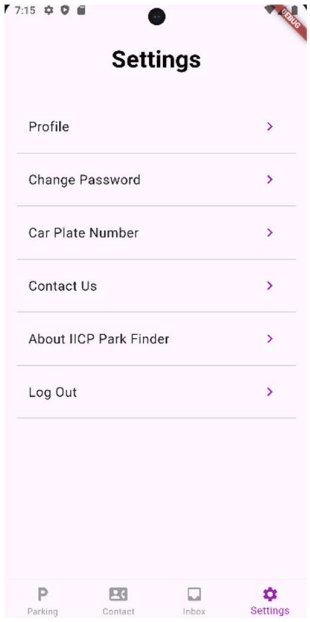


Figure 4- 29 Settings Page

Figure 4-29 shows the Settings page where the user can navigate to the profile, change password, car plate number, contact AFM, and terms and conditions page. The user can also log out from the application on this page.

4.2.8 Profile Page

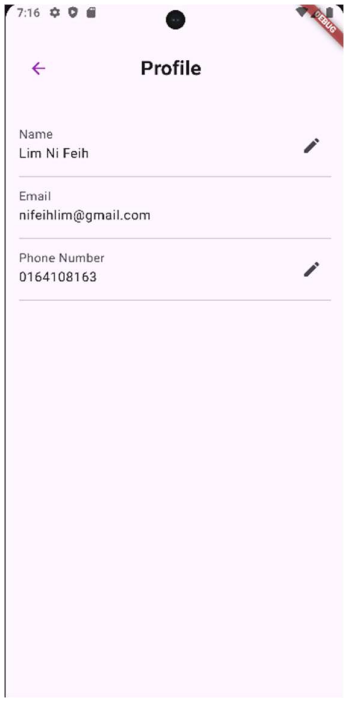


Figure 4- 30 Profile Page

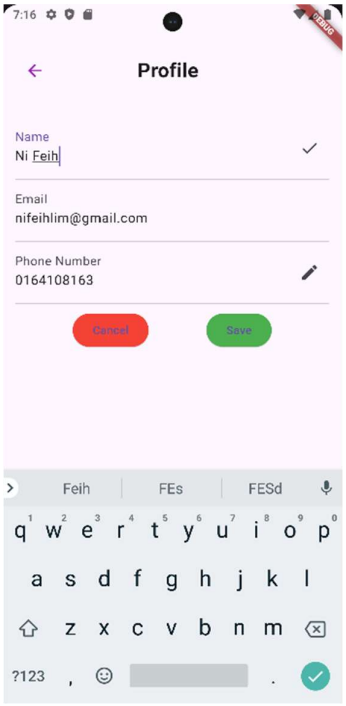


Figure 4- 31 Edit Profile Details

Figure 4-30 shows the profile page which displays the user's name, email, and phone number. The pen icon allows the user to edit the name and phone number if there are any changes to be made. The email is not editable as the user's student email will never change thus making it redundant to be editable. The email used in this example is a personal email used for testing and showcasing purposes only.

4.2.9 Change Password Page

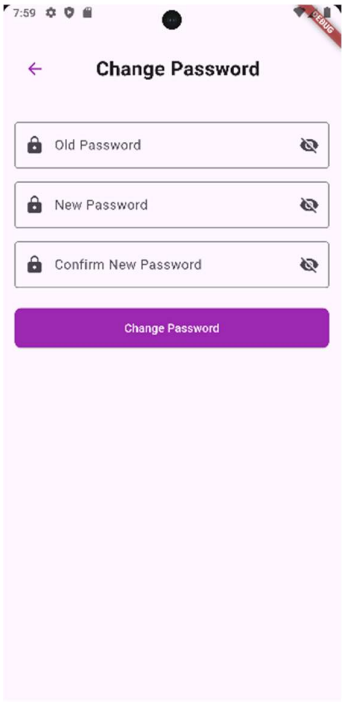


Figure 4- 32 Change Password Page

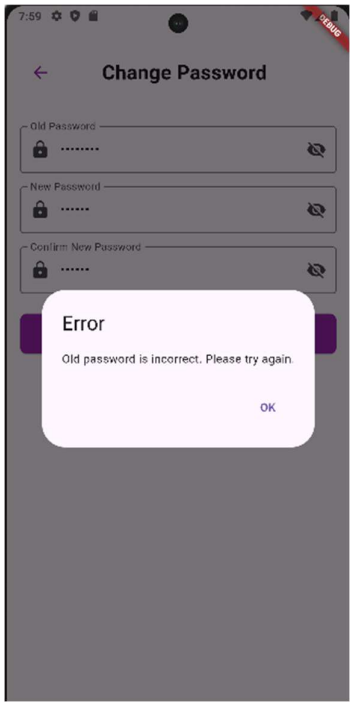


Figure 4- 33 Error Old Password Invalid

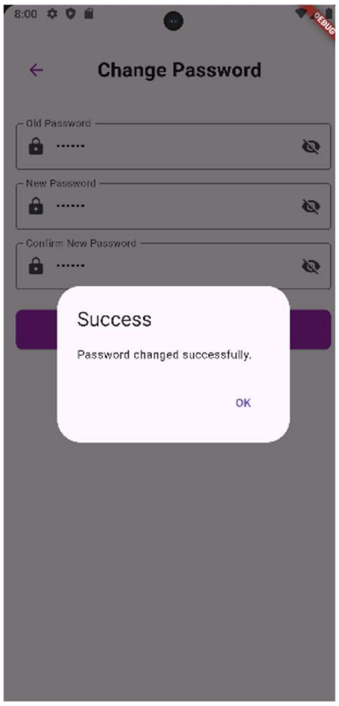


Figure 4- 34 Dialog Box Success in Changing Password

Figure 4-32 shows the change password page which allows the user to change their password. There are four validations for this feature which include checking for empty fields, the old password keyed in must be correct, the new password keyed in must be at least 6 characters long, and the new and confirm password must match. Figure 4-33 shows an example of an error message when the current password keyed in is wrong. Figure 4-34 shows the dialog box when the password has been successfully changed.

4.2.10 Car Plate Number Page



Figure 4- 35 Car Plate Numbers Page

Figure 4- 35 shows the car plate numbers page listing all the plate numbers of the user. The user can tap on the '+' floating button to navigate to the add car plate number page.

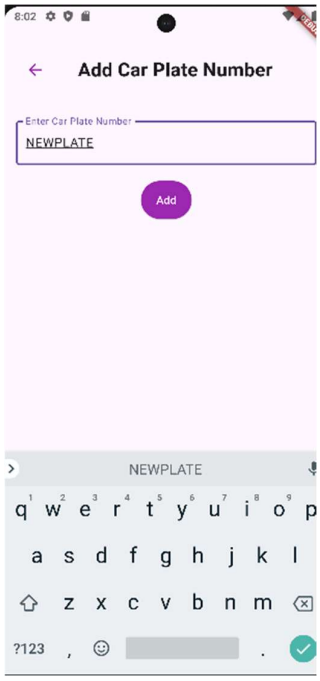


Figure 4- 36 Add Car Plate Numbers Page



Figure 4- 37 Updated List of Car Plate Numbers

Figure 4-36 shows the add car plate numbers page where the user can key in the other plate numbers that are owned. The field validates to check if the field is empty and if the car plate number already exist in the database. When successful, the app will navigate back to the list

of car plate numbers with the updated list and show the snack bar that the car plate has been added.



Figure 4- 38 Edit and Delete Car Plate Numbers

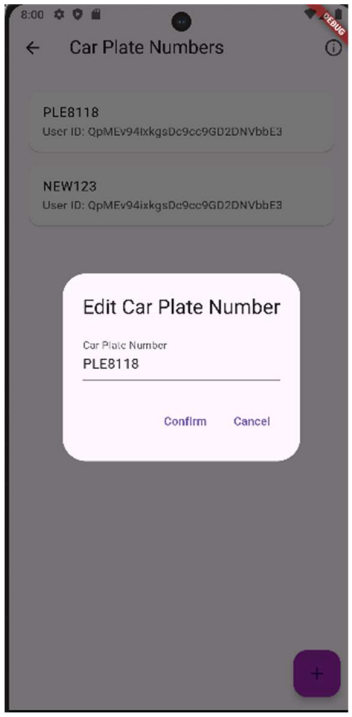


Figure 4- 39 Edit Car Plate Number

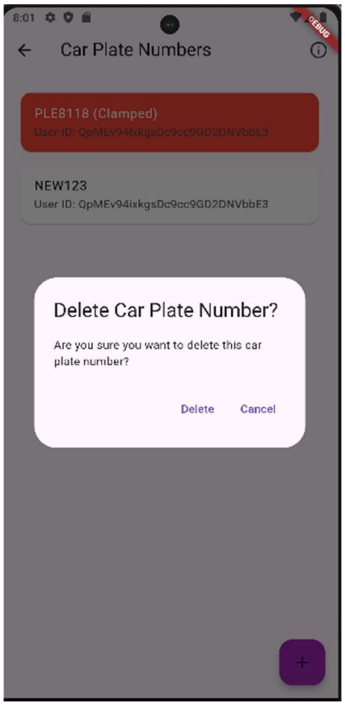


Figure 4- 40 Confirmation Dialog to Delete Car Plate Number

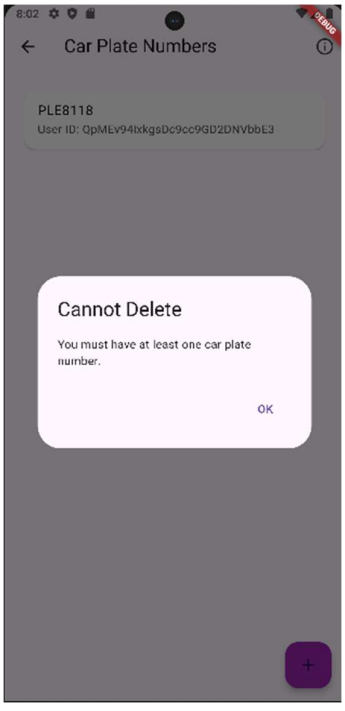


Figure 4- 41 Cannot Delete with One Car Plate Number

Figure 4-38 shows the how the car plate number can be edited or deleted by swiping the card to the left. Figure 4-39 shows the edit dialog to edit the selected car plate number. Figure 4-40 shows the confirmation dialog to delete the car plate number. Deleting the car plate number consists of validation where the user must have at least one car plate number



Figure 4- 42 List with Clamped Car Plate Number

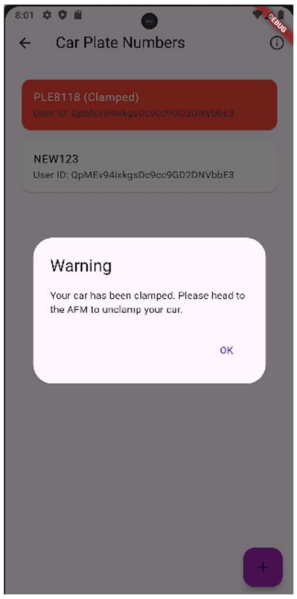


Figure 4- 43 Warning Dialog When User Edit or Delete Clamped Plate Number

When the car plate number is clamped, the tile will turn red with a (‘Clamped’) font next to the plate number as shown in Figure 4-42. A warning dialog will prompt if the user tried to edit or delete the car plate number that is clamped ensuring data integrity within the system.

4.2.11 Contact Us Page (AFM)

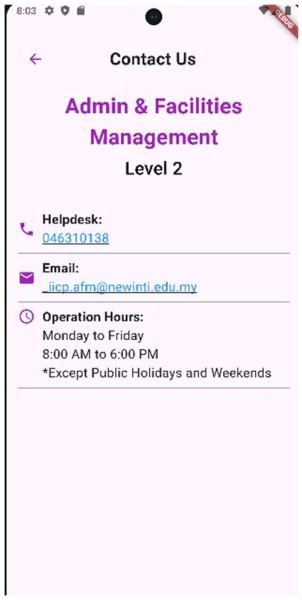


Figure 4- 44 Contact Us Page (AFM)

Figure 4-44 shows the contact us page where the user can see the details of the AFM department if the user needs to contact or go to the office.

4.2.12 About IICP Park Finder Page (Terms and Conditions)

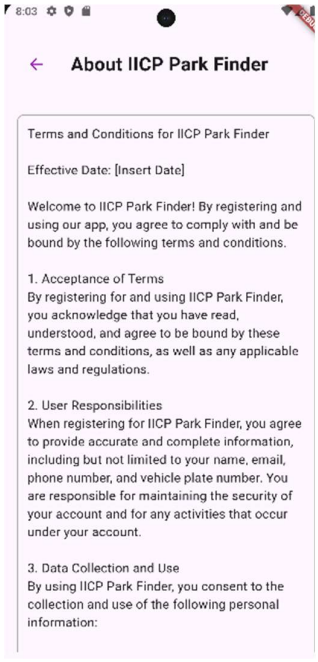


Figure 4- 45 About IICP Park Finder Page (T&C)

Figure 4-45 shows the About IICP Park Finder page where the user can read the terms and conditions of using the mobile application. The T&C written is only a draft.

4.2.13 Logout

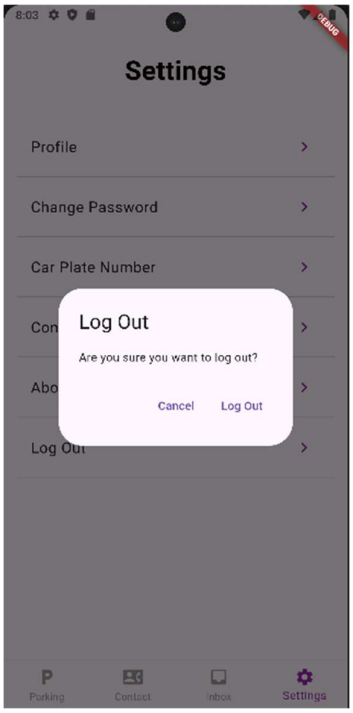


Figure 4- 46 Logout Confirmation Dialog

Figure 4-46 shows the confirmation dialog prompting the user to confirm if the user wants to log out from the application, logging the user out from the application and database connection.

4.3 Web Application

This section documents the development of the web application in Visual Studio Code. The web application is developed with Vue.js on a local host.

4.3.1 Login

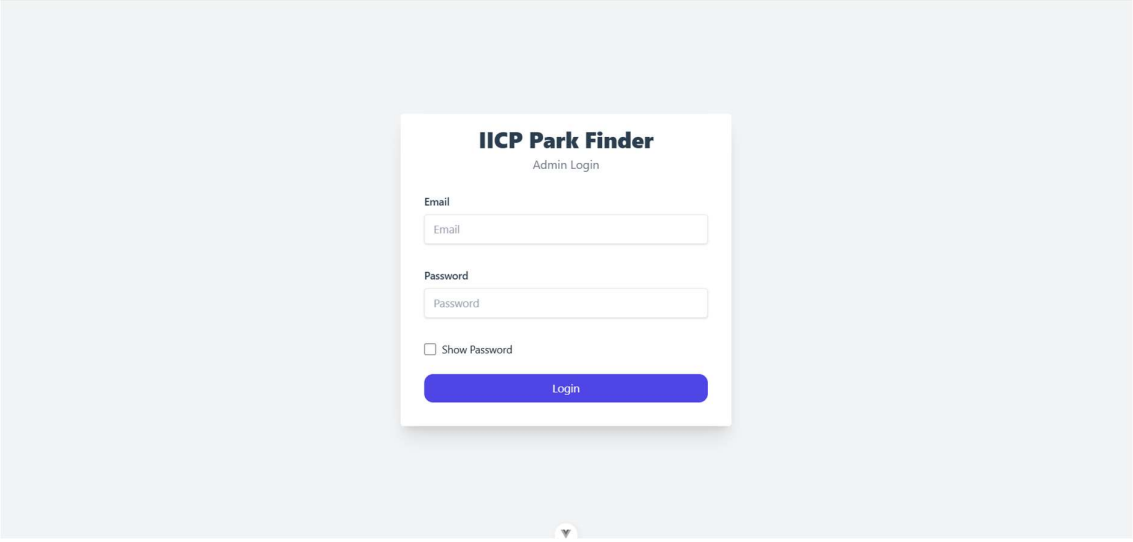


Figure 4- 47 Admin Login Page

The landing page for the web application is the login page where the admin must enter the email and password. The validations for the login page consists of fields not empty and a registered email and password. Admin can tick the show password to unhash the password.

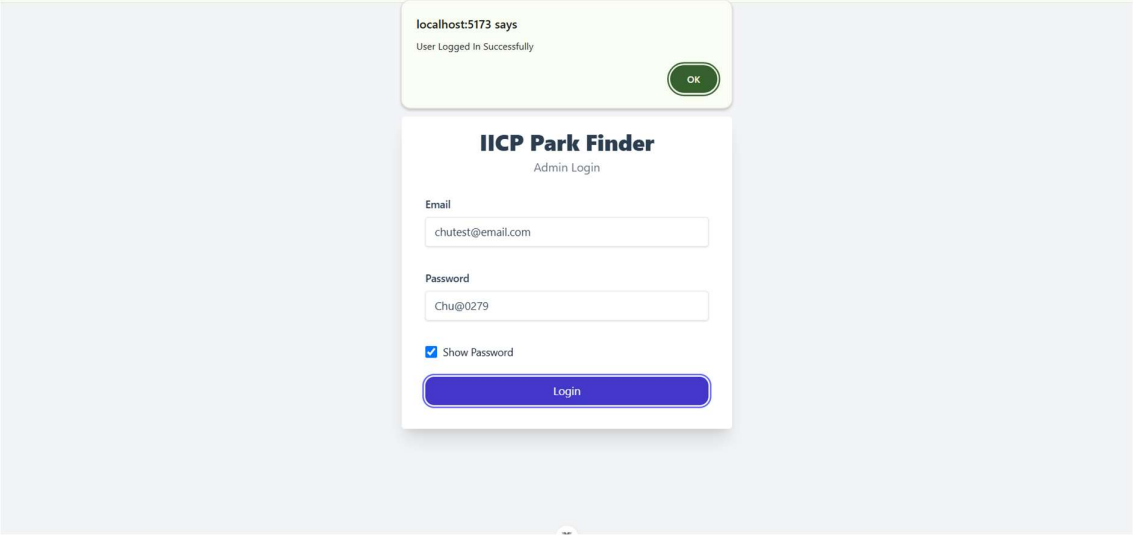


Figure 4- 48 Admin Login Successful

Once login attempt is successful, the admin web will show a success dialog and navigate to the home page of the web application.

4.3.2 Parking Statistics Page

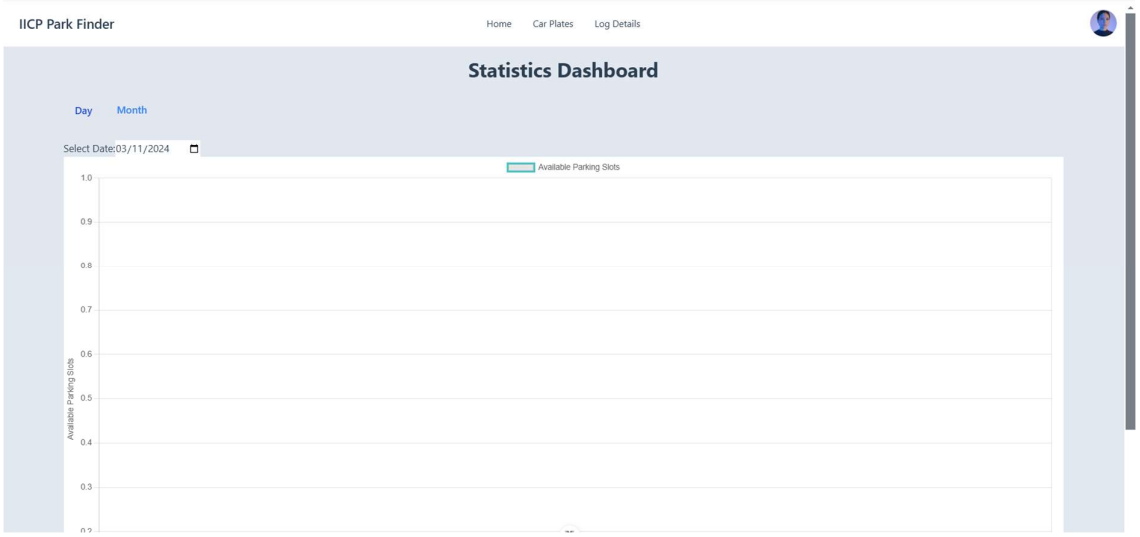


Figure 4- 49 Home Page with Statistics Dashboard

The home page for the web application is the statistics dashboard which will show the line graph of available parking spaces on campus on the current date. The statistics consists of day and month categories. These statistics will allow the AFM department to determine the peak hour when students come to campus and plan the traffic efficiently.

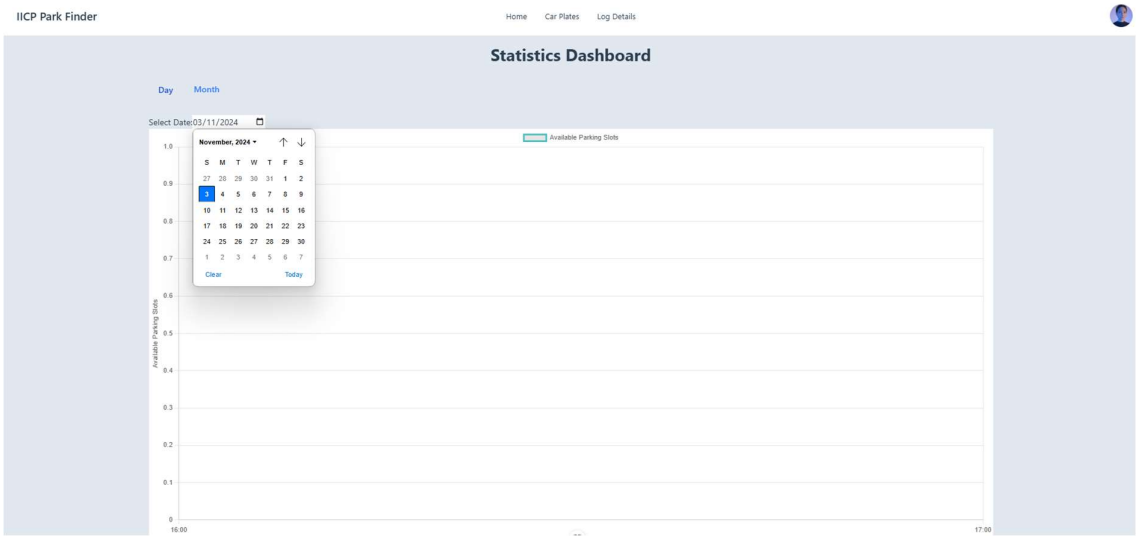


Figure 4- 50 Data Picker to View Day Statistics

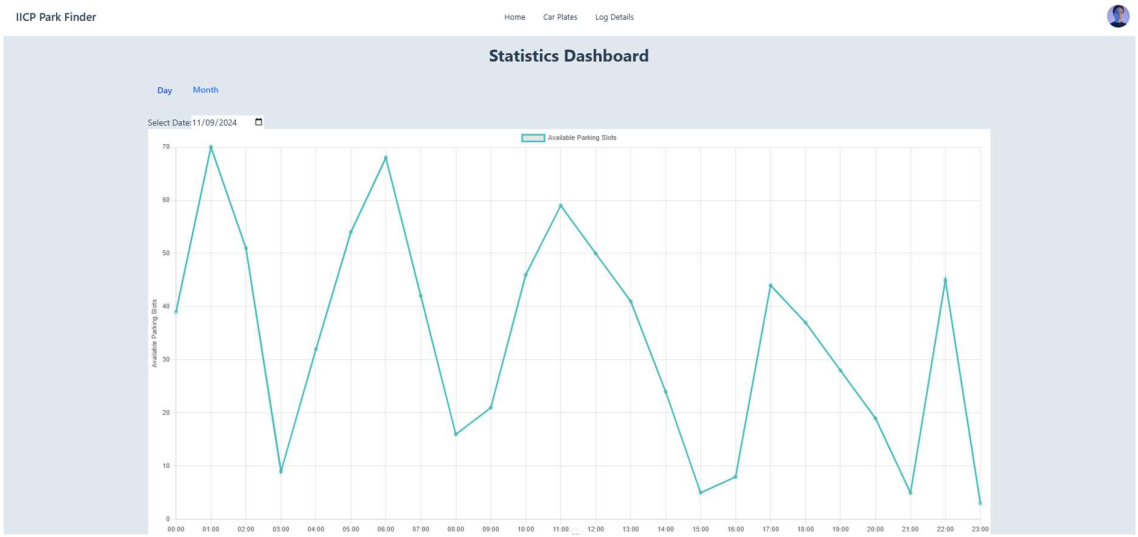


Figure 4- 51 Statistics for Chosen Date

Figure 4-50 shows the date picker to pick the date to view the available parking throughout the day. Figure 4-51 shows the available parking spaces throughout the day from 12 a.m. until 11 p.m. for the chosen date.

```
const createChart = () => {
  const ctx = document.getElementById('parkingChart').getContext('2d');


  const labels = parkingData.value.map(data => {
    const [day, month, year] = data.date.split('/');
    return new Date(`${year}-${month}-${day}T${data.time}:00`).toISOString();
  });

  const dataPoints = parkingData.value.map(data => data.availableparkinglot);


  if (chartInstance) {
    chartInstance.destroy();
  }

  // Initialize the chart
  chartInstance = new Chart(ctx, {
    type: 'line',
    data: {
      labels: labels,
      datasets: [{
        label: 'Available Parking Slots',
        data: dataPoints,
        borderColor: 'rgba(75, 192, 192, 1)',
        fill: false,
      }]
    },
```

Figure 4- 52 Code Snippet for Displaying the Line Graph

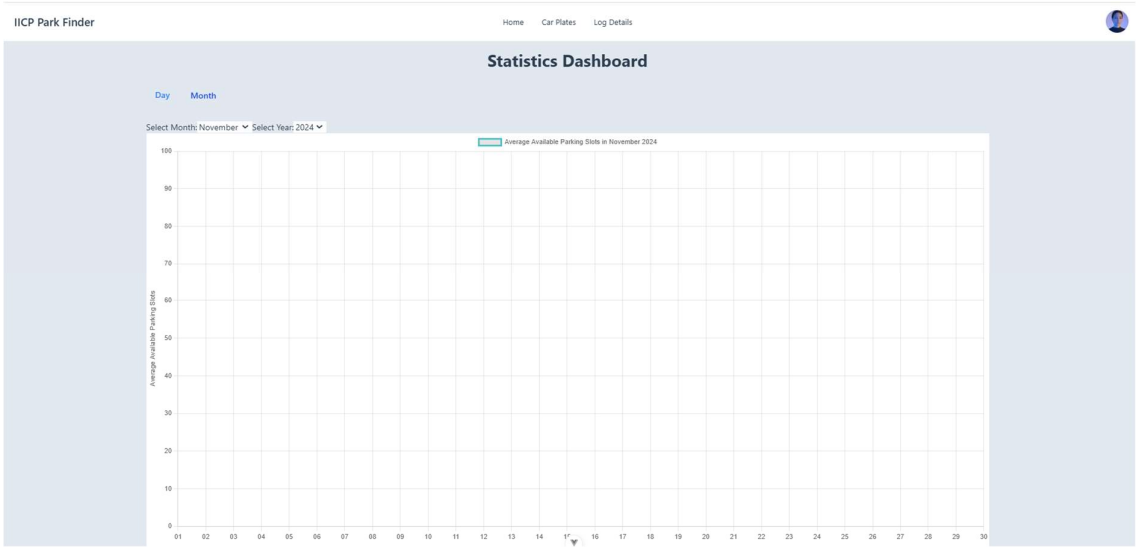


Figure 4- 53 Statistics Dashboard for Month

When the admin clicks on the 'Month' the statistics for the current month will show. It is currently empty as no data for this month has been added in the database.

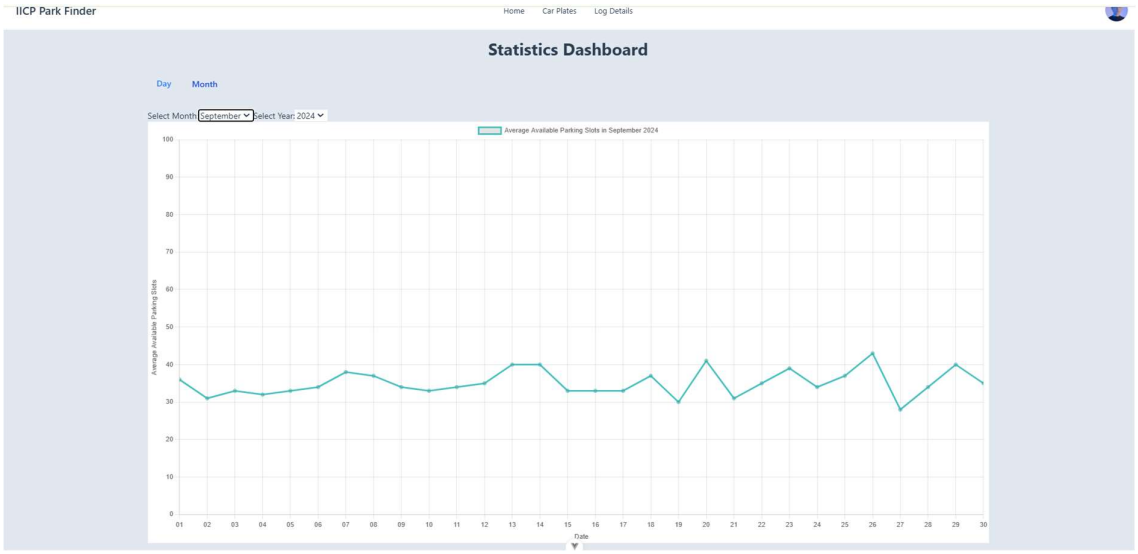


Figure 4- 54 Statistics Dashboard for Selected Month and Year

The admin can select the month and year in the top right corner above the graph to get the average available parking lot everyday throughout the month. This can give the admin a bigger picture of when in the month do students come to campus the most.

```
const calculateAverages = data => {
  const groupedData = {}

  data.forEach(entry => {
    const date = entry.date
    if (!groupedData[date]) {
      groupedData[date] = []
    }
    groupedData[date].push(entry.availableparkinglot)
  })

  // Calculate averages
  return Object.keys(groupedData).map(date => {
    const total = groupedData[date].reduce((sum, value) => sum + value, 0)
    const average = total / groupedData[date].length
    const roundedAverage = Math.round(average)

    return { date, roundedAverage }
  })
}
```

Figure 4- 55 Code Snippet to Calculate Average Available Parking Lot for Each Day in the Month

4.3.3 Student Lists Page

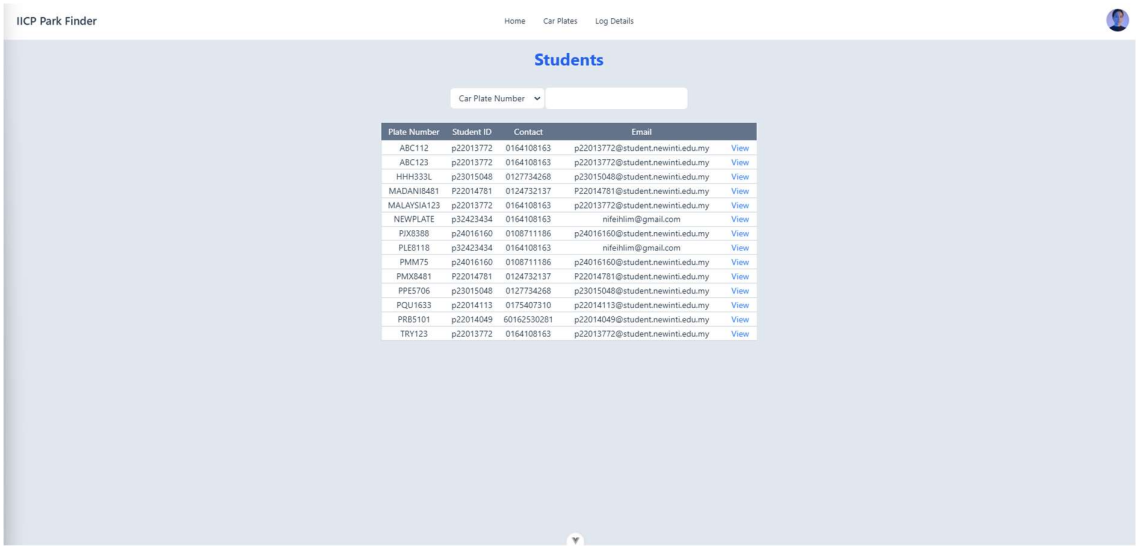


Figure 4- 56 Student Lists Page

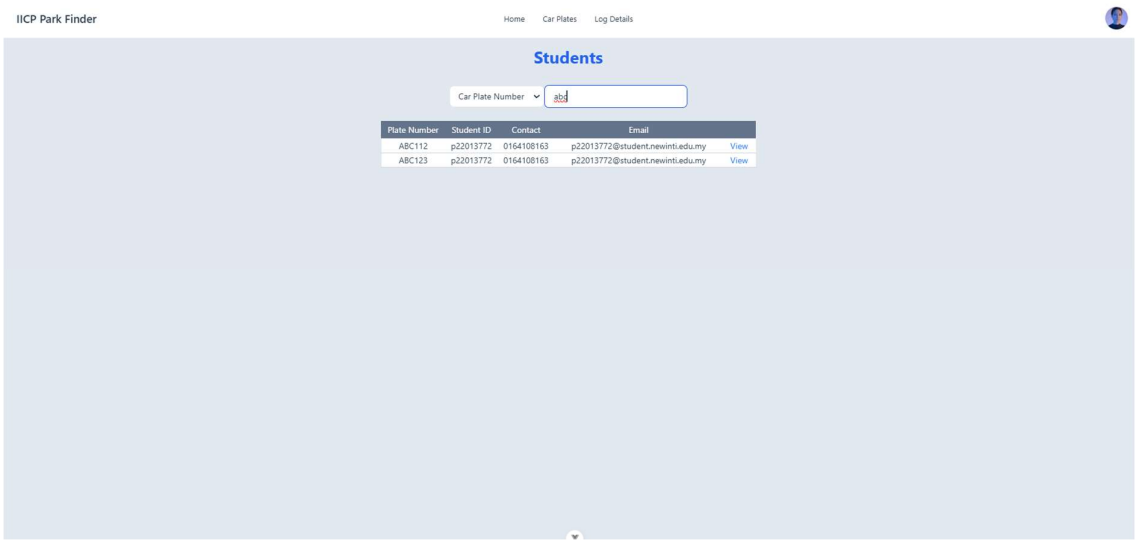


Figure 4- 57 Search Student List by Car Plate Number

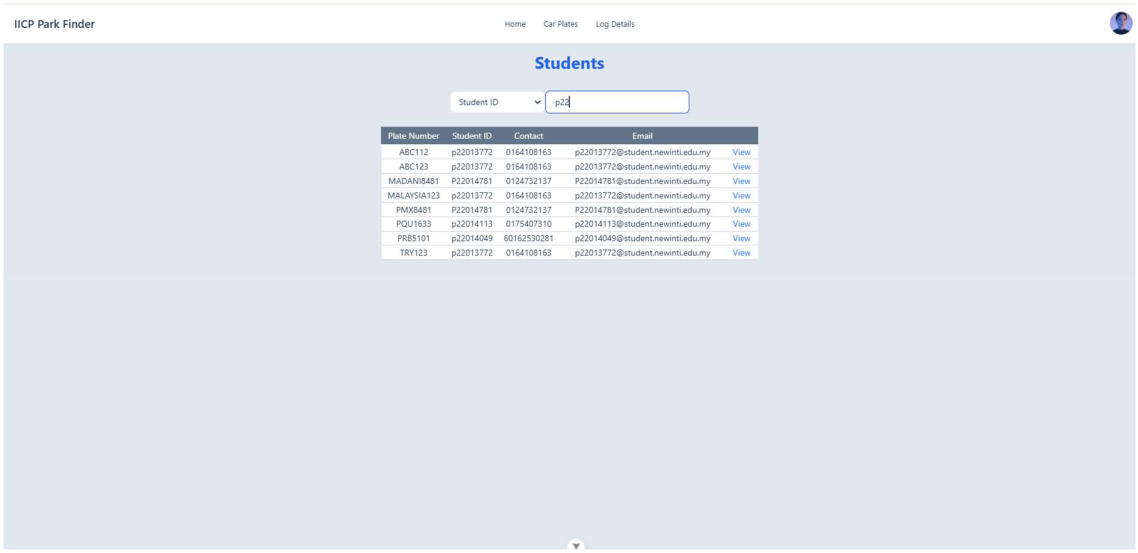


Figure 4- 58 Search Student List by Student ID

Figure 4-56 shows the student lists based on the car plate number. The admin can search the list by car plate number or student ID as shown in Figure 4-57 and Figure 4-58. To view the fines for the car plate numbers, the admin can click on ‘View’ to see the fine for the selected car plate.

4.3.4 Fines Page

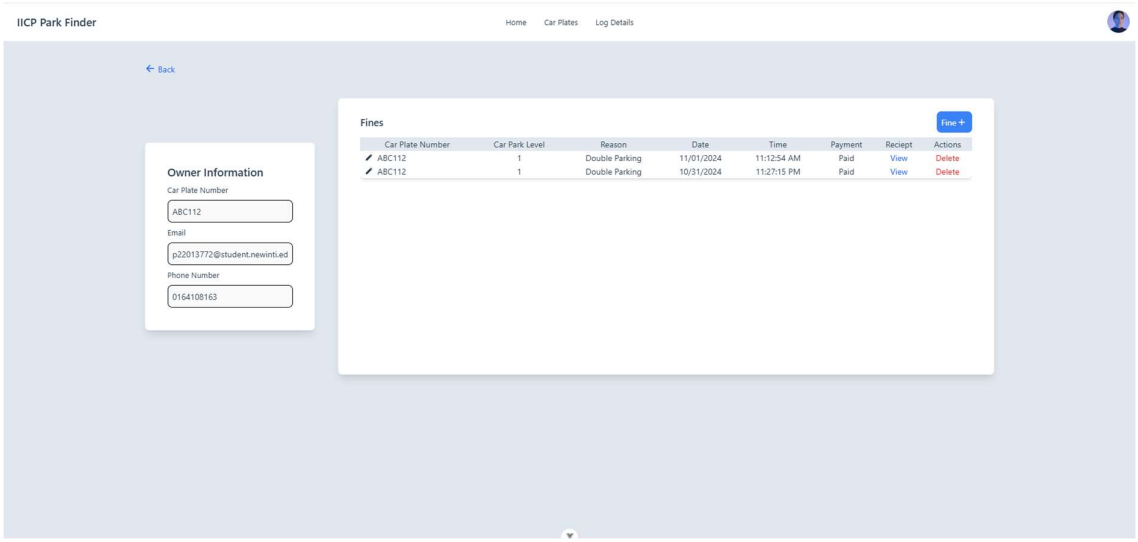


Figure 4- 59 Fine Page

After clicking on the View in the Student Lists page, the user will navigate to the Fines page which shows the fine details accumulated by the vehicle.

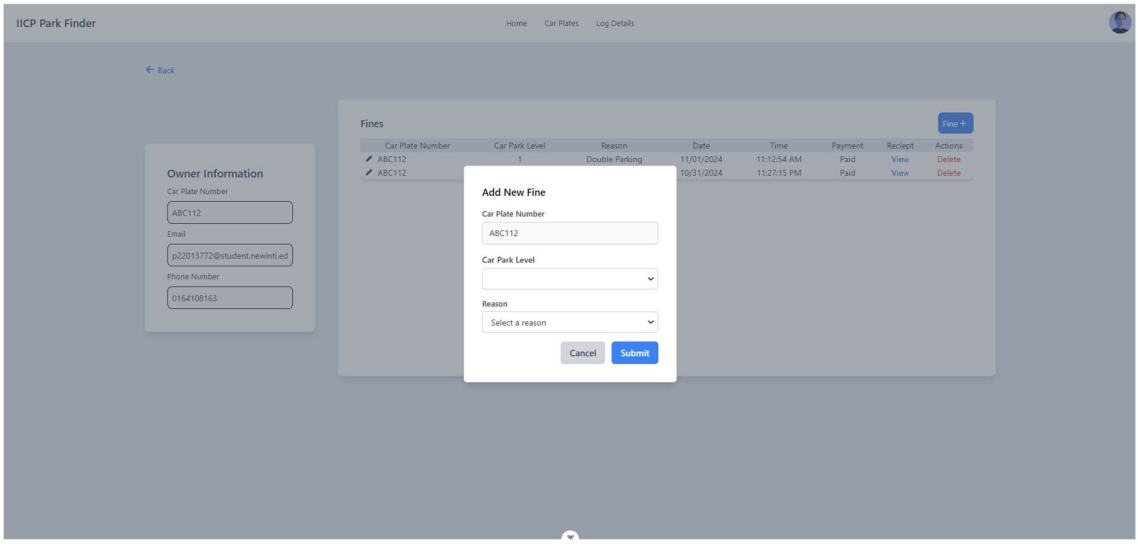


Figure 4- 60 Add Fine

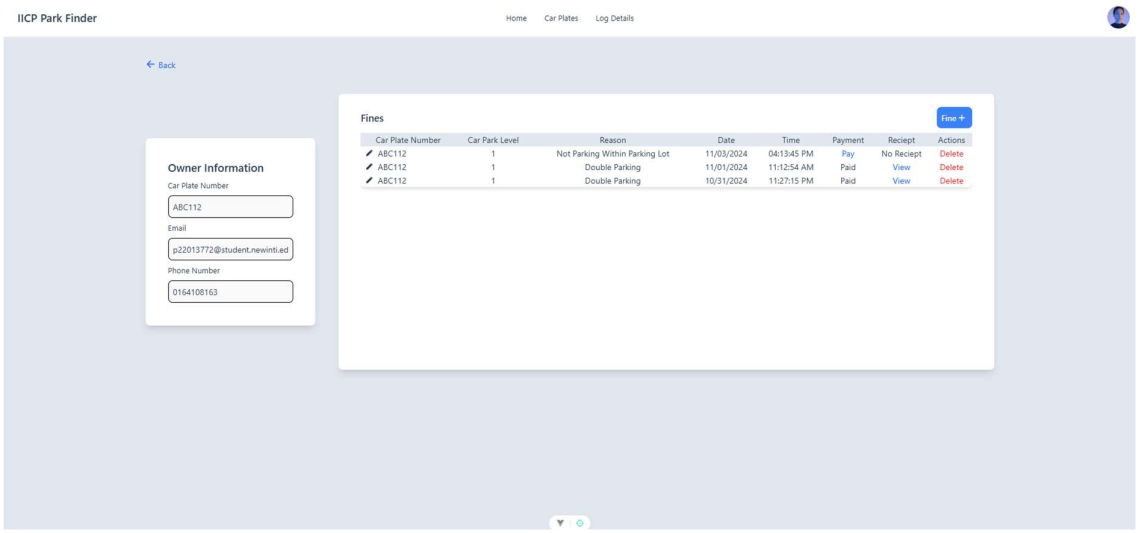


Figure 4- 61 Updated Fine List after Adding Fine

When the admin clicks on the Fine+ button on the top right corner, the add new fine modal will pop up allowing the admin to select the car park level and reason of fining the vehicle. The list will automatically update after the fine is added.

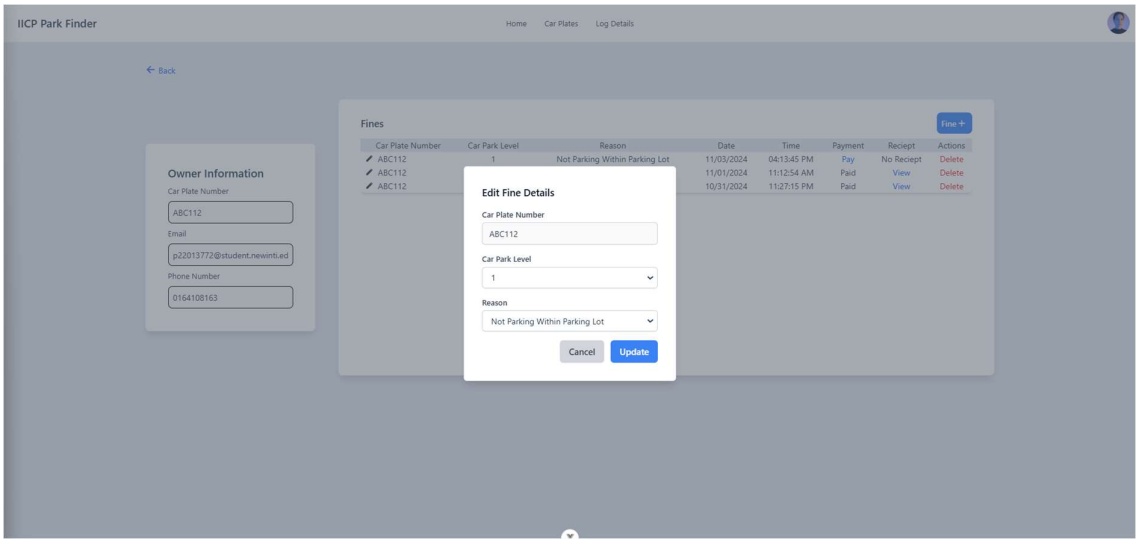


Figure 4- 62 Edit Fine

The admin can also edit the fine by clicking on the pen icon on the left of the fine. The list will automatically update after the fine has been edited.

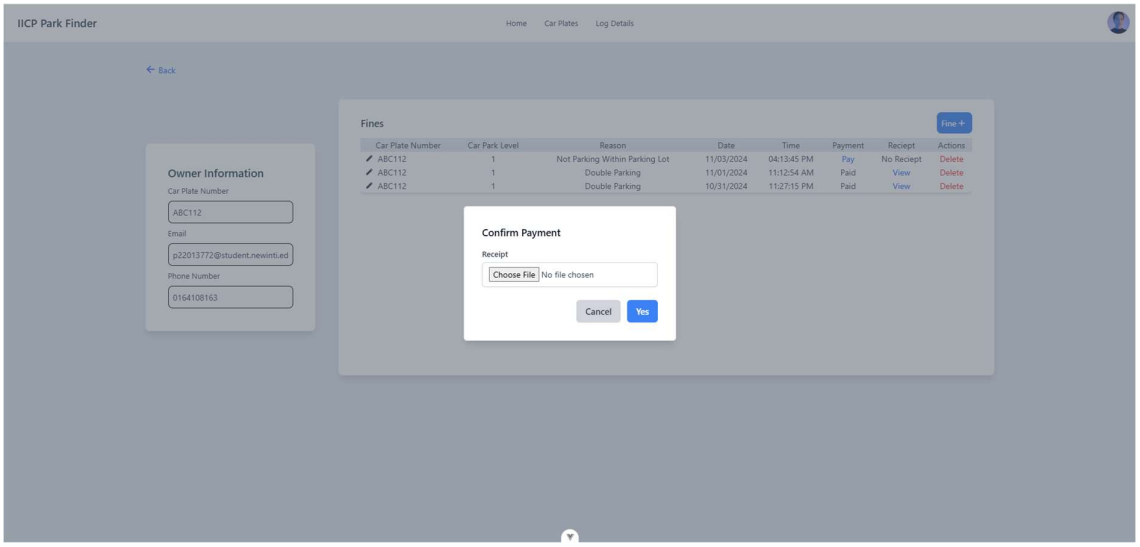


Figure 4- 63 Pay Fine

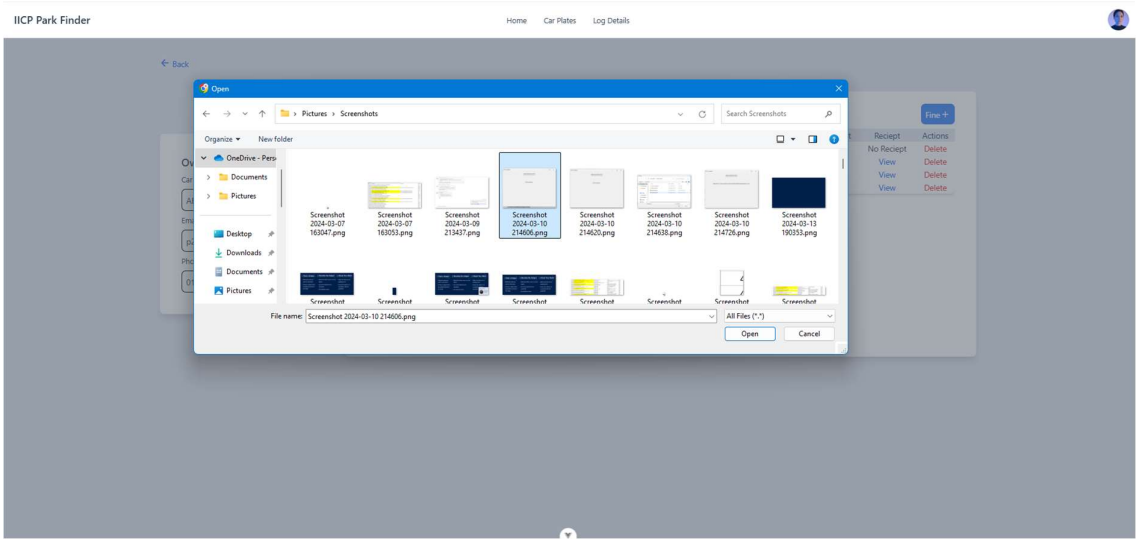


Figure 4- 64 Select File to Upload as Receipt

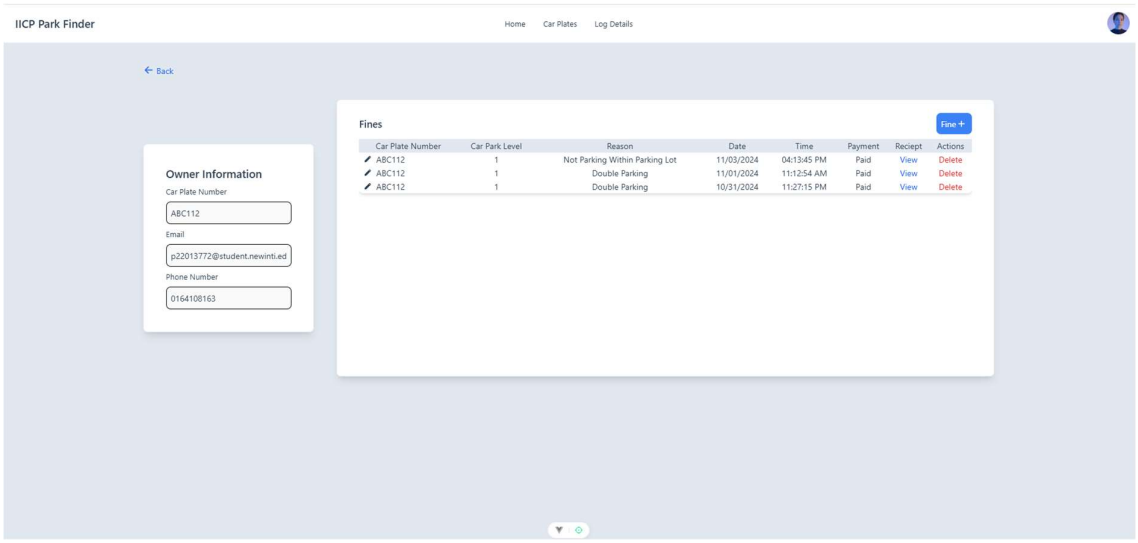


Figure 4- 65 Fine Mark as Paid

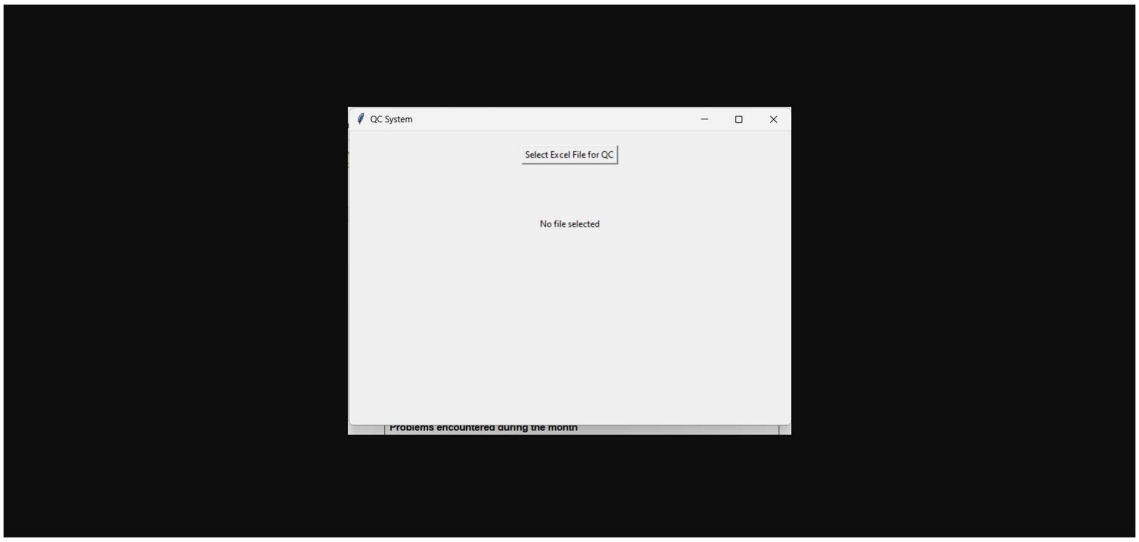


Figure 4- 66 Receipt Uploaded

When the vehicle owner has paid the fine, the admin can Click on the ‘Pay’ to add the receipt to the database. After the admin has uploaded the receipt, the fine will be mark as paid and the admin can click on the ‘View’ to open the receipt.

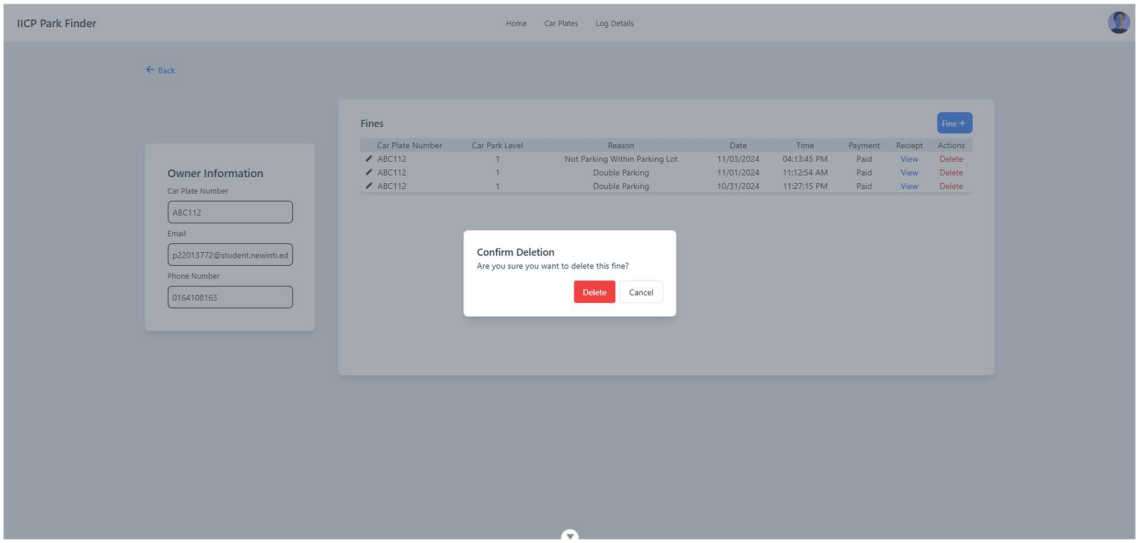


Figure 4- 67 Delete Confirmation Dialog

Sometimes when new students park in campus, the students might not be familiar with the parking rules and regulations. In these cases, the AFM department might consider to delete the parking fine and give a warning to the student. Thus, the delete function is there to delete the fine in such cases.

4.3.5 Log Details Page

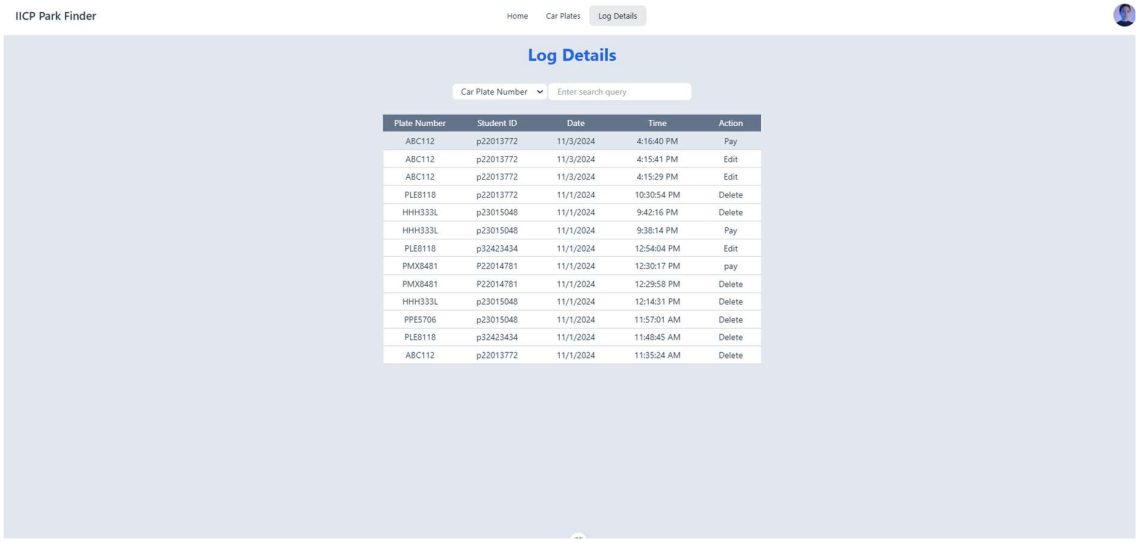


Figure 4- 68 Log Details Page

When the admin clicks on the Log Details at the top navigation bar, the admin will navigate to the Log Details page where the admin can see all the pay, edit, and delete fine transactions that has been made. This function is to ensure that the admin will know if the student has been fined before or not (given warning). This function also allows the admin to know if any unauthorised user has tampered with the system.

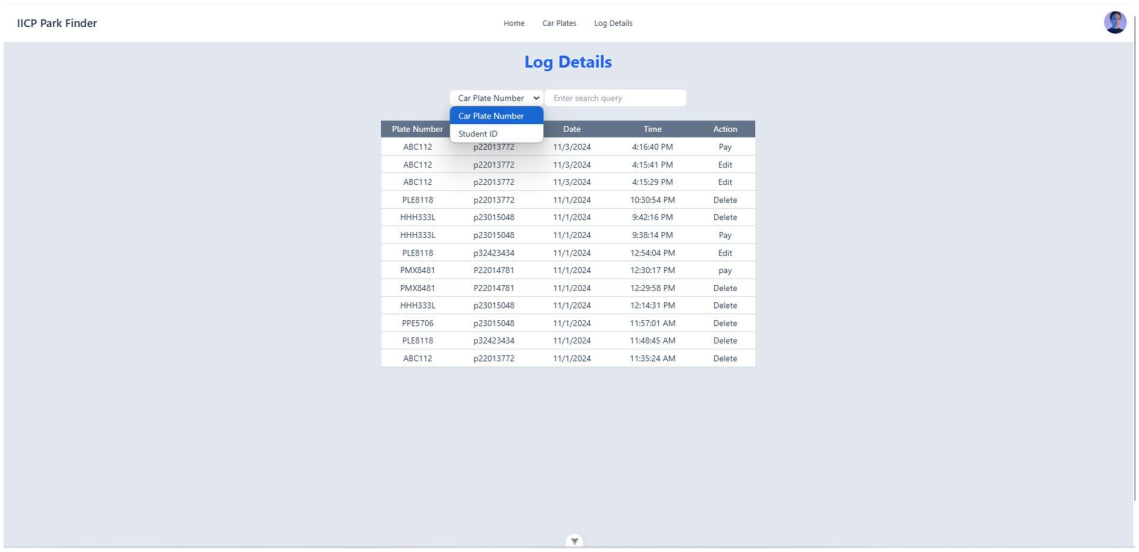


Figure 4- 69 Search Category for Log Details

As shown in Figure 4-69, the admin can search for log details based on car plate numbers and student ID for easy access.

4.3.6 Logout

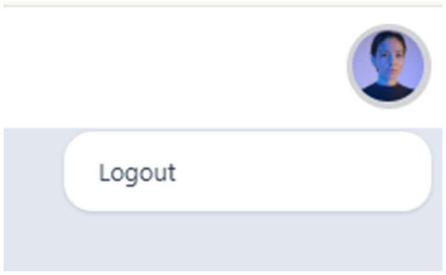
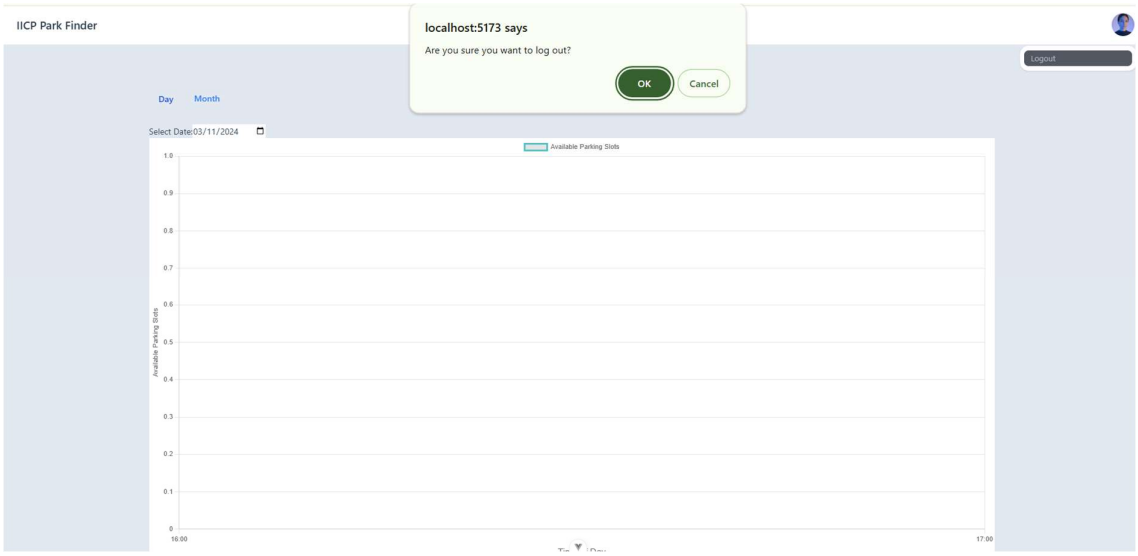


Figure 4- 70 Admin Logout

To logout from the system, the admin can click on the round icon on the top right of the page and click on the logout button.



A confirmation dialog will prompt the admin to confirm if the admin wants to logout from the system.

4.4 Object Detection Model

This section will document the development of the object detection model which is used to detect cars and compare it to the parking lot bounding box to determine the vacancy status of each parking lot.

4.4.1 Yolov8 Implementation (Detect Objects)

```
class DetectionPredictor(BasePredictor):

    def __init__(self, cfg, parking_lots):
        super().__init__(cfg)
        self.parking_lots = parking_lots

    def get_annotator(self, img):
        return Annotator(img, line_width=self.args.line_thickness, example=str(self.model.names))

    def preprocess(self, img):
        img = torch.from_numpy(img).to(self.model.device)
        img = img.half() if self.model.fp16 else img.float()  # uint8 to fp16/32
        img /= 255  # 0 - 255 to 0.0 - 1.0
        return img

    def postprocess(self, preds, img, orig_img):
        preds = ops.non_max_suppression(preds,
                                        self.args.conf,
                                        self.args.iou,
                                        agnostic=self.args.agnostic_nms,
                                        max_det=self.args.max_det)

        for i, pred in enumerate(preds):
            shape = orig_img[i].shape if self.webcam else orig_img.shape
            pred[:, :4] = ops.scale_boxes(img.shape[2:], pred[:, :4], shape).round()

        return preds
```

Figure 4- 71 Code Snippet of Modified Yolov8 DetectionPredictor Class (Part 1)

```
class DetectionPredictor(BasePredictor):
    def write_results(self, idx, preds, batch):
        all_outputs = []
        log_string = ""
        if len(im.shape) == 3:
            im = im[None]  # expand for batch dim
        self.seen += 1
        im0 = im0.copy()
        if self.webcam:  # batch_size >= 1
            log_string += f'{idx}: '
            frame = self.dataset.count
        else:
            frame = getattr(self.dataset, 'frame', 0)

        self.data_path = p
        save_path = str(self.save_dir / p.name)  # im.jpg
        self.txt_path = str(self.save_dir / 'labels' / p.stem) + ('' if self.dataset.mode == 'image' else f'_{frame}')
        log_string += '%gx%g ' % im.shape[2:]  # print string
        self.annotator = self.get_annotator(im0)

        det = preds[idx]
        all_outputs.append(det)
        if len(det) == 0:
            return log_string
        for c in det[:, 5].unique():
            n = (det[:, 5] == c).sum()  # detections per class
            log_string += f"{n} {self.model.names[int(c)]}{'s' * (n > 1)}, "
        # write
        gn = torch.tensor(im0.shape)[[1, 0, 1, 0]]  # normalization gain whwh
        xywh_bboxs = []
        confs = []
```

Figure 4- 72 Code Snippet of Modified Yolov8 DectectionPredictor Class (Part 2)

```
    log_string += f"{n} {self.model.names[int(c)]}{'s' * (n > 1)}, "
# write
gn = torch.tensor(im0.shape)[[1, 0, 1, 0]]  # normalization gain whwh
xywh_bboxs = []
confs = []
oids = []
outputs = []
for *xyxy, conf, cls in reversed(det):
    x_c, y_c, bbox_w, bbox_h = xyxy_to_xywh(*xyxy)
    xywh_obj = [x_c, y_c, bbox_w, bbox_h]
    xywh_bboxs.append(xywh_obj)
    confs.append([conf.item()])
    oids.append(int(cls))
xywhs = torch.Tensor(xywh_bboxs)
confss = torch.Tensor(confs)

outputs = deepsort.update(xywhs, confss, oids, im0)
if len(outputs) > 0:
    bbox_xyxy = outputs[:, :4]
    identities = outputs[:, -2]
    object_id = outputs[:, -1]

    # Draw the detected boxes and parking lot boxes
    draw_boxes(im0, bbox_xyxy, self.model.names, object_id, identities, parking_lots=self.parking_lots)

return log_string
```

Figure 4- 73 Code Snippet of Modified Yolov8 DetectionPredictor Class (Part 3)

The Yolov8 model used for the object detection model is a pre-trained yolov8l.pt model that comes by default when downloading the Ultralytics package (package for Yolo model). Usually developers will train the model with custom data but due to insufficient images of cars on the internet to train the model, custom training for object detection model is not used in the development. Thus, Deep SORT tracking algorithm is integrated with the Yolov8 model for a more accurate object detection. This is done by passing the DectectionPredictor class in the Ultralytics package and modifying the class in the predictor file to integrate Deep SORT from PyTorch library as shown in Figure 4-71 to Figure 4-73.

4.4.2 Deep SORT PyTorch Implementation (Track Objects)

Deep SORT is a tracking algorithm that tracks objects after it has been detected, increasing the accuracy of detecting objects even when it is occluded for a few seconds.

```
DEEPSORT:
  REID_CKPT: "deep_sort_pytorch/deep_sort/deep/checkpoint/ckpt.t7"
  MAX_DIST: 0.2
  MIN_CONFIDENCE: 0.3 # sets the minimum confidence threshold for detections.
  NMS_MAX_OVERLAP: 0.5 #maximum allowed overlap (using Non-Maximum Suppression) between bounding boxes
  MAX_IOU_DISTANCE: 0.7 # Higher values for objects are close together but need to be treated separat
  MAX_AGE: 70 # how long a track can be kept without seeing the object in new frames
  N_INIT: 3 # number of consecutive frames that an object must be detected to initialize a new track
  NN_BUDGET: 100 #manage memory usage
```

Figure 4- 74 Parameter to Tune the Deep SORT Algorithm

Figure 4-74 shows parameters used to tune the Deep SORT algorithm to increase the accuracy of the tracking model in a yaml file.

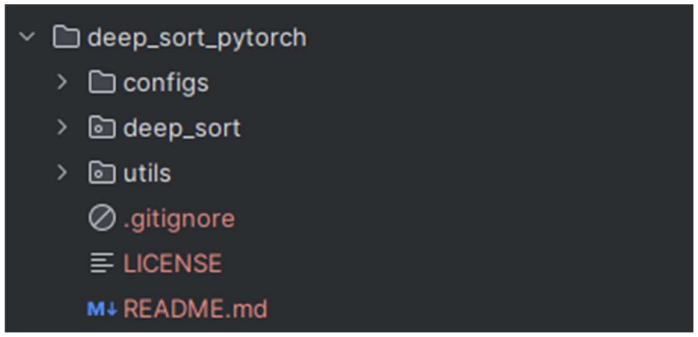


Figure 4- 75 Deep SORT PyTorch Package Files

```
from deep_sort_pytorch.utils.parser import get_config
from deep_sort_pytorch.deep_sort import DeepSort
from collections import deque
import numpy as np
```

Figure 4- 76 Import Deep SORT to Integrate with Yolov8

Figure 4-75 shows the Deep SORT package files that are downloaded to integrate with Yolov8 while Figure 4-76 shows the import of the files in the predict file.

```
Codeium: Refactor | Explain | Docstring | ×
def init_tracker():
    global deepsort
    cfg_deep = get_config()
    cfg_deep.merge_from_file("deep_sort_pytorch/configs/deep_sort.yaml")

    deepsort = DeepSort(cfg_deep.DEEPSORT.REID_CKPT,
                        max_dist=cfg_deep.DEEPSORT.MAX_DIST, min_confidence=cfg_deep.DEEPSORT.MIN_CONFIDENCE,
                        nms_max_overlap=cfg_deep.DEEPSORT.NMS_MAX_OVERLAP,
                        max_iou_distance=cfg_deep.DEEPSORT.MAX_IOU_DISTANCE,
                        max_age=cfg_deep.DEEPSORT.MAX_AGE, n_init=cfg_deep.DEEPSORT.N_INIT,
                        nn_budget=cfg_deep.DEEPSORT.NN_BUDGET,
                        use_cuda=True)
```

Figure 4- 77 Setting Parameters for Deep SORT

The parameters of the Deep SORT tracking algorithm are initialized with the parameter in the yaml file.

```
outputs = deepsort.update(xywhs, confss, oids, im0)
```

Figure 4- 78 Implement Deep SORT Algorithm

The line outputs = deepsort.update(xywhs, confss, oids, im0) is a part of the tracking process, where DeepSORT processes the detections in the frame and maintains object identities across multiple frames. Here, xywhs represents the bounding boxes in a format indicating the center coordinates, width, and height of each detected object, while confss contains the confidence scores for these detections, and oids represents the object classes (e.g., car, person, etc.). Using these inputs, along with the current image frame im0, the update function performs object matching and tracking. It compares the detected objects to those from previous frames using a combination of appearance features (extracted by a re-identification model) and motion-based predictions. If a match is found, the tracker updates the position of the object; otherwise, it may create a new track with a unique ID. Additionally, if an object has

not been matched for a set number of frames (as defined by the MAX_AGE parameter), it is removed from tracking. The result, stored in outputs, is a list of tracked objects, each with updated bounding box coordinates, a consistent identity ID, and a class ID. This process ensures that each object is assigned a unique, consistent identifier across frames, allowing for continuous and reliable multi-object tracking.

4.4.3 Parking Lot Bounding Box Drawing

```
def draw_rectangle(event, x, y, flags, param):
    global ix, iy, drawing, parking_lots, img

    if event == cv2.EVENT_LBUTTONDOWN:
        # Start drawing the rectangle
        drawing = True
        ix, iy = x, y

    elif event == cv2.EVENT_MOUSEMOVE:
        if drawing:
            # If drawing, display the rectangle being drawn
            img_copy = img.copy()
            cv2.rectangle(img_copy, (ix, iy), (x, y), (0, 255, 0), 2)
            cv2.imshow( winname: "Draw Parking Lots", img_copy)

    elif event == cv2.EVENT_LBUTTONUP:
        # Finalize the rectangle
        drawing = False
        # Ask for the name of the parking lot
        parking_lot_name = get_parking_lot_name()

        if parking_lot_name:
            # Save the parking lot data if the name is provided
            parking_lots.append([parking_lot_name, ix, iy, x, y])
            # Draw the final rectangle on the resized image
            cv2.rectangle(img, (ix, iy), (x, y), (0, 255, 0), 2)
            # Put the parking lot name above the rectangle
            cv2.putText(img, parking_lot_name,  org: (ix, iy - 10),
                        cv2.FONT_HERSHEY_SIMPLEX,  fontScale: 0.5,  color: (0, 255, 0),  thickness: 1, cv2.LINE_AA)
```

Figure 4- 79 Code Snippet for Drawing Parking Lot Bounding Box

```python
def update_database(db, parking_lots):
    """Update the parking lot data in Firebase Firestore."""
    for parking_lot in parking_lots:
        name = parking_lot[0]  # Extract the parking lot name
        # Update Firestore
        doc_ref = db.collection('ParkingLot').document(name)
        doc_ref.set({
            'level': 2,
            'vacant': True
        })
```

Figure 4- 80 Add Parking Lot to Database Collection

```python
while True:
    cv2.imshow( winname: "Draw Parking Lots", img)
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('s'):
        # Save the parking lot data to a JSON file
        try:
            with open("parking_lots.json", "w") as f:
                json.dump(parking_lots, f, indent=4)
            print(f"Saved {len(parking_lots)} parking lots to 'parking_lots.json'.")
            # Update the Firestore database
            update_database(db, parking_lots)
            print("Updated the Firebase Firestore database.")
        except Exception as e:
            print(f"Error saving parking lots: {e}")
        break
```

Figure 4- 81 Save Parking Lot Drawing and Update Database

py | val.py | tasks.py | deep_sort.yaml | .gitkeep | draw_parkng_lots.py | parking_lots.json

```
1  [["A5", 641, 116, 641, 116], ["A1", 319, 67, 418, 234], ["A3", 545, 69, 656, 239], ["A4", 652, 75, 779, 239]]
```

Figure 4- 82 Parking Lots Coordinates in parking_lots.json File

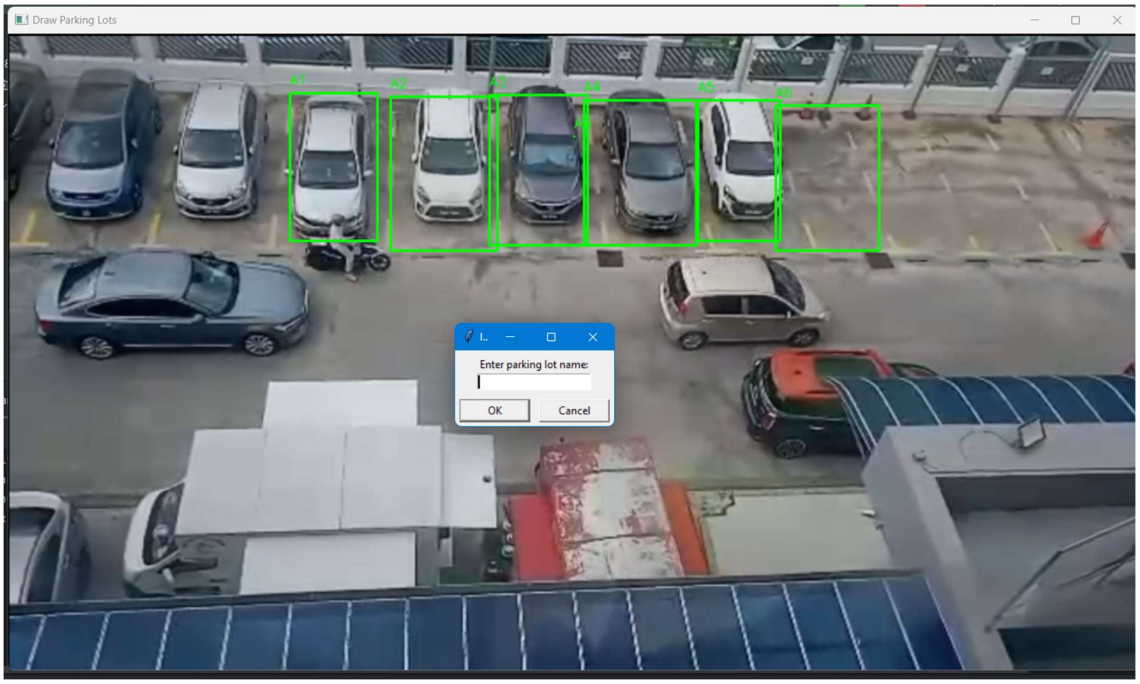


Figure 4- 83 Image Used for Drawing Parking Lots

Figure 4-79 shows the code for drawing the parking lot in an image. The image is a screenshot of the recorded video so that the parking lot drawn is accurate in coordinates as shown in Figure 4-83. When the image is opened, a parking lot can be drawn by clicking the left button and dragging the mouse. Then, a Tkinter dialog box will prompt the user to key the parking lot name. A parking lot box can removed by right clicking the box. To save the changes, the user must type ‘s’ and to discard changes the user has to type ‘q’ as shown in Figure 4-81. If the user decides to save the changes, the coordinates of the parking lots will be saved to the parking_lots.json file as shown in Figure 4-82 and the parking lots details will be updated to the database.

4.4.3 Firebase Connection (Update Vacancy Status of Parking Lots)

```
def load_parking_lots(filename):
    with open(filename, 'r') as f:
        parking_lots = json.load(f)
    return parking_lots
```

Figure 4- 84 Code Snippet to Load Parking Lot Boxes in Video

```python
def draw_boxes(img, bbox, names, object_id, identities=None, offset=(0, 0), parking_lots=[]):
    height, width, _ = img.shape

    # Draw parking lot boxes and calculate areas
    parking_lot_areas = {}
    for lot in parking_lots:
        lot_name, x1, y1, x2, y2 = lot
        # Determine color based on the vacant status
        doc = db.collection('ParkingLot').document(lot_name).get()
        if doc.exists:
            vacant_status = doc.to_dict().get('vacant', True)  # Default to True if not found
            color = (0, 255, 0) if vacant_status else (0, 0, 255)  # Green if vacant, red if not
        else:
            color = (0, 255, 0)  # Default to green if document does not exist

        cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)  # Draw in the determined color
        cv2.putText(img, lot_name, org: (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, fontScale: 0.5, color, thickness: 2)
        parking_lot_area = (x2 - x1) * (y2 - y1)  # Area of parking lot
        parking_lot_areas[lot_name] = parking_lot_area
```

Figure 4- 85 Code Snippet to Draw Parking Lot and Bounding Box on Detected Cars

```python
# Update Firestore database based on intersection areas
for lot_name, intersection_area in intersection_areas.items():
    parking_lot_area = parking_lot_areas[lot_name]
    if intersection_area / parking_lot_area >= 0.5:  # If more than 50% of the parking lot is occupied
        db.collection('ParkingLot').document(lot_name).set( document_data: {'vacant': False}, merge=True)
    else:
        db.collection('ParkingLot').document(lot_name).set( document_data: {'vacant': True}, merge=True)
```

Figure 4- 86 Code Snippet to Compare Detected Car Bounding Box and Parking Lot

The coordinates in the parking_lots.json file will be called when the object detection model starts running the video. The parking lots coordinates will be used to draw the parking lot in the video where the bounding box of the detected cars will be measured against the parking lot. If the intersection of bounding box of detected car and parking lot is more than 50%, the parking lot will be labelled as occupied and update the database accordingly.

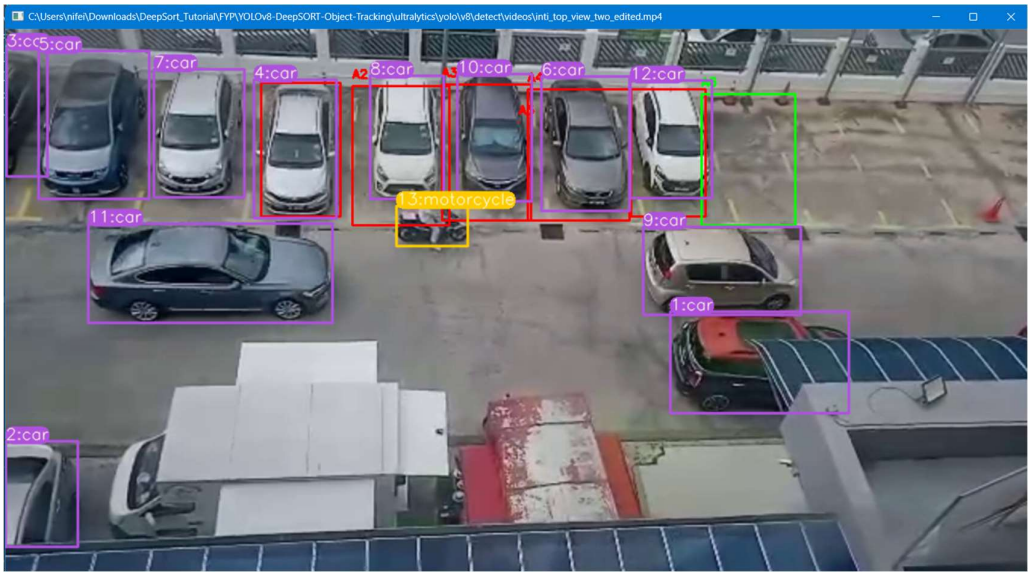


Figure 4- 87 Object Detection Model Running on Recorded Video

Figure 4-87 shows the output when the object detection model is run on the recorded video of the parking lot in INTI in front of the Level 2 lobby. The red and green bounding boxes are the parking lot boxes drawn where red means occupied and green means vacant. The model will keep comparing the bounding box of the cars and the parking lot to determine if the parking lot is occupied or not and updates the color of the parking lot boxes and the vacancy status of the parking lot in the database.

CHAPTER 5: TESTING AND EVALUATION

5.1 Introduction

This chapter outlines the systematic approach taken to verify and validate the system's functionality and performance which is the fourth phase of the Waterfall model. To achieve a comprehensive assessment, both functional and non-functional testing are conducted. The functional testing involved unit testing and integration testing, focusing on individual components and the interactions to ensure each function operates as intended. Non-functional testing is addressed through user acceptance testing, evaluating whether the overall system meets end-user expectations and usability standards. These testing are executed to identify potential issues early and confirm the system’s reliability, usability, and overall effectiveness.

5.2 Functional Testing

5.2.1 Unit Testing

The unit testing approach involves the isolation of each unit to ensure that it performs as expected under various conditions. By executing specific test cases designed for each unit, errors can be identified and rectified, enhancing the reliability and maintainability of the software. Unit testing not only fosters code quality but also facilitates smoother integration in subsequent testing phases. This section details the conducted test cases for each page within the system in tables.

5.2.1.1 Mobile Application

This section documents the unit testing carried out for the mobile application of IICP Park Finder.

5.2.1.1 (a) Login Page

Table 5. 1 Unit Testing For Login Page

Test Case	Expected Results	Screenshot	Status
Successful Login	-It should navigate user to the Parking Page.	Figure 5-1Figure 5 - 3 Validate Login Email and Password	Pass
Empty Field	- It should show error message to field in all the fields.	Figure 5-2	Pass
Incorrect Email or Password	- It should show error message that the information is incorrect.	Figure 5-3	Pass

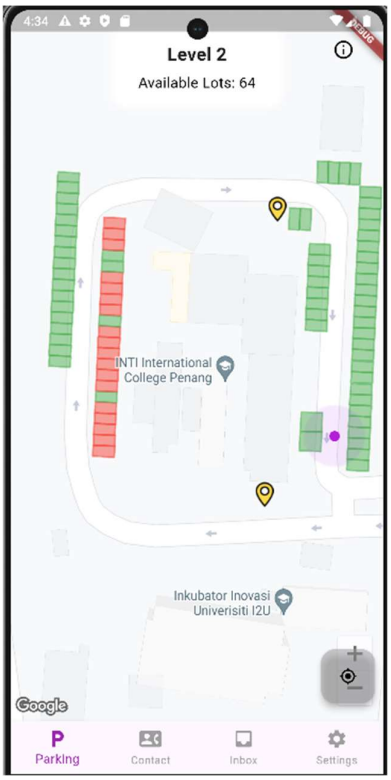


Figure 5 - 1 Load to Parking Page After Successful Login



Figure 5 - 2 Validate Login Empty Field

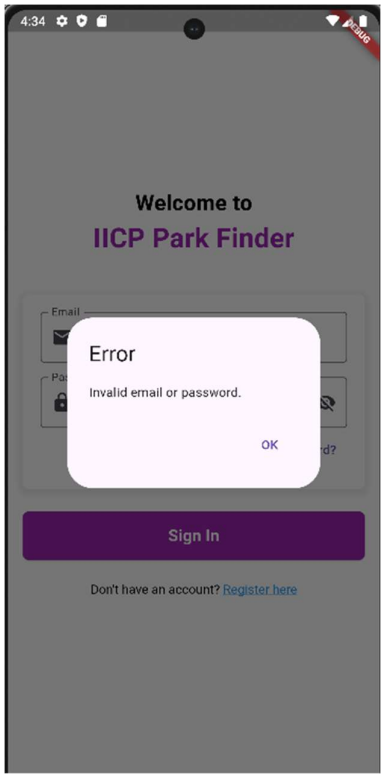


Figure 5 - 3 Validate Login Email and Password

Table 5.1 shows the unit testing for the login feature includes three essential test cases focused on functionality and user error handling. The first test case, *Successful Login*, verifies that when valid login credentials are entered, the system navigates the user to the Parking Page, which is successfully demonstrated in Figure 5-1. The second test case, *Empty Field*, checks that if any login fields are left blank, the system displays an error message instructing the user to fill in all required fields; this is confirmed in Figure 5-2. Lastly, the *Incorrect Email or Password* test case ensures that when incorrect login details are provided, an error message appears to inform the user of the mistake, as shown in Figure 5-3. All test cases have passed, indicating the login functionality works as expected and handles user errors appropriately.

5.2.1.1 (b) Register Page

Table 5. 2 Unit Testing for Register Page

Test Case	Expected Results	Screenshot	Status
Successful Register	-It should send an email verification link for user to register. User must verify account. After verify account, application show success message.	Figure 5-4 Figure 5-5 Figure 5-6 Figure 5-7	Pass
Empty Field	-It should show error message to fill in all the fields.	Figure 5-8	Pass
Not Student Email Used to Register	-It should show error message that email must be student email.	Figure 5-9	Pass
Password Do Not Match	-It should show error message that password should match	Figure 5-10	Pass
Email Account Already Registered	-It should show error message that email account already registered.	Figure 5-11	Pass
Car Plate Number Already Registered	-It should show error message that car plate number already registered.	Figure 5-12	Pass
Password must be at least 6 characters long	-It should show error message that password must be at least 6 characters long	Figure 5-13	Pass

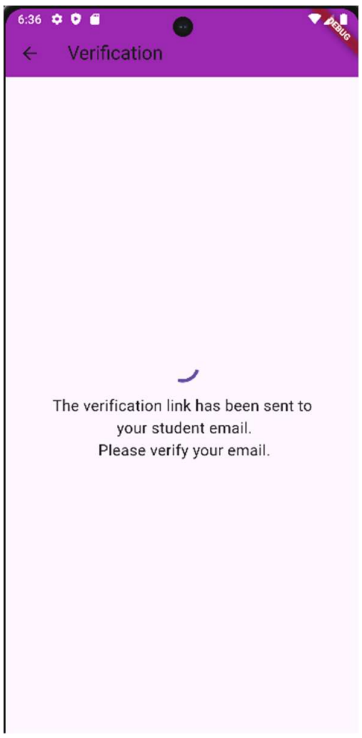


Figure 5 - 4 Registration Successful, Waiting for Verification

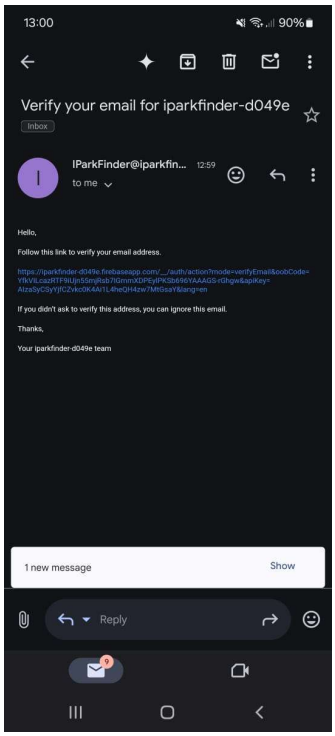


Figure 5 - 5 Verification Email for Registration



Figure 5 - 6 Confirm Email Verified

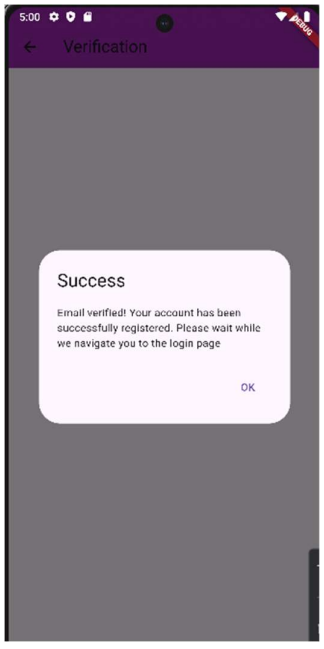


Figure 5 - 7 Account Registered Successfully

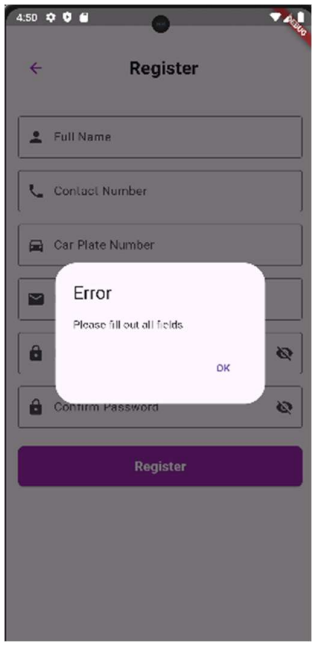


Figure 5 - 8 Reject Empty Field

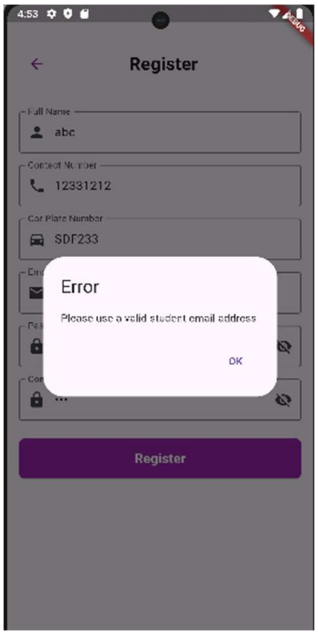


Figure 5 - 9 Reject Invalid Email Address

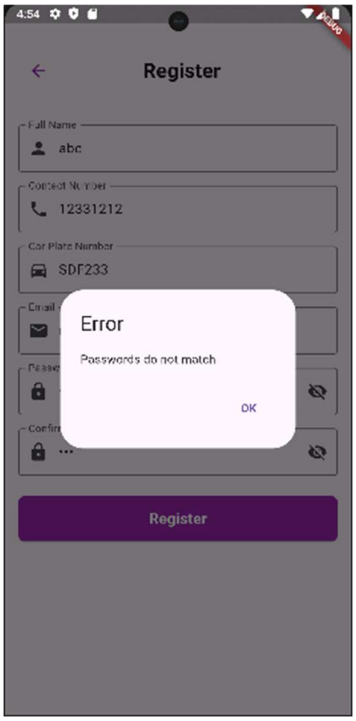


Figure 5 - 10 Reject Mismatched Password

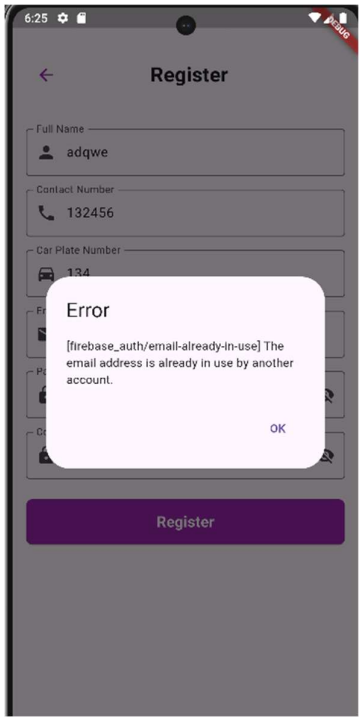


Figure 5 - 11 Reject Email Already Registered

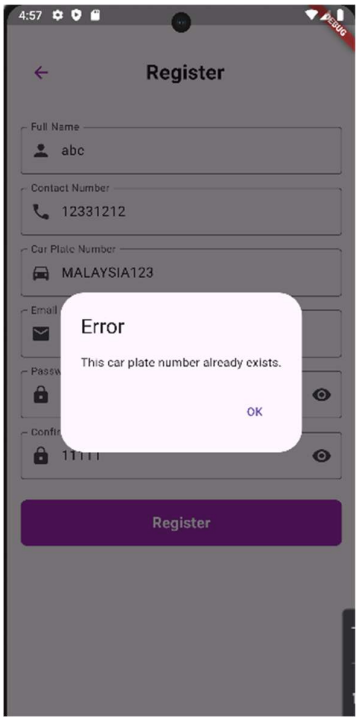


Figure 5 - 12 Reject Car Plate Already Registered

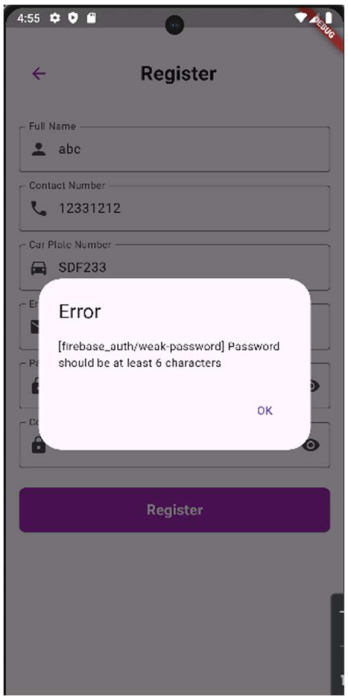


Figure 5 - 13 Reject Password Too Short

The unit testing for the registration feature includes several key scenarios to validate functionality and handle user input errors as shown in Table 5.2. In the *Successful Register* test case, when users enter valid information, the system sends an email verification link, requires account verification, and displays a success message upon completion. This process is demonstrated in Figures 5-4 through 5-7. The *Empty Field* case ensures that any blank registration fields trigger an error message prompting users to complete all fields, as seen in Figure 5-8. The system also verifies email eligibility in the *Non-Student Email Used to Register* test case; if a non-student email is used, an error message appears, as shown in Figure 5-9. Additionally, if passwords do not match, an error message alerts the user, as demonstrated in Figure 5-10. Duplicate entries are handled in the *Email Account Already Registered* and *Car Plate Number Already Registered* cases, where error messages inform users of already-registered details, illustrated in Figures 5-11 and 5-12, respectively. Finally, the system enforces a minimum password length of six characters; if this requirement is not met, an error message appears, shown in Figure 5-13. All test cases have passed, confirming the registration feature's effectiveness in enforcing data requirements and guiding users through error corrections as needed.

5.2.1.1 (c) Forgot Password Page

Table 5. 3 Unit Testing for Forgot Password Page

Test Case	Expected Results	Screenshot	Status
Successful Change Password	-It should send a reset password link to user email. User must reset the password in the link.	Figure 5-14 Figure 5-15 Figure 5-16 Figure 5-17	Pass
Empty Field	-It should show error message to fill in the field.	Figure 5-18	Pass

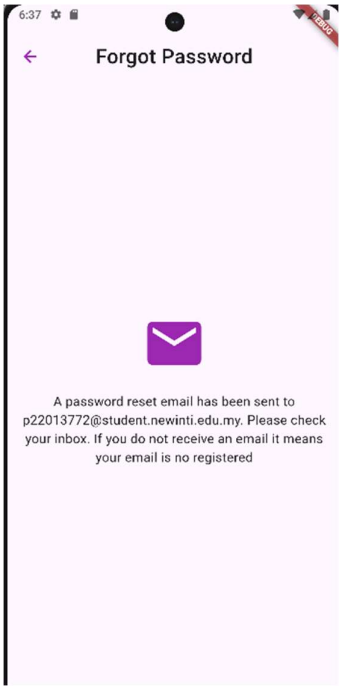


Figure 5 - 14 Navigate to Waiting Page for User to Reset Email

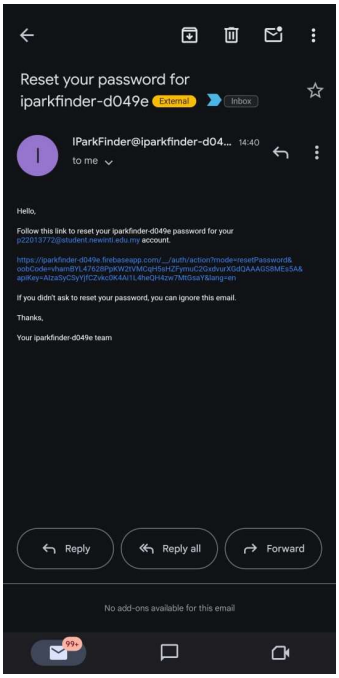


Figure 5 - 15 Email Sent to Reset Password



Figure 5 - 16 Reset Password Page from Firebase



Figure 5 - 17 Password Reset Successful



Figure 5 - 18 Reject Empty Field

The unit testing for the *Change Password* feature is shown in table 5.3 which includes two primary scenarios to ensure both functionality and proper error handling. In the *Successful*

Change Password test case, when a user requests a password reset, the system sends a reset link to their email. The user is then required to click the link and reset their password, with this flow demonstrated across Figures 5-14 to 5-17. For the *Empty Field* scenario, if any required fields are left blank during the reset process, an error message instructs the user to fill in the missing information, as shown in Figure 5-18. Both test cases passed, verifying that the *Change Password* feature operates as expected and provides clear guidance for users when errors occur

5.2.1.1 (d) Parking Page

Table 5. 4 Unit Testing for Parking Page

Test Case	Expected Results	Screenshot	Status
View Vacant and Occupied Spaces	-It should show the parking lots with green and red color to indicate vacant and occupied space.	Figure 5-19	Pass
View Number of Available Lots	-It should show the number of available lots on campus.	Figure 5-19	Pass
Navigate to Parking Lot	-It should show confirmation dialog to navigate user to the selected parking lot. If parking lot is occupied, a snackbar should display stating the parking lot is occupied. Navigation starts when user confirms to navigate.	Figure 5-20 Figure 5-21 Figure 5-22	Pass
Stop Navigation	- It should show the confirmation dialog to stop the navigation when user tap on the red button. Navigation should stop when user confirms.	Figure 5-23 Figure 5-24	Pass
Navigate to Nearest Parking Lot of an Entrance.	-It should show the confirmation dialog to navigate to the nearest vacant parking lot to the entrance. Navigation to the parking lot starts when the user confirms.	Figure 5-25	Pass
Recenter Button When Not Navigating	-It should show the map with INTI International College Penang in the center.	Figure 5-26 Figure 5-27	Pass
Recenter Button When Navigating	-It should show the map with user location in the center.	Figure 5-28 Figure 5-19	Pass

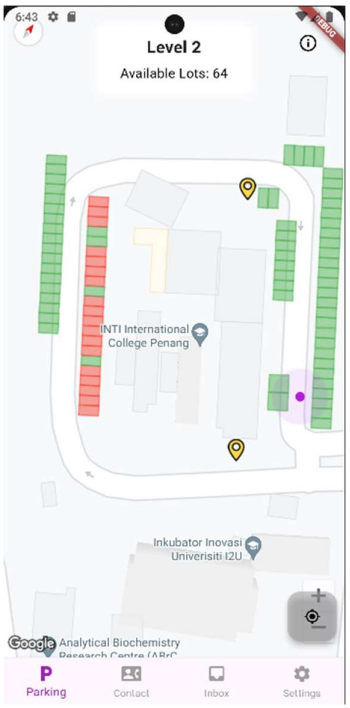


Figure 5 - 19 Show Vacant and Occupied Parking Lots, and Available Lots in Parking Page

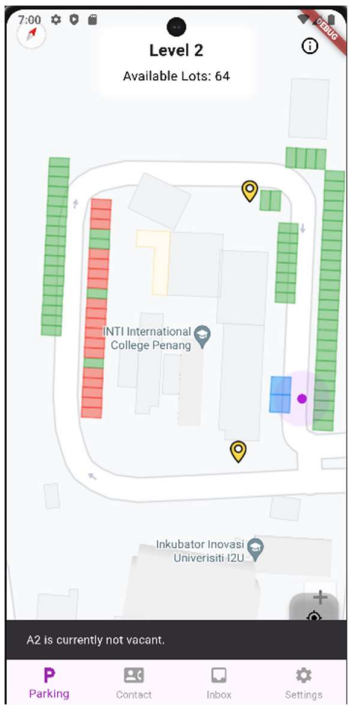


Figure 5 - 20 Snackbar Shown When Parking Lot Chosen is Occupied

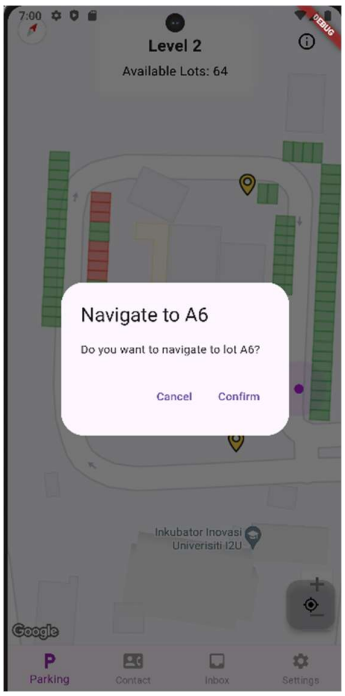


Figure 5 - 21 Confirm Navigation

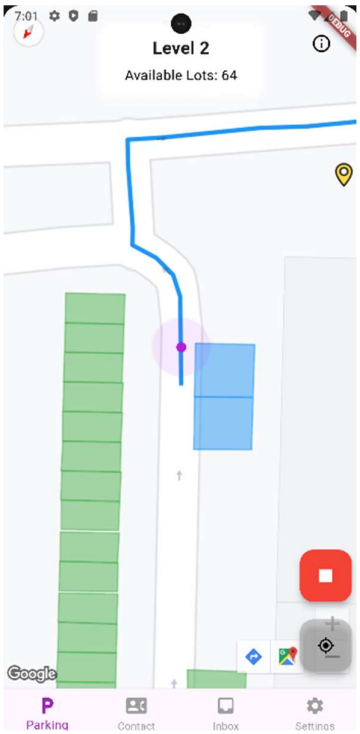


Figure 5 - 22 Start Navigation

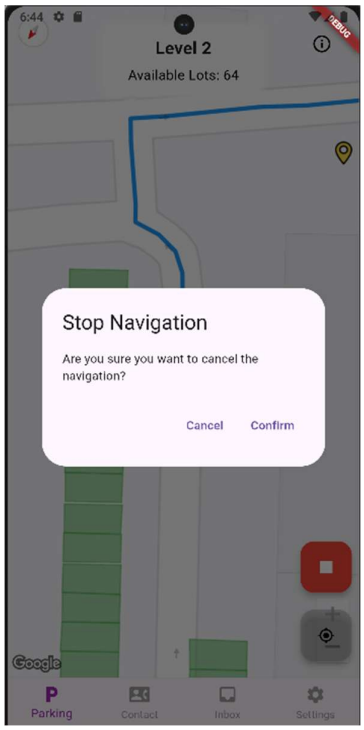


Figure 5 - 23 Confirm Stop Navigation

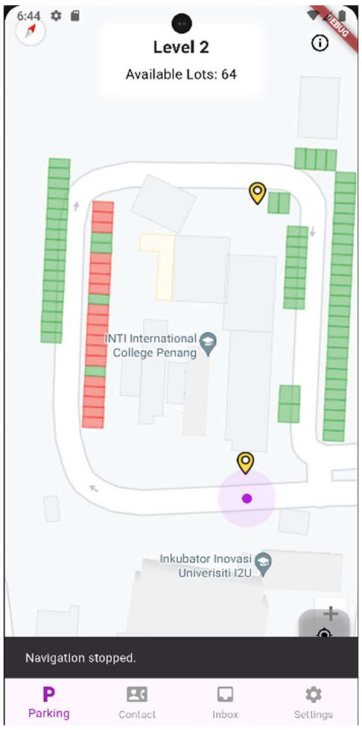


Figure 5 - 24 Navigation Stops and Snackbar Shown

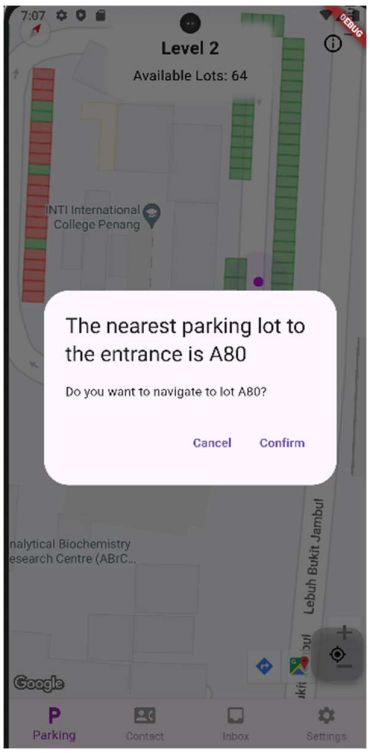


Figure 5 - 25 Confirm to Navigate to Nearest rking Lot of Chosen Entrance

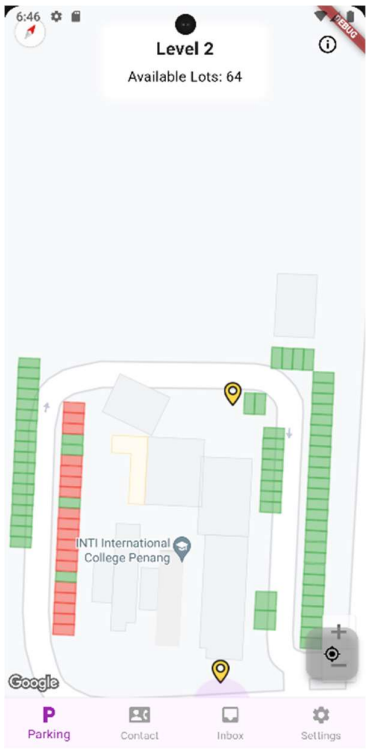


Figure 5 - 26 Map Not Centered

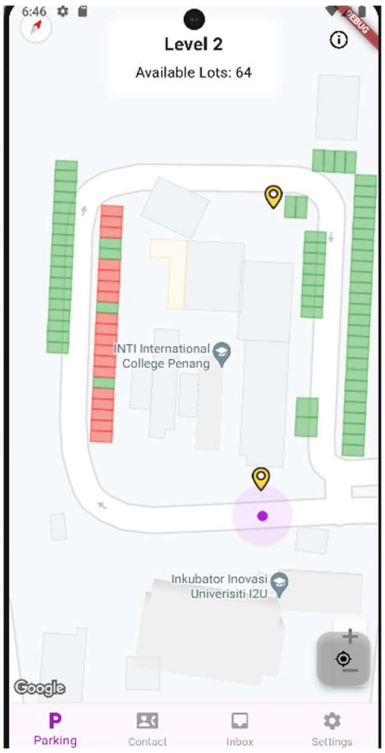


Figure 5 - 27 Map After Tapping on Recenter Button

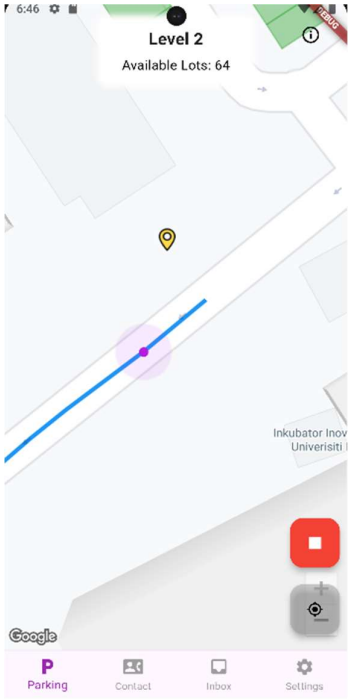


Figure 5 - 28 Map Not Centered When Navigating

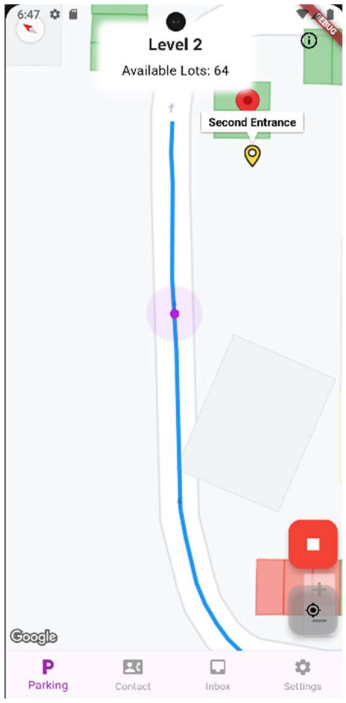


Figure 5 - 29 Map After Tapping on Recenter Button When Navigating

The unit testing for parking space functionality covers various scenarios to ensure accurate visualization and navigation within the parking system as shown in Table 5.4. The *View Vacant and Occupied Spaces* test case verifies that the parking lots display with green and red colors to indicate vacant and occupied spaces, as shown in Figure 5-19. The *View Number of Available Lots* test case ensures the system shows the total number of available parking lots on campus, also demonstrated in Figure 5-19. For *Navigate to Parking Lot*, a confirmation dialog prompts the user to navigate to a selected parking lot, displaying a snackbar notification if the lot is occupied, as seen in Figures 5-20 through 5-22. The *Stop Navigation* test case confirms that a dialog appears, allowing the user to stop navigation upon tapping the red button, as shown in Figures 5-23 and 5-24. The *Navigate to Nearest Parking Lot of an Entrance* test case verifies that the system displays a confirmation dialog for directing users to the nearest vacant parking lot to an entrance, seen in Figure 5-25. The *Recenter Button When Not Navigating* test case checks that the map centers on INTI International College Penang, illustrated in Figures 5-26 and 5-27. Meanwhile, the *Recenter Button When Navigating* case ensures the map centers on the user's current location during navigation, as shown in Figures 5-28 and 5-19. All test cases passed, confirming that the parking functionality works as intended, providing clear visual cues and user-friendly navigation controls.

5.2.1.1 (e) Contact Page

Table 5. 5 Unit Testing for Contact Page

Test Case	Expected Results	Screenshot	Status
Toggle Show Contact Number	-It should update the toggle UI and update the show value in database to true.	Figure 5-30 Figure 5-31	Pass
Toggle Hide Contact Number	-It should update the toggle UI and update the show value in database to false.	Figure 5-32 Figure 5-33	Pass
Search Car Plate When Field Empty	- It should show error message to prompt enter a car plate number in the field.	Figure 5-34	Pass
Car Plate Number Does Not Exist in Database	- It should show error message stating no user found for car plate number.	Figure 5-35	Pass
Car Plate Number is Hidden By Owner	- It should show error message user does not have contact info visible.	Figure 5-36	Pass
Find Contact of Car Plate Number Success	- It should show a dialog box with the contact number of the owner of the vehicle allowing the user to call the number. The phone	Figure 5-37 Figure 5-38	Pass

	app will launch when the user decides to call.		

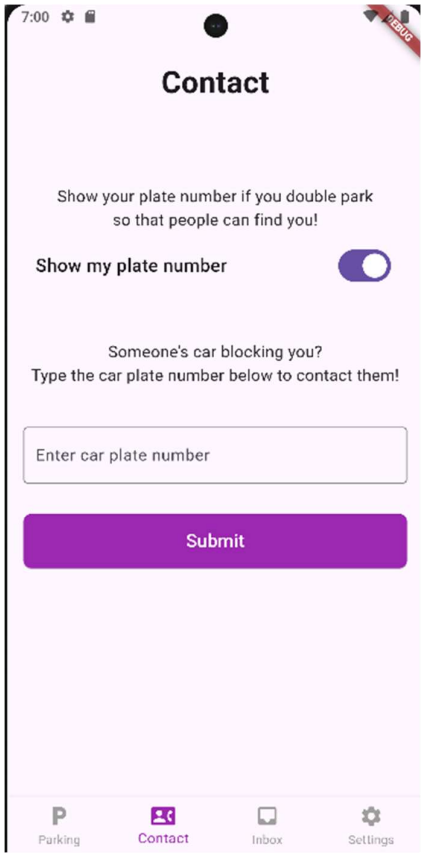


Figure 5 - 30 Toggle Update When Show

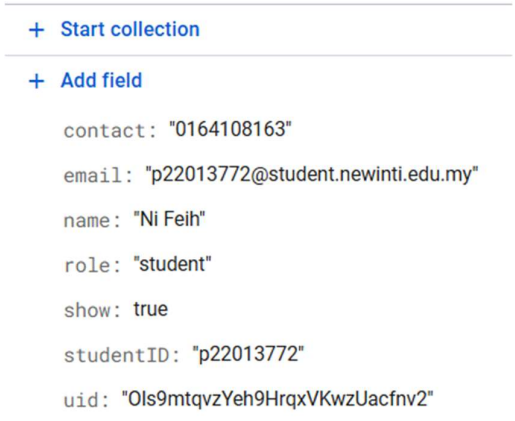


Figure 5 - 31 Show Field in Database Update to True

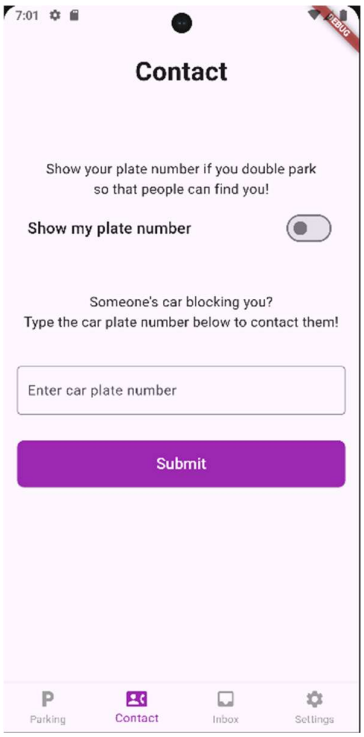


Figure 5 - 32 Toggle UI Update to Hide Car Plate Number



Figure 5 - 33 Show Field in Database Update to False

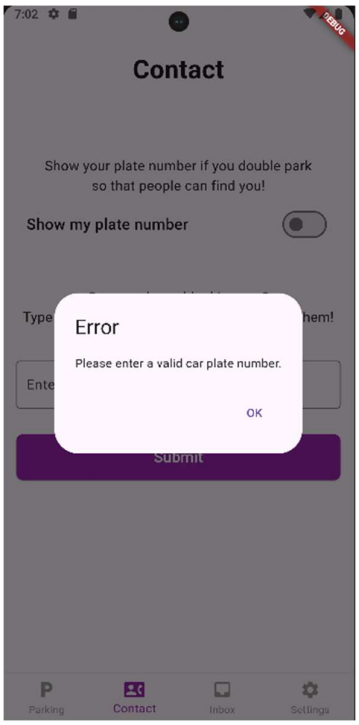


Figure 5 - 34 Reject Empty Field

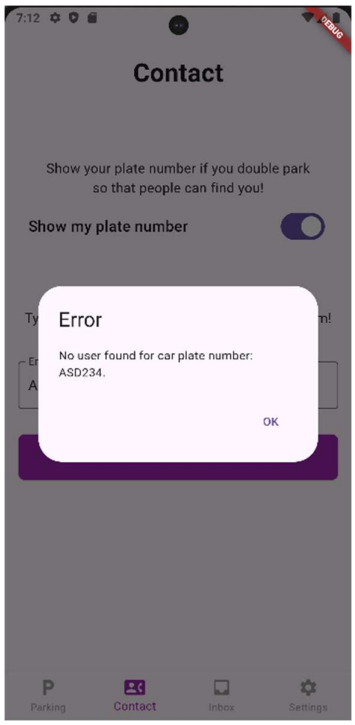


Figure 5 - 35 Car Plate Number Does Not Exist

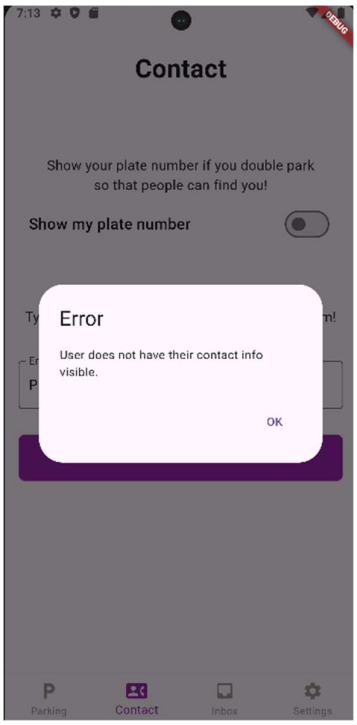


Figure 5 - 36 Owner Hid Car Plate Number

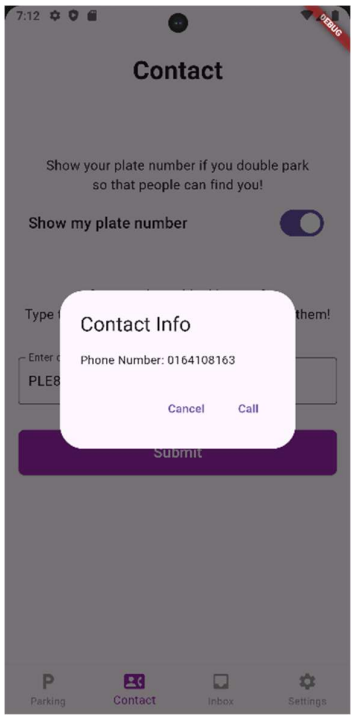


Figure 5 - 37 Car Plate Number Found

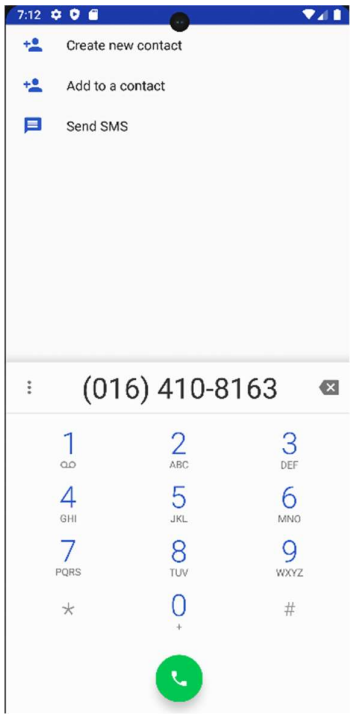


Figure 5 - 38 Call Car Plate Number

The unit testing for contact visibility and search functionality includes several key scenarios to ensure the toggle and search features operate smoothly as tabulated in Figure 5.5. In the *Toggle Show Contact Number* test case, the UI toggle updates to display the contact number, setting the "show" value in the database to true, as illustrated in Figures 5-30 and 5-31. Conversely, the *Toggle Hide Contact Number* test case hides the contact number by updating the toggle UI and setting the "show" value in the database to false, shown in Figures 5-32 and 5-33. When attempting to *Search Car Plate When Field is Empty*, the system displays an error message prompting the user to enter a car plate number, as seen in Figure 5-34. If the *Car Plate Number Does Not Exist in Database*, an error message notifies the user that no owner information is available, as illustrated in Figure 5-35. For cases where the *Car Plate Number is Hidden by Owner*, an error message informs the user that the owner's contact information is not visible, shown in Figure 5-36. Finally, the *Find Contact of Car Plate Number Success* test case successfully retrieves the contact information, displaying it in a dialog box with an option to call, which launches the phone app upon selection, as seen in Figures 5-37 and 5-38. All test cases passed, confirming the reliability of the contact toggle and search features.

5.2.1.1 (f) Inbox Page

Table 5. 6 Unit Testing for Inbox Page

Test Case	Expected Results	Screenshot	Status
View List of Notifications	-It should show the list of notifications received by the user.	Figure 5-39	Pass
View Details of Notifications	-It should navigate to show the details of the notification tapped by user.	Figure 5-40	Pass
Notification List Color Update	-Notifications that have been viewed should be grey and not viewed should be white.	Figure 5-41	Pass

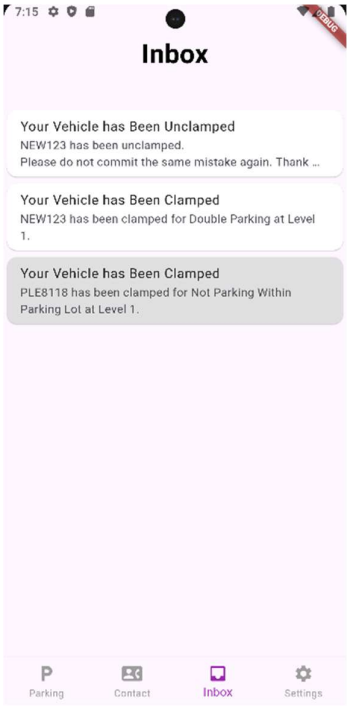


Figure 5 - 39 Show List of Notifications For User

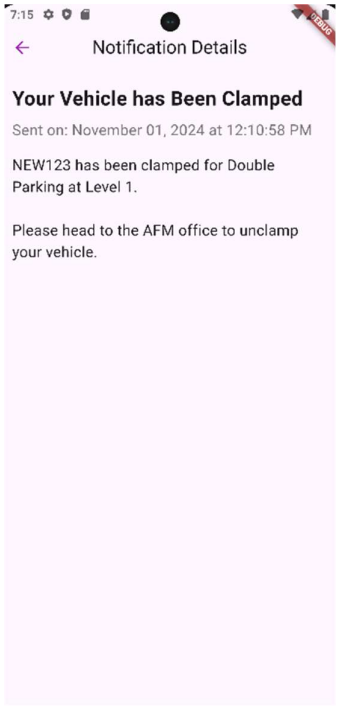


Figure 5 - 40 Show Details of Notification Selected by User

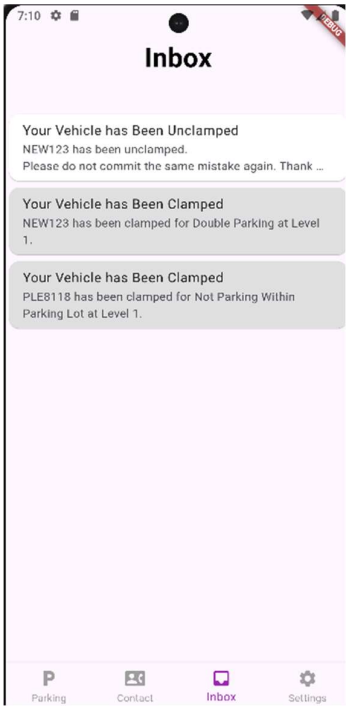


Figure 5 - 41 List Updates Color After Reading the Notification

The unit testing seen in Table 5.6 is for the notification feature which includes three main scenarios to ensure users can effectively view and interact with notifications. The *View List of Notifications* test case confirms that the system displays all notifications received by the user, as shown in Figure 5-39. In the *View Details of Notifications* test case, selecting a notification navigates the user to a detailed view, demonstrated in Figure 5-40. Additionally, the *Notification List Color Update* test case ensures that viewed notifications appear in grey, while unviewed notifications remain white, as illustrated in Figure 5-41. All test cases passed,

verifying that the notification feature functions correctly and provides clear visual cues for viewed and unviewed notifications.

5.2.1.1 (g) Profile Page

Table 5. 7 Unit Testing for Profile Page

Test Case	Expected Results	Screenshot	Status
View Name, Email, and Contact Number of User	-It should show the name, email and contact number of user.	Figure 5-42	Pass
Edit Contact Number	-It should save the contact number and display correctly after editing the number.	Figure 5-43 Figure 5-45	Pass
Edit Name	-It should save the name and display correctly after editing the name.	Figure 5-44 Figure 5-45	Pass

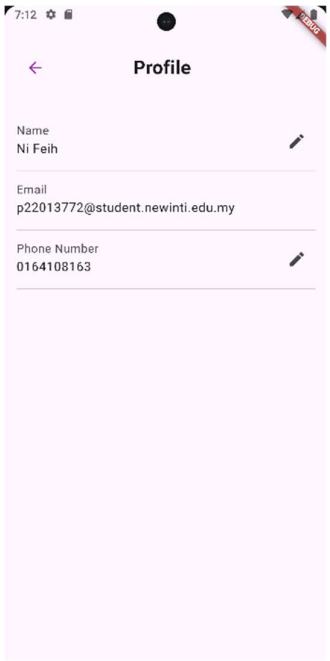


Figure 5 - 42 Show Details of User Profile Detail

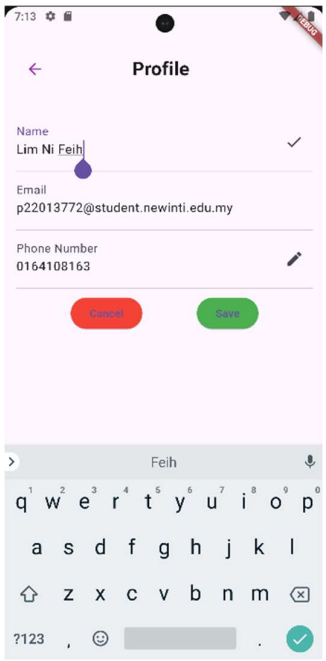


Figure 5 - 43 Edit Name

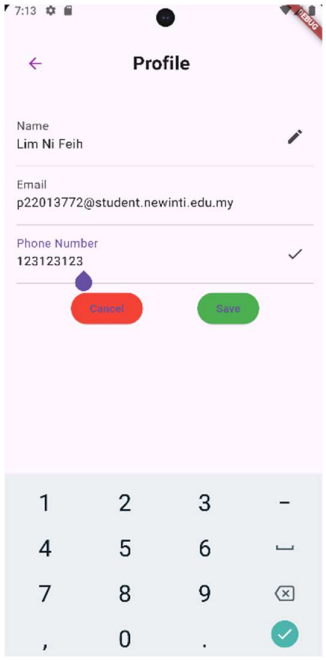


Figure 5 - 44 Edit Phone Number

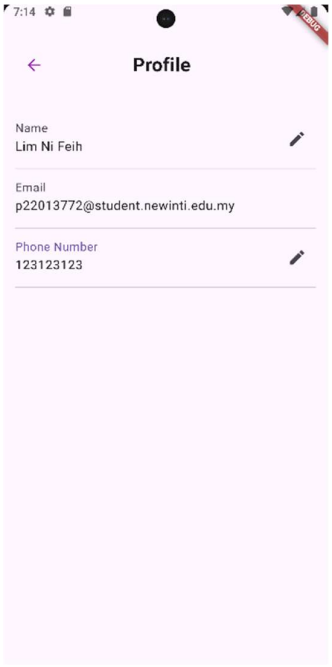


Figure 5 - 45 Profile Changes Saved Successfully

Table 5.7 shows the unit testing for the user profile information feature including three key scenarios to confirm accurate display and edit functionality. In the *View Name, Email, and Contact Number of User* test case, the system displays the user's name, email, and contact number, as shown in Figure 5-42. The *Edit Contact Number* test case verifies that, upon editing, the new contact number is saved and displayed correctly, demonstrated in Figures 5-43 and 5-45. Similarly, the *Edit Name* test case ensures that when the name is edited, the updated name is saved and displayed accurately, as seen in Figures 5-44 and 5-45. All test cases passed, confirming that the profile information display and editing features function as expected.

5.2.1.1 (h) Change Password Page

Table 5. 8 Unit Testing for Change Password Page

Test Case	Expected Results	Screenshot	Status
Fields Empty	-It should show error message to fill in all the fields.	Figure 5-46	Pass
Old Password is Incorrect	-It should show error message that the old password keyed in is incorrect.	Figure 5-47	Pass
New Password is not at least 6 Characters Long	-It should show error message that the new password must be at least 6 characters long.	Figure 5-48	Pass

Confirm Password and New Password Does Not Match	-It should show error message that the new and confirm password do no match.	Figure 5-49	Pass
Password Saved Correctly	-It should show success message.	Figure 5-50	Pass

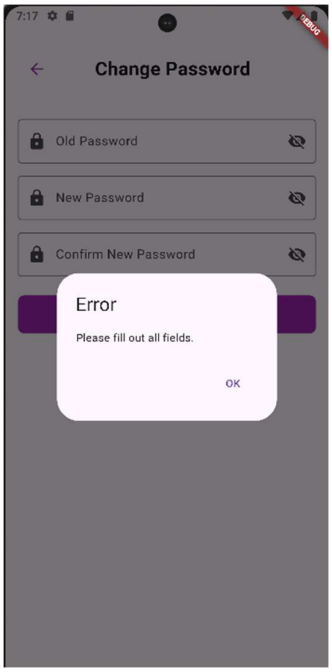


Figure 5 - 46 Reject Empty Field

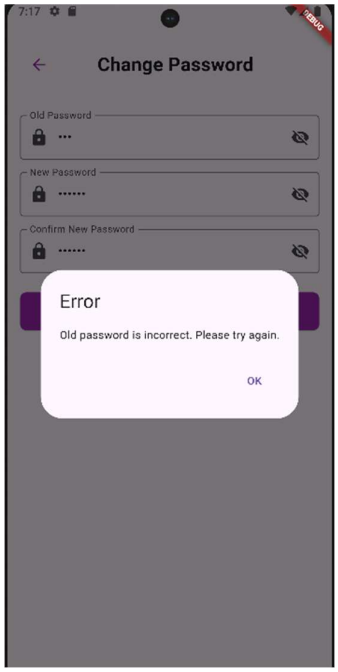


Figure 5 - 47 Reject Incorrect Current Password

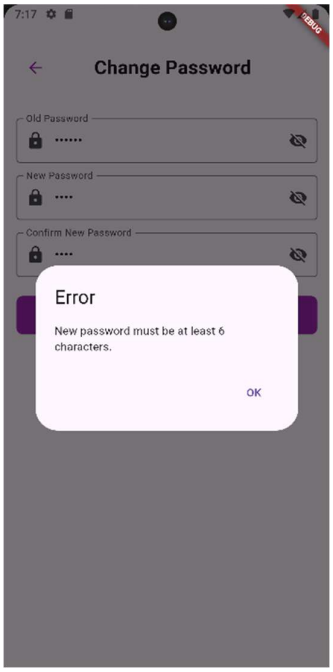


Figure 5 - 48 Reject New Password Shorter Than 6 Characters

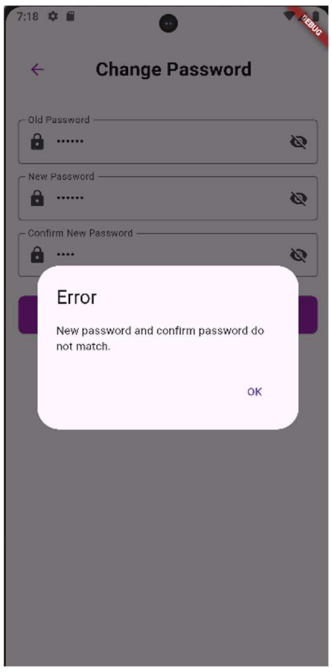


Figure 5 - 49 Reject Mistmatched Password

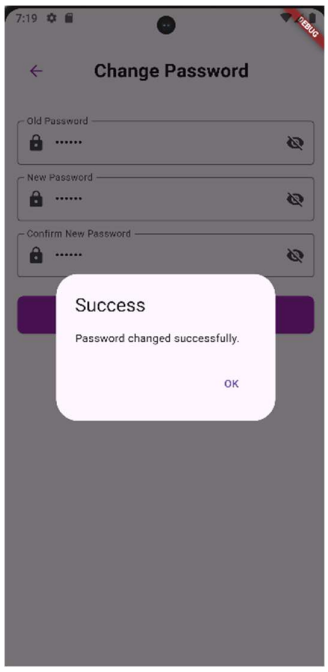


Figure 5 - 50 Password Changes Successfully

The unit testing for the password update feature includes five key scenarios to ensure proper validation and saving of the new password as shown in Table 5.8. The *Fields Empty* test case checks that if any required fields are left blank, an error message prompts the user to complete all fields, as seen in Figure 5-46. In the *Old Password is Incorrect* test case, an error message informs the user if the old password entered is incorrect, demonstrated in Figure 5-47. The *New Password is Not at Least 6 Characters Long* test case ensures that an error message appears if the new password does not meet the six-character minimum, shown in Figure 5-48. Additionally, if the *Confirm Password and New Password Do Not Match*, an error message alerts the user, as illustrated in Figure 5-49. Finally, in the *Password Saved Correctly* test case, a success message confirms that the new password has been saved, as shown in Figure 5-50. All test cases passed, confirming the password update feature's functionality and error handling.

5.2.1.1 (i) Car Plate Numbers Page

Table 5. 9 Unit Testing for Car Plate Numbers Page

Test Case	Expected Results	Screenshot	Status
View List of Car Plate Numbers	-It should show the list of car plate numbers owned by user.	Figure 5-51	Pass
Edit Car Plate Number	-It should show a dialog box allowing user to edit the car	Figure 5-52 Figure 5-53	Pass

	plate number. Car plate number edited should show the new value.		
New Car Plate Number Already Exist	-It should show error message that car plate number already exist.	Figure 5-54	Pass
Delete Car Plate Number	-It should show a dialog box prompting the user to confirm deleting the car plate number.	Figure 5-55 Figure 5-56	Pass
Car Plate Number Clamped	-Car plate number should be red in color with '(Clamped)' next to the number. -Should show error message if user tries to edit or delete the car plate number.	Figure 5-51 Figure 5-57	Pass
Add Car Plate Number	-It should navigate user to the Add Car Plate Number Page.	Figure 5-58	Pass
Car Plate Number Added Already Exist	-It should show error message that car plate number already exist.	Figure 5-59	Pass
Add Car Plate Number Success	-It should show a success message that car plate number is added.	Figure 5-60	Pass



Figure 5 - 51 List of Car Plate Numbers Owned by User

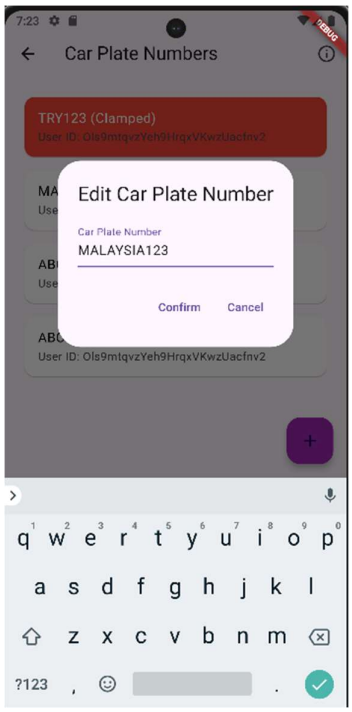


Figure 5 - 52 Edit Car Plate Number (MALAYSIA123)



Figure 5 - 53 List After 'MALAYSIA123' Edited to 'MALAYSIA'

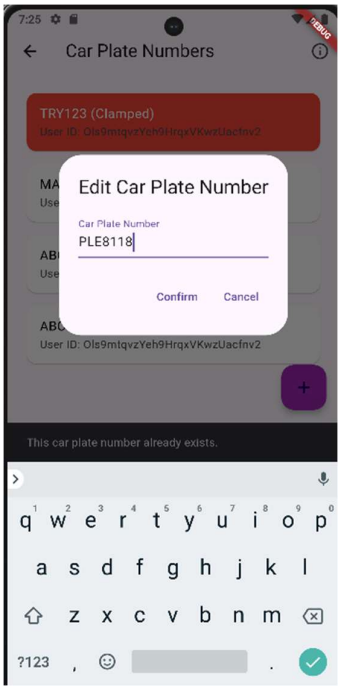


Figure 5 - 54 Snackbar Shown When User Key In Existing Car Plate Number

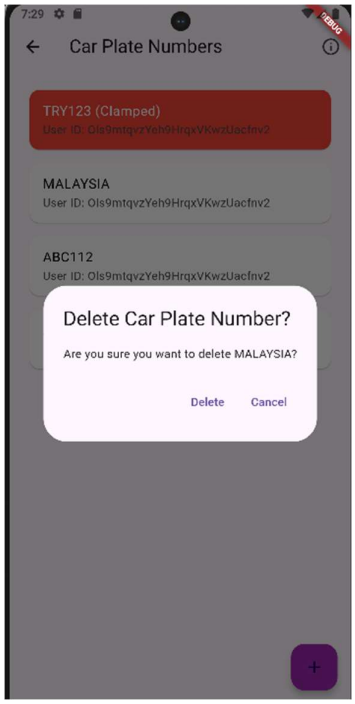


Figure 5 - 55 Delete Car Plate Number



Figure 5 - 56 List After 'MALAYSIA' Deleted

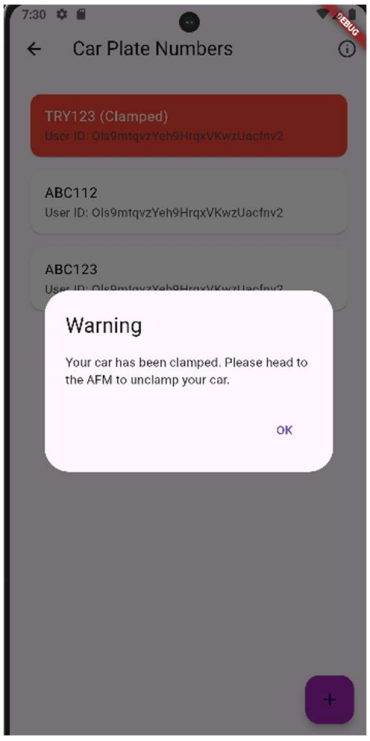


Figure 5 - 57 Warning When User Edit or Delete Clamped Car Plate Number

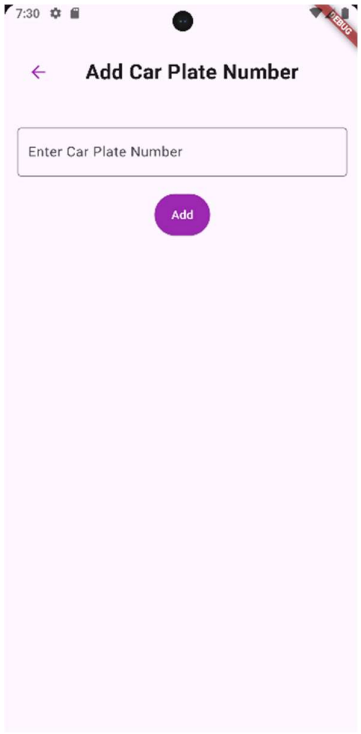


Figure 5 - 58 Navigate to Add Car Plate Number Page

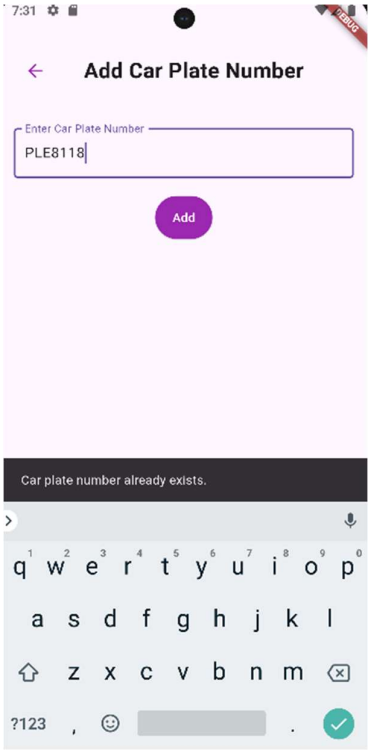


Figure 5 - 59 Error When Car Plate Number Added Already Exist



Figure 5 - 60 List Updated After 'MALAYSIA123' Is Added

The unit testing for managing car plate numbers covers a range of actions to ensure proper display, editing, deletion, and addition of car plate numbers as displayed in Table 5.9. In the *View List of Car Plate Numbers* test case, the system displays all car plate numbers owned by the user, as shown in Figure 5-51. The *Edit Car Plate Number* test case allows users to

modify a car plate number through a dialog box, with the new value displayed upon saving, demonstrated in Figures 5-52 and 5-53. For the *New Car Plate Number Already Exists* scenario, an error message alerts the user if the car plate number is a duplicate, as seen in Figure 5-54. The *Delete Car Plate Number* test case provides a confirmation dialog before deleting a car plate number, shown in Figures 5-55 and 5-56. When a *Car Plate Number is Clamped*, the number appears in red with "(Clamped)" displayed next to it, and attempts to edit or delete it result in an error message, illustrated in Figures 5-51 and 5-57. For adding a car plate number, the *Add Car Plate Number* case navigates the user to the add page, as seen in Figure 5-58, while *Car Plate Number Added Already Exists* displays an error for duplicates, shown in Figure 5-59. Finally, *Add Car Plate Number Success* confirms successful addition with a success message, demonstrated in Figure 5-60. All tests passed, verifying the accuracy and reliability of car plate number management functionalities.

5.2.1.1 (j) Contact Us Page

Table 5. 10 Unit Testing for Contact Us Page

Test Case	Expected Results	Screenshot	Status
View Details of AFM department	-It should show the helpdesk, email, and operation hours details.	Figure 5-61	Pass
Call AFM number	-It should navigate to phone app with AFM number.	Figure 5-62	Pass
Copy AFM Email Address	-It should copy email to clipboard when long press.	Figure 5-63	Pass

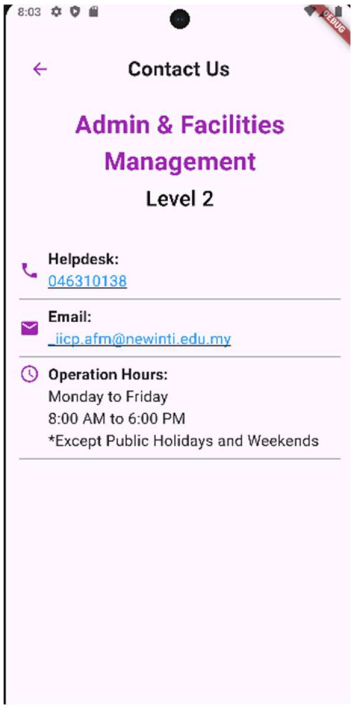


Figure 5 - 61 Show AFM Details

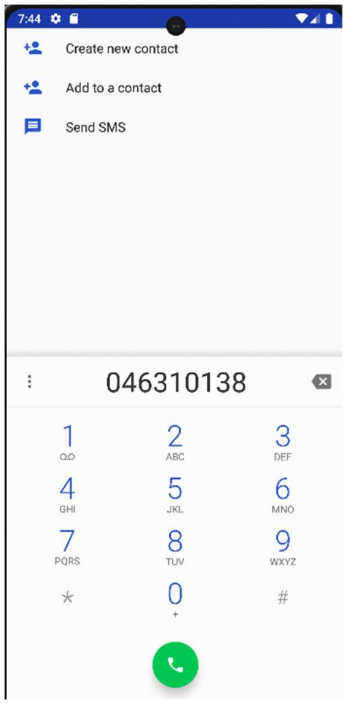


Figure 5 - 62 Launch Phone App With AFM Number

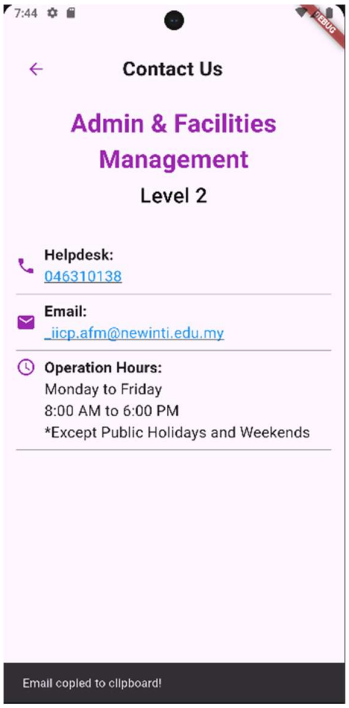


Figure 5 - 63 Email Copied to Clipboard After Long Press

The unit testing for the Contact Us page includes three key scenarios to ensure accessible and interactive contact options as tabulated in Table 5.10 . In the *View Details of AFM Department* test case, the system displays the helpdesk contact, email, and operation hours information, as shown in Figure 5-61. The *Call AFM Number* test case verifies that selecting the phone option navigates the user to the phone app with the AFM number ready for dialing, demonstrated in Figure 5-62. Lastly, the *Copy AFM Email Address* test case ensures that a long press on the email address copies it to the clipboard, as illustrated in Figure 5-63. All test cases passed, confirming the feature's functionality in providing contact details and easy interaction options for users.

5.2.1.1 (k) About IICP Park Finder Page

Table 5. 11 Unit Testing for About IICP Park Finder Page

Test Case	Expected Results	Screenshot	Status
View Terms and Conditions	-It should show the Terms and Conditions for using the mobile application.	Figure 5-64	Pass
Scroll the Terms and Conditions	-It should allow user to scroll the text.	Figure 6-65	Pass

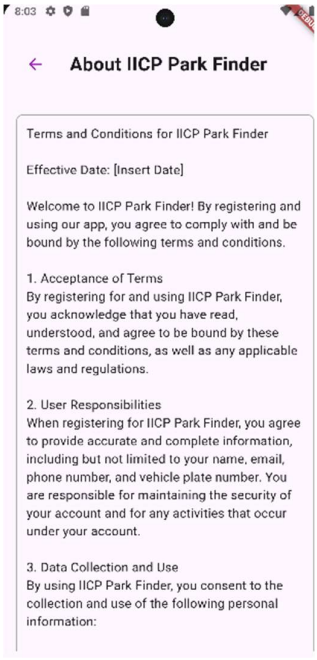


Figure 5 - 64 Show Terms and Conditions

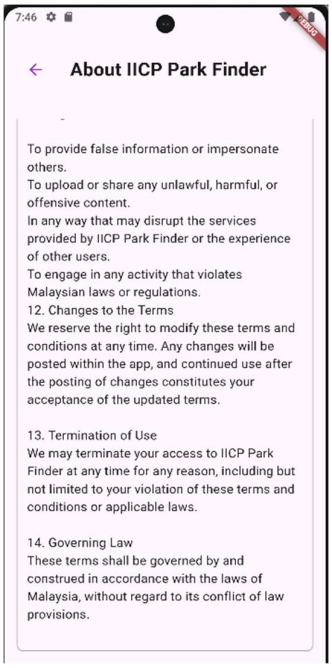


Figure 5 - 65 Scroll Terms and Conditions to the Bottom

The unit testing for the About IICP Park Finder page covers two scenarios to ensure readability and accessibility as shown in Table 5.11. The *View Terms and Conditions* test case verifies that the full text of the Terms and Conditions is displayed for users, as shown in Figure 5-64. The *Scroll the Terms and Conditions* test case ensures that users can scroll through the text, allowing easy reading of the content, as demonstrated in Figure 6-65. Both test cases passed, confirming the feature's effectiveness in providing accessible Terms and Conditions.

5.2.1.1 (l) Logout

Table 5. 12 Unit Testing for Logout

Test Case	Expected Results	Screenshot	Status
Confirmation Dialog to Log Out	-It should show the confirmation dialog to prompt the user whether to log out or not.	Figure 5-66	Pass

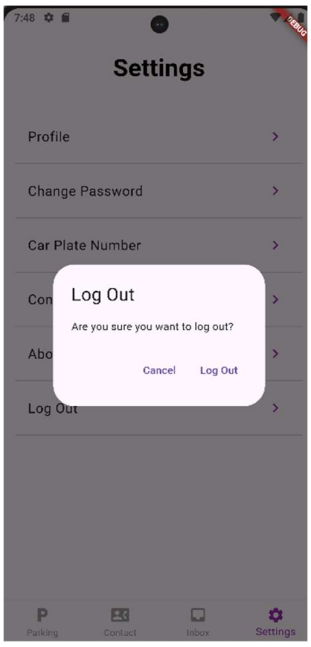


Figure 5 - 66 Prompt User to Confirm Logout

The unit testing for the logout feature includes a single test case, *Confirmation Dialog to Log Out*, which ensures that a confirmation dialog appears to prompt the user to confirm or cancel the logout action. This functionality is demonstrated in Figure 5-66, and the test case passed, verifying the logout feature's user confirmation process works as intended.

5.2.1.2 Web Application

This section documents the unit testing carried out for the web application of IICP Park Finder which only allows Admin access.

5.2.1.2 (a) Login Page

Table 5. 13 Unit Testing for Web App Login Page

Test Case	Expected Results	Screenshot	Status
Login Success	-It should show a success message when login is successful.	Figure 5-67	Pass
Incorrect Email or Password	-It should show an error message that email or password is incorrect	Figure 5-68	Pass
User Not Admin	-It should show error message that the user is not an admin	Figure 5-69	Pass

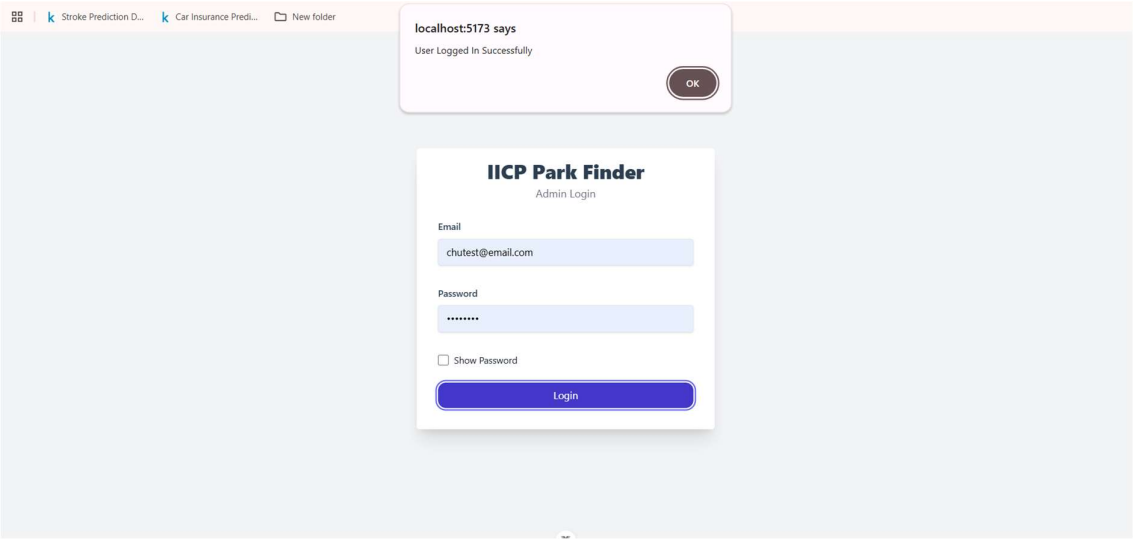


Figure 5 - 67 Successful Admin Login

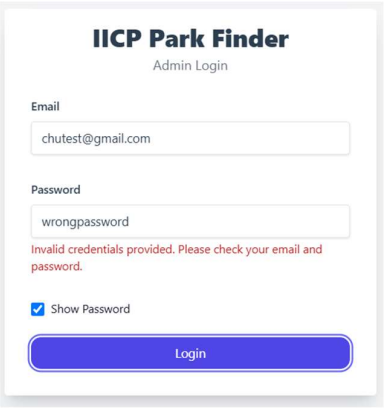


Figure 5 - 68 Reject Incorrect Password and Email

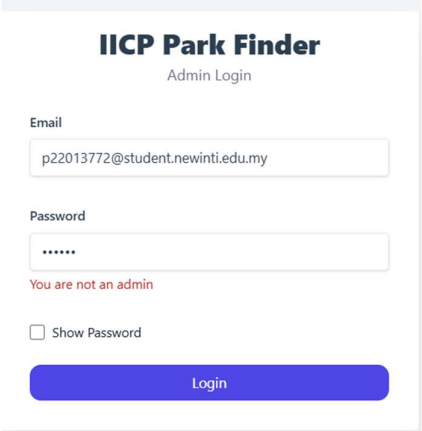


Figure 5 - 69 Reject Unauthorised User

The unit testing for the login feature in the web application includes three scenarios to ensure appropriate feedback for different login outcomes as tabulated in Table 5.13. The *Login Success* test case verifies that a success message is displayed upon successful login, as

shown in Figure 5-67. In the *Incorrect Email or Password* scenario, an error message informs the user if the email or password is incorrect, as illustrated in Figure 5-68. Lastly, the *User Not Admin* test case ensures that a non-admin user attempting to access an admin area receives an error message indicating their lack of admin privileges, shown in Figure 5-69. All test cases passed, confirming the login feature's effectiveness in handling various login scenarios with clear user feedback.

5.2.1.2 (b) Parking Statistics Page

Table 5. 14 Unit Testing for Web Parking Statistics Page

Test Case	Expected Results	Screenshot	Status
View Parking Lot Statistics Today	-It should show the line graph for available parking lots for today.	Figure 5-70	Pass
View Parking Lot Statistics for Chosen Date	- It should show the line graph for available parking lots for selected date.	Figure 5-71 Figure 5-72	Pass
View Parking Lot Statistics for This Month	-It should show the line graph of average available parking lot of everyday for this month.	Figure 5-73	Pass
View Parking Lot Statistics for Selected Month and Year	-It should show the line graph of average available parking lot of everyday for selected month and year.	Figure 5-74	Pass



Figure 5 - 70 Show Line Graph for Available Parking Lots of Current Date

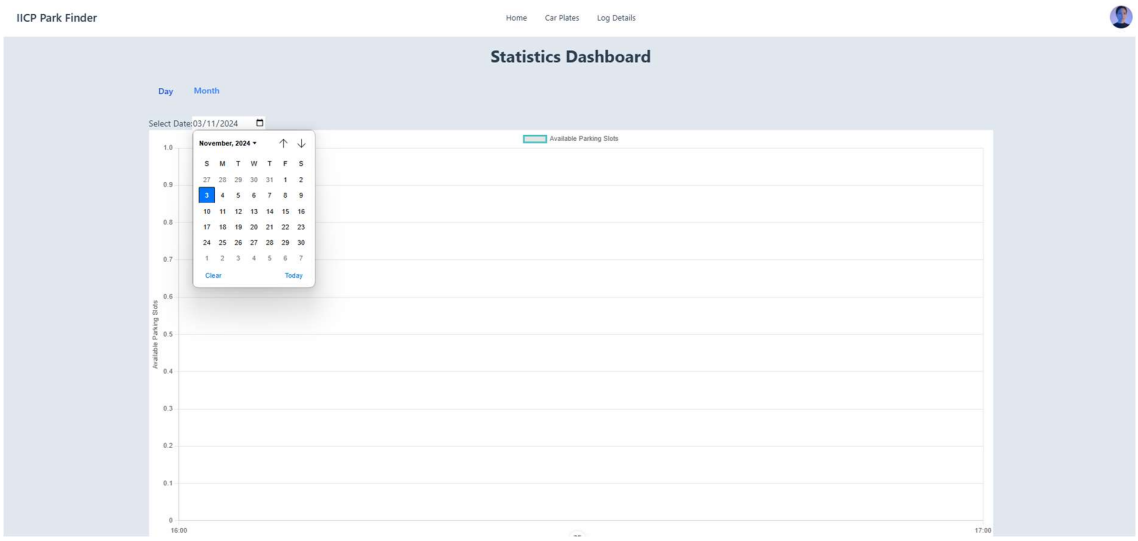


Figure 5 - 71 Select Date

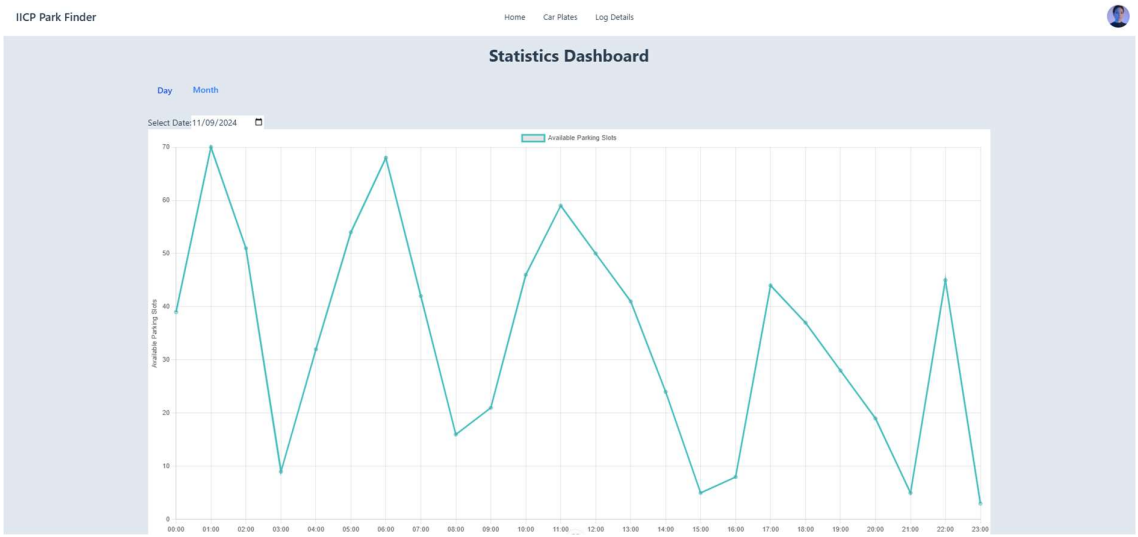


Figure 5 - 72 Show Available Parking Lots Throughout the Date

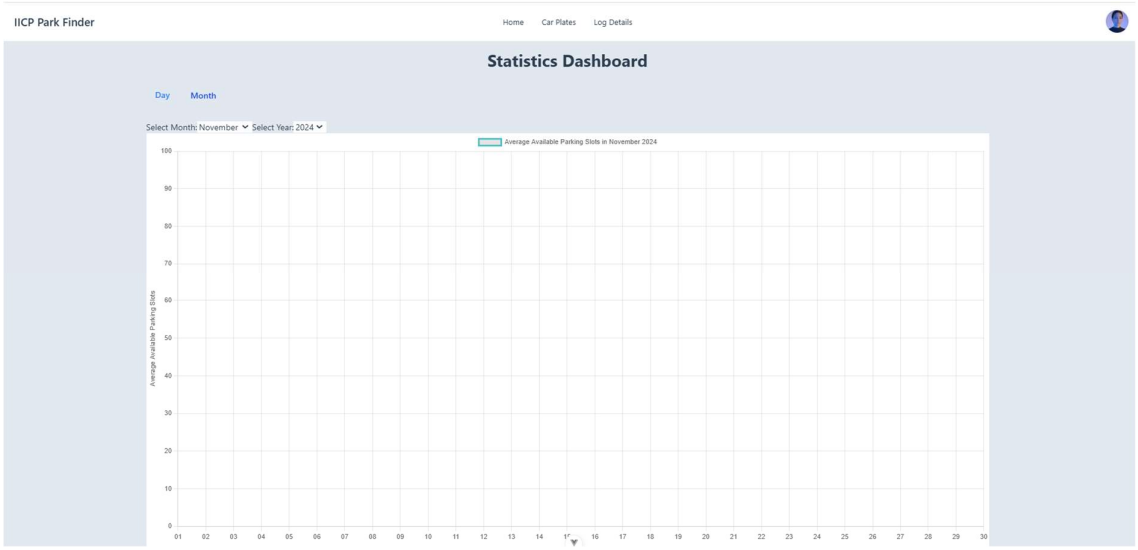


Figure 5 - 73 Show Line Graph of Average Available Parking Lot throughout Current Month

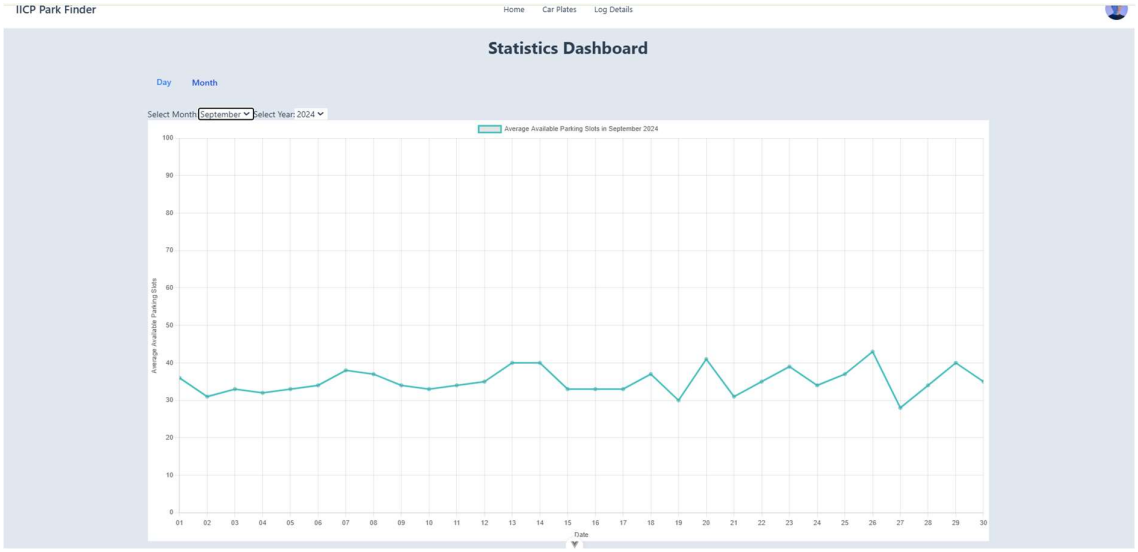


Figure 5 - 74 Show Line Graph of Average Available Parking Lots throughout Chosen Month

The unit testing for viewing parking lot statistics covers four scenarios to ensure accurate graphical representation of parking data over different time frames which is shown in Table 5.14. The *View Parking Lot Statistics Today* test case confirms that a line graph showing available parking lots for the current day when Statistics page is displayed, as seen in Figure 5-70. The *View Parking Lot Statistics for Chosen Date* test case verifies that selecting a specific date generates a line graph for that day's parking availability, demonstrated in Figures 5-71 and 5-72. For the *View Parking Lot Statistics for This Month* test case, the system displays a line graph showing the daily average of available parking lots throughout the current month, as shown in Figure 5-73. Lastly, the *View Parking Lot Statistics for Selected Month and Year* test case ensures that a similar line graph is generated for any chosen month and

year, illustrated in Figure 5-74. All test cases passed, confirming that the feature effectively provides detailed parking statistics across different time periods.

5.2.1.2 (c) Student Lists Page

Table 5. 15 Unit Testing for Web App Student Lists Page

Test Case	Expected Results	Screenshot	Status
View Lists of Student Details	-It should show a table with student details based on car plate number in ascending order.	Figure 5-75	Pass
Search for Student Details Based on Car Plate Number	-It should show table with students details based on car plate number query.	Figure 5-76	Pass
Search for Student Details Based on Student ID	- It should show table with students details based on student ID query.	Figure 5-77	Pass

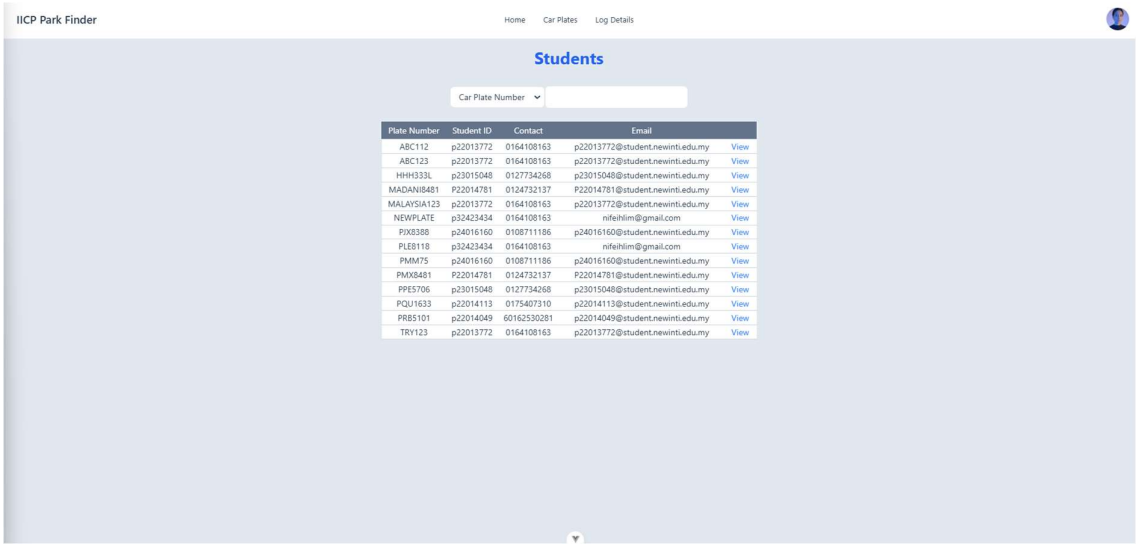


Figure 5 - 75 Show List of Car Plate Numbers and Student Information In Ascending Order

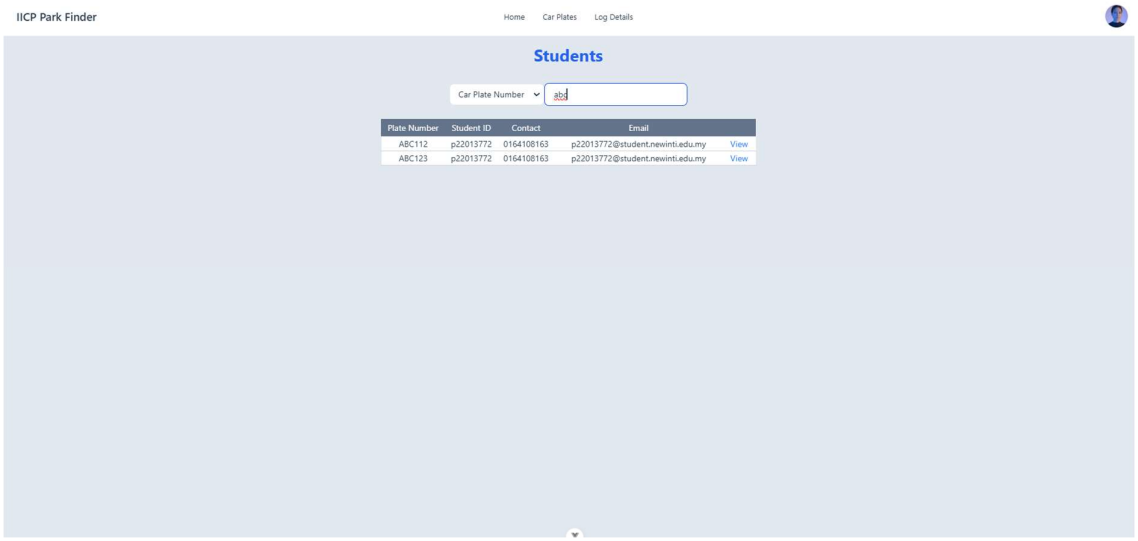


Figure 5 - 76 Show List of Details Based on Car Plate Number Query

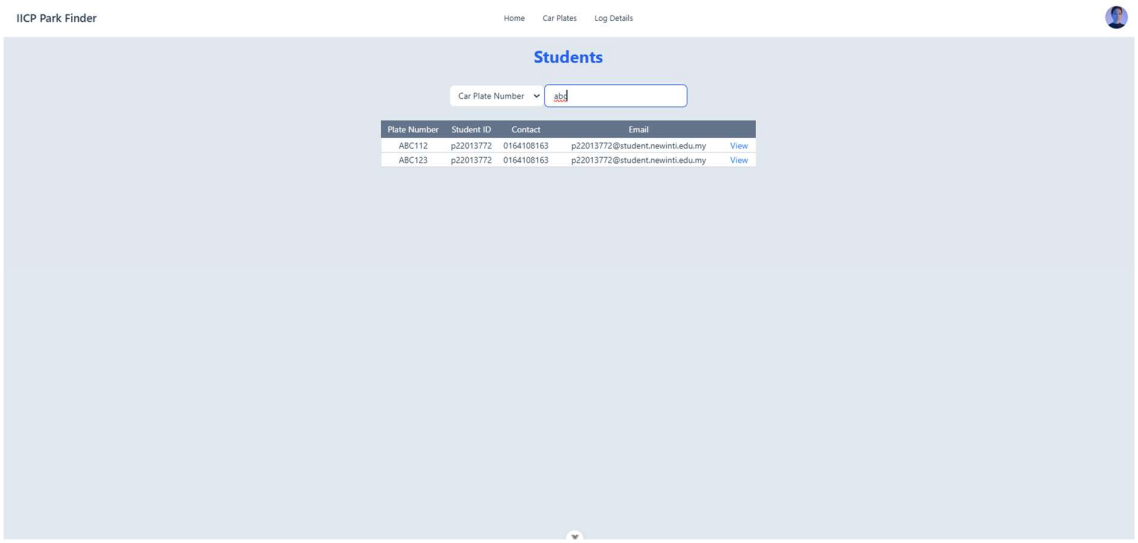


Figure 5 - 77Show List of Details Based on Student ID Query

Table 5.15 shows the unit testing for viewing and searching student details focusing on accurate display and query functionality. The *View Lists of Student Details* test case ensures that a table listing student details is shown in ascending order by car plate number, as illustrated in Figure 5-75. In the *Search for Student Details Based on Car Plate Number* test case, the system retrieves and displays student details in a table format according to the car plate number entered, demonstrated in Figure 5-76. The *Search for Student Details Based on Student ID* test case similarly retrieves student information based on a student ID query, shown in Figure 5-77. All test cases passed, confirming that the feature accurately lists and searches student details based on the specified criteria.

5.2.1.2 (d) Fines Page

Table 5. 16 Unit Testing for Web App Fines Page

Test Case	Expected Results	Screenshot	Status
View Fines for Vehicle.	-It should show the car plate number, email of owner, phone number and fines imposed on the vehicle.	Figure 5-78	Pass
Add Fine	-It should show an add fine modal where admin needs to select the car park level and reason of fine. -It should show error message if field not chosen.	Figure 5-79 Figure 5-80 Figure 5-81	Pass
Edit Fine	-It should show an edit modal where admin can update the car park level and reason of fine.	Figure 5-82 Figure 5-83	Pass
Pay Fine	-It should show a confirm payment modal which allows the user to choose receipt file. -It should show error if file is not chosen. -Payment status should update to Paid.	Figure 5-84 Figure 5-85 Figure 5-86 Figure 5-87	
Delete Fine	-It should show a confirmation dialog to delete the fine.	Figure 5-88 Figure 5-89	Pass
View Receipt	-It should open the receipt file in another tab.	Figure 5-90	Pass

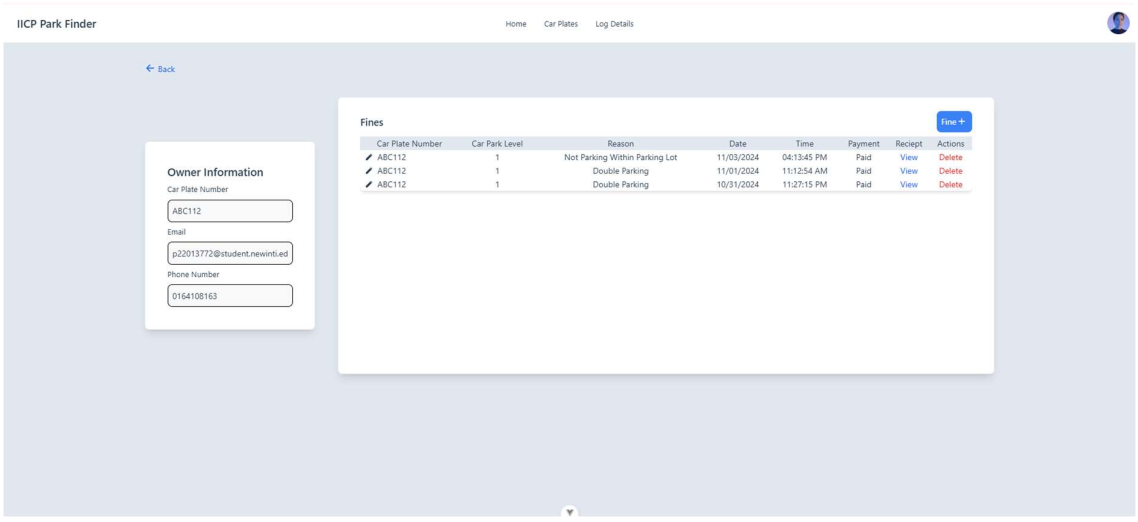


Figure 5 - 78 Show Fines for Vehicle

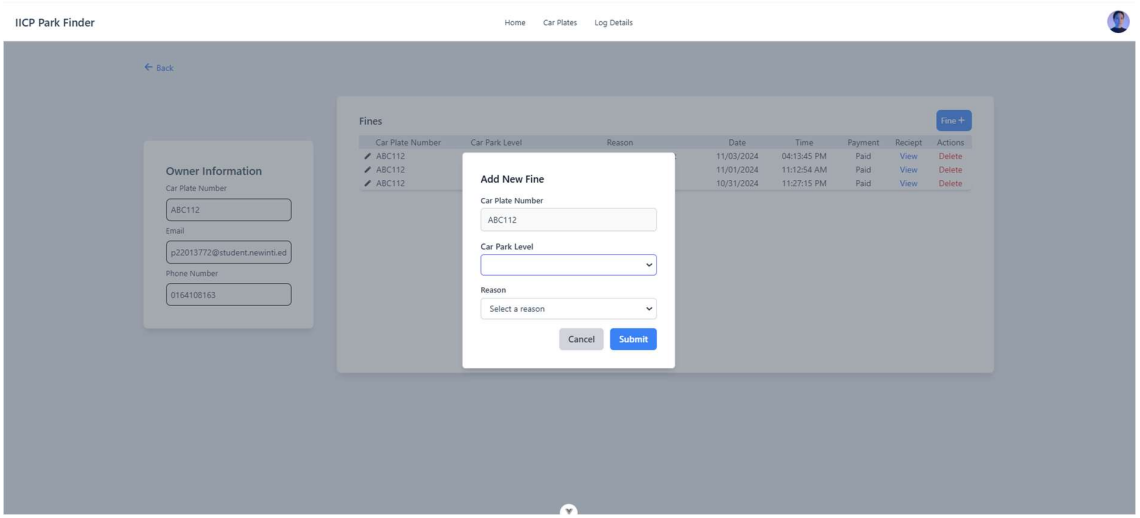


Figure 5 - 79 Add FIne Modal

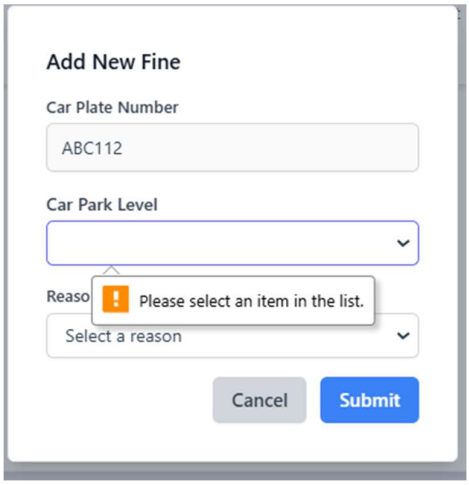


Figure 5 - 80 Reject Empty Fields



Figure 5 - 81 Fine List Update After Add

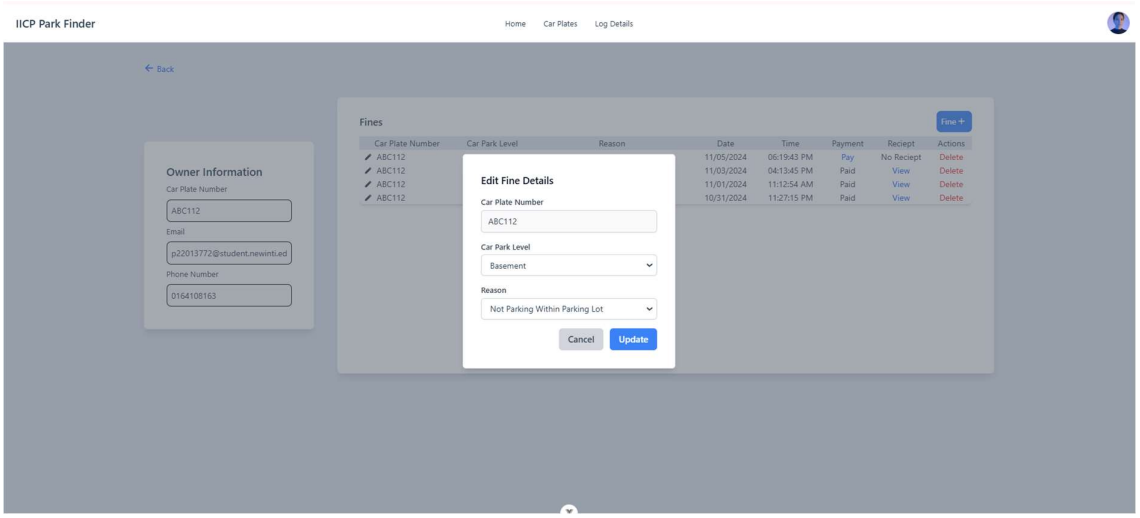


Figure 5 - 82 Edit Fine Modal



Figure 5 - 83 Reason and Car Park Level Changed to Updated Value

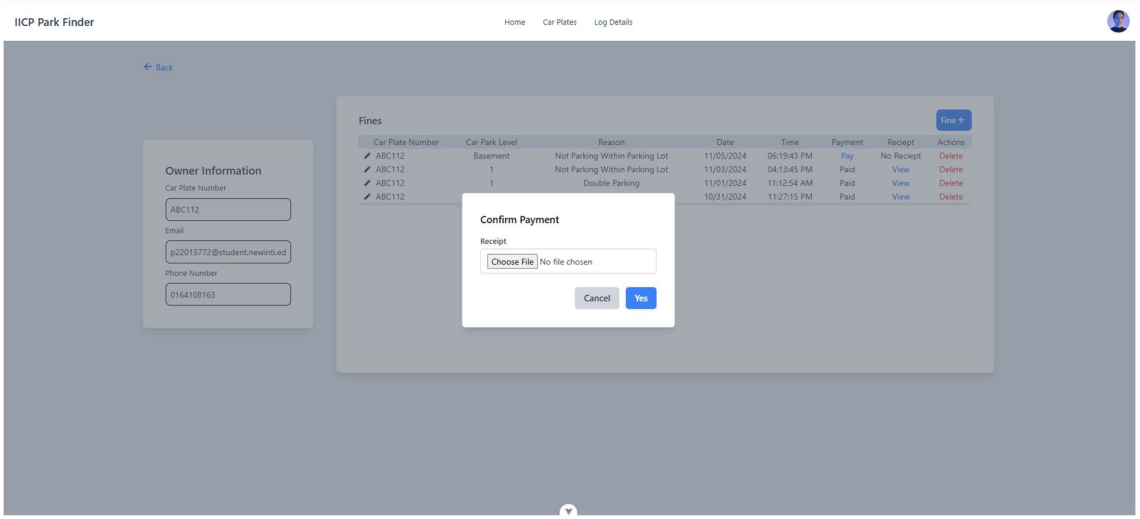


Figure 5 - 84 Add Receipt File

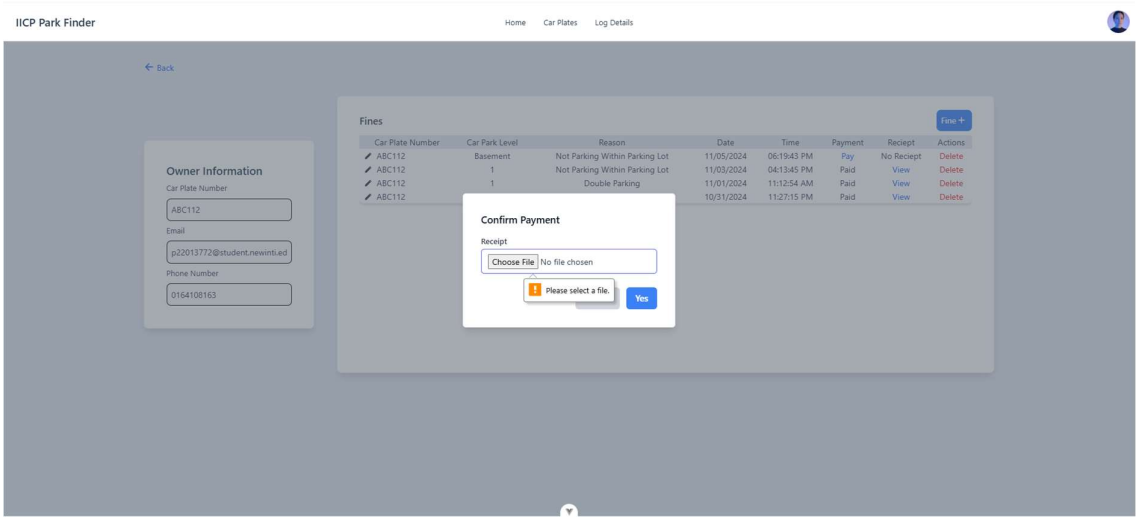


Figure 5 - 85 Reject Empty File View

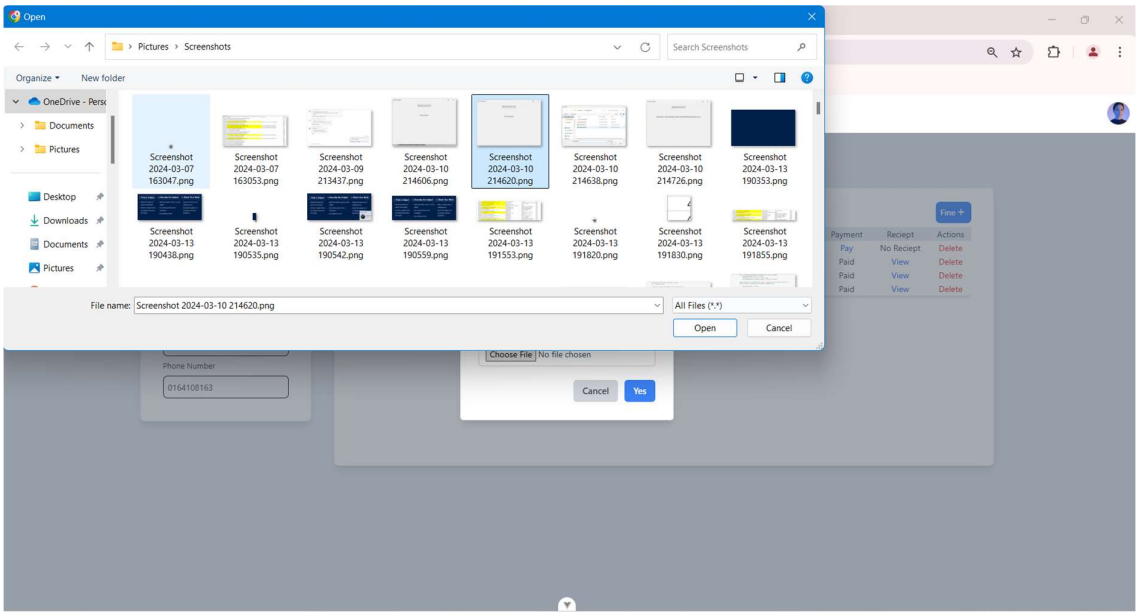


Figure 5 - 86 Choose Receipt File

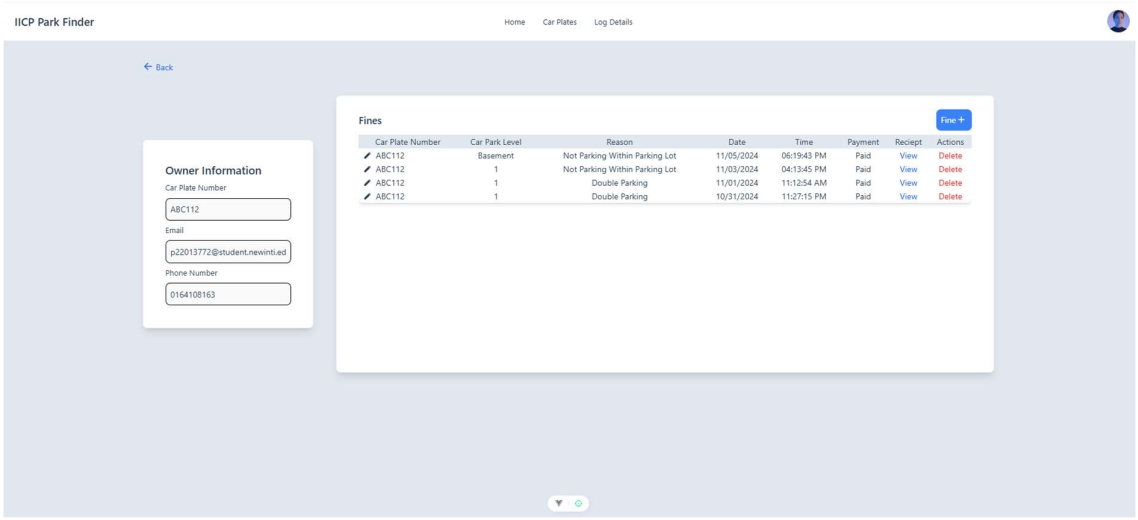


Figure 5 - 87 Payment Status and Receipt Status Changed After Payment

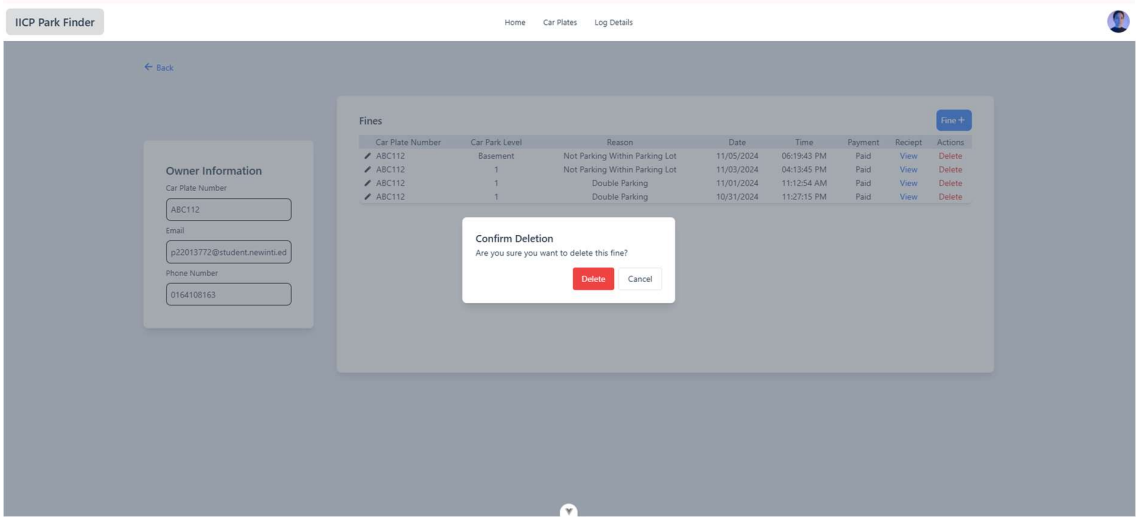


Figure 5 - 88 Confirm Delete Fine

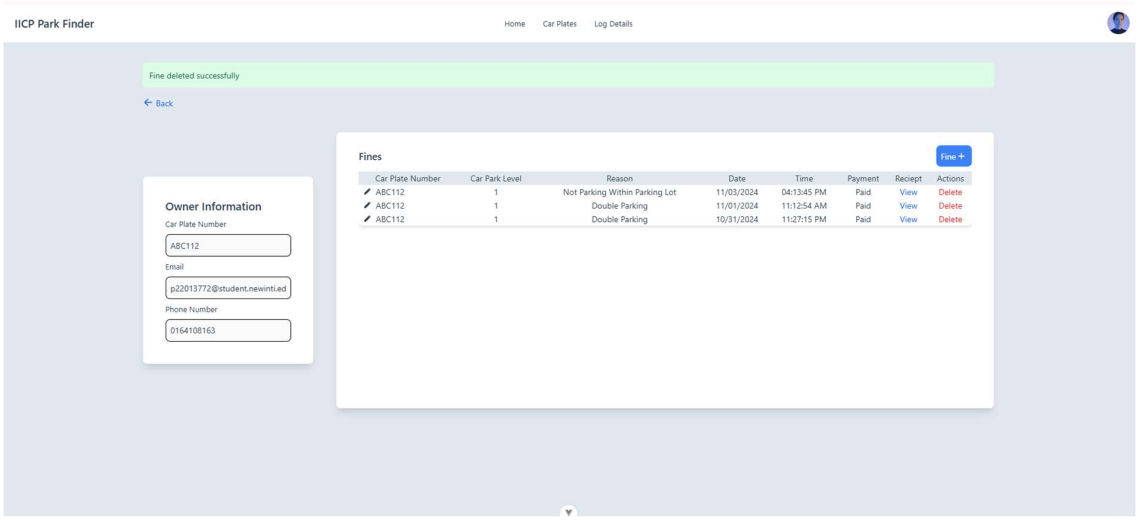


Figure 5 - 89 Fine Deleted

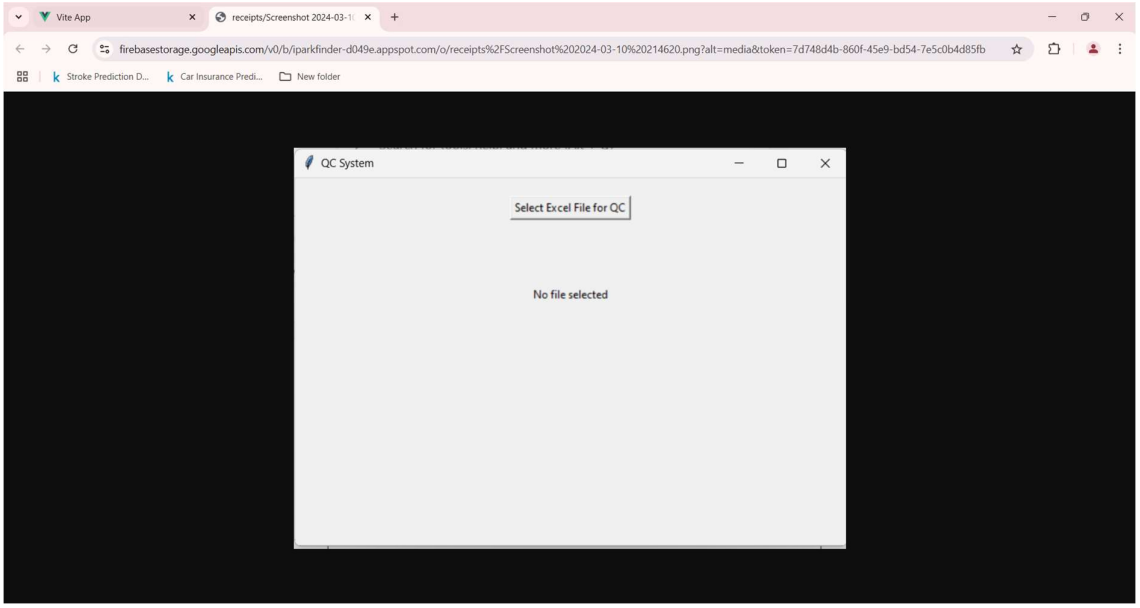


Figure 5 - 90 View Fine Receipt

The unit testing for managing vehicle fines includes a range of scenarios to ensure correct viewing, adding, editing, paying, and deleting of fines, along with receipt handling as displayed in Table 5.16. The *View Fines for Vehicle* test case confirms that the system displays details such as the car plate number, owner's email, phone number, and any fines imposed, as shown in Figure 5-78. In the *Add Fine* test case, an admin can add a fine through a modal requiring selection of car park level and reason; an error message appears if fields are left blank, as demonstrated in Figures 5-79 to 5-81. The *Edit Fine* case allows admins to update fine details in an edit modal, shown in Figures 5-82 and 5-83. For the *Pay Fine* test case, a payment confirmation modal lets users choose a receipt file, displays an error if no file is chosen, and updates the payment status to "Paid," as seen in Figures 5-84 to 5-87. The *Delete Fine* test

case includes a confirmation dialog for deleting a fine, illustrated in Figures 5-88 and 5-89. Lastly, *View Receipt* allows users to open the receipt file in a new tab, as shown in Figure 5-90. All test cases passed, confirming reliable functionality for fine management and payment processing.

5.2.1.2 (e) Log Details Page

Table 5. 17 Unit Testing for Web App Log Details Page

Test Case	Expected Results	Screenshot	Status
View Lists of Log Details	-It should show a table of logs of edit, delete and pay fine transactions.	Figure 5-91	Pass
Search for log Based on Car Plate Number	-It should show table with log details based on car plate number query.	Figure 5-92	Pass
Search for Student Details Based on Student ID	- It should show table with log details based on student ID query.	Figure 5-93	Pass

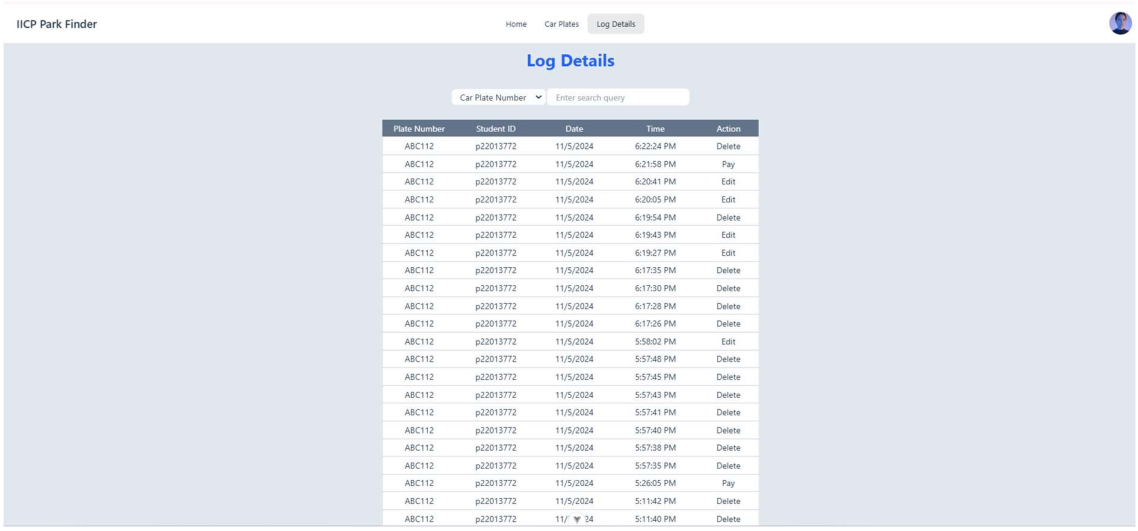


Figure 5 - 91 Show Log Details



Figure 5 - 92 Show Log Details on Searched Car Plate Number

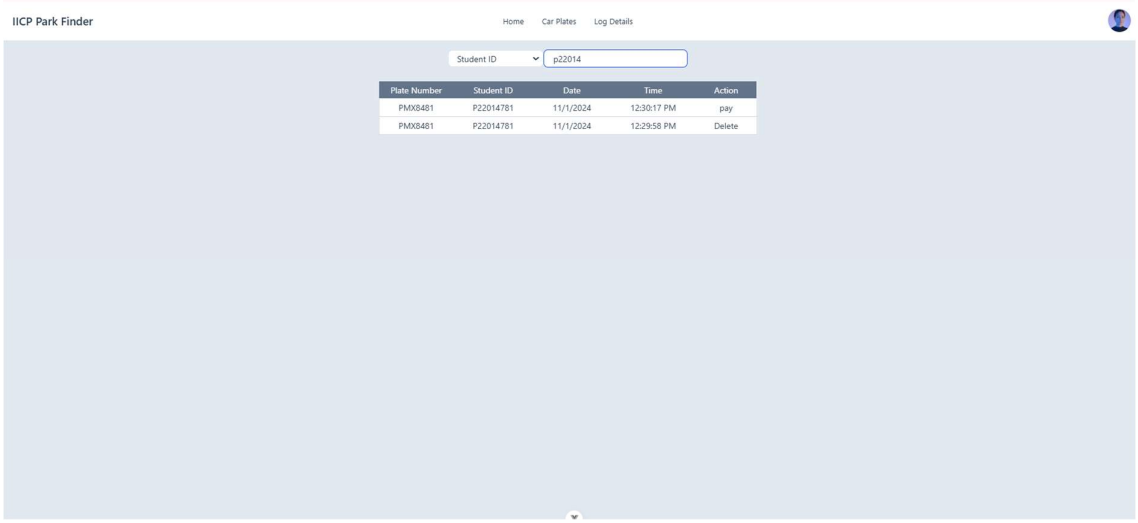


Figure 5 - 93 Show Log Details on Searched Student ID

Table 5.17 shows the unit testing for viewing and searching log details covers key scenarios to verify the accurate display and retrieval of transaction logs. The *View Lists of Log Details* test case ensures that a table displays logs for actions such as editing, deleting, and paying fines, as shown in Figure 5-91. The *Search for Log Based on Car Plate Number* test case enables users to filter logs by car plate number, displaying relevant entries in a table format, as seen in Figure 5-92. Similarly, the *Search for Student Details Based on Student ID* test case retrieves and displays log details based on a student ID query, illustrated in Figure 5-93. All test cases passed, confirming effective functionality for viewing and searching transaction logs.

5.2.1.2 (f) Logout

Table 5. 18 Unit Testing for Web App Logout

Test Case	Expected Results	Screenshot	Status
Log Out from System	-It should show a confirmation dialog to ask if user confirms to log out.	Figure 5-94 Figure 5-95	Pass

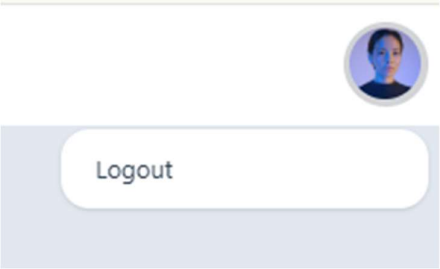


Figure 5 - 94 Logout Button

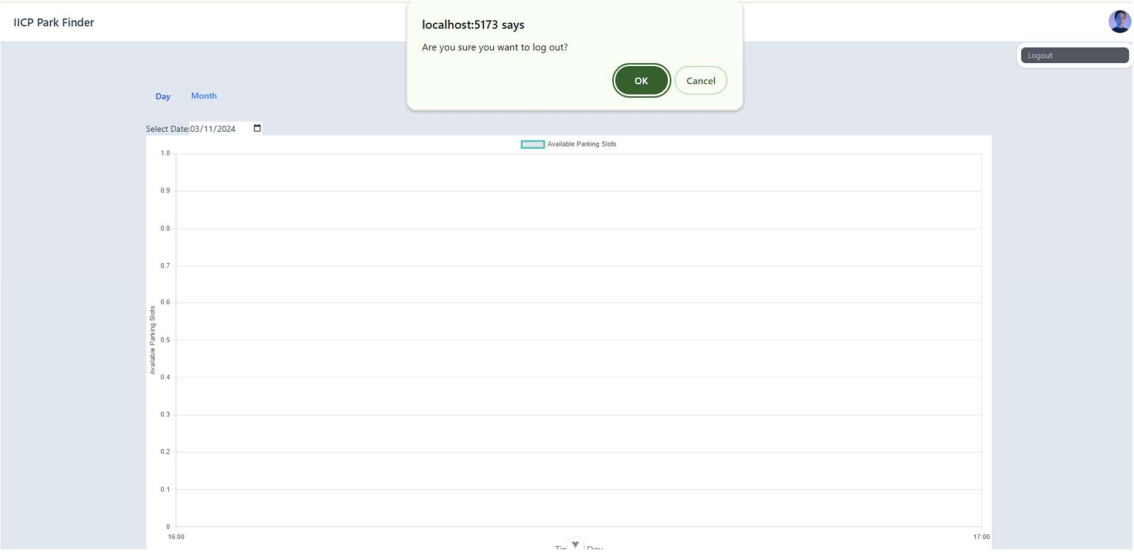


Figure 5 - 95 Confirm Logout

The unit testing for the logout feature includes a single test case, *Log Out from System*, which verifies that a confirmation dialog appears to prompt the user to confirm or cancel the logout action ass tabulated in Table 5.18. This functionality is illustrated in Figures 5-94 and 5-95, and the test case passed, confirming that the logout process provides appropriate user confirmation.

5.2.2 Integration Testing

The integration testing process aims to verify that combined parts of the application function correctly together, ensuring data flows seamlessly across modules and that integrated components perform as intended. The testing will be conducted and recorded in the tables for both the mobile and web applications.

5.2.2.1 Mobile Application

5.2.2.1 (a) Login Activity to Register Activity

Table 5. 19 Integration Testing for Mobile App Login Activity to Register Activity

Scenario	Expected Result	Status
From the login page, user taps on the Register Here link.	App navigates to the first page of the registration module where user can read and scroll the Terms and Conditions text. Only upon scrolling all the way down can the user click on the Accept and Continue Button.	Pass
From the terms and conditions page, user scrolls the text all the way down and taps on the 'Accept and Continue' button.	App navigates to the register page where the user must fill in all the text fields and tap on the 'Submit' button to register.	Pass
In the register page, user keys in all the fields and submit.	The application validates to make sure all the fields are filled, the email is a student email and is not registered yet, the car plate number is not registered yet, and the password and confirm password matches. The application then sends a request to Firebase to send a verification link to the registered email. .	Pass
User opens the verification email to tap on the link and return to the application.	Upon verifying the email, the application will show a success message and navigate the user back to the login page. The details of the user will be saved to the database.	Pass

Table 5.21 shows the integration testing for the mobile application's login activity to register activity. This integration testing validates the user registration process from the login page. Starting from the login page, users access the registration module by tapping "Register Here," where they must scroll through and accept the Terms and Conditions to proceed. On the registration page, users fill in required fields, with validations ensuring all fields are completed, the email is a unique student email, the car plate number is unregistered, and passwords

match. A verification email is sent via Firebase to the registered email; upon verifying, users receive a success message and are redirected to the login page, with the details saved in the database. All tests passed, confirming a smooth registration workflow.

5.2.2.1 (b) Login Activity to Forgot Password Activity

Table 5. 20 Integration Testing for Mobile App Login Activity to Forgot Password Activity

Scenario	Expected Result	Status
From the login page, user taps on the Forgot Password link.	App navigates to the forgot password page where the user must fill in the field with the student email.	Pass
In the forgot password page, the user keys in the email and click on 'Submit'.	App will validate the entry to make sure it is not empty, and the email has been registered before. App will show an error message if email is not a registered account. If success, the application will make a request to Firebase to send a reset password to the email. The app will navigate to a page telling the user to open the reset password link sent to the email.	Pass
User opens the reset password email and reset the password.	User keys in the new password which will validate to make sure the password is at least 6 characters long. Once successful, the user will be notified at the password has been successfully changed.	Pass

Table 5.20 tabulate the integration testing for the mobile app login activity to forgot password activity. This integration testing verifies the forgot password functionality. Starting from the login page, users can access the forgot password page to enter the registered email. The app validates the email input, ensuring it's not empty and matches a registered account, displaying an error if not. Upon success, a password reset request is sent via Firebase, and users are directed to check their email. Once the reset email is opened, users can set a new password (validated for a minimum of 6 characters) and receive a success notification upon completion. All tests passed, confirming reliable password recovery flow.

5.2.2.1 (c) Login Activity to Parking Activity

Table 5. 21 Integration Testing for Mobile App Login Activity to Parking Activity

Scenario	Expected Result	Status
In the login page, user keys in the email and password to sign in.	App will validate to make sure that the email and password fields are not empty, the email and password are registered and correct. Upon successful login, the application will sign the user in and navigate to the Parking page. In the parking page, the user will see the vacant and occupied parking lots color-coded green and red respectively.	Pass
User tap on a parking lot in the parking page and confirm navigation.	App will check if the parking lot tapped is vacant or not. If occupied, the app will tell the user that the parking lot is not vacant. If vacant, the app will show a confirmation dialog box asking if the user wants to navigate to the parking lot. Upon confirmation, the app will navigate the user to the parking lot. If while navigating, the parking lot is occupied, the application will stop the navigation and let the user know that the parking lot is occupied. Once the user reaches the parking lot, the application will stop the navigation and show a dialog box stating the user has reached the destination.	Pass
User tap on an entrance and confirm navigation.	App will show a confirmation dialog showing the parking lot name that is nearest to the chosen entrance. Upon confirmation, the app will navigate the user to the parking lot. If while navigating, the parking lot is occupied, the application will stop the navigation and let the user know that the parking lot is occupied. Once the user reaches the parking lot, the application will stop the navigation and show a dialog box stating the user has reached the destination.	Pass

Table 5.21 shows the results of integration testing from the login activity to the parking activity. This integration testing validates login, parking lot selection, and navigation to the parking lot. After successful login, users access a parking page where vacant and occupied lots are color-coded. When a user taps a parking lot, the app checks availability and, if vacant, confirms navigation to it, updating the user if the lot becomes occupied mid-route and confirming arrival upon reaching it. For entrances, users can select one, and the app identifies the nearest vacant lot, guiding users similarly with real-time availability updates and confirmation on arrival. All tests passed, confirming smooth login, parking navigation, and entrance guidance.

5.2.2.1 (d) Parking Activity to Contact Activity

Table 5. 22 Integration Testing for Mobile App Parking Activity to Contact Activity

Scenario	Expected Result	Status
From the parking page, the user taps on the contact page in the navigation bar.	App will navigate to the contact page showing the status of the show car plate number set by the user. .	Pass
User toggle to hide car plate number.	App will update the UI and set the show value to false. User's contact number can no longer be found when other user key in the user's car plate number.	Pass
User toggle to show car plate number.	App will update the UI and set the show value to true. User's contact number can be found when other user key in the user's car plate number.	Pass
User search for vehicle owner's contact number.	App will validate the field to make sure the field is not empty. If the car plate number is not registered, or the owner hid the car plate number, a dialog box will pop up to let the user know. If the car plate number is registered and shown by the owner, a dialog box will pop up allowing the user to tap on the 'Call' option. The application will open the phone app with the contact number of the vehicle owner.	Pass

Table 5.22 tabulates the integration testing for parking activity to contact activity. This integration testing focuses on user interactions with car plate number visibility and contact search functionalities on the contact page from the parking page. Users can toggle their car plate number visibility, choosing to either hide or show their contact details when others search

by car plate number. When a user searches for a vehicle owner's contact, the app checks if the field is filled, displays a message if the car plate is unregistered or hidden, and provides a call option if contact info is available. All tests passed, confirming smooth toggling, visibility updates, and contact search functionality.

5.2.2.1 (e) Contact Activity to Inbox Activity

Table 5. 23 Integration Testing for Mobile App Contact Activity to Inbox Activity

Scenario	Expected Result	Status
From the contact page, the user taps on the inbox page in the navigation bar.	App will navigate to the inbox page showing the list of notifications received from the user. The notifications consist of the fines accumulated and paid by the user. Notifications that have been read has a grey background while unread notifications have a white background.	Pass
From the inbox page, the user taps on one of the notifications.	The app will redirect to the notification details page showing the notification title, time and message.	Pass

Table 5.23 shows the integration testing result for mobile app contact activity to inbox activity. This integration testing verifies navigation and notification handling within the inbox activity. It confirms that from the contact page, users can access the inbox, which displays notifications related to accumulated and paid fines. Notifications are visually distinguished by background color (grey for read, white for unread). Tapping on a notification redirects the user to a detailed view displaying the notification's title, time, and message. All tests passed, confirming effective navigation and notification management.

5.2.2.1 (f) Inbox Activity to Settings Activity

Table 5. 24 Integration Testing for Mobile App Inbox Activity to Settings Activity

Scenario	Expected Result	Status
From the inbox page, the user taps on the settings page in the navigation bar.	App will navigate to the settings page showing a list of pages the user can navigate to which consist of Profile page, Change Password Page, Car Plate Numbers Page, Contact Us Page, About IICP Page and Logout.	Pass
From the settings page, the user taps on the Profile tile.	App will navigate to the profile page where the user can view the name, email and contact number of the user. The user	Pass

	can edit and save the name and contact number. The database will be updated if changes are saved, and the new data will automatically show on the page.	
From the settings page, the user taps on the Change Password tile.	App will navigate to the change password page where the user can fill in the current password, new password, and confirm password. The app will validate the fields to make sure no field is empty, the old password keyed in is correct, the new password consists of at least 6 characters, and the new password, and confirm password matches. An error message will show if the validations are not met. Upon successful change, the app will show a success message to let the user know the password has been changed successfully and return to the settings page.	Pass
From the settings page, the user taps on the Car Plate Numbers tile.	App navigates to the Car Plate Numbers page where the user can view all the car plate numbers owned by the user. Car plates that are clamped will have a red background. If user tries to edit or delete the clamped car plate, a warning message will pop up letting the user know the car plate is clamped and to go to the AFM office to unclamp. User can swipe left on the car plate number to edit or delete the car plate number. Upon edit, the app will validate to see if the field is empty and if the car plate number already exists. Error message will show if validation is not successful. Error message will show if user has only one car plate number tries to delete it.	Pass

	From the Car Plate Numbers page, the user taps on the '+' floating button and navigates to the add car plate numbers page. The field will check if empty and if car plate number already exists. Error message will show if validation is not met. If adding is successful, a snackbar will show 'Car Plate Number Added'. User can return to the car plate number page by tapping on the back button.	
From the settings page, the user taps on the Contact Us tile.	App will navigate to the Contact Us page where the user can view the level of AFM office, phone number, email address, and operation hours. User can tap on the phone number and the app will launch the phone app with AFM's phone number. User can long tap on the email address to copy to the address to clipboard.	Pass
From the settings page, the user taps on the About IICP Park Finder page.	App will navigate to the About IICP Park Finder page and view the Terms and Conditions of using the application. User can scroll to read to the bottom of the page.	Pass
From the settings page, the user taps on the Logout tile.	App will show a confirmation dialog box for the user to confirm the logout activity. If user accepts, the application will log the user out and navigate to the login page.	Pass

Table 5.24 shows the integration testing for inbox activity to settings activity. This integration testing assesses navigation and functionality within the settings page of an application. Key actions tested include navigating to various pages: **Profile** (view and edit user info with automatic updates on save), **Change Password** (validates password fields and updates on successful change), **Car Plate Numbers** (view, add, edit, delete car plates, with alerts for clamped plates and validation checks), **Contact Us** (displays contact details, tap-to-call, and copy-to-clipboard features), **About IICP Park Finder** (viewing Terms and Conditions), and **Logout** (confirmation dialog for logging out). All tests passed, confirming seamless navigation and functionality across settings options.

5.2.2.2 Web Application

5.2.2.2 (a) Login Activity to Parking Statistics Activity

Table 5. 25 Integration Testing for Web App Login Activity to Parking Statistics Activity

Scenario	Expected Result	Status
In the login page, user keys in the email and password to sign in.	App will validate to make sure that the email and password fields are not empty, the email and password are registered and correct, and the account belongs to an admin. Error message will show if the requirements are not met. Upon successful login, the application will sign the user in and show a success notification. Then, the application will navigate to the parking statistics page. In the parking statistics page, the user will see the line graph for available parking lots throughout the current day.	Pass
User selects a date from date picker.	The chart will show the line graph of available parking lot throughout the selected date.	Pass
User clicks on the 'Month' tab.	The chart will show the line graph of average available parking lot everyday throughout the current month.	Pass
User selects a month and date.	The chart will show the line graph of average available parking lot everyday throughout the selected month.	Pass

Table 5.25 shows the test case, expected results and test status for integration from the login activity to parking statistics activity in the web application. This integration testing validates the login process and parking statistics display for an admin user. It confirms that the app checks for valid email, password, and admin status during login, showing an error if requirements aren't met. Upon successful login, the user is directed to a parking statistics page displaying a line graph of available parking lots for the current day. Additional tests allow users to view parking availability by selecting specific dates or viewing monthly averages. All tests passed, confirming accurate login, navigation, and chart functionalities for parking statistics.

5.2.2.2 (b) Parking Statistics Activity to Student Lists Activity

Table 5. 26 Integration Testing for Web App Parking Statistics Activity to Student Lists Activity

Scenario	Expected Result	Status
User clicks on the 'Car Plates' tab	App will navigate to the students lists details page where the are tabled and displayed in ascending order of the car plate numbers.	Pass
User selects Car Plate Number category for searching.	User can key in car plate numbers and the table will update the list accordingly. If no car plate is found, 'No data found' will be displayed.	Pass
User selects Student ID category for searching.	User can key in student ID and the table will update the list accordingly. If no student ID is found, 'No data found' will be displayed.	Pass

Table 5.26 shows the tabulated data for integration testing between the parking statistics activity and student lists activity. This integration testing focuses on the user's ability to navigate and search within the car plate listings. It verifies that users can access the students list by clicking on the "Car Plates" tab, displaying records in ascending order of car plate numbers. Additionally, users can search by car plate number or student ID, with the table updating based on input; if no matches are found, a "No data found" message appears. All tests passed, confirming proper navigation and search functionality in the car plates section.

5.2.2.2 (c) Student Lists Activity to Fines Activity

Table 5. 27 Integration Testing for Web App Student Lists Activity to Fines Activity

Scenario	Expected Result	Status
User clicks on the 'View' link on a car plate number row in the Students Lists page.	App will navigate to the fine page where user can see the student email of the owner, contact number, and fines imposed on the vehicle.	Pass
User clicks on the pen icon.	An edit modal will display where user can edit the fine reason and car park level. Changes made and logging will be updated to database. The fine detail on the web page will update automatically.	Pass
User clicks on 'pay'.	A pay fine modal will display where user can select a receipt to upload. The modal will close, receipt uploaded to database, and the payment status changed to 'Paid'. User can click on the 'View; in the	Pass

	receipt column to view the receipt uploaded for the fine.	
User clicks on the 'Delete'.	A confirmation dialog should show to ensure if the user wants to delete the fine. Once deleted, the table will reload displaying the new list. Log will be added to database.	Pass

Integration testing for the web applications student lists activity to fine activity is tabulated in table 5.27. Key tests include viewing fine details, editing fines, paying fines, and deleting fines. Each action is designed to trigger specific modals or confirmation dialogs where users can view or modify information, upload receipts, and confirm deletions. Database updates and page refreshes are validated to ensure changes are accurately saved and displayed in real-time. All tests passed successfully, confirming that the integration of these features works as expected.

5.2.2.2 (d) Student Lists Activity to Log Details Activity

Table 5. 28 Integration Testing for Web App Student Lists Activity to Log Details Activity

Scenario	Expected Result	Status
User clicks on the 'Log Details' in the navigation tab.	App will navigate to the log details page where the user can see the edit, pay and delete fine transactions that has been done.	Pass
User selects Car Plate Number category for searching.	User can key in car plate numbers and the table will update the list accordingly. If no car plate is found, 'No data found' will be displayed.	Pass
User selects Student ID category for searching.	User can key in student ID and the table will update the list accordingly. If no student ID is found, 'No data found' will be displayed.	Pass

Table 5.28 shows the testing conducted for integration between the student lists activity and log details in the web application. This integration testing ensures that users can effectively view and search fine transaction logs in the system. Key tests include navigating to the log details page, where users can view all edit, pay, and delete fine actions in a table, and testing search functionalities by car plate number and student ID. When searching, the table updates based on input, displaying "No data found" if there are no matching records. All tests passed, confirming that both navigation and search features work as intended.

5.2.2.2 (e) Home Activity to Log Out Activity

Table 5. 29 Integration Testing for Web App Home Activity to Log Out Activity

Scenario	Expected Result	Status
User clicks on the profile icon in the top right of the page.	App will show a logout button. User will be shown a confirmation dialog to confirm log out from system. App will navigate to login page if confirm	Pass

Table 5.29 shows the table for integration testing for the web application from the home activity to log out activity where the user clicks on the profile icon from the home page. The application passes the testing as the confirmation dialog pops up when the logout button is clicked. The application then navigates to the login page when the user click on confirm.

5.3 Non-Functional Testing

Non-functional testing focuses on evaluating the performance, usability, reliability, and other attributes of a software application that are not related to specific functionalities. Unlike functional testing, which verifies what the system does, non-functional testing assesses how the system performs under various conditions and its overall user experience. This type of testing is essential for identifying potential bottlenecks, ensuring scalability, and guaranteeing that the application meets the performance requirements and user expectations. User acceptance testing will be conducted for the non-functional testing phase.

5.3.1 User Acceptance Testing

User acceptance testing (UAT) ensures that the software application meets the needs and expectations of its end-users and stakeholders before it goes live. UAT is typically the final step in the testing process, providing a platform for users to validate the system's functionality, usability, and overall performance in real-world scenarios. The primary goal of UAT is to confirm that the application aligns with business requirements and user specifications.

During this phase, actual users engage with the system to perform tasks that they would typically undertake in their daily operations. The feedback is invaluable for identifying any potential issues or areas for improvement that may not have been previously addressed in functional testing. The testing is done for both mobile and web application of IICP Park Finder to ensure that the software delivers a positive user experience and meets the desired quality standards, ultimately leading to higher user satisfaction and smoother deployment. The results of the testing will be tabulated and discussed in this section.

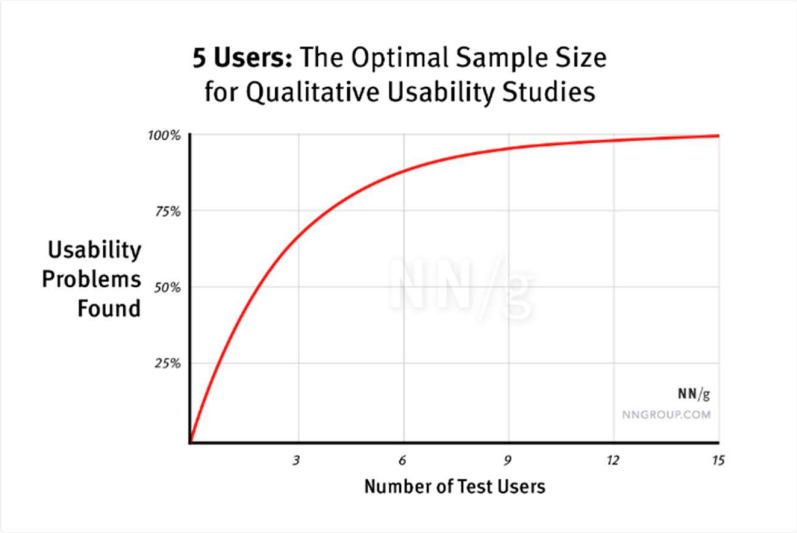


Figure 5 - 96 Graph for Optimal Sample Size for Qualitative Usability Studies

According to Nielsen J. (2000), although an old research, is a usability pioneer with a Ph.D. in human-computer interaction who states that (Nielsen Norman Group, 2000) only five users are needed to effectively uncover the majority of usability issues during user testing as sown in Figure 5-96. This is based on the principle that the first few users typically identify a large percentage of the most critical problems, with diminishing returns as more users are added. Nielsen's research indicates that testing with five users can reveal approximately 85% of usability issues, making it a cost-effective and efficient approach for gathering user feedback. This is because each additional user contributes to a smaller fraction of insights compared to the initial participants possibly resulting in redundant feedbacks which have been discovered by the previous testers. More than 5 users are needed when the product has a wide range of target user group. Therefore, with the mobile application only targeting student group, approximately five users will be chosen to test IICP Park Finder for an efficient testing process.

5.3.1.1 Mobile Application

The UAT for the mobile application is conducted by installing the application onto my phone with developer options in android. As the target audience of the mobile application are students of INTI International College Penang, the UAT is conducted by five INTI students from different courses to test the application to determine if the performance of the user experience and whether the application meets the requirements of the students. The test will be separated into two sections, the interview with the students where the students are asked to perform certain tasks (remarks and problems faced are tabulated), and the questionnaire survey that documents the general comments and opinions of the testers regarding the application.

5.3.1.1 (a) Interview with Participant Number One

The first participant to test the mobile application is a student studying bachelor's in electrical engineering. The test was conducted on campus on 30/10/2024 at 12:00 p.m. The table below shows participants number one's compiled feedback for the application.

Table 5. 30 User Acceptance Testing with Participant Number One

Test	Scenario	Participant's Comments	Test Results
1	Student registers account in application	Good and straightforward.	Pass
2.	Student logs into the application	-Easy to login. -Hope can have auto login. -UI looks impressive and neat.	Pass
3.	Student navigates to a parking lot	-Did not know can navigate to parking lot. -Suggestion: Should include an information button.	Pass

4.	Student navigates to parking lot nearest to entrance	Same comment as test no.3.	Pass
5.	Student shows and hides car plate number	Easy to use after playing with the application.	Pass
6.	Student calls owner of car plate number (MALAYSIA123)	Nice to use.	Pass
7.	Student receives notifications of clamping in inbox page.	Good because receives notification immediately.	Pass
8.	Student views and edit profile details	Update is fast.	Pass
9.	Student changes password and tries log in with new password	Clear.	Pass
10.	Student adds, edit and delete car plate number.	-Should add clamped word next to car plate number. -Include information button for edit and delete.	Pass
11.	Student views Contact Us page	Would like to include 'Level 2'.	Pass
12.	Student views About IICP Park Finder page	Clear.	Pass
13.	Student logs out of the application	Good.	Pass
14.	Student tries forgot password function.	Easy	Pass

The overall impression of participant one towards the application is that it is easy to use and straightforward except for the hidden UI's like navigating and needing to swipe to edit and delete the plate number in which the participant recommended to add information button to teach the users on how to utilize the functions. The participant also mentioned that the user interface is very neat and user friendly as the buttons and text fields are simple to read and understand.

5.3.1.1 (b) Interview with Participant Number Two

Participant number two is a student studying diploma in business at INTI International College Penang at the time of the testing. The test was conducted in campus on 30/10/2024 at 1.00 p.m. and the comments are tabulated as shown below.

Table 5. 31 User Acceptance Testing with Participant Number Two

Test	Scenario	Participant's Comments	Test Results
1	Student registers account in application	No problems and very clear.	Pass
2.	Student logs into the application	Would like an autofill function.	Pass
3.	Student navigates to a parking lot	-Did not know can navigate to parking lot.	Pass

		-Participant knew how to navigate once the participant knew there is a navigation feature.	
4.	Student navigates to parking lot nearest to entrance	Okay.	Pass
5.	Student shows and hides car plate number	Very useful and easy.	Pass
6.	Student calls owner of car plate number (MALAYSIA123)	Straightforward.	Pass
7.	Student receives notifications of clamping in inbox page.	Did not know can tap to view notifications.	Pass
8.	Student views and edit profile details	Easy and fast update.	Pass
9.	Student changes password and tries log in with new password	Works well.	Pass
10.	Student adds, edit and delete car plate number.	Had trouble figuring out the edit and delete function.	Pass
11.	Student views Contact Us page	Okay. Like that can call by tapping phone number.	Pass
12.	Student views About IICP Park Finder page	Okay.	Pass
13.	Student logs out of the application	Easy.	Pass
14.	Student tries forgot password function.	Works well. Easy to reset.	Pass

As shown in the table above, the overall comments from participant two are similar to participant one even though both students are from different courses. This shows that the navigation functions, edit and delete car plate numbers features are not intuitive to users and have a high learning curve for the users to use the features.

Thus, information buttons are added to the parking page and car plate numbers before continuing the user acceptance testing with participant number three. The ‘Clamped’ word is also added next to car plate numbers that have been clamped as the participant did not know the car plate number was clamped with just the red background.

5.3.1.1 (c) Interview with Participant Number Three

As mentioned above, information buttons are added to the parking page and car plate numbers to see if this approach will help the users learn the functions better. Participant number three is a student studying bachelor’s in mechanical engineering in INTI International

College Penang. The test is conducted in campus on 30/10/2024 at 2.00 p.m. The table below shows the feedback from the participant for each test case.

Table 5. 32 User Acceptance Testing with Participant Number Three

Test	Scenario	Participant's Comments	Test Results
1	Student registers account in application	Okay, no problem. Many validations.	Pass
2.	Student logs into the application	Straightforward.	Pass
3.	Student navigates to a parking lot	User found the information button in the top corner and figured out the navigation feature.	Pass
4.	Student navigates to parking lot nearest to entrance	Okay, no problem.	Pass
5.	Student shows and hides car plate number	Straightforward.	Pass
6.	Student calls owner of car plate number (MALAYSIA123)	Easy to call. Likes the feature.	Pass
7.	Student receives notifications of clamping in inbox page.	No problem. Fast update when vehicle is clamped.	Pass
8.	Student views and edit profile details	Easy to edit and changes are immediately reflected.	Pass
9.	Student changes password and tries log in with new password	No problem.	Pass
10.	Student adds, edit and delete car plate number.	User had difficulty finding the edit and delete button. User thought the information button is the same as in parking page. Suggestion: Use animation instead of font for information button.	Pass
11.	Student views Contact Us page	Easy to understand.	Pass
12.	Student views About IICP Park Finer page	Okay.	Pass
13.	Student logs out of the application	No problem.	Pass
14.	Student tries forgot password function.	Easy to reset password. No issues.	Pass

As seen in the table above, the feedback from the participant is similar to previous participants which is mostly positive except for the navigation and edit and delete car plate numbers function white requires extra effort to understand. Although with the information button in the parking page, the user managed to navigate without help. However, the participant thought the information button in the car plate numbers page is the same in the parking page so the participant suggested to use animation showing swipe left on the car plate number instead of using font. Besides these functions, the user was happy with the features and like the concept of the application.

5.3.1.1 (d) Interview with Participant Number Four

Participant number four is a student in the bachelor's of electrical and electronic engineering course at INTI International College Penang. The test was conducted at campus on 30/10/2024 at 2:30 p.m. The feedbacks of the participant is tabulated as shown below.

Table 5. 33 User Acceptance Testing with Participant Number Four

Test	Scenario	Participant's Comments	Test Results
1	Student registers account in application	No issues, very smooth.	Pass
2.	Student logs into the application	Straightforward and simple. Would like an auto-fill feature.	Pass
3.	Student navigates to a parking lot	Learned through the information button. Suggestion: should show the information button when first time login with 'Do not show again' check box to teach users.	Pass
4.	Student navigates to parking lot nearest to entrance	No issues.	Pass
5.	Student shows and hides car plate number	Easy to use. Likes that it protects user's privacy.	Pass
6.	Student calls owner of car plate number (MALAYSIA123)	No problem.	Pass
7.	Student receives notifications of clamping in inbox page.	Very fast receive notification.	Pass
8.	Student views and edit profile details	Easy.	Pass
9.	Student changes password and tries log in with new password	Easy to change password.	Pass

10.	Student adds, edit and delete car plate number.	No problem to add. Thought the information button is the same in parking page. Suggestion: Change to long tap for edit and delete.	Pass
11.	Student views Contact Us page	No problem.	Pass
12.	Student views About IICP Park Finder page	Okay.	Pass
13.	Student logs out of the application	Straightforward and easy.	Pass
14.	Student tries forgot password function.	Easy to reset password. No problem.	Pass

The participant's comment for the application is mostly positive and the setbacks faced are covered by the previous participants. The participant added a suggestion to change the logic for editing and deleting the car plate number to long tap instead of swipe. Additionally, the participant also mentioned that the participant probably will not use the navigation function but use the parking page only to see where the vacant and occupied spaces are. Besides that, the participant faced no issues and is satisfied with the application.

5.3.1.1 (e) Interview with Participant Number Five

Table 5. 34 User Acceptance Testing with Participant Number Five

Test	Scenario	Participant's Comments	Test Results
1	Student registers account in application	No issues. Very easy to register.	Pass
2.	Student logs into the application	Easy and straightforward. Would like an auto-fill feature.	Pass
3.	Student navigates to a parking lot	Easy to navigate.	Pass
4.	Student navigates to parking lot nearest to entrance	Easy to navigate.	Pass
5.	Student shows and hides car plate number	Easy to understand.	Pass
6.	Student calls owner of car plate number (MALAYSIA123)	No problem.	Pass
7.	Student receives notifications of clamping in inbox page.	Very fast receive.	Pass
8.	Student views and edit profile details	Okay.	Pass

9.	Student changes password and tries log in with new password	No problem. Can log in.	Pass
10.	Student adds, edit and delete car plate number.	Found the edit and delete button without information button.	Pass
11.	Student views Contact Us page	Likes the UI.	Pass
12.	Student views About IICP Park Finder page	Okay.	Pass
13.	Student logs out of the application	No issues.	Pass
14.	Student tries forgot password function.	Fast and straightforward.	Pass

Unlike the previous participants, participant number 5 has no issues in navigating, editing, and deleting the car plate numbers. Similar to participant number one, participant number five also commented that there should be an autofill feature because the student email is very long and takes time to type every time for login.

As the interviews are currently receiving repetitive feedback, the user acceptance testing stop with five participants to avoid redundant comments following Nielsen's study and finding of only requiring 5 testers for sufficient user insight.

5.3.1.1 (f) Questionnaire Survey

At the end of the interview, the participants filled in the questionnaire survey to get the participants' opinions and comments on how useful the functionalities are and how easy it is for the user to use the application.

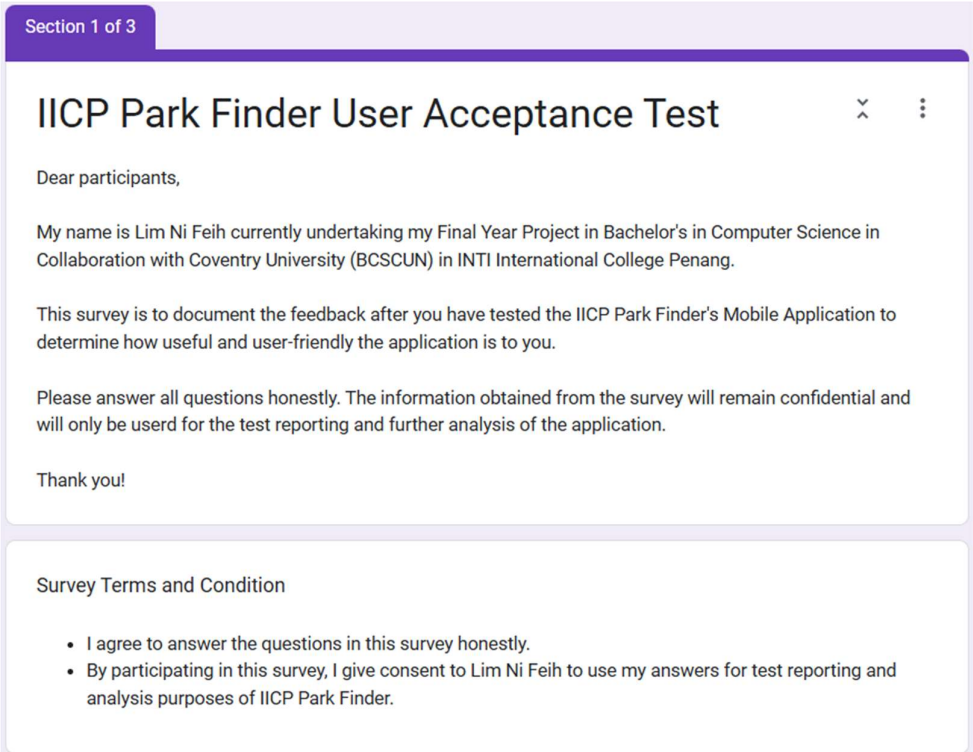


Figure 5 - 97 Terms and Conditions of User Acceptance Testing Survey

Figure 5-97 shows the terms and conditions that must be accepted by the participants when carrying out the user acceptance testing of IICP Park Finder. The terms and conditions are shown at the beginning of the survey.

The survey is separated into two sections, first based on functionalities, and second based on overall satisfaction and suggestions to improve the app.

Functionalities Section:

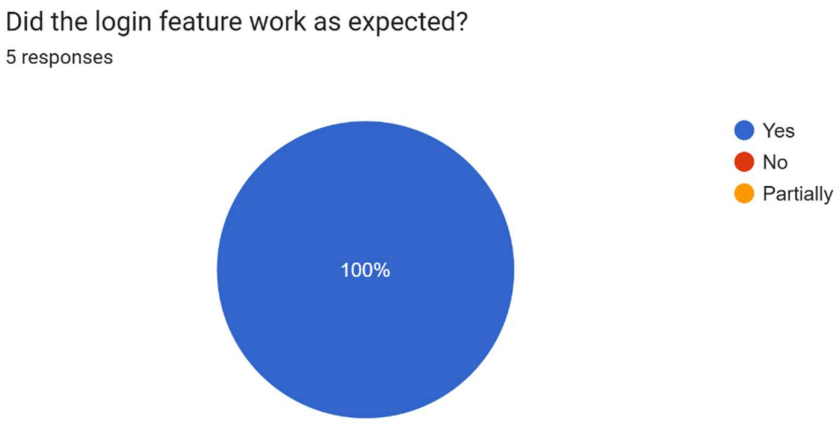


Figure 5 - 98 Expectation of Login Feature

Figure 5-98 shows the result for the first question in Section 1. It can be seen that 100% of the users find that the login feature is fulfils the user requirement to log into the application and do not have any flaws. It can be said that the login feature is functional and meets the user's expectations.

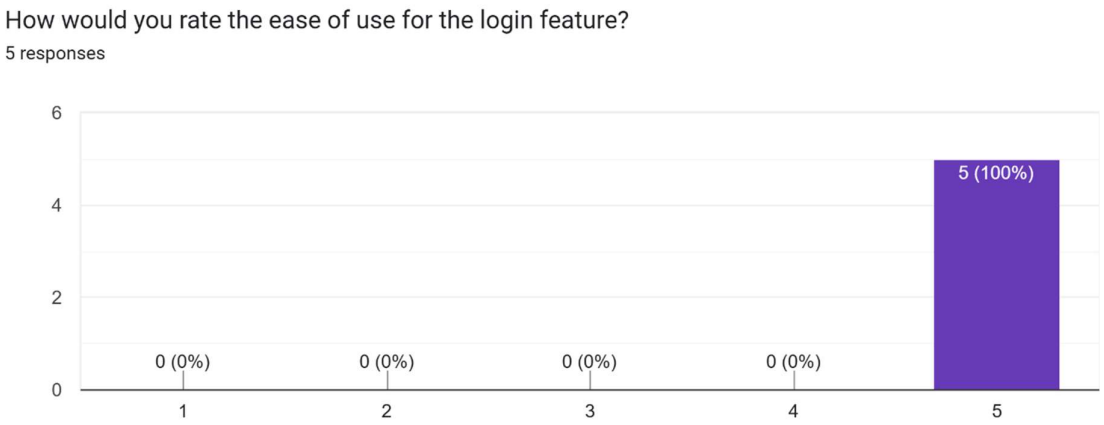


Figure 5 - 99 Ease of Using Login Feature

Figure 5-99 shows the ease of using the login feature where all the participants voted 5 with 1 being very difficult and 5 being very easy. This shows that the users have no problems with logging into the application although some suggested auto fill in to ease the login process.

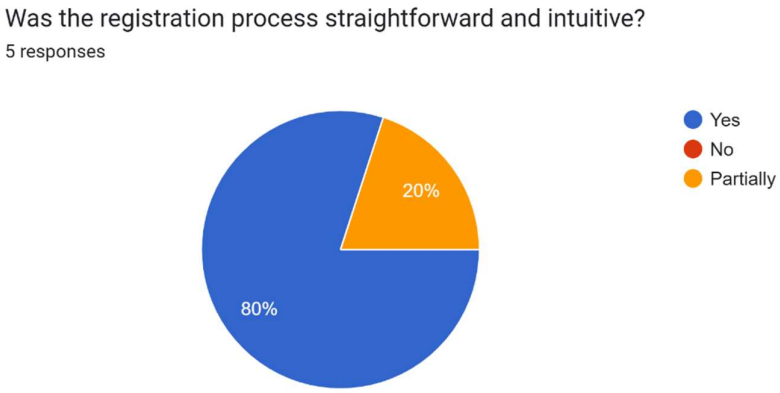


Figure 5 - 100 Ease of Registering Account

Figure 5-100 shows the result for ease of registering an account where 80% (4 out of 5) of the participants voted yes while 20% (1 out of 5) voted partially. The participant that voted partially said that there were many fields to fill in while most apps only require three fields.

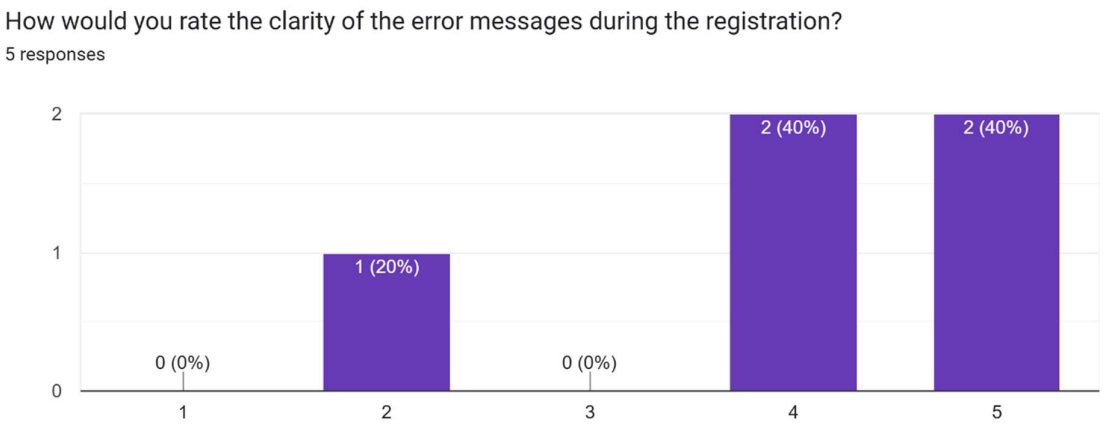


Figure 5 - 101 Clarity of Error Messages for Registration

Figure 5-101 shows the voting for the clarity of error messages for registration with 1 being Very Unclear and 5 being Very Clear. 80% (4 out of 5) voted for 4 and 5 indicating that the error messages were clear. The 20% (1 out of 5) voted for not very clear because the participant did not face any error messages during the registration process.

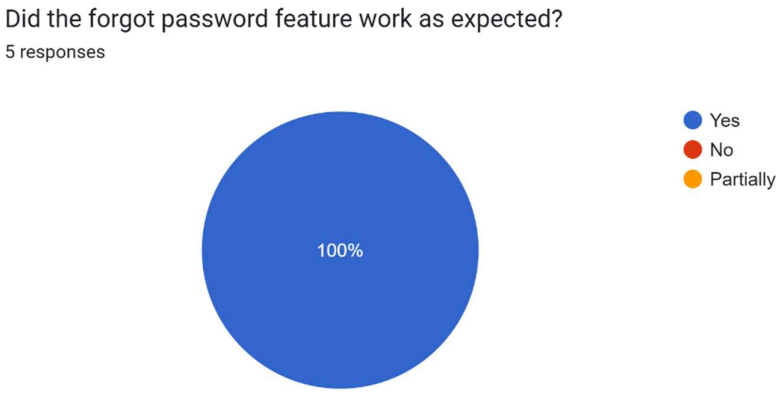


Figure 5 - 102 Forgot Password Functionality

Figure 5-102 shows the vote results for the forgot feature working as expected by the participants. As shown in the image, 100% of the participants agree that the forgot password feature work as expected and fulfils the requirements of the users.

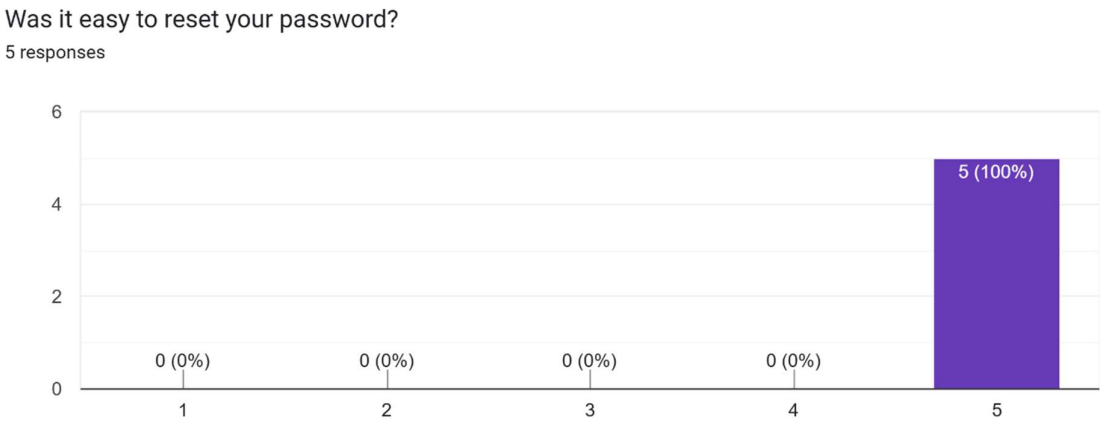


Figure 5 - 103 Ease of Resetting Password

Figure 5-103 shows the result of how easy It is for the user to reset the password with 1 being very difficult and 5 being very easy. 100% of the participants voted 5 which shows that resetting the password is very easy despite the verification requiring a few steps.

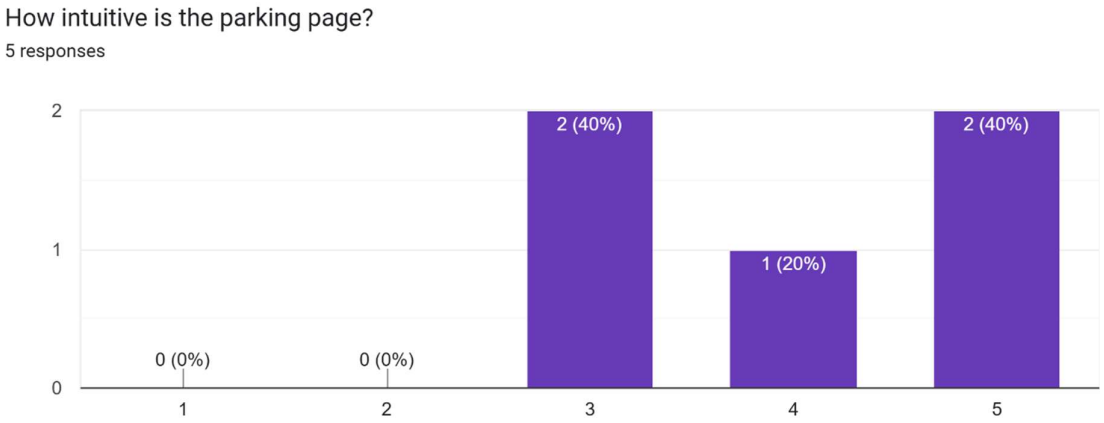


Figure 5 - 104 Intuitiveness of Parking Page

Figure 5-104 shows the results of the intuitiveness of the parking page. From the interview with the participants in section 5.3.1.1 above, most of the participants faced problems navigating due to now knowing the feature exists. Figure 5-103 reflects difficulty faced by the users where 2 participants voted 3, 1 participant voted 4, and 2 participants voted 5. The participants who voted 3 and 4 mentioned that the navigation was not intuitive while the participants to voted 5 mentioned that although the navigation was not intuitive, the green, red and blue parking lots were easy to understand thus the vote for 5.

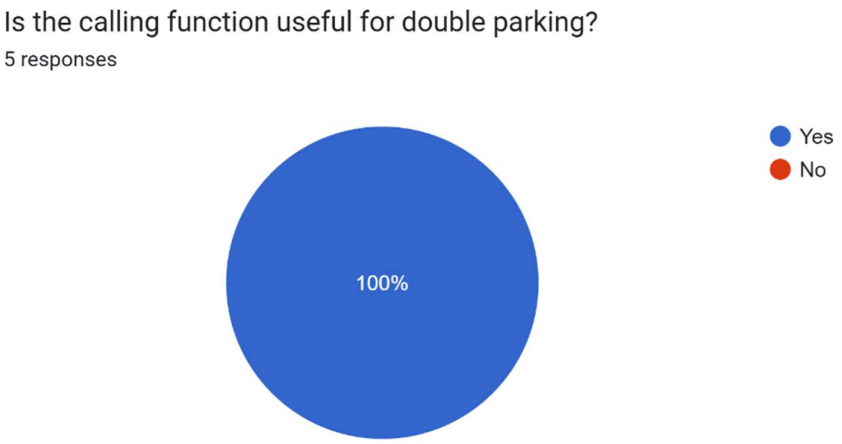


Figure 5 - 105 Usefulness of Double-Parking Function

Figure 5-105 shows the result for the usefulness the contact function. The participants found the calling function very useful as they no longer have to keep writing their phone number every time they double park. Not only that, even when they forgot to put the phone number they can do it from the classrooms through the application.

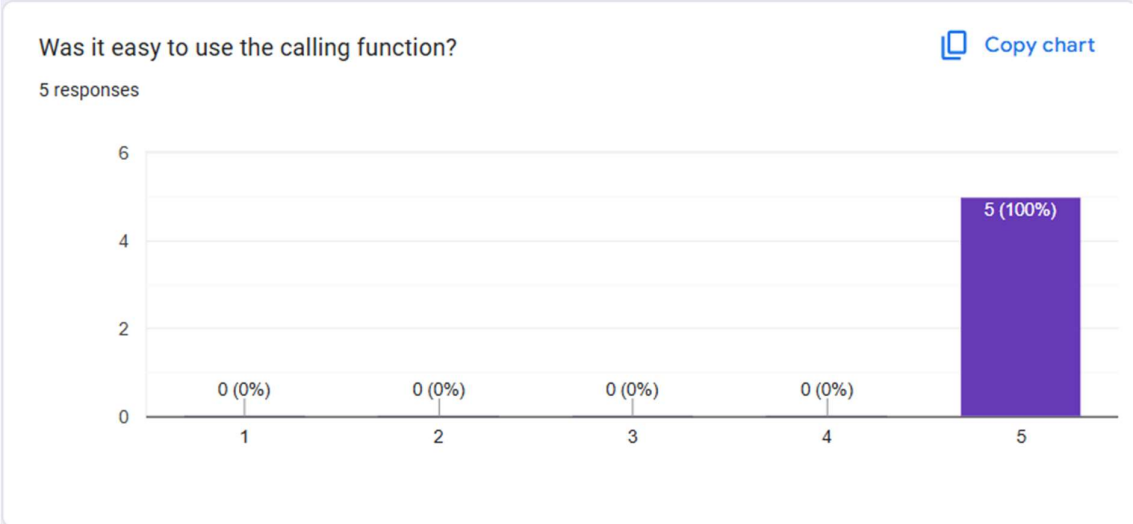


Figure 5 - 106 Ease of Calling Function

Figure 5-106 shows the easy of using the contact function with 1 being very difficult and 5 being very easy. All the participants found the contact function easy to use and fulfils their requirements.

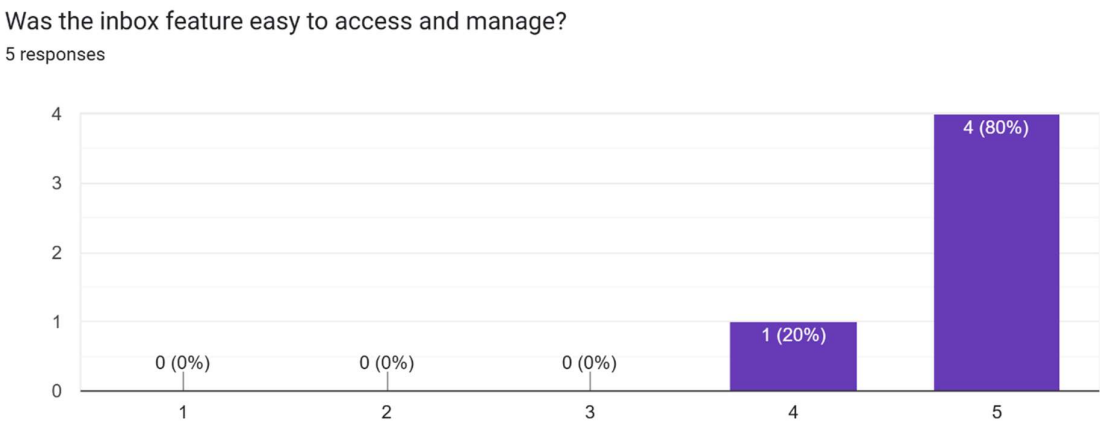


Figure 5 - 107 Ease of Accessing Inbox Feature

Figure 5-107 shows the results for ease of accessing the inbox feature with 1 being very difficult and 5 being very easy. Most of the participants voted for 5 and one voted for 4 which is still in the easy category. Thus, it is safe to say that the inbox feature is easy to access and manage.

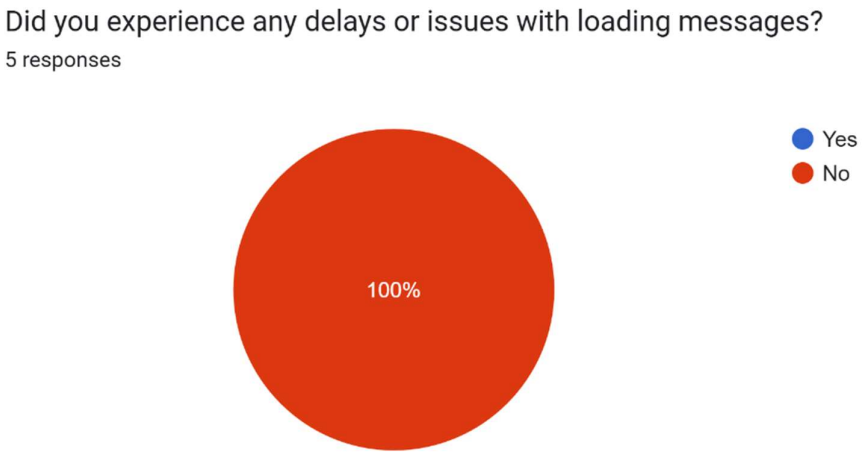


Figure 5 - 108 Delay in Loading Messages

Figure 5-107 shows the result of whether the participants faced an delay with loading messages and changes in the application. As mentioned in the tables for the interview section, most of the participants mentioned that they were impressed that the message loading and updates were fast.

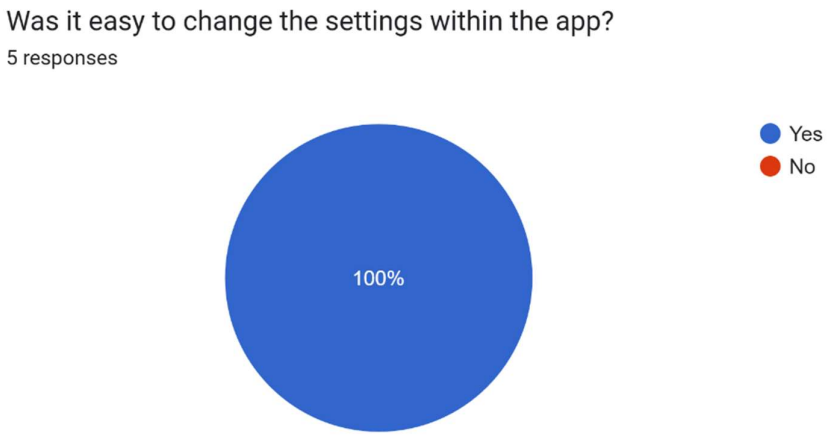


Figure 5 - 109 Ease of Changing Settings

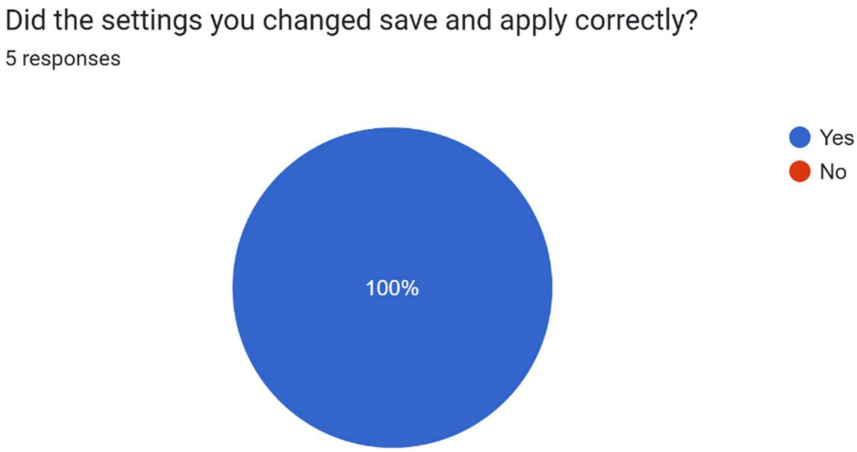


Figure 5 - 110 Settings Saved and Applied Correctly

Figure 5-109 shows the ease of changing the settings details while Figure 5-110 shows whether the changes made were saved and applied correctly. Based on the figures, it is safe to say that the participants had no issue with changing the features in the settings except for the edit and delete function for car plate numbers.

Section 2: Overall Satisfaction and Suggestions

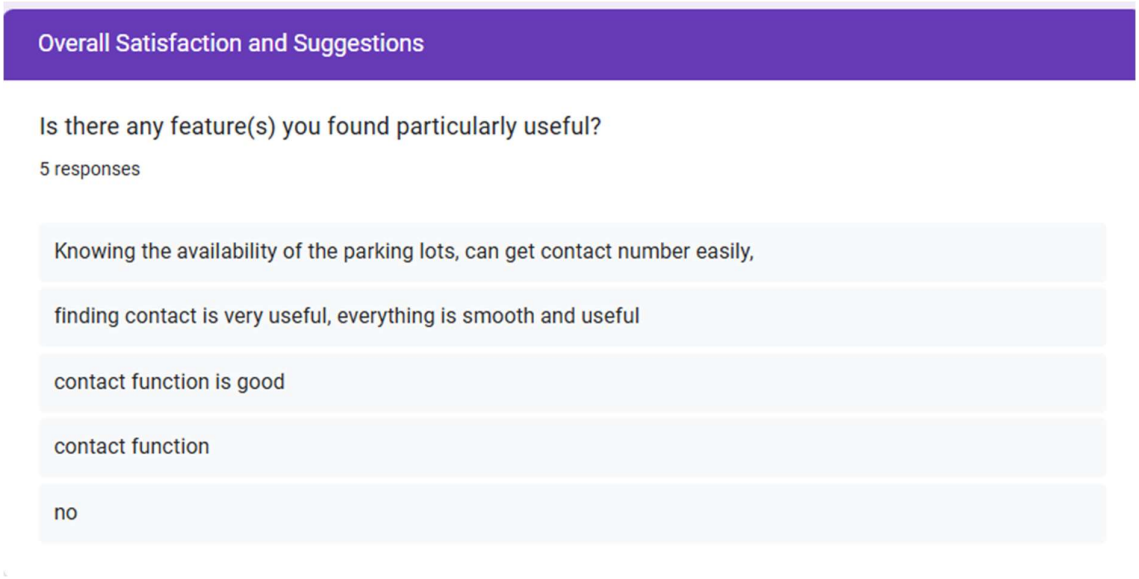


Figure 5 - 111 Useful Features

Figure 5-111 shows the feedback by the participants on which features they found particularly useful which is the contact functionality and knowing the availability of the parking lots. The users also find the application smooth.

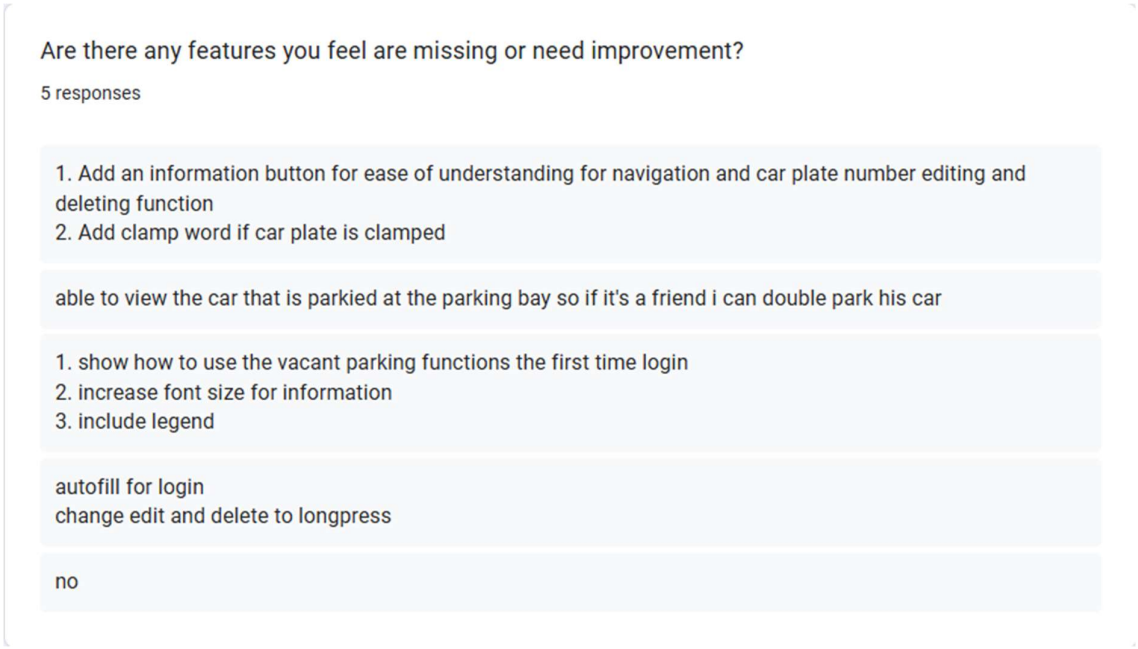


Figure 5 - 112 Suggestions for App Flaws

Figure 5-112 shows the feedback from the participants about the features that are missing and need improvement where most of the comments are about the navigation and the edit and delete button. The feedback about the clamp has been implemented.

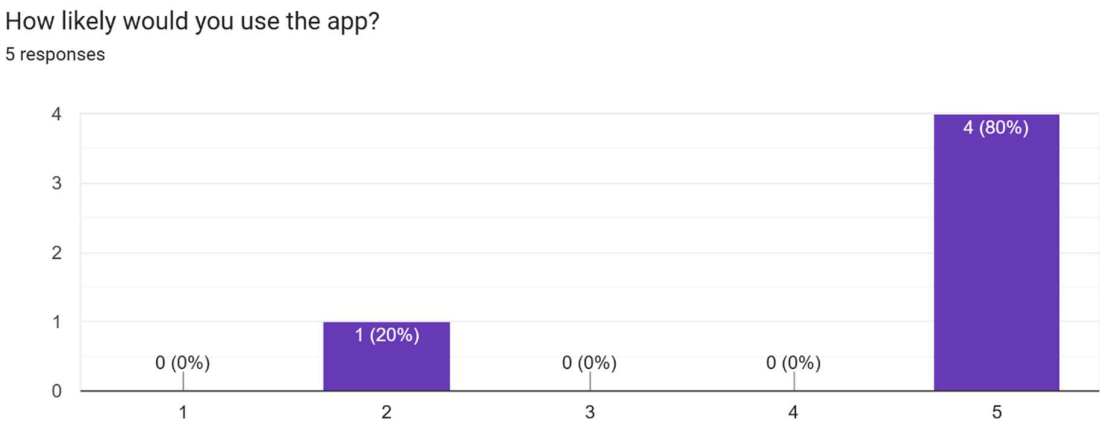


Figure 5 - 113 Likeliness to Use the Application

Figure 5-113 shows the likeliness of the participants using the application if deployed with 1 being not likely and 5 being very likely. 80% of the participants responded with very likely thus proving the usefulness of the applications to the students at INTI International College Penang.

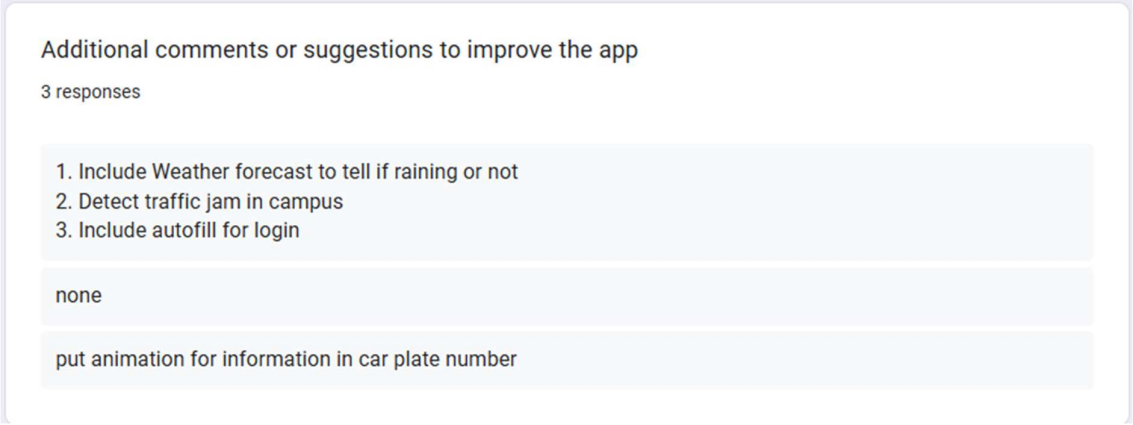


Figure 5 - 114 Suggestions for Additional Functionalities

Figure 5-114 shows the suggestions given by the participants on additional functionalities. The unique feedback is to include weather forecast and detect traffic jam in campus which will be considered for the future enhancements of the application.

5.3.1.1 (h) Conclusion for User Acceptance Testing for Mobile Application.

Based on the participants' answers in the interview and questionnaire survey, it can be concluded that IICP Park Finder's features are valid and useful with the overall impression that the application received mostly positive responses. Important notes for improvement are

mostly focused on teaching the user on the navigation features and the editing and deleting of car plate numbers where the participants have given suggestions to enhance the user experience. With this, although most of the feedback are positive, there are still room for improvement of the mobile application.

5.3.1.2 Web Application

The UAT for the web application is conducted by running the web application as localhost on my laptop. As the web application is only for the Head of AFM Department's use, Mr. Syazwan, who is the current head of the department will test the web application to determine the performance of the user experience and whether the application meets the requirements of the admin role. The test will be separated into two sections, the interview where Mr. Syazwan is asked to perform certain tasks and the questionnaire survey that documents the general comments and opinions regarding the application.

5.3.1.2 (a) Interview with Head of Admin and Facilities Department



Figure 5 - 115 User Acceptance Testing with Head of Admin and Facilities Department

The interview with Mr. Syazwan, the Head of Admin and Facilities Department was conducted on 1/11/2024 at 9:30 a.m. as shown in Figure 5-115 above. The comments and problems faced are tabulated below.

Table 5. 35 User Acceptance Testing with Head of Admin and Facilities Department

Test	Scenario	Participant's Comments	Test Results
1	Admin logs into the application	Easy.	Pass
2.	Admin selects a date to view the line graph of available parking lots for the date.	Chart shows the data correctly for the selected date.	Pass
3.	Admin selects a month to view the line graph of average available parking lots for the date.	Chart shows the data correctly for the selected month.	Pass
4.	Admin searches for student list details based on car plate number and student ID.	Table updates as I type, Data shown is accurate.	Pass
5.	Admin views the fine for a car plate number.	Same as test no.4.	Pass
6.	Admin adds, edit, delete, and pay fine for the car plate number.	All functions work properly.	Pass
7.	Admin view receipt for the fine.	Receipt added is correctly displayed.	Pass
8.	Admin views the log details.	Log details are correct.	Pass
9.	Admin searches for log details based on car plate number and student ID.	Table updates as I type and data shown is accurate. Proper validation when no data is found.	Pass
10.	Admin logs out from the application.	Good.	Pass

The table above shows the comments of Mr. Syazwan when testing the web application. There were no issues found, and he mentioned that the user interface is easy to navigate. Mr. Syazwan also mentioned that the application fulfills the requirements of AFM department in terms of fining the student and the statistics to determine when the peak hour of students parking in and leaving campus. The log details are useful to know if anyone has tampered with the application. It also serves the purpose of deleting fines as a mean to give warning to students who are new to campus and have been fined.

5.3.1.2 (b) Questionnaire Survey

Functionality

1. **Core Functions**: Do all core functions (like adding, updating, deleting records) work as expected? Yes, the core functions meet the expectations.
2. **Navigation**: Is the navigation intuitive and does it take the user where they expect to go? Yes, the navigation is user friendly
3. **Role-Based Access**: Are access controls working correctly for different admin roles, if applicable? On the control for administration system is fully under the admin control.
4. **Form Validation**: Are form validation messages accurate, and do they appear where and when expected? Yes, there is notification appear to indicate the admin user.
5. **Error Handling**: Does the system handle errors gracefully, providing useful messages and guidance? Yes, the system shoe error massage when there is an error.

Usability

6. **Ease of Use**: Are you able to understand and perform tasks without additional help or training? Yes.
7. **Interface Clarity**: Is the information displayed clearly and consistently across different pages? Yes.
8. **Response Time**: Are pages loading within an acceptable timeframe? Yes.

Data Integrity

9. **Data Accuracy**: Does data entered and displayed maintain accuracy without alterations? Yes, on the admin site student data is not editable and follow the privacy policy
10. **Data Security**: Is sensitive information properly secured, especially for locally hosted environments? Yes.

Performance

13. **Database Connection**: Is the database stable, and are there any noticeable lags or issues when performing CRUD operations? Yes

Compliance and Feedback

15. **Compliance with Requirements**: Does the website meet all specified business requirements? Yes, Follow as per the regulation on the organization and personal data.
16. **User Feedback**: Are there any suggestions from users regarding the interface, features, or other areas for improvement? Can add on for the statistic summary for reason of fine.

Figure 5 - 116 Questionnaire Survey for Web Application

Figure 5-116 shows the questionnaire survey answered by Mr.Syazwan to determine if the application fulfils the requirements in terms of functionality, usability, data integrity,

performance and compliance with the business requirements. As shown in the figure above, the web application receives mostly positive feedback indicating that the web application fulfils the requirements of the administration role of the system.

5.3.1.1 (c) Conclusion of User Acceptance Testing for Web Application

The User Acceptance Testing (UAT) conducted with Mr. Syazwan, the Head of the Admin and Facilities Department, yielded positive results, demonstrating that the web application meets the specified requirements and functions effectively. All test scenarios were executed successfully, with Mr. Syazwan providing favourable feedback on the application's usability and functionality. He expressed that the user interface is intuitive and easy to navigate, enhancing the overall user experience.

The application accurately displays data relevant to parking availability, fines, and log details, thus fulfilling the needs of the Admin and Facilities Department. Furthermore, it provides essential statistics to identify peak parking hours, which is crucial for efficient management of campus facilities. The logging feature ensures accountability and transparency, allowing the department to monitor any potential tampering with the system.

Overall, the successful outcomes of the testing and the positive comments received indicate that the application only meets the functional requirements of the AFM department but also contributes to improved operational efficiency and better service delivery for students. The findings from this UAT confirm that the application is compliant with the business requirements and effectively supports the department’s objectives.

5.4 Conclusion

This chapter provides an overview of the testing and evaluation processes conducted for both the mobile and web applications of the IICP Park Finder. Through functional testing, including unit and integration testing, the applications were assessed to ensure the specified requirements are met. The results from the various test scenarios confirm that the applications function correctly, with all key features operating as intended.

The User Acceptance Testing (UAT) conducted with users, including interviews and questionnaire surveys, validated the effectiveness of the applications. Feedback from participants confirms that the applications meet user expectations and provide useful operations for the users be it student or admin.

Overall, the testing and evaluation processes demonstrated that both applications are functional, user-friendly, and ready for deployment. The outcomes of the UAT, combined with the absence of critical issues during functional testing, affirm that the IICP Park Finder

applications are equipped to serve the intended purposes. The implementation of monitoring and user feedback mechanisms will ensure ongoing improvements and adaptations to meet evolving user needs.

CHAPTER 6: DISCUSSION

6.1 Achievements of this Project

The IICP Park Finder project is deemed a success as it has met all the proposed objectives. The mobile application developed enables INTI students to find vacant parking spaces nearest to entrance points effectively, utilizing Dijkstra's shortest path algorithm. This feature not only optimizes parking for users but also enhances their overall experience on campus by minimizing the time spent searching for parking.

Additionally, the integration of object detection within the parking facilities successfully updates available parking spaces to users. This easy to access parking availability system enhances the functionality of the application, providing users with up-to-date information that facilitates quick finding of parking spaces in campus especially during peak hours.

The implementation of line graphs in the web application allows administrative staff to strategize effectively on reducing wait times and improving traffic flow during peak hours. This analytical feature equips staff with valuable insights into parking patterns, aiding in operational efficiency and planning.

Furthermore, the project was successful in catering to different user roles, including students and administrative staff. The user interface was designed with usability in mind, and feedback from usability testing indicated that users found the application intuitive and straightforward. This positive reception affirms that the design and functionality align well with user expectations.

User acceptance testing demonstrated strong support for the IICP Park Finder system among users. Feedback from both pre-acceptance interviews and usability assessments indicated a clear demand for the application within the campus community, highlighting its necessity for enhancing the parking management process.

In summary, the project successfully addressed its objectives, implemented essential features for various user roles, and received positive validation from users, indicating that IICP Park Finder will effectively serve the parking management needs of INTI students and staff.

6.2 Limitations

6.2.1 Recorded Video Limitations

The use of recorded video for object detection presents inherent limitations, primarily concerning the timeliness and relevance of the data captured. The reason for using recorded video or the project is due to concerns of security breach which is discussed in Chapter 3.2.1 Requirements Phase. Since the video footage is not real-time, it does not accurately reflect the current availability of parking spaces, leading to discrepancies between the recorded information and the actual conditions on the ground.

6.2.2 Lack of Training Data for Object Detection Model

Another significant limitation of this project is the absence of sufficient dataset of cars necessary for training the object detection model. Without an adequate amount of labelled data, the model is not trained on a custom dataset relying solely on the pre-trained model on the default Coco128 dataset. This shortfall could result in decreased detection accuracy and performance, limiting the effectiveness of the parking space detection feature. The lack of model training may hinder the ability to generalize the model's performance, making it less reliable in real-world applications where variability in car types, colors, and parking conditions is common.

6.2.3 Fixed User Location for Navigation

The navigation functionality of the application operates based on a fixed user location, which means that while the navigation process itself is not hindered, the movement of the user is predetermined. Users are guided along a specified path rather than having their movements tracked in real time. This setup can limit the responsiveness of the navigation system to sudden changes in the environment, such as unexpected traffic. Although the application can still provide directions effectively, the fixed nature of the user's movement may not account for dynamic situations, potentially leading to a less flexible navigation experience. This limitation highlights the importance of integrating real-time data for a more adaptive navigation system in future developments.

6.3 Future Enhancements

6.3.1 Implement Real-Time Video Footage

To address the limitations posed by using recorded video footage, a future enhancement could involve implementing real-time video streaming capabilities. This would allow the system to provide live updates on parking space availability, ensuring that users receive the most current information. Integrating IP cameras capable of streaming video directly to the application could facilitate immediate data capture, which would enhance the accuracy of parking availability updates.

6.3.2 Collect Custom Data for Model Training

To overcome the challenge of insufficient training data for the object detection model, future enhancements should focus on developing a comprehensive dataset. This could be achieved by collecting and labeling a diverse array of images and video footage of cars in various conditions, environments, and angles. Furthermore, implementing a continuous learning mechanism where the model is updated regularly with new data from user interactions or real-time footage could significantly improve its accuracy and robustness in detecting and tracking vehicles.

6.3.3 Implement Real-Time Location for Navigation

To enhance the navigation experience and move beyond the limitations of fixed user movement, the application could be upgraded to include real-time location tracking capabilities. By integrating GPS functionality, the system could provide dynamic navigation that adjusts based on the user's actual movement and position. This would enable the application to respond to changing conditions and offer more precise guidance, such as rerouting users to available parking spaces as the users navigate the campus.

6.3.4 Improve User Interface for Mobile Application

As brought up by most of the participants during the User Acceptance Testing, the navigation, editing and deleting car plate numbers lacks the intuitiveness for user experience. Thus, a more user-friendly approach such as implementing animated guidelines when first logging into the application and when tapping the information button can be integrated into the application.

6.4 Conclusion

Overall the result of the project is a success as IICP Park Finder was able to meet the objectives with positive results from the functional and non-functional tests. Since the system

was conducted in a short amont of time with limited resources, the system still have limitations are discussed in this chapter which are aimed to be addressed in future releases.

CHAPTER 7: REFLECTION

7.1 Personal Reflection

When I began my journey to select a project for my final year, I initially considered various options before settling on the IICP Park Finder. My decision was heavily influenced by my own experiences as a student at INTI International College Penang, where not only I but students of INTI International College Penang often faced challenges in finding available parking spaces. As I used to be a member of the student council of INTI International College Penang, I often receive complains from students that it is very difficult to find parking in campus. This personal connection to the problem inspired me to create a solution that could genuinely improve the parking experience for my fellow students. Through this project, I rediscovered my passion for coding and the impact it can have on people's lives. It reinforced my belief that software development is not just about writing code; it is about addressing real-world issues and creating systems that make a difference to give back to the community.

The development of IICP Park Finder allowed me to enhance my skills not only in both web and mobile application development, which I had previously found challenging but also in machine learning when developing an object detection model which integrates tracking algorithm for a better accuracy. I had the opportunity to work with various technologies and frameworks, such as Firebase and Google Maps, expanding my knowledge beyond what I learned in my courses. This project pushed me out of my comfort zone and encouraged me to experiment with new ideas and solutions, ultimately enhancing my problem-solving abilities.

Additionally, this experience taught me invaluable lessons about project management and collaboration. I honed my time-management skills by balancing multiple tasks, from gathering requirements to implementing feedback from user testing and working with Mr. Syazwan from the first phase of the project until the end. The importance of clear communication became evident during the usability tests, where I learned to actively listen to users and incorporate their feedback into the project. This interaction deepened my understanding of the user experience and emphasized the importance of user-centric design in software development.

Overall, IICP Park Finder has been more than just a project for me; it has been a journey of self-discovery, learning, and growth. I feel better equipped for the challenges of the real world as I transition into my career, and I am excited to apply the skills and insights gained from this project to future endeavours.

7.2 Challenges and Solutions

The development of the IICP Park Finder system presented several challenges, particularly concerning its limitations. One major limitation was the reliance on recorded video footage for object detection due to concerns of security breach in INTI International College Penang's network. This limitation was not known during the start of the project and several steps were taken. It stared from finding using a CCTV camera to implement real-time object detection which lead to needing a wireless camera that is accessible remotely, so decided to use IP Camera, which then led to needing the higher authorities' permissions as port forwarding is required which potentially will lead to rejection as it concerns the security of the network and lengthen the requirements phase. Thus, it is decided to use a recorded video instead.

Another significant challenge arose from the lack of training data for the object detection model. Without a comprehensive dataset of car images, the model struggled to accurately recognize and track vehicles within the parking lot. This shortfall compromised the overall effectiveness of the system. To mitigate this issue, Deep SORT tracking algorithm is integrated with YOLO to accurately detect the cars in the parking lots.

Lastly, the navigation aspect of the IICP Park Finder system faced limitations due to its reliance on a user's real-time location, which necessitated continuous testing and code updates while driving. This approach posed significant risks, as focusing on coding and system performance during driving could distract me, increasing the likelihood of accidents and compromising the safety of both the developer and other road users. Moreover, the unpredictable nature of real-world driving conditions, including weather changes and varying traffic patterns, added complexity to the development process. To mitigate these risks, simulation techniques to test the navigation algorithms without the need for actual driving, allowing for assessments of system responsiveness under controlled conditions are implemented.

In summary, while developing the IICP Park Finder system faced significant challenges, there are viable solutions to address these limitations. By actively solving problems as the problems arise, the IICP Park Finder managed to mitigate the challenges and meet the objectives of the project. These experiences concurrently enhance my problem-solving skills, contributing to my overall development

7.3 Conclusion

In conclusion, the journey of developing the IICP Park Finder has been profoundly enriching, allowing me to not only address a significant issue faced by students but also to grow personally and professionally. This project reinforced my belief in the power of technology to create meaningful solutions and deepened my understanding of software development

beyond mere coding. The challenges encountered, from limitations in data acquisition to the complexities of real-time navigation, provided invaluable learning experiences that enhanced my problem-solving abilities and project management skills. As I transition into my career, I feel well-prepared to tackle future challenges, armed with the insights and knowledge gained from this project, and I am eager to apply these skills in my future endeavours.

CHAPTER 8: CONCLUSION

In conclusion, this report comprehensively details the development and implementation of the IICP Park Finder system, aimed at alleviating parking challenges faced by students at INTI International College Penang. The project effectively addressed the needs identified through stakeholder input, particularly through user surveys that highlighted the significant difficulties students experienced in locating available parking spaces. By integrating various technologies, including object detection and machine learning, along with effective navigation algorithms, the IICP Park Finder has demonstrated its capability to enhance the parking experience on campus. Challenges encountered throughout the project, such as limitations in video data and the absence of a training dataset for the object detection model, were tackled with innovative solutions, ensuring the project's objectives were met. Furthermore, the emphasis on user-centric design, supported by extensive usability testing and feedback, underscored the system's potential for improving user satisfaction and operational efficiency. Overall, the IICP Park Finder not only fulfils its intended purpose but also provides valuable insights into the ongoing development of smart solutions for campus management. The skills and knowledge acquired during this project have prepared myself for future endeavours in the field of software development, reinforcing my commitment to creating impactful technological solutions.

References

AKE Parking Technology, 2020. *Indoor PGS-Ultrasonic Series.* [Online]
Available at:
https://www.akeparking.com/parkingsolutions/indoorparkingguidance/UltrasonicSeries/list/37.html
[Accessed 6 June 2024].

Amazon Web Services, Inc. , 2024. *Mongo DB Atlas (pay-as-you-go).* [Online]
Available at: https://aws.amazon.com/marketplace/pp/prodview-pp445qepfdy34
[Accessed 5 June 2024].

Amgad, M. et al., 2017. *A Survey of Shortest-Path Algorithms.* [Online]
Available at: https://arxiv.org/pdf/1705.02044
[Accessed 3 Juna 2024].

Andrew, V. G. & Tomasz, R., 1993. *A Heuristic Improvement of Bellman-Ford Algorithm.* [Online]
Available at: https://apps.dtic.mil/sti/tr/pdf/ADA322742.pdf
[Accessed 5 June 2024].

Balasubramanian, R., Beaulah, J. & Anbarasa, P., 2016. *A Survey: Analysis on Pre-processing and Segmentation Techniques for Medical Images.* [Online]
Available at: chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.researchgate.net/profile/Anbarasa-Pandian/publication/305502844_A_Survey_Analysis_on_Pre-processing_and_Segmentation_Techniques_for_Medical_Images/links/5792520a08aed51475aed3f5/A-Survey-Anal
[Accessed 20 May 2024].

Ben, H., 2023. *The Pros and Cons of Firestore and Realtime Database in Firebase.* [Online]
Available at: https://giraffemindset.medium.com/the-pros-and-cons-of-firestore-and-realtime-database-in-firebase-fbf812d3438#:~:text=Firestore%20also%20has%20a%20more,it%20easier%20to%20work%20with.&text=The%20main%20disadvantage%20of%20Firestore,more%20difficult%20to%
[Accessed 8 June 2024].

Buchanan Technologies, 2024. *Buchanan Technologies.* [Online]
Available at: https://www.buchanan.com/advantages-of-cloud-database/
[Accessed 7 June 2024].

Buletin Mutiara, 2023. *Smart sensors for designated MBPP parking lots installed.* [Online]
Available at: https://www.buletinmutiara.com/smart-sensors-for-designated-mbpp-parking-lots-installed/
[Accessed 9 June 2024].

Cloudinary, 2024. *Region-Based Segmentation.* [Online]
Available at: https://cloudinary.com/glossary/region-based-segmentation#:~:text=At%20its%20core%2C%20region%2Dbased,manageable%20and%20easier%20to%20analyze.
[Accessed 21 May 2024].

Codecademy, 2024. *A* Search.* [Online]
Available at: https://www.codecademy.com/resources/docs/ai/search-algorithms/a-star-search
[Accessed 4 June 2024].

Dhruv, K., 2021. *Introduction to Waterfall Model of SDLC (Software Development Life Cycle).* [Online]
Available at: https://luciferrocks.medium.com/introduction-to-waterfall-model-of-sdlc-software-development-life-cycle-a01c7ec5e2ef
[Accessed 11 June 2024].

Dong, P. T., 2013. A Review on Image Feature Extraction and Representation Techniques. *International Journal of Multimedia and Ubiquitous Engineering ,* 8(4), p. 12.

Engati Technologies, 2024. *Convolutional neural network.* [Online]
Available at: https://www.engati.com/glossary/convolutional-neural-network
[Accessed 2 June 2024].

Ezarik, M., 2022. *Perceived Campus Parking Problems and the Factors That Influence Them.* [Online]
Available at: https://www.insidehighered.com/news/2022/12/14/campus-parking-problems-common-yet-not-universal-infographic
[Accessed 13 April 2024].

Filzani, I. I., Muhammad, F. & Myzatul, A. K., 2020. *Parking Spaces in Taylor's University: Problems and solutions.* Kuala Lumpur, e-International Publishing House.

Flutter, 2024. *Flutter Documentation.* [Online]
Available at: https://docs.flutter.dev/
[Accessed 12 June 2024].

Franz, J. O., Nicola, U. & Hermann, N., n.d. *AnEfficient A* Search Algorithm for Statistical Machine Translation,* Aachen: Computer Science Department RWTH Aachen- University of Technology.

GeeksforGeeks, 2024. *A* Search Algorithm.* [Online]
Available at: https://www.geeksforgeeks.org/a-search-algorithm/
[Accessed 6 June 2024].

GeeksforGeeks, 2024. *Bellman Ford Algorithm (Simple Implementation).* [Online]
Available at: https://www.geeksforgeeks.org/bellman-ford-algorithm-simple-implementation/
[Accessed 2 June 2024].

GeeksforGeeks, 2024. *Bellman-Ford Algorithm.* [Online]
Available at: https://www.geeksforgeeks.org/bellman-ford-algorithm-dp-23/
[Accessed 4 June 2024].

GeeksforGeeks, 2024. *Convolutional Neural Network (CNN) in Machine Learning.* [Online]
Available at: https://www.geeksforgeeks.org/convolutional-neural-network-cnn-in-machine-learning/
[Accessed 28 May 2024].

GeeksforGeeks, 2024. *Introduction to Convolution Neural Network.* [Online]
Available at: https://www.geeksforgeeks.org/introduction-convolution-neural-network/
[Accessed 27 May 2024].

GeeksforGeeks, 2024. *Time and Space Complexity of Dijkstra's Algorithm.* [Online]
Available at: https://www.geeksforgeeks.org/time-and-space-complexity-of-dijkstras-algorithm/
[Accessed 4 June 2024].

Ghassem, T., Soosan, B., Kaamran, R. & Helia, M., 2013. *Hand posture recognition using K-NN and Support Vector Machine classifiers evaluated on our proposed HandReader dataset.* Toronto, ResearchGate.

GitHub, 2024. *Github Docs.* [Online]
Available at: https://docs.github.com/en
[Accessed 12 June 2024].

Google Cloud, 2024. *What is a Cloud Database.* [Online]
Available at: https://cloud.google.com/learn/what-is-a-cloud-database
[Accessed 7 June 2024].

Google for Developers, 2024. *Cloud Firestore.* [Online]
Available at: https://firebase.google.com/docs/firestore
[Accessed 8 June 2024].

GoogleforDevelopers, 2024. *Google Maps Platform.* [Online]
Available at: https://developers.google.com/maps/support/
[Accessed 10 October 2024].

Graham, C., 2024. *A* Pathfinding Algorithm.* [Online]
Available at: https://www.baeldung.com/cs/a-star-algorithm#:~:text=The%20time%20complexity%20of%20A,nodes%20on%20the%20resulting%20path.
[Accessed 6 June 2024].

iStock, 2024. *Empty parking lots.* [Online]
Available at: https://www.istockphoto.com/photo/empty-parking-lots-aerial-view-gm636444558-112911121
[Accessed 20 May 2024].

Jamil, A. M. S. & Mahgoub, H. H., 2016. Gradient Based Image Edge Detection. *International Journal of Engineering and Technology ,* 8(3), p. 5.

Jashojit, M. et al., 2016. Grayscale Conversion of Histopathological Slide Images as a Preprocessing Step for Image Segmentation. *International Journal of Software Engineering and Its Applications ,* 10(1), p. 12.

Jati, N. L. M. .. & Arrie, K., n.d. *Comparison of CNN and SVM for Ship Detection in,* s.l.: s.n.

Javatpoint, 2024. *A* Search Algorithm in Artificial Intelligence.* [Online]
Available at: https://www.javatpoint.com/ai-informed-search-algorithms
[Accessed 6 June 2024].

JavatPoint, 2024. *Classification Algorithm in Machine Learning.* [Online]
Available at: https://www.javatpoint.com/classification-algorithm-in-machine-learning
[Accessed 27 May 2024].

Jonathan, T., 2024. *MongoDB: Advantages and Disadvantages Your Enterprise Should Consider.* [Online]
Available at: https://www.koombea.com/blog/mongodb-advantages-and-disadvantages/#:~:text=MongoDB%20is%20a%20NoSQL%20database,or%20complex%20relational%20data%20modeling.
[Accessed 9 June 2024].

Junzhao, L., Mohamed, M. & Mohamed, D., 2013. *A Multi-Classifier Image Based Vacant Parking Detection System,* Saudi Arabia: King Fahd University of Petroleum and Minerals.

Kaan, E., 2023. *Introduction to Object Detection: Vehicle Detection with OpenCV and Cascade Classifiers.* [Online]
Available at: https://medium.com/@kaanerdenn/introduction-to-object-detection-vehicle-detection-with-opencv-and-cascade-classifiers-8c6834191a0b
[Accessed 25 May 2024].

Klause, R. D. & Andreas, G., 2001. *Component Database Systems.* s.l.:s.n.

Krig, S., 2016. Image Pre-Processing. In: *Computer Vision Metrics.* s.l.:Springer International Publishing Switzerland, p. 45.

Lev, C., 2024. *convolutional neural network (CNN).* [Online]
Available at: https://www.techtarget.com/searchenterpriseai/definition/convolutional-neural-network
[Accessed 2 June 2024].

Li, Z., Xiaochen, H., Liying, J. & Jing, Z., 2016. *A Color Image Edge Detection Algorithm Basedon Color Difference.* [Online]
Available at:
https://www.researchgate.net/publication/306533781_A_Color_Image_Edge_Detection_Algorithm_Based_on_Color_Difference
[Accessed 24 May 2024].

Magzhan, K. & Hajar, M. J., 2013. A Review and Evaluations of Shortest Path Algorithms. *International Journal of Scientific & Technology Research ·,* 2(6), p. 7.

MatBotix, 2019. *Ultrasonic Sensors: Advantages and Limitations.* [Online]
Available at: https://maxbotix.com/blogs/blog/advantages-limitations-ultrasonic-sensors
[Accessed 10 June 2024].

Mei, R. Q., 2021. *Easily Find Parking & Transfer Credit With Penang Smart Parking's Latest Updates.* [Online]
Available at: https://penangfoodie.com/easily-find-parking-transfer-credit-with-penang-smart-parkings-latest-updates/
[Accessed 5 June 2024].

Mikhail, B. & Partha, N., 2008. Towards a theoretical foundation for Laplacian-based manifold methods. *Journal of Computer and System Sciences 74,* p. 20.

MongoDB Atlas, 2024. *MongoDB Atlas. The multi-cloud developer data platform.* [Online]
Available at: https://www.mongodb.com/atlas
[Accessed 9 June 2024].

Munira, O., 2018. *An Intro to Git and GitHub for Beginners Part I (Tutorial).* [Online]
[Accessed 12 June 2024].

Neowin, 2024. *Google Earth 7.1.4.1529.* [Online]
Available at: https://www.neowin.net/software/google-earth-7141529/
[Accessed 12 October 2024].

Nicholas, T., n.d. *Vacant Parking Space Detection in Static Images,* San Diego: Univaersity of California.

Nielsen Norman Group, 2000. *Why You Only Need to Test with 5 Users.* [Online]
Available at: https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/
[Accessed 3 November 2024].

Norlia, M., 2016. Smart Parking System (SPS) Architecture Using Ultrasonic Detector. *International Journal of Software Engineering and Its Applications ,* 6(3), p. 9.

OpenCV, 2024. *Canny Edge Detection.* [Online]
Available at: https://docs.opencv.org/4.x/da/d22/tutorial_py_canny.html
[Accessed 29 May 2024].

Oracle, 2020. *What Is a Database?.* [Online]
Available at: https://www.oracle.com/my/database/what-is-database/#:~:text=A%20database%20is%20an%20organized,database%20management%20system%20(DBMS).
[Accessed 7 June 2024].

Oracle, n.d. [Online].

Paula, T. et al., 2019. AnImageFeature-Based Method for Parking Lot Occupancy. *Future Internet ,* 11(169), p. 17.

Phoaivu, 2011. *Extended GMM for Background Subtraction on GPU.* [Online]
Available at: https://www.codeproject.com/Articles/142859/Extended-GMM-for-Background-Subtraction-on-GPU
[Accessed 22 May 2024].

Piccardi, M., 2004. *Background subtraction techniques: a review.* Sydney, IEEE International Conference on Systems, Man and Cybernetics .

Priyanka, M. & Bidyut, B. C., 2014. A Survey of Hough Transform. *Pattern Recognition,* p. 18.

QGIS, 2024. *Visual Style Guide.* [Online]
Available at: https://qgis.org/styleguide/
[Accessed 20 October 2024].

Ravikiran A, S., 2024. *A* Algorithm Concepts and Implementation.* [Online]
Available at: https://www.simplilearn.com/tutorials/artificial-intelligence-tutorial/a-star-

algorithm
[Accessed 6 June 2024].

Sabih, M., 2022. *Background subtraction in computer vision.* [Online]
Available at: https://medium.com/@muhammadsabih56/background-subtraction-in-computer-vision-402ddc79cb1b
[Accessed 29 May 2024].

Satya, B. V., n.d. *Edge detection and line detection in Image processing.* [Online]
Available at: chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.bbau.ac.in/dept/CS/TM/Edge%20detection%20.pdf
[Accessed 24 May 20224].

ScienceDirect, 2023. *Dijkstra Algorithms.* [Online]
Available at: https://www.google.com/url?sa=i&url=https%3A%2F%2Fwww.sciencedirect.com%2Ftopics%2Fcomputer-science%2Fdijkstra-algorithms&psig=AOvVaw3bGQopvozxCAw_HU1EZjZM&ust=1718544124483000&source=images&cd=vfe&opi=89978449&ved=0CBEQjRxqFwoTCNDAw_TZ3YYDFQAAAAAdAAAAABA
[Accessed 1 June 2024].

Sents, R., 2023. *With over 25,000 spaces, students continue to struggle with campus parking.* [Online]
Available at: https://thecrimsonwhite.com/108029/top-stories/with-over-25000-spaces-students-continue-to-struggle-with-campus-parking/
[Accessed 6 April 2024].

Sharma, P., 2023. *A Step-by-Step Introduction to the Basic Object Detection Algorithms (Part 1).* [Online]
Available at: https://www.analyticsvidhya.com/blog/2018/10/a-step-by-step-introduction-to-the-basic-object-detection-algorithms-part-1/
[Accessed 1 June 2024].

Sumitkrsharma, 2023. *Feature Detection and Description in Computer Vision.* [Online]
Available at: https://sumit-kr-sharma.medium.com/feature-detection-and-description-in-computer-vision-657171e43d19
[Accessed 26 May 2024].

Sungchul, K. & Hwangjo, Y., 2012. Classification, Regression, and Ranking. *SVM Tutorial*, January, p. 33.

Supun, M., 2020. *A practical guide to Firestore.* [Online]
Available at: https://medium.com/@supun.muthutantrige/a-practical-guideline-to-firestore-50281732bcbc
[Accessed 5 June 2024].

Tutorialspoint, 2024. *SDLC - Waterfall Model.* [Online]
Available at: https://www.tutorialspoint.com/sdlc/sdlc_waterfall_model.htm
[Accessed 11 June 2024].

Ultralytics Inc., 2024. *Brand Guidelines.* [Online]
Available at: https://www.ultralytics.com/brand
[Accessed 21 October 2024].

Ultralytics, 2024. *Ultralytics.* [Online]
Available at: https://github.com/ultralytics/ultralytics
[Accessed 20 October 2024].

Vipin, 2024. *A Guide to Edge Detection and Line Detection in Computer Vision using OpenCV.* [Online]
Available at: https://medium.com/@vipinra79/a-guide-to-edge-detection-and-line-detection-in-computer-vision-using-opencv-dba499b5b4b3
[Accessed 25 May 2024].

Vue.js, 2024. *The Progressive.* [Online]
Available at: https://vuejs.org/
[Accessed 10 October 2024].

Wojciech, B., Szymon, G. & Wojciech, R., 2007. *Image Preprocessing for Improving OCR Accuracy ,* UKRAINE: s.n.

Yeshwanth, S. C., Nanda, I. & Rao, K. P., 2020. *Image Classification using SVM and CNN,* s.l.: University of Southern Queensland.

Yong, M., Yangguo, L., Lin, Z. & Yuanlong, C., 2021. *Research Review on Parking Space Detection Method.* [Online]
Available at:
https://www.researchgate.net/publication/348454518_Research_Review_on_Parking_Space_Detection_Method
[Accessed 10 June 2024].

Zaid, H. G., Ahmed, A. R. & Ayad, H. Q., 2021. Comparison Between Convolutional Neural Network CNN and. *JOURNAL OF TECHNIQUES,* p. 8.

Appendix

Link to Github Repo for Mobile Application:

https://github.com/NiFeih/ipark_finder1

Link to Github Repo for Web Application:

https://github.com/NiFeih/IICP-Park-Finder-Admin

Link to Github Repo for Object Detection Model:

https://github.com/NiFeih/IICP_Park_Finder_Object_Detection

Safe Assign Report:

